

Python Programming

High performance. Delivered.

consulting | technology | outsourcing



Goals

- Modules Basics
- Design Pattern
- Rest API best practices
- Microservice using Django in depth

Python's Benevolent Dictator For Life



Guido van Rossum





Python Basics

- ★ 1. What is Python?
- Python is a **high-level, interpreted, general-purpose** programming language.
- **Key characteristics**
- **Interpreted** → runs line-by-line (no compilation step).
- **Dynamically typed** → variables do not need type declarations.
- **High-level** → closer to human language.
- **Object-oriented** → everything in Python is an object.
- **Cross-platform** → runs on Windows/Linux/macOS.
- **Huge ecosystem** → NumPy, Pandas, Django, Flask, TensorFlow, etc.



Python Basics

- ★ 2. Python Execution Model
- ♦ Interpreter Flow
- You write code → .py file
- Python converts code → **bytecode (.pyc)**
- Python Virtual Machine (PVM) executes bytecode
- Results shown
- Python does **not** create machine code directly like C/C++.

PYTHON MEMORY MANAGEMENT



- Python memory handling is done through:
 - **Memory Architecture**
 - **Reference Counting**
 - **Garbage Collection (GC)**
 - **Pointers and Object Model**
 - **Stack vs Heap**
 - **Memory Pools / Allocators (PyMalloc)**
 - **Interning**

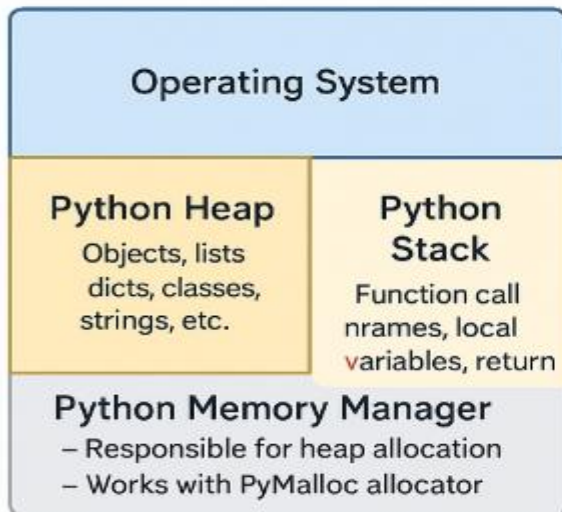
PYTHON MEMORY MANAGEMENT



- Python memory handling is done through:
 - **Memory Architecture**
 - **Reference Counting**
 - **Garbage Collection (GC)**
 - **Pointers and Object Model**
 - **Stack vs Heap**
 - **Memory Pools / Allocators (PyMalloc)**
 - **Interning**

1. Python Memory Architecture

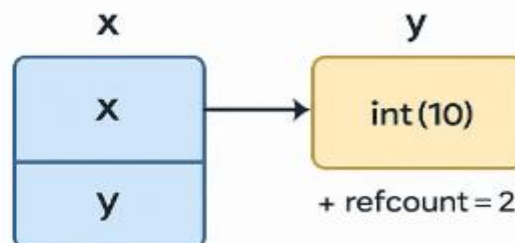
High-Level Memory Layout



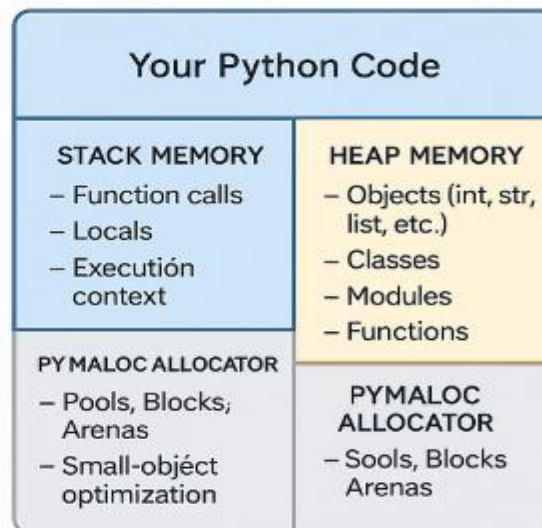
Generation-Based Garbage Collection



Stack vs Heap in Python



MASTER DIAGRAM



2. Stack vs Heap in Python

-  **Stack Memory**
- Used for:
 - Function call frames
 - Local variables (but they only store *references*)

```
def add(a, b):  
    c = a + b  
    return c
```

STACK (LIFO)

```
+-----+  
| Frame: add() |  
| a -> ref to int |  
| b -> ref to int |  
| c -> ref to int |  
+-----+
```

2. Stack vs Heap in Python

- **◆ Heap Memory**
- Where the actual objects live

HEAP

```
+-----+  
| int object 10 |  
| int object 20 |  
| int object 30 |  
+-----+
```

3. Python Objects & Pointers

- Every Python variable is just a **pointer/reference** to an object in the heap.

```
x = 10
```

```
y = x
```

STACK

```
x _____  
y _____
```

HEAP

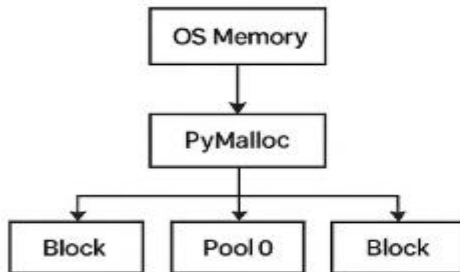
```
+-----+  
| int(10) |  
+-----+  
| refcount = 2 |  
+-----+
```



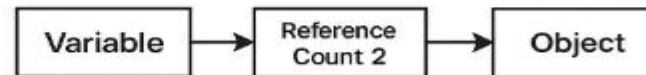
2. All Python Memory Management Diagrams



PyMalloc Architecture



Reference Counting



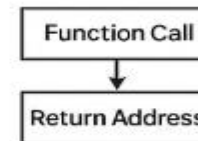
Circular Reference Detection



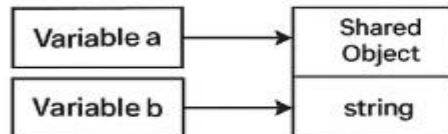
Circular Reference Detection



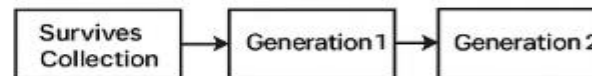
Stack Frame



Object Lifecycle



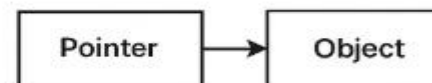
Generational Garbage Collection



Interning



Heap Memory



3. Python Objects & Pointers

- Why refcount = 2?

```
x = 10
```

```
y = 10
```

- Python interns small integers (from -5 to 256).
So, the value 10 exists once in memory, and both variables x and y point to the same object.
- So, Python keeps:
- Object: 10
- Reference count: 2 <-- because x and y point here



4. Reference Counting (CORE of CPython)



- Every object has a **refcount**:
“How many references are pointing to me?”

```
ini
```

```
x = [1,2,3]
y = x
z = x
```

REFCOUNT = 3

Diagram

```
lua
```

```
STACK
```

```
+-----+ +-----+
| x ----> | LIST OBJECT |
+-----+ | [1,2,3]      |
| y ----> | refcount = 3 |
+-----+ +-----+
| z ----> |
+-----+
```

5. Garbage Collection — Generation Based

- Python has **three generations** of garbage.
- **WHY?**
- Most objects die young → optimize cleanup.
- **Generations:**
- **Gen0** → youngest (frequently collected)
- **Gen1**
- **Gen2** → oldest (rarely collected)

GC HEAP

```
+-----+ +-----+ +-----+
| Gen 0   | | Gen 1   | | Gen 2   |
| (new objs) | | survivors | | long-lived |
+-----+ +-----+ +-----+
```



6. How Python Determines What to Delete

- **Two phases:**
- **1 Reference counting**
- Immediate deletion when `refcount = 0`
- **2 Cycle Detection**
- For objects referencing each other (common in lists, classes)



6. How Python Determines What to Delete

Example circular reference:

```
go  
  
a = []  
b = []  
a.append(b)  
b.append(a)
```

refcount never becomes 0 → GC must detect.



6. How Python Determines What to Delete

Diagram

lua

a ----> [b]

^

|

|

v

[b] <---- a

GC breaks these cycles.



7. PyMalloc — Python's Memory Allocator

- Python does **NOT** use OS malloc() for every object.
- It uses:
 - **PyMalloc** for small objects
 - Blocks: 8 KB
 - Pools of 64-byte classes
 - Arenas (256 KB)
 - This makes Python memory **fast, cached, reusable**.

OS MEMORY

|

v

+-----+

| ARENA 0 (256 KB) |

| + Pool 0 |

| + Pool 1 |

| + Pool 2 |

+-----+

| ARENA 1 (256 KB) |

| + Pool 3 |

| + Pool 4 |

+-----+



7. PyMalloc — Python's Memory Allocator



- Python does **NOT** use OS malloc() for every object.
- It uses:
 - **PyMalloc** for small objects
 - Blocks: 8 KB
 - Pools of 64-byte classes
 - Arenas (256 KB)

8. Small Object Optimization / Interning

- Python optimizes memory by *reusing* objects for:
-  **Small integers (between -5 to 256)**
- These are **preallocated**.

These are preallocated.

```
ini
```

```
a = 10
```

```
b = 10
```

8. Small Object Optimization / Interning

sql

STACK

a ————
|
b ————

HEAP

```
+-----+  
| int 10 |  
| refcount = 2 |  
+-----+
```

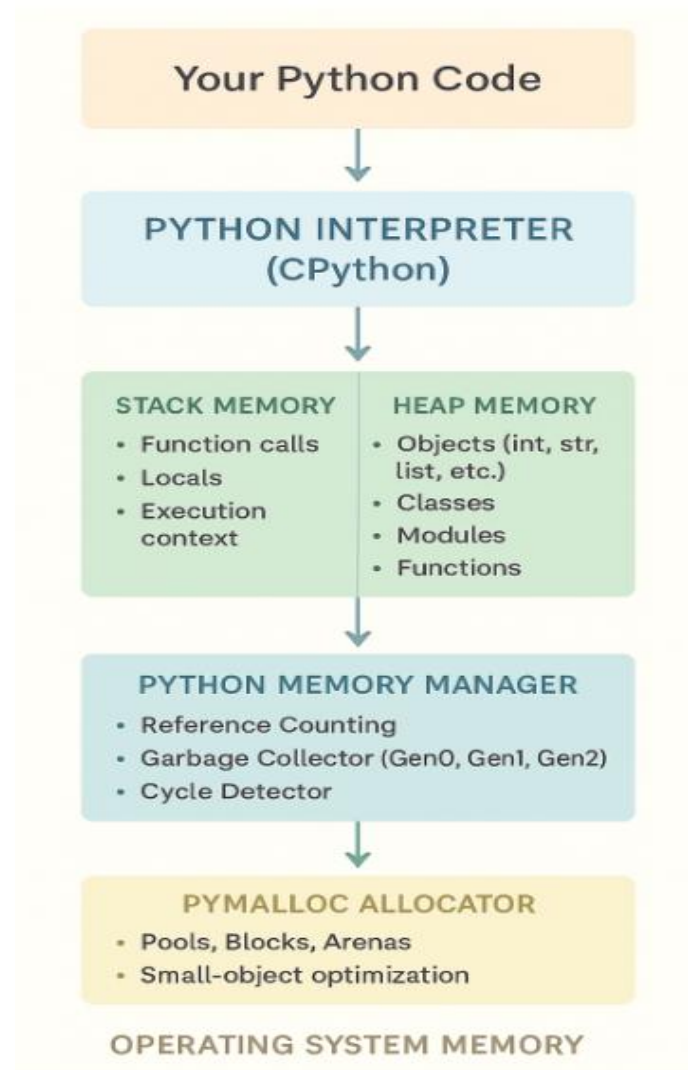
- ◆ String interning
- Common strings reused:
- identifiers
 - small strings
 - compile-time constants

9. Memory Leaks in Python (Yes, they happen)

- Situations causing leaks:
- Global variables holding references
- Circular references with `__del__`
- Large objects kept in caches
- Unclosed file handles
- Third-party C-extensions

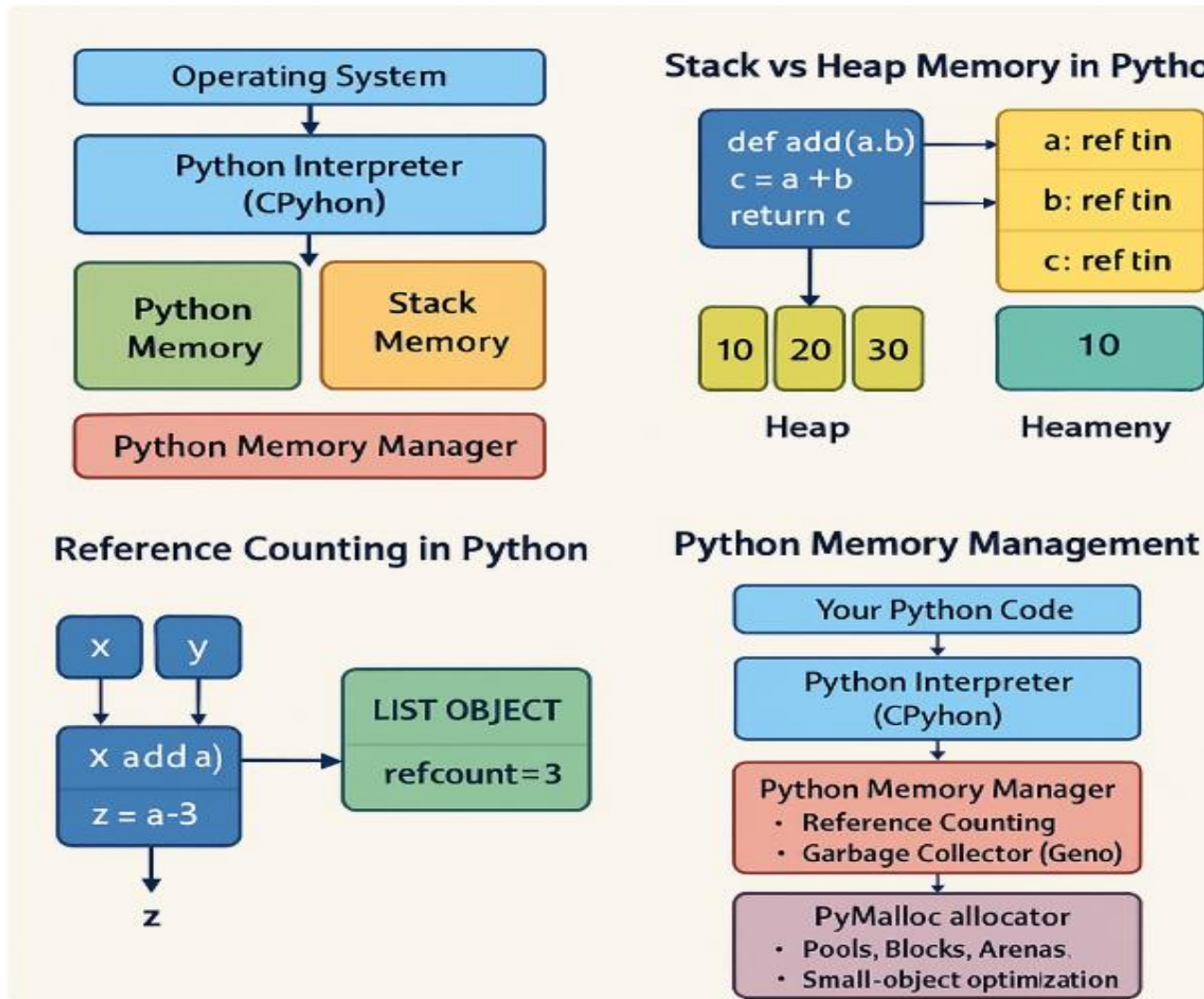


10. Putting It All Together (MASTER DIAGRAM)





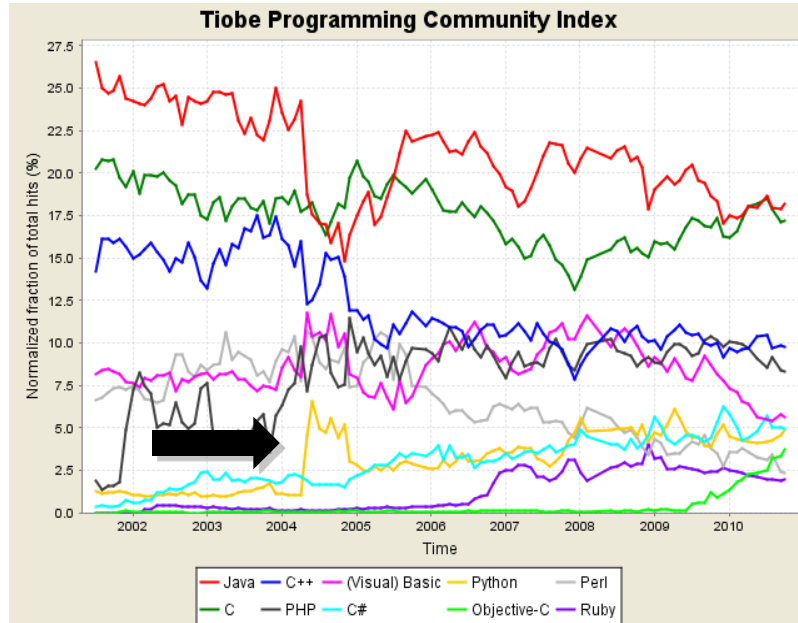
All Python Memory Management Diagrams



Python's place in the Market



- TIOBE has been collecting data on programming language “popularity” for many years
- Counts results for a query like "<language> programming" on popular search engines



Position Oct 2010	Position Oct 2009	Delta in Position	Programming Language	Ratings Oct 2010	Delta Oct 2009	Status
1	1	=	Java	18.166%	-0.48%	A
2	2	=	C	17.177%	+0.33%	A
3	4	↑	C++	9.802%	-0.08%	A
4	3	↓	PHP	8.323%	-2.03%	A
5	5	=	(Visual) Basic	5.650%	-3.04%	A
6	6	=	C#	4.963%	+0.55%	A
7	7	=	Python	4.860%	+0.96%	A
8	12	↑↑↑↑	Objective-C	3.706%	+2.54%	A
9	8	↓	Perl	2.310%	-1.45%	A
10	10	=	Ruby	1.941%	-0.51%	A
11	9	↓↓	JavaScript	1.659%	-1.37%	A
12	11	↓	Delphi	1.558%	-0.58%	A
13	17	↑↑↑↑	Lisp	1.084%	+0.48%	A-
14	24	↑↑↑↑↑↑↑↑	Transact-SQL	0.820%	+0.42%	A-
15	15	=	Pascal	0.771%	+0.10%	A-
16	18	↑↑	RPG (OS/400)	0.708%	+0.12%	A-
17	29	↑↑↑↑↑↑↑↑	Ada	0.704%	+0.40%	A-
18	14	↓↓↓	SAS	0.664%	-0.14%	B
19	19	=	MATLAB	0.627%	+0.05%	B
20	-	↑↑↑↑↑↑↑↑	Go	0.626%	+0.63%	B



The screenshot shows the Python.org website in a web browser. The browser's address bar displays 'python.org'. The website features the Python logo and a search bar. A left sidebar contains navigation links such as 'ABOUT', 'NEWS', 'DOCUMENTATION', 'DOWNLOAD', 'COMMUNITY', 'FOUNDATION', and 'CORE DEVELOPMENT'. The main content area is titled 'Python Programming Language – Official Website' and includes a bold introductory paragraph, a list of supported operating systems, a statement on the language's freedom of use, and information about the Python Software Foundation. A right sidebar contains a 'Python 3 Poll', a 'NASA uses Python...' section with an image of an astronaut, and a 'What they are saying...' section featuring a quote from ITA Software.

Python Programming Language – Official Website

Python is a programming language that lets you work more quickly and integrate your systems more effectively. You can learn to use Python and see almost immediate gains in productivity and lower maintenance costs.

Python runs on Windows, Linux/Unix, Mac OS X, and has been ported to the Java and .NET virtual machines.

Python is free to use, even for commercial products, because of its OSI-approved [open source license](#).

New to Python or choosing between Python 2 and Python 3? Read [Python 2 or Python 3](#).

The [Python Software Foundation](#) holds the intellectual property rights behind Python, underwrites the [PyCon conference](#), and funds other projects in the Python community.

[Read more](#), -or- [download Python now](#)

» **Python 3.2 alpha 3 released**
The [third and final alpha release of Python 3.2](#) has been released for testing.
Published: Tue, 12 Oct 2010, 8:30 +0200

Python 3 Poll
I wish there was Python 3 support in

(enter [PyPI](#) package name)
[Vote](#) [Results](#)

NASA uses Python...

... joining users such as [Rackspace](#), [Industrial Light and Magic](#), [AstraZeneca](#), [Honeywell](#), and many others.

What they are saying...
ITA Software:
Since then, we've changed how we use Python a ton internally. We have lots more production software written in Python.



Why Python?

- **1 Super easy to read & write**
- Python's whole vibe is:
👉 *"Let me not torture you."*
- You can learn the syntax in a weekend and still build serious apps.
- for item in items:
- print(item)
- That's it. No ceremony. No boilerplate.



Why Python?

- **2** Massive ecosystem — libraries for everything
- Python is like a supermarket where every shelf is full.
- Want AI? → TensorFlow, PyTorch
Data analysis? → Pandas, NumPy
Web apps? → Django, Flask, FastAPI
Automation? → Selenium, PyAutoGUI
APIs? → FastAPI, Flask
Machine learning? → Scikit-Learn
DevOps? → Ansible, Fabric
Visualization? → Matplotlib, Seaborn, Plotly
- Whatever you want to build, someone already created a Python library for it.



Why Python?

- **③ Perfect for beginners & experts**
- Beginners love it because it's simple.
Experts love it because it's powerful.
That balance is rare.



Why Python?

- **4 Python is everywhere**
- Industries using it right now:
 - AI & Machine Learning
 - Data Science
 - Finance & Banking
 - Automation / DevOps
 - Web Development
 - Cybersecurity
 - IoT
 - Scientific Computing
- Python isn't a "nice to have" — it's a golden ticket skill.



Why Python?

- **5** Huge community & support
- If you get stuck, someone else has already solved that problem.
- StackOverflow alone is overflowing with Python answers.
Libraries have great documentation.
You will *never* feel alone in Python.



Why Python?

- **6 Cross-platform**
- Write once → run anywhere.
- Windows, macOS, Linux, Raspberry Pi, cloud VMs — Python doesn't care.



Why Python?

- **7 Ideal for rapid development**
- You can prototype insanely fast.
- Business loves Python because:
- Faster prototypes → faster feedback → faster delivery
- Less code → fewer bugs → cheaper maintenance



Why Python?

- **8 It plays well with others**
- Python integrates with:
- C / C++
- Java
- Rust
- SQL databases
- Cloud services
- REST APIs
- Microservices
- It's like that friendly coworker who gets along with everybody.



Why Python?

- **📌 Perfect for automation & scripting**
- If you want your computer to do your boring work:
- Rename files
- Read Excel sheets
- Automate browser clicks
- Run DevOps tasks
- Create cron jobs
- Python is literally the king of automation.



Why Python?

- **10 Strong job market**
- Python is consistently in the top 3 globally for:
- Most in-demand skill
- Highest paying tech roles
- Most used languages
- Most loved by developers



Why Python?

- **10 Strong job market**
- Python developers work in:
- Google
- Netflix
- Amazon
- NASA
- JP Morgan
- Meta
- Infosys, TCS, Accenture
- Every AI startup EVER



Python Is Free and Open Source

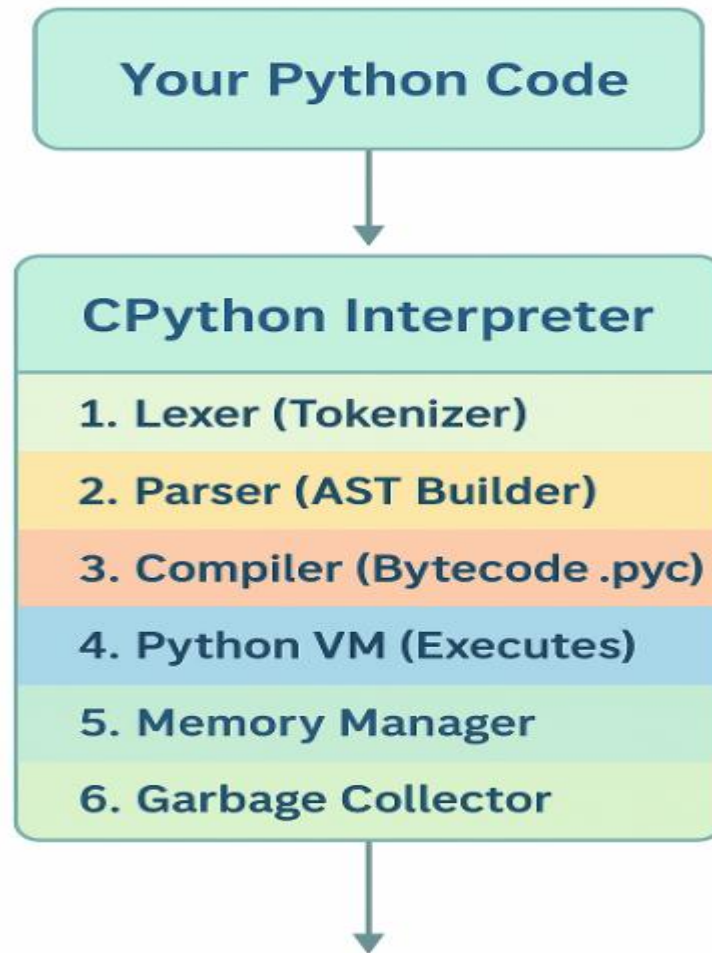
- **Open source:** Publicly available; open-source software typically programmed by volunteers; anyone can use source code without fee
- Can modify or even resell Python
- Embracing open-source ideals is part of what makes Python successful



- Java
C
C++
Javascript
PHP
Python.....
SQL
Perl
C#



The Python Interpreter





The Python Interpreter

- The Python interpreter is the software that:
- **1** Reads your Python code
- **2** Converts it into something the computer can understand
- **3** Executes it step-by-step
- The most widely used interpreter is **CPython**, the official one written in C.



The Python Interpreter

-  **How the Python Interpreter Works (The Full Pipeline)**
- When you run:
 - `python app.py`
- The interpreter does all this:



The Python Interpreter

- **1 Lexing / Tokenization**
- Your code → split into *tokens* (keywords, identifiers, numbers, operators)
- Example:
 - `x = 10 + 5`
 - Tokens:
 - `NAME(x)`, `EQUAL`, `NUMBER(10)`, `PLUS`, `NUMBER(5)`



The Python Interpreter

- **2 Parsing**
- Tokens $\rightarrow$ syntax tree (AST)
- Python checks:
 - is the syntax valid?
 - do commas, brackets, indentation all match?
- Example AST snippet:
 - Assign
 - $\mid$ Name(x)
 - $\perp$ BinOp(10 + 5)



The Python Interpreter

- **3 Compilation to Bytecode**
- Python compiles the AST into **bytecode** (low-level instructions).
- This creates .pyc files inside `__pycache__` directory.
- Bytecode example:
 - `LOAD_CONST 10`
 - `LOAD_CONST 5`
 - `BINARY_ADD`
 - `STORE_NAME x`



The Python Interpreter

- **4 Execution by the Python Virtual Machine (PVM)**
- The bytecode runs inside the PVM — a loop that executes opcode by opcode.
- Think of it like:
 - while True:
 - instruction = get_next_bytecode()
 - execute(instruction)
- That's literally what the interpreter does behind the scenes.

The Python Interpreter

- **★ Where Does the Interpreter Live?**
- The interpreter includes:
 - **✓ Parser**
 - **✓ Compiler**
 - **✓ Bytecode VM**
 - **✓ Memory Manager**
 - **✓ Garbage Collector**
 - **✓ Standard Library Loader**
 - **✓ Built-in object model**



What CPython Really Is

- CPython = Python language implementation in C.
- It provides:
 - C API (PyObject*)
 - GIL (Global Interpreter Lock)
 - Reference counting
 - PyMalloc memory allocator
- CPython powers 99% of Python installations.



⚡ Other Python Interpreters (Not Just One!)

- Python has multiple implementations:
-  **CPython**
 - Default, official, fastest to maintain
 - Written in C
 - Has the GIL
-  **PyPy**
 - Super fast, uses JIT (Just-In-Time compilation)
 - Great for long-running programs

⚡ Other Python Interpreters (Not Just One!)

-  **Jython**
 - → Runs on JVM
 - → Integrates with Java libraries
-  **IronPython**
 - → Runs on .NET CLR
 - → Integrates with C# libraries
-  **MicroPython**
 - → For microcontrollers (ESP32, Arduino, etc.)



What is the Python Interpreter?

- It is the engine that:
 - ✓ Reads your code
 - ✓ Translates it to bytecode
 - ✓ Runs that bytecode on a virtual machine
 - ✓ Manages memory & garbage collection
 - ✓ Handles errors, imports, objects, functions
- It's the “brain” of Python.



Installing

-  **1. Install Python on Windows**
- **Step 1 — Download**
- Go to the official site:
-  <https://www.python.org/downloads/>
- Click **“Download Python 3.x.x”**



Installing

- **Step 2 — RUN the Installer**
- **VERY IMPORTANT:**
 - ✓ **Check the box: “Add Python to PATH”**
- **Then click:**
 - **Install Now**



Installing

- **Step 3 — Verify Installation**
- Open CMD and type:
- `python --version`

4. Create a Virtual Environment (Recommended)

- This keeps project packages separate.
- `python3 -m venv myenv`
- Activate:
- **Windows**
- `myenv\Scripts\activate`
- **macOS/Linux**
- `source myenv/bin/activate`

Python IDEs and Shells



Install Python

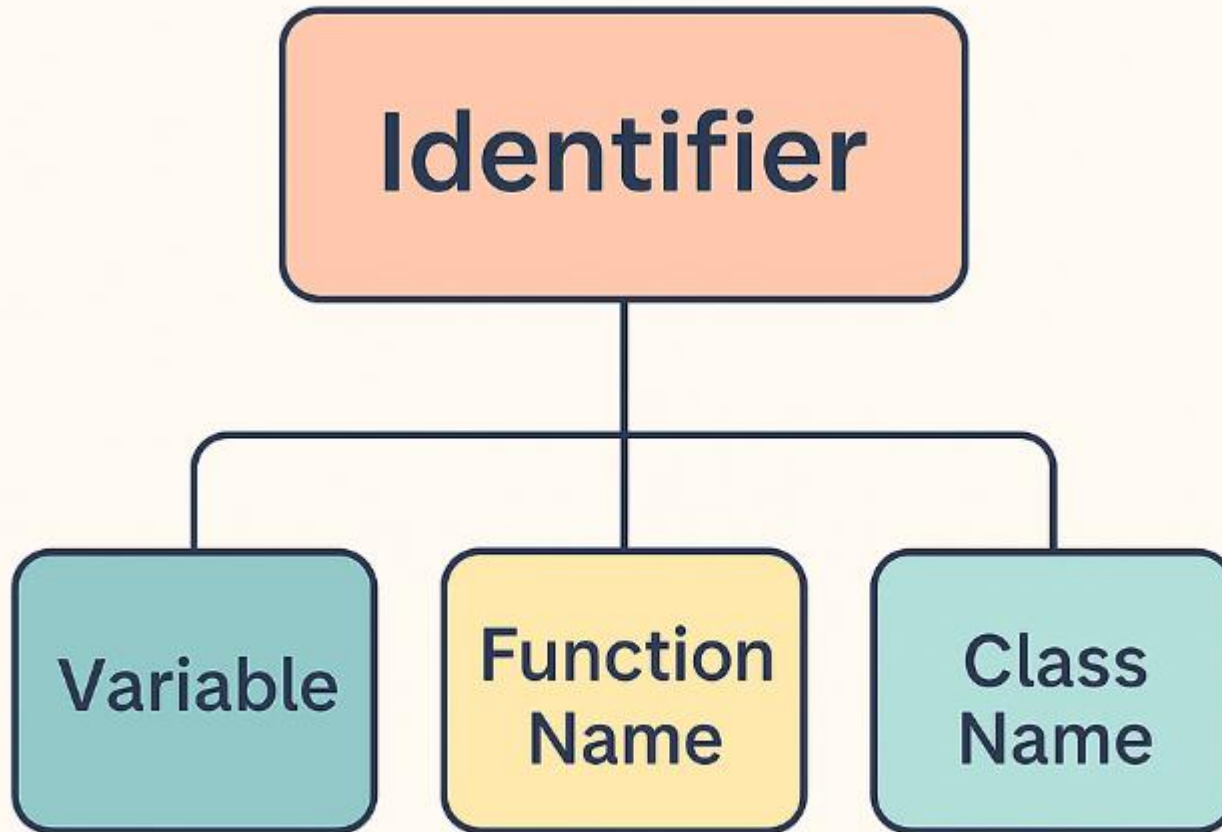




What Are Identifiers in Python?

- An **identifier** is simply the **name** you give to things in Python:
- variables
- functions
- classes
- modules
- packages
- constants
- objects
- Basically — *anything you name is an identifier.*

What Are Identifiers in Python?





What Are Identifiers in Python?

- `name = "Alice"` `# 'name' is an identifier`
- `def greet():` `# 'greet' is an identifier`
- `pass`
- `class Car:` `# 'Car' is an identifier`
- `pass`



Rules for Writing Identifiers

- ✓ **1. Identifiers can contain:**
- Letters (a–z, A–Z)
- Digits (0–9)
- Underscore _
- count1
- _user_name
- age2

Rules for Writing Identifiers

- ✗ 2. Identifiers **cannot start with a digit**
- ⚡ Invalid:
 - 1value = 10
- ✔ Valid:
 - value1 = 10



Rules for Writing Identifiers

- ✓ **3. Identifiers are case-sensitive**
- Age = 10
- age = 20
- These are **two different** variables.

Rules for Writing Identifiers

- **✗ 4. You cannot use Python keywords as identifiers**
- Keywords like:
 - if, while, return, class, def, pass, break, continue, True, False, None
- **⊘ Invalid:**
 - `class = 10`
- **✓ Valid:**
 - `class_name = 10`



Rules for Writing Identifiers

- ✓ **5. Identifiers can be of any length**
- Python does not limit length (but keep them readable).

★ Best Practices for Identifiers (Recommended)

- ✓ **Use meaningful names**
- `total_marks = 90` # Good
- `tm = 90` # Bad
- ✓ **Follow Python naming conventions (PEP 8)**
- **Variables & functions**
- → `snake_case`
- `user_name`
- `calculate_total()`
- **Constants**
- → `UPPERCASE`
- `MAX_SPEED = 120`

★ Valid vs Invalid Identifier Examples

- ✓ **Valid identifiers**
- x
- value123
- `_my_var`
- `first_name`
- `MAX_VALUE`
- `get_data`

★ Valid vs Invalid Identifier Examples

- ✗ **Invalid identifiers**
- 1name
- first-name
- total%value
- class # keyword
- for # keyword

★ Valid vs Invalid Identifier Examples

- ★ **Special Identifiers (Python Internals)**
- You will notice strange names like:
- `__init__`
- `__str__`
- `__main__`
- These are **dunder identifiers** (“double underscore”) and they are used internally by Python.
- Example:
- `__init__` → constructor method
- `__str__` → string representation
- `__main__` → entry point script



Identifier naming convention for Python

- Class names start with an uppercase letter and all other identifiers with a lowercase letter.
- Starting an identifier with a single leading underscore indicates by convention that the identifier is meant to be private.
- Starting an identifier with two leading underscores indicates a strongly private identifier.
- If the identifier also ends with two trailing underscores, the identifier is a language-defined special name.

■ 1. `_variable` (Single Leading Underscore)

- A name that **starts with one underscore** → `_name` is a **weak internal-use** indicator.
- ✓ **Meaning:**
- “This is **private**. Don’t use it outside the module/class.”
- Python does **NOT** enforce privacy — it’s only a **convention**, not a rule.
- ✓ **Example:**
- class Demo:
- `def __init__(self):`
- `self._internal_value = 100`

■ 1. `_variable` (Single Leading Underscore)

- Developers understand that:
- `_internal_value` is not part of public API
- It may change at any time
- **✓ Used for:**
 - Internal variables
 - Temporary variables
 - Non-public methods
- Example in modules:
 - `_var = 10` # treated as internal

2. `__main__` (Double Leading & Trailing Underscores)



- This is a **special Python identifier** (called a dunder).
- `__main__` refers to:
- ✓ **The top-level script being executed**
- OR
- ✓ **The name of the current module when directly run**
- **When you run:**
- `python myscript.py`
- Inside that file:
- `__name__ == "__main__"`
- So, Python knows:
- This file is being run directly
- Not imported as a module

2. `__main__` (Double Leading & Trailing Underscores)



- 🧠 **Use Case: The Famous `if __name__ == "__main__":`**
- `def run():`
- `print("Running script...")`
- `if __name__ == "__main__":`
- `run()`
- This block only runs when the file is executed directly.
- If the same file is imported:
- `import myscript`
- `run()` **will NOT run automatically**, because:
- `myscript.__name__ = "myscript"`

■ `_variable` vs ■ `__main__` (Side-by-Side Comparison)



Feature	<code>_variable</code>	<code>__main__</code>
Starts with	1 underscore	double underscores
Ends with	nothing	double underscores
Type	Normal identifier	Special Python reserved name
Meaning	Private / internal use	Script entry point
Enforced?	✗ No	✓ Yes (special behavior)

`__main__`

- Since there is no `main()` function in Python, when the command to run a python program is given to the interpreter, the code that is at level 0 indentation is to be executed.
- Will define a few special variables. `__name__` is one such special variable.
- If the source file is executed as the main program, the interpreter sets the `__name__` variable to have a value `"__main__"`.
- If this file is being imported from another module, `__name__` will be set to the module's name.
- `__name__` is a built-in variable which evaluates to the name of the current module.

__main__

```
print("File1 __name__ = %s" % __name__)

if __name__ == "__main__":
    print("File1 is being run directly")
else:
    print("File1 is being imported")
```



Reserved Words

and	exec	not
assert	finally	or
break	for	pass
class	from	print
continue	global	raise
def	if	return
del	import	try
elif	in	while
else	is	with
except	lambda	yield



Python Engineering Standards

- 🖌️ PEP 8 (Python Style Guide)
- 🔑 **Key Rules**
- **1. Indentation**
- `def greet():`
 `print("Hello")` # 4 spaces (NOT tabs)
- **2. Line Length**
- **Max: 79 characters** (recommended)

```
# Good
```

```
total = price + tax
```

```
# Bad (too Long)
```

```
total_price_with_tax_and_discount_applied = price + tax - discount
```

3. Naming Conventions

Type	Style	Example
Variable	snake_case	user_name
Function	snake_case	get_data()
Class	PascalCase	UserService
Constant	UPPER_CASE	MAX_SIZE



4. Imports

```
# Standard  
import os  
import sys  
  
# Third-party  
import requests  
  
# Local  
from mymodule import helper
```

5. Spaces

Good

x = 10

y = x + 5

Bad

x=10

y=x+5

6. Blank Lines

```
# 2 blank lines before top-level functions  
def func1():  
    pass  
  
def func2():  
    pass
```



7. Comments

```
# This function calculates total price  
def calculate_total(price, tax):  
    return price + tax
```


Python Typing (Type Hints)

- Typing improves **readability, tooling, and bug detection**.

Basic Types

Python

```
def greet(name: str) -> str:  
    return "Hello " + name
```

Python Typing (Type Hints)

- Typing improves **readability, tooling, and bug detection**.

Common Types

⟨> Python

```
from typing import List, Dict, Tuple

def process(items: List[int]) -> int:
    return sum(items)

def get_user() -> Dict[str, str]:
    return {"name": "John"}
```

Python Typing (Type Hints)

- Typing improves **readability, tooling, and bug detection**.

Optional & Union

</> Python

```
from typing import Optional, Union

def find_user(id: int) -> Optional[str]:
    return None # or string

def parse(value: Union[int, str]) -> str:
    return str(value)
```

Modern Python (3.10+)

- Use | instead of Union:

```
def parse(value: int | str) -> str:  
    return str(value)
```



Custom Types



```
class User:
    def __init__(self, name: str):
        self.name = name

def get_user() -> User:
    return User("Alice")
```

Type Aliases

```
UserId = int
```

```
def get_user(user_id: UserId) -> str:  
    return "User"
```



Typed Functions Example (Best Practice)

```
from typing import List

def average(scores: List[float]) -> float:
    if not scores:
        return 0.0
    return sum(scores) / len(scores)
```



Best Practices (PEP 8 + Typing)

- ✓ Use **type hints everywhere** (functions at least)
 - ✓ Follow **snake_case** naming
 - ✓ Keep functions **small & readable**
 - ✓ Use tools:
- flake8 → style check
- black → auto-format
- mypy → type checking



Example (Clean + Typed + PEP8)

```
from typing import List

def calculate_average(numbers: List[int]) -> float:
    """Calculate average of a list of numbers."""
    if not numbers:
        return 0.0
    return sum(numbers) / len(numbers)
```



Lines and Indentation

- ◆ 1. Indentation (Very Important)
-  **Rule:**
- Use **4 spaces per indentation level**
- **Never mix tabs and spaces**

```
#  Correct  
def greet():  
    print("Hello")  
    if True:  
        print("Inside block")
```

```
#  Wrong (tabs or inconsistent spaces)  
def greet():  
    print("Hello")  
    print("Bad indentation")
```



Why Indentation Matters

- Python uses indentation to define blocks (instead of {} like Java)

```
# Example
if True:
    print("Valid block")
print("Outside block")
```

Indentation for Continuation Lines

- When a line continues, indent properly:

```
#  Aligned with opening bracket  
result = my_function(  
    arg1,  
    arg2,  
    arg3,  
)
```

```
#  Hanging indent  
result = my_function(  
    arg1, arg2,  
    arg3  
)
```

Multi-line Conditions

```
if (  
    condition_one  
    and condition_two  
    and condition_three  
):  
    print("Valid")
```

Indentation in Lists / Dicts

```
#  Clean formatting  
data = [  
    "apple",  
    "banana",  
    "cherry",  
]
```

```
user = {  
    "name": "John",  
    "age": 30,  
}
```

Common Mistakes

-  Mixing tabs and spaces
-  Uneven indentation
-  Long unreadable lines
-  Using \ instead of parentheses

✓ Best Practice Summary

- ✓ Always use **4 spaces**
- ✓ Keep lines **≤ 79 chars**
- ✓ Use **parentheses for long lines**
- ✓ Keep code visually clean and aligned

What is a Docstring?

- A **docstring** is a string literal used to document:
 - Modules
 - Functions
 - Classes
 - Methods

```
def greet(name: str) -> str:  
    """Return a greeting message."""  
    return f"Hello {name}"
```



Why Docstrings Matter

- ✓ Improves readability
 - ✓ Helps tools (`help()`, IDEs, Swagger-like docs)
 - ✓ Useful for teams & production code

Basic Rules (PEP 257)

-  1. Use triple quotes

```
"""This is a docstring"""
```

✓ 2. First line = short summary

- One line
- Starts with capital letter
- Ends with period

```
"""Calculate the total price."""
```

3. Add blank line before details

```
def calc_total(price: float, tax: float) -> float:
    """
    Calculate total price.

    Adds tax to base price and returns final value.
    """
    return price + tax
```

Function Docstring (Best Practice)

```
def divide(a: float, b: float) -> float:
    """
    Divide two numbers.

    Args:
        a (float): Numerator.
        b (float): Denominator.

    Returns:
        float: Result of division.

    Raises:
        ValueError: If b is zero.
    """
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

Common Docstring Styles

- 1. Google Style (Most Popular 👍)

```
def add(a: int, b: int) -> int:
    """
    Add two numbers.

    Args:
        a (int): First number.
        b (int): Second number.

    Returns:
        int: Sum of numbers.
    """
    return a + b
```

Common Docstring Styles

- 2. NumPy Style

```
def add(a: int, b: int) -> int:
    """
    Add two numbers.

    Parameters
    -----
    a : int
    b : int

    Returns
    -----
    int
    """
    return a + b
```




Common Docstring Styles

- 3. reStructuredText (Sphinx)
- reStructuredText, usually written as **reST**, is a lightweight markup language used a lot in Python documentation.
- **Sphinx** is a documentation generator that commonly uses reStructuredText files as input.



Common Docstring Styles

- **What reStructuredText is**
- It lets you write plain text with simple formatting for:
 - headings
 - bullet lists
 - code blocks
 - links
 - tables
 - notes and warnings
 - API documentation structure

Common Docstring Styles

```
</> rst
```

- Item 1
- Item 2

Here is code:

```
.. code-block:: python
```

```
def hello():  
    print("Hello")
```



Common Docstring Styles

- What Sphinx is
- Sphinx takes .rst files and builds documentation into formats like:
 - HTML
 - PDF
 - EPUB
- It is widely used for:
 - Python project docs
 - API reference docs
 - user guides
 - technical manuals



Common Docstring Styles

- What Sphinx is
- Sphinx takes .rst files and builds documentation into formats like:
 - HTML
 - PDF
 - EPUB
- It is widely used for:
 - Python project docs
 - API reference docs
 - user guides
 - technical manuals



Common Docstring Styles

- **Why people use Sphinx**
- Sphinx is popular because it supports:
 - automatic table of contents
 - cross-references between pages
 - syntax-highlighted code blocks
- extensions
 - autodoc for Python docstrings
 - searchable HTML output
 - themes like Read the Docs

Common Docstring Styles

```
</> rst
```

```
Main Title
```

```
=====
```

```
Section
```

```
-----
```

```
Subsection
```

```
~~~~~
```

Common Docstring Styles

- 3. reStructuredText (Sphinx)

```
def add(a: int, b: int) -> int:
    """
    Add two numbers.

    :param a: First number
    :param b: Second number
    :return: Sum
    """
    return a + b
```




When to use it

- Use reStructuredText with Sphinx when you want:
 - professional project documentation
 - Python API docs from docstrings
 - multi-page technical manuals
 - versioned docs



Document Generator using Sphinx

- 1. install sphinx as optional dependency
- 2. install sphinx as cli tool – `pip install sphinx`
- 3. Test Sphinx
 - `sphinx-build –version`
- 4. run `sphinx-quickstart`
- 5. It will create `.rst` file under source folder
- 6. To generate html document
 - `sphinx-build –b html source build\html`

Class Docstring

```
class User:
    """
    Represents a user.

    Attributes:
        name (str): User name.
        age (int): User age.
    """

    def __init__(self, name: str, age: int):
        """Initialize user."""
        self.name = name
        self.age = age
```



Module Docstring

- Top of the file

```
"""  
User management module.  
  
This module handles user creation and validation.  
"""
```



One-line Docstring

```
def square(x: int) -> int:  
    """Return square of a number."""  
    return x * x
```

Common Mistakes

-  No docstring
-  Too long first line
-  Missing period
-  Not explaining parameters
-  Writing comments instead of docstrings

✓ Best Practices

- ✓ Use **Google style** (widely used in industry)
- ✓ Keep **summary short and clear**
- ✓ Document **inputs, outputs, exceptions**
- ✓ Keep it **updated with code**



Example (Clean + Typed + Documented)



```
from typing import List

def average(scores: List[float]) -> float:
    """
    Calculate the average of scores.

    Args:
        scores (List[float]): List of numeric scores.

    Returns:
        float: Average value.
    """
    if not scores:
        return 0.0
    return sum(scores) / len(scores)
```


linting/formatting (flake8, black)

- 🖌️ What is Linting vs Formatting?

Type	Tool	Purpose
Linting	flake8	Finds errors, style issues, bad practices
Formatting	black	Automatically formats code

1. Flake8 (Linter)

-  **What it does**
 - Checks **PEP 8 violations**
 - Detects unused imports, variables
 - Finds bugs early

1. Flake8 (Linter)

-  Install

```
pip install flake8
```

-  Run

- flake8 your_file.py

-  Example Output

- test.py:1:1: F401 'os' imported but unused
 - test.py:3:5: E303 too many blank lines

Config (.flake8)

```
[flake8]
max-line-length = 88
ignore = E203, W503
exclude = .git,__pycache__,venv
```

Config (.flake8)

- **1. max-line-length = 88**
- Sets the maximum allowed characters per line
- Default in Flake8 is **79** (PEP 8)
- **88 is commonly used with Black**
- 👉 Why 88?
- Black uses 88 by default
- Slightly longer lines = better readability without horizontal scrolling

Config (.flake8)

- **2. ignore = E203, W503**
- These are specific rule codes from Flake8 (actually from pycodestyle).
- **✗ E203 – whitespace before :**
- Example:
- `my_list[1 : 3]`
- PEP 8 says avoid space before :
- But **Black formats it this way intentionally**
- 👉 So we ignore it to avoid conflict with Black

Config (.flake8)

-  **W503 – line break before binary operator**
- Example:
 - total = (
 a
 + b
)
- Old PEP 8 discouraged this
- New recommendation prefers this style
-  Black uses this style → so we ignore W503

Config (.flake8)

- **3. exclude = .git,__pycache__,venv**
- **Flake8 will skip checking these folders:**

Folder	Reason
.git	Git metadata (not code)
__pycache__	Compiled Python files
venv	Virtual environment (external packages)

-  **This avoids:**
 - unnecessary linting
 - performance slowdown
 - irrelevant errors

Flake 8

```
pyproject.toml  .flake8  config.py  main.py

. .flake8
1  #create rules for flake 8
2  [flake8]
3  #ignore rules
4  ignore = E203, E266, E501, W503, F403, F401
5  #max line length
6  max-line-length = 88
7  #exclude files and folders
8  exclude = .git, __pycache__, build, dist, .venv
9  #select rules
10 select = B,C,E,F,W,T4,B9
11
12
13
14
```

Flake 8



```
(sdkenv) E:\Datascience2026\sdkdemo>flake8 main.py
```

```
(sdkenv) E:\Datascience2026\sdkdemo>flake8 config.py
```

```
(sdkenv) E:\Datascience2026\sdkdemo>
```

2. Black (Formatter)

-  **What it does**
- Auto-formats code
- Enforces consistent style
- Removes debates 😄

2. Black (Formatter)

-  **Install**
- pip install black
-  **Run**
- black your_file.py

2. Black (Formatter)

-  **Before vs After**
-  **Before**
 - `def add(a,b): return a+b`
-  **After (Black)**
 - `def add(a, b):
 return a + b`

2. Black (Formatter)

-  **Config (pyproject.toml)**
- [tool.black]
line-length = 88
target-version = ["py310"]

2. Black (Formatter)

-  **Config (pyproject.toml)**
- [tool.black]
line-length = 88
target-version = ["py310"]

2. Black (Formatter)



Feature	flake8	black
Fix code	✗ No	✓ Yes
Detect issues	✓ Yes	✗ No
Enforce style	✓ Yes	✓ Yes

2. Black (Formatter)

```
(sdkenv) E:\Datascience2026\sdkdemo>black main.py
All done! ✨ 🍰 ✨
1 file left unchanged.

(sdkenv) E:\Datascience2026\sdkdemo>black config.py
All done! ✨ 🍰 ✨
1 file left unchanged.

(sdkenv) E:\Datascience2026\sdkdemo>
```

Recommended Setup (Best Practice)

- **1. Install all tools**
 - pip install black flake8 isort
- **2. Add isort (for imports)**
 - pip install isort
- Run:
 - isort .



Recommended Setup (Best Practice)

```
(sdkenv) E:\Datascience2026\sdkdemo>isort config.py --check-only
ERROR: E:\Datascience2026\sdkdemo\config.py Imports are incorrectly sorted and/or formatted.
(sdkenv) E:\Datascience2026\sdkdemo>
```

```
(sdkenv) E:\Datascience2026\sdkdemo>isort config.py --diff
--- E:\Datascience2026\sdkdemo\config.py:before 2026-04-09 12:10:50
+++ E:\Datascience2026\sdkdemo\config.py:after 2026-04-09 18:29:11.748358
@@ -1,4 +1,5 @@
     import os
+
+    from dotenv import load_dotenv
+
+    load_dotenv()
```

Recommended Setup (Best Practice)

- 🔍 **What isort is complaining about**
- Diff shows:
 - `import os`
 - `+`
 - `from dotenv import load_dotenv`
- 👉 The issue is **missing blank line between import groups**

Recommended Setup (Best Practice)

-  **Rule isort enforces**
- Imports must be grouped like this:
 - **Standard library**
 - **Third-party**
 - **Local imports**
 - **And each group must be separated by a blank line**

Recommended Setup (Best Practice)

-  **Fix automatically**
- Just run:
 - isort config.py

```
(sdkenv) E:\Datascience2026\sdkdemo>isort config.py
Fixing E:\Datascience2026\sdkdemo\config.py

(sdkenv) E:\Datascience2026\sdkdemo>isort config.py
```

What is pyproject.toml?

-  It is the **main configuration file for a Python project**
- Think of it like:
 - Blueprint of your project 
- It tells Python:
 - what your project is
 - what dependencies it needs
 - how to build/install it

What is pyproject.toml?

- 👉 **TOML = Tom's Obvious Minimal Language**
- Created by **Tom Preston-Werner** (GitHub co-founder)
- Designed to be:
 - simple ✓
 - readable ✓
 - less error-prone ✓

3. Why Python uses TOML

- Python chose TOML for:
 - readability (better than JSON)
 - less indentation errors (better than YAML)
 - standardization
- 👉 That's why we have:
- `pyproject.toml`



4. Structure of .toml (core concepts)

◆ A. Key-Value pairs

⟨/⟩ TOML

```
name = "dsapp"  
version = "1.0.0"
```

👉 Format:

```
key = value
```

◆ B. Data types

1. String

⟨/⟩ TOML

```
name = "myapp"
```

2. Integer

⟨/⟩ TOML

```
count = 10
```

4. Structure of .toml (core concepts)

3. Boolean

```
</> TOML  
  
debug = true
```

4. Array (IMPORTANT)

```
</> TOML  
  
dependencies = [  
    "pandas",  
    "numpy"  
]
```

👉 MUST:

- use `[ ]`
- close properly **!**

5. Table (section)

```
</> TOML  
  
[project]  
name = "dsapp"
```

👉 `[project]` = section

6. Nested table

```
</> TOML  
  
[tool.setuptools]  
package-dir = {"" = "src"}
```

Combined Config (pyproject.toml)

- [tool.black]
line-length = 88

```
[tool.isort]  
profile = "black"
```

```
[tool.flake8]  
max-line-length = 88  
ignore = ["E203", "W503"]
```

Project layout for platform repos; reusable modules and SDKs



- For **platform repos**, the goal is usually:
 - keep shared code reusable,
 - avoid tight coupling between services,
 - make versioning and releases predictable,
 - let teams find things quickly.
- A good layout depends on whether you have:
 - one repo for a single platform product,
 - a monorepo with many services,
 - a repo dedicated to shared SDKs/modules.



1. Recommended high-level structure

- Layout



1. Recommended high-level structure

- This separates:
 - **apps** = runnable products
 - **packages** = internal reusable building blocks
 - **sdk**s = external-consumer libraries
 - **contracts** = shared interfaces
- That separation is one of the most important design choices.



2. Internal reusable modules vs SDKs

- These are not the same.
- **Reusable internal modules**
 - Used by your own platform teams.
- Examples:
 - logging helpers
 - auth middleware
 - config loading
 - retry utilities
 - telemetry wrappers
 - database abstractions



2. Internal reusable modules vs SDKs

- These should live under something like:
 - packages/
- **SDKs**
 - Used by other teams, customers, or downstream services to consume your platform.
- Examples:
 - Python SDK for your product API
 - Java client for internal platform services
 - TypeScript frontend SDK
 - These should live under:
 - sdks/



2. Internal reusable modules vs SDKs

- Why separate them:
 - internal modules can change faster,
 - SDKs need stronger compatibility guarantees,
 - SDKs need clearer docs, changelogs, and versioning.

3. Python layout for reusable modules

For each module, use a **src** layout.

Example:

```
packages/logging/  
├── pyproject.toml  
├── README.md  
├── src/  
│   ├── platform_logging/  
│   │   ├── __init__.py  
│   │   ├── logger.py  
│   │   ├── formatters.py  
│   │   └── context.py  
├── tests/  
│   ├── test_logger.py  
│   └── test_context.py
```



3. Python layout for reusable modules

- Why src/ layout is good:
 - prevents accidental imports from the repo root,
 - makes packaging cleaner,
 - works better for reusable libraries.

4. Python layout for an SDK

```
sdk/python-sdk/  
├── pyproject.toml  
├── README.md  
├── CHANGELOG.md  
├── src/  
│   ├── platform_sdk/  
│   │   ├── __init__.py  
│   │   ├── client.py  
│   │   ├── auth.py  
│   │   ├── models/  
│   │   ├── exceptions.py  
│   │   └── resources/  
├── tests/  
│   ├── unit/  
│   └── integration/
```



4. Python layout for an SDK

- A good SDK usually contains:
 - `client.py` for the main entry point,
 - `auth.py` for token/session handling,
 - `models/` for typed request/response objects,
 - `resources/` for domain grouping like users, orders, projects,
 - `exceptions.py` for stable error handling.



5. Suggested structure inside shared modules

- A reusable module should be small and focused.
- Example for packages/config:

```
packages/config/  
├── src/platform_config/  
│   ├── __init__.py  
│   ├── loader.py  
│   ├── models.py  
│   ├── validators.py  
│   └── env.py  
├── tests/  
└── pyproject.toml
```



5. Suggested structure inside shared modules

- Guideline:
 - one module = one responsibility,
 - avoid giant “common” or “utils” dumping grounds.
- Bad:
 - packages/common/
packages/helpers/
packages/utils/



5. Suggested structure inside shared modules

- Better:
 - packages/logging/
packages/config/
packages/http/
packages/auth/
packages/events/

6. Dependency direction

- A healthy platform repo usually follows this dependency rule:
- apps -> packages
sdks -> contracts
apps -> contracts
packages -> lower-level packages only
- Try to avoid:
- packages -> apps
sdk -> app internals
high-level package -> unrelated high-level package

6. Dependency direction

- A useful layering model:

```
contracts
  ↓
foundation packages  (config, logging, errors)
  ↓
infra packages      (db, messaging, http, cache)
  ↓
domain packages     (billing, identity, tenant)
  ↓
apps
```

This prevents circular dependencies and tangled architecture.

7. Keep contracts separate

- For platform teams, shared contracts deserve their own place.

```
contracts/  
├─ openapi/  
│   ├─ platform-api.yaml  
│   └─ admin-api.yaml  
├─ protobuf/  
└─ jsonschema/
```



7. Keep contracts separate

- Why:
 - SDK generation becomes easier,
 - interface ownership is clearer,
 - breaking changes are easier to review.
- If possible:
 - generate SDK code from contracts,
 - do not manually duplicate API models everywhere.



8. Versioning strategy

- For reusable modules and SDKs, versioning matters a lot.
- **Internal packages**
 - Can use faster-moving versions:
 - 0.x during active evolution
 - independent versioning per package if tooling supports it



8. Versioning strategy

- **SDKs**
- Use stronger semantic versioning:
 - MAJOR.MINOR.PATCH
 - document breaking changes carefully
- Good practice:
 - internal packages may be released often,
 - SDKs should have stricter release gates and changelogs.



9. Naming conventions

- Use consistent, explicit names.
- Good:
 - platform_logging
 - platform_config
 - platform_auth
 - platform_sdk
- Less good:
 - utils
 - helpers
 - shared
 - core unless “core” is very clearly defined



9. Naming conventions

- Repo folders should also be obvious:
 - auth-service
 - billing-api
 - python-sdk
- not:
 - service1
 - common2
 - newmodule

10. Test layout

For shared modules:

```
tests/  
├─ unit/  
└─ integration/
```

For simpler packages:

```
tests/  
└─ test_*.py
```

In platform repos, you often need multiple test levels:

```
tests/  
├─ contract/  
├─ integration/  
├─ e2e/  
└─ smoke/
```



10. Test layout

- Recommended use:
 - **unit**: per package
 - **integration**: package/service interaction
 - **contract**: schema/API compatibility
 - **e2e**: full platform flows



11. Documentation layout

- Each reusable module or SDK should have its own small README.
- Example:
 - packages/logging/README.md
 - sdks/python-sdk/README.md

11. Documentation layout

- At repo root:
 - overall architecture
 - repo map
 - development workflow
 - release process
- Per package:
 - purpose
 - install/use
 - public API
 - examples
 - release notes if needed



11. Documentation layout

- For SDKs especially, include:
 - quickstart
 - authentication example
 - error handling example
 - pagination/retries/timeouts example

Example full Python platform repo

```
platform-repo/  
├── apps/  
│   ├── gateway-api/  
│   │   ├── pyproject.toml  
│   │   ├── src/gateway_api/  
│   │   └── tests/  
│   └── worker-sync/  
│       ├── pyproject.toml  
│       ├── src/worker_sync/  
│       └── tests/  
└──
```

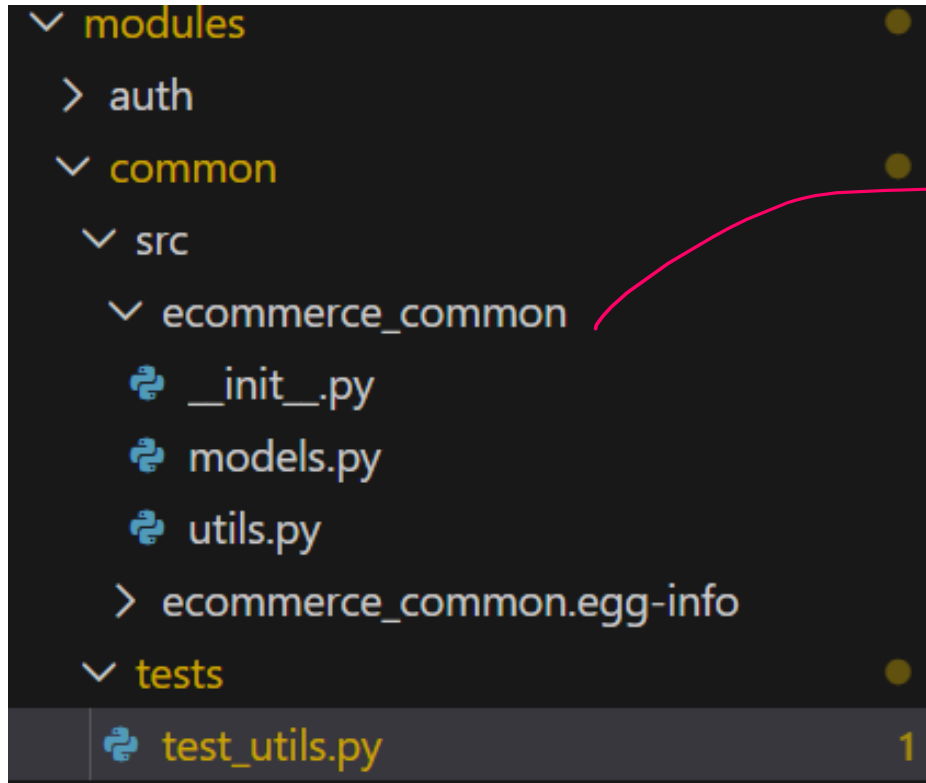
Example full Python platform repo

```
├─ packages/  
│   ├── platform_config/  
│   │   ├── pyproject.toml  
│   │   ├── src/platform_config/  
│   │   └── tests/  
│   ├── platform_logging/  
│   │   ├── pyproject.toml  
│   │   ├── src/platform_logging/  
│   │   └── tests/  
│   ├── platform_auth/  
│   │   ├── pyproject.toml  
│   │   ├── src/platform_auth/  
│   │   └── tests/  
│   └── platform_http/  
│       ├── pyproject.toml  
│       ├── src/platform_http/  
│       └── tests/  
└─
```


Example full Python platform repo

```
├── tests/
├── sdks/
│   ├── python-sdk/
│   │   ├── pyproject.toml
│   │   ├── src/platform_sdk/
│   │   └── tests/
├── contracts/
│   ├── openapi/
│   └── protobuf/
├── docs/
├── scripts/
├── tests/
│   ├── integration/
│   ├── contract/
│   └── e2e/
├── pyproject.toml
├── README.md
└── Makefile
```

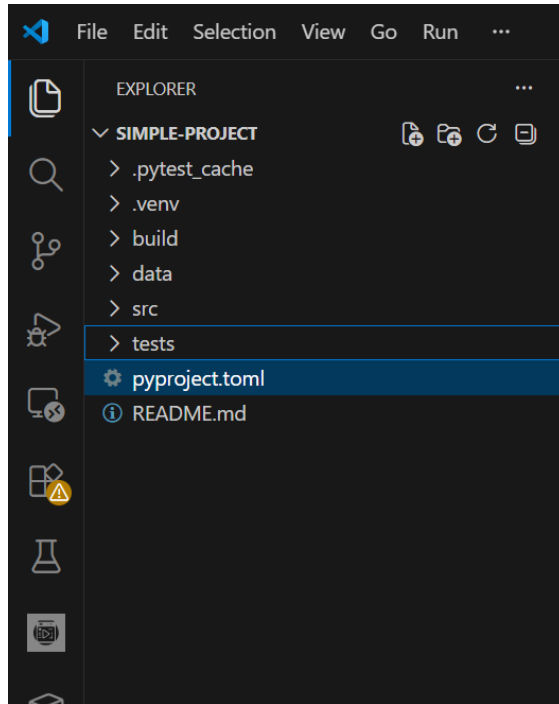
Install modules to access in test



~~pip~~ install -e modules/common

Then restart vscode

Create Simple Project



- Create venv
- `Python -m venv .venv`
- `#activate`
- `.venv\Scripts\activate`
- If it is bash shell in github codespaces
- `source venv/bin/activate`
- `Ctrl+shift+p` open folder for specific
- `#install dependencies from pyproject.toml`
- `Pip install -e .`
- `# run pytest`
- `pytest`

Test Case Result



The screenshot shows the Visual Studio Code interface with a project named 'simple-project'. The Explorer sidebar on the left shows the project structure, including a 'tests' directory. The main editor displays the 'pyproject.toml' file, which defines the project name as 'dsapp', version '0.1.0', and lists dependencies: 'pandas>=2.0.0' and 'pytest>=7.0.0'. The bottom panel shows the 'TERMINAL' output of a 'pytest' command. The output indicates that the test session started successfully, collected 1 item, and passed 1 test in 6.48 seconds. A red bracket on the right side of the terminal output highlights the success message. The status bar at the bottom shows the current file is 'test_analysi' and the Python version is 3.13.5.

```
pyproject.toml
[project]
name = "dsapp"
version = "0.1.0"
description = "Simple Data Science project"
requires-python = ">=3.10"

dependencies = [
    "pandas>=2.0.0",
    "pytest>=7.0.0"
]
```

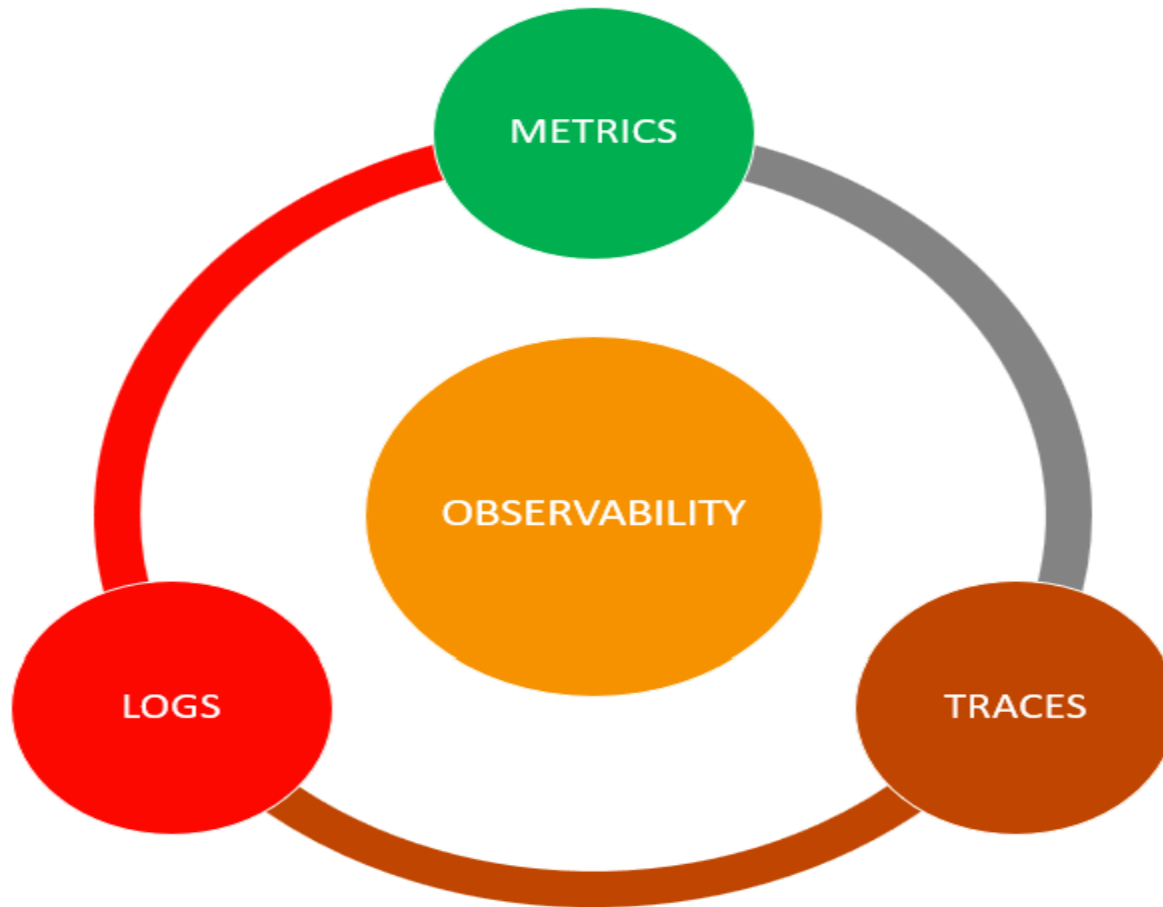
```
(.env) E:\Datascience2026\simple-project>pytest
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0
rootdir: E:\Datascience2026\simple-project
configfile: pyproject.toml
collected 1 item

tests\test_analysis.py . [100%]

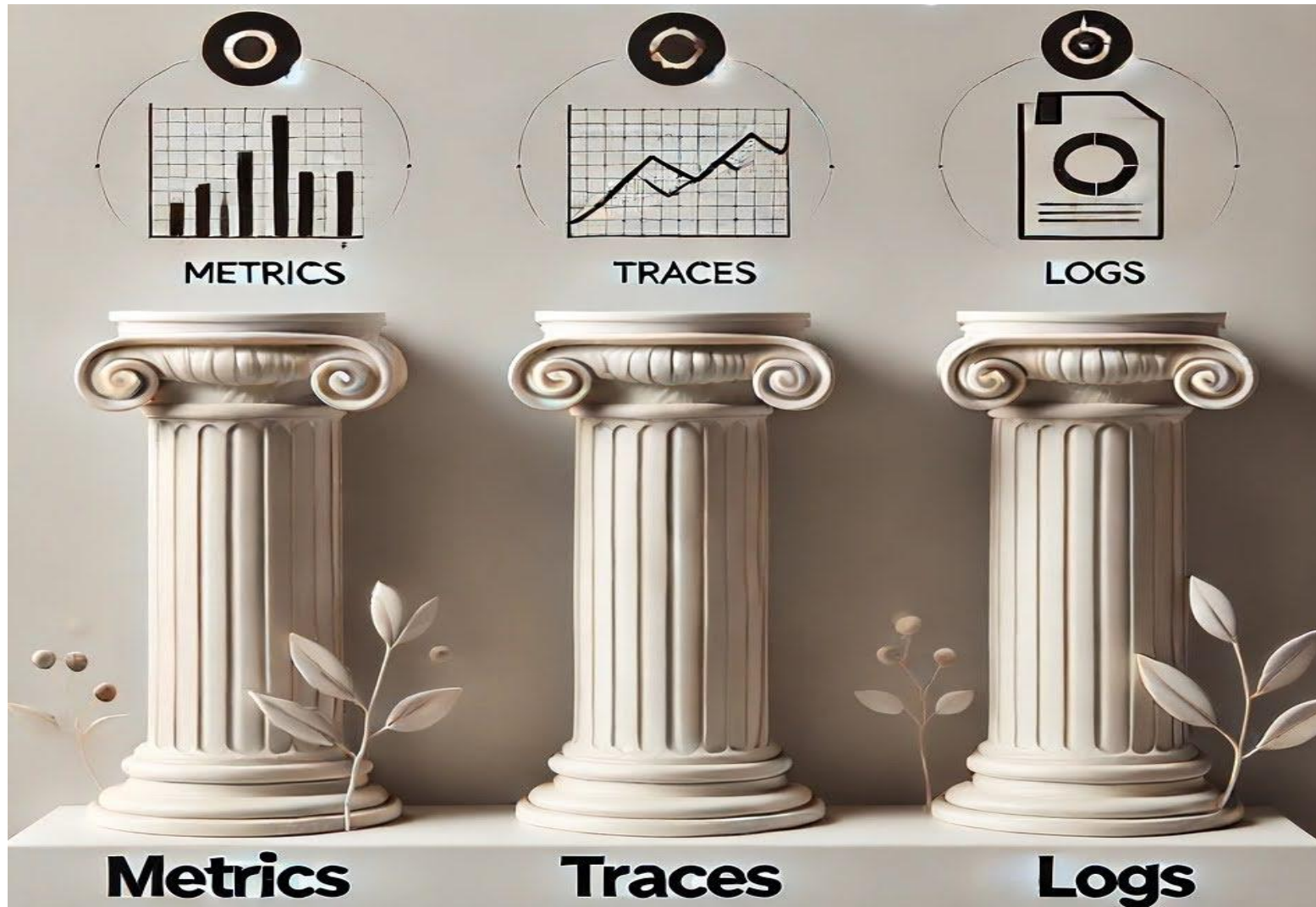
===== 1 passed in 6.48s =====
(.env) E:\Datascience2026\simple-project>
```



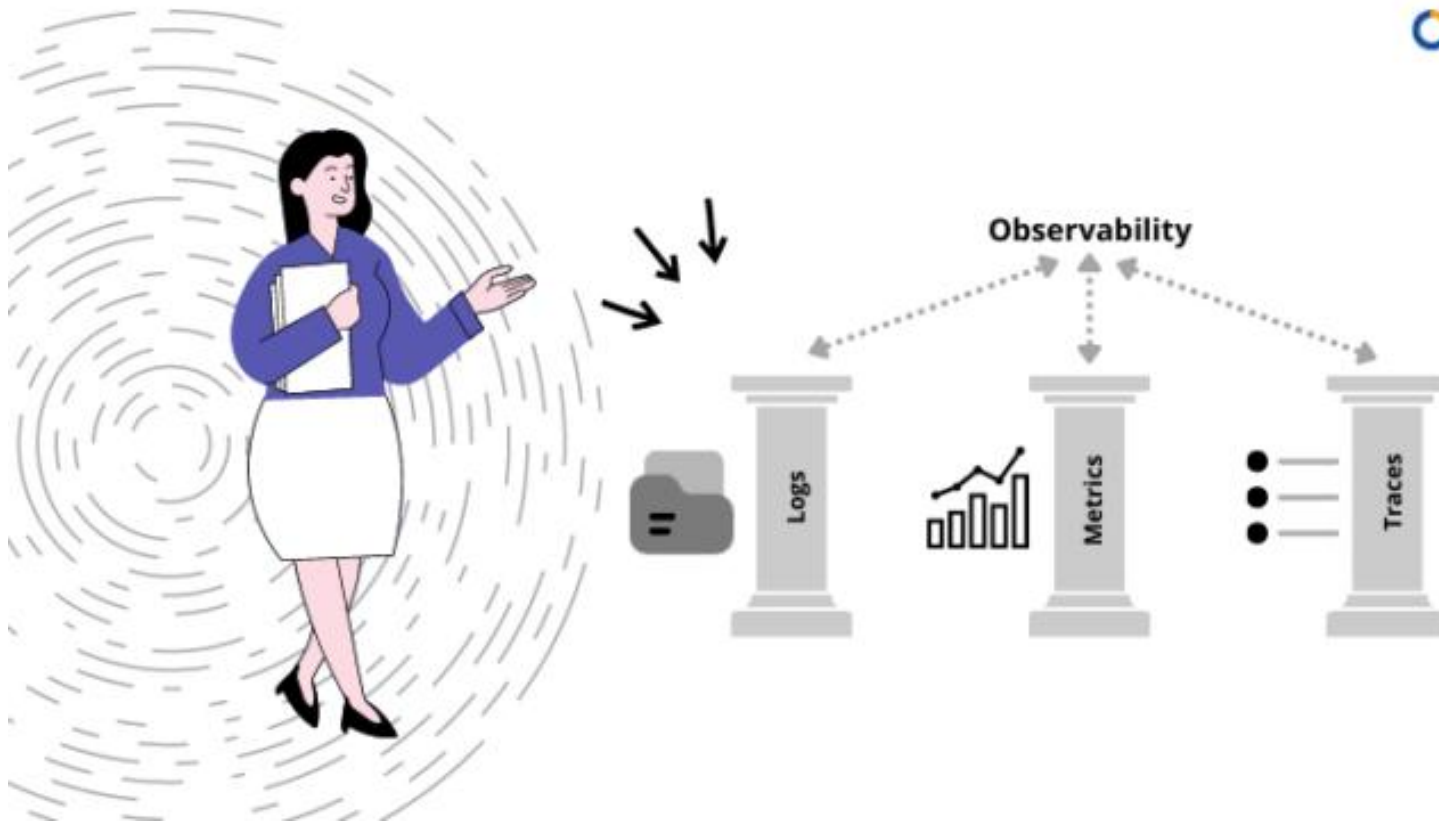
- Observability is the measure of how well the internal states of a system can be inferred from knowledge of its external outputs.
- Often confused with monitoring, observability goes a step further by identifying issues in a system and why those issues occur.
- It's from control theory and has become super crucial in software engineering, especially with the rise of complex, distributed systems and microservices.
- Observability collects data from metrics, logs, and traces, the three pillars, to give a complete view of a system's health and performance.



Observability



Observability

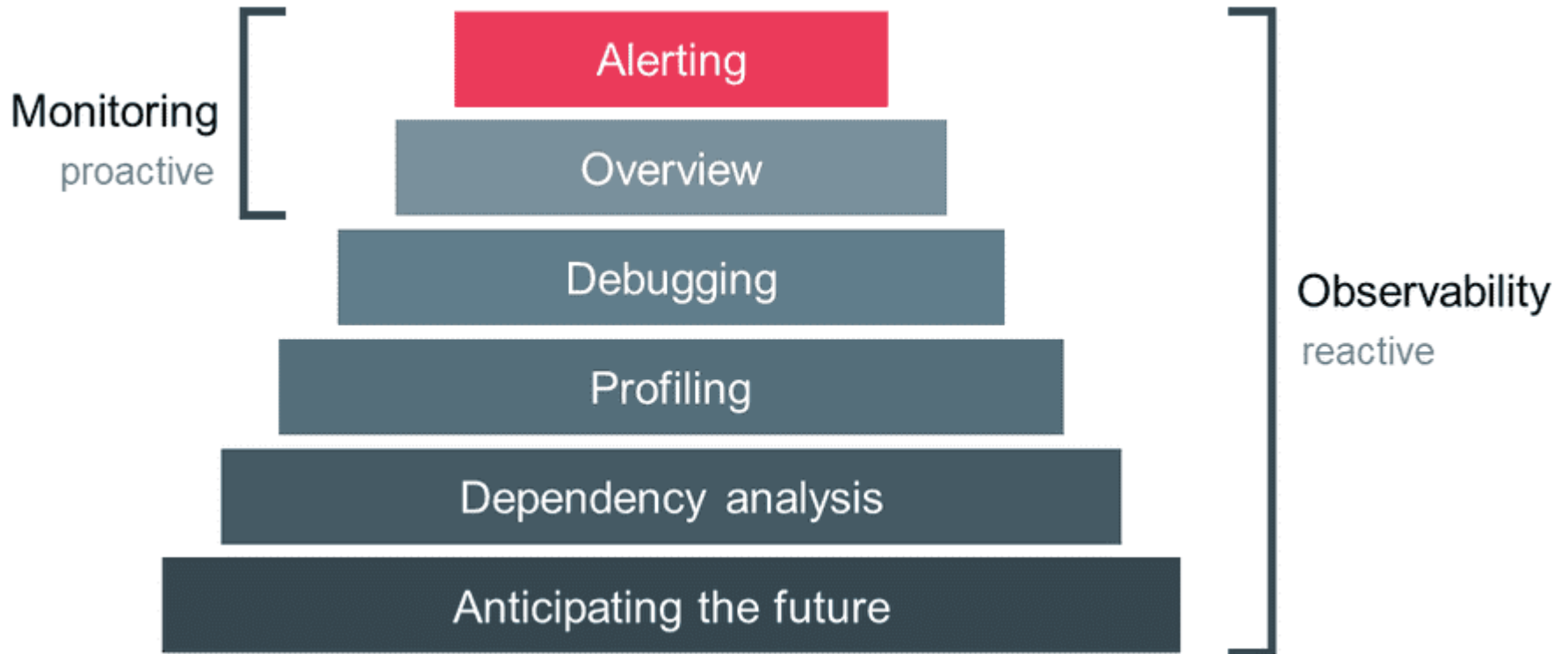




Need for Comprehensive Observability

- Less downtime: Faster detection and fixing means reduced downtime and more customer service.
- Better decisions: With visibility into system performance, management can make better decisions on resource allocation and strategic improvements.
- Develop and run faster: Developers and ops teams can catch bottlenecks or failures before they become big problems.
- Better customer experience: Smoother system operation means less customer friction and better service overall.

Observability vs Monitoring



Observability vs Monitoring

Observability

- Tells you why a system is at fault
- Acts as a knowledge base in defining what to monitor
- Focuses on giving context to the data
- Gives a more complete assessment of the overall environment
- Observability is a traversable map
- It gives you complete Information
- Observability creates the potential to monitor different events

VS

Monitoring

- Notifies you that a system is at fault
- Focuses on monitoring the systems & discovering faults
- Focuses on collecting data
- Focuses on monitoring KPIs
- Monitoring is a single plane
- It gives you limited Information
- Monitoring is the process of using observability

Profiling



Microservices

Aspect	Application Profiling	Observability
Focus	Code-level performance and resource usage (CPU, memory, etc.)	System-wide behavior and health (logs, metrics, traces)
Purpose	Identify performance bottlenecks in specific functions or code	Provide real-time monitoring and insights into system health
Granularity	Highly granular, focused on individual threads or functions	Broader scope, focusing on entire applications or services
Timing	Typically done offline or in development	Continuous, real-time monitoring in production environments
Data Type	Low-level data such as CPU cycles, memory usage, and I/O	High-level data like logs, metrics, and distributed traces
Use Case	Debugging, performance optimization, resource tuning	Troubleshooting, detecting issues, and ensuring reliability
Tool Examples	Profilers (e.g., CPU profilers, memory profilers)	Monitoring tools (e.g., Prometheus, Grafana, Datadog)
Scope	Specific code segments or modules	Entire application stack or distributed systems
Insight	Provides detailed insights into code behavior	Provides insights into how the system behaves as a whole
Outcome	Identifies inefficiencies at the code level ↓	Monitors and maintains system performance and availability



Metrics

- Metrics" refers to quantifiable measurements used to track and assess the performance or progress of a specific process, activity, or objective.
- Metrics are essential in various fields, including business, technology, healthcare, and education, to provide objective data for decision-making, analysis, and improvement.
- There are many types of metrics, depending on the context.

What metrics does APM track?



Application Performance Management Solution Includes



Individual
Performance
web request



All application
usage &
performance



Detailed
transaction
trace



Code-level
performance
profiling



Basic server
metrics



Application
server metrics



Custom
Application
metrics



Application
log data



Application
error

Types of Metrics

- **Business Metrics:**
 - **Revenue Growth:** Measures the increase in revenue over a certain period.
 - **Profit Margin:** Percentage of profit generated from total revenue.
 - **Customer Acquisition Cost (CAC):** The cost of acquiring a new customer.
 - **Net Promoter Score (NPS):** Measures customer satisfaction and likelihood to recommend a service/product.
 - **Churn Rate:** The percentage of customers who stop using a service/product.



Types of Metrics

- **Financial Metrics:**
 - **Return on Investment (ROI):** Measures profitability relative to the investment cost.
 - **Gross Profit Margin:** A company's total sales revenue minus its cost of goods sold (COGS).
 - **Debt-to-Equity Ratio:** A measure of a company's financial leverage.



Types of Metrics

- **Project Management Metrics:**
 - **On-time Delivery:** The percentage of projects delivered by the scheduled due date.
 - **Budget Variance:** The difference between the budgeted cost and the actual cost.
 - **Scope Changes:** Number or percentage of changes made to the project scope.



Types of Metrics

- **Technology/Software Development Metrics:**
 - **Velocity:** Measures the amount of work completed during a sprint (agile metric).
 - **Mean Time to Repair (MTTR):** Average time to fix a system after a failure.
 - **Defect Density:** The number of defects found in each amount of code.

Types of Metrics

- **Healthcare Metrics:**
 - **Patient Satisfaction Score:** Measures patient satisfaction with healthcare services.
 - **Average Length of Stay (ALOS):** The average number of days a patient spends in the hospital.
 - **Readmission Rates:** The percentage of patients who return to the hospital after discharge.



Types of Metrics

- **Marketing Metrics:**
 - **Conversion Rate:** The percentage of visitors who take a desired action (e.g., purchasing).
 - **Click-through Rate (CTR):** The percentage of people who clicked on a link after seeing an ad or email.
 - **Cost Per Lead (CPL):** The average cost of acquiring a lead.



Types of Metrics

- **Human Resources Metrics:**
 - **Employee Turnover Rate:** The percentage of employees who leave a company over a given period.
 - **Absenteeism Rate:** Measures employee absences as a percentage of the workforce.
 - **Employee Engagement Score:** A measure of employees' emotional investment in their work.



Importance of Metrics

- Performance Measurement:
 - Metrics allow organizations to gauge how well they are meeting objectives or standards.
 - Decision-Making: With data from metrics, businesses can make informed decisions regarding strategy, resource allocation, and priorities.
 - Benchmarking: Metrics help compare performance over time or against competitors.
 - Continuous Improvement: Metrics enable tracking of progress and identification of areas needing improvement.

Understand logs and events

- Logs and events are essential components in monitoring and analyzing systems, networks, and applications.
- They provide critical insights into the performance, security, and operational status of infrastructure, helping in troubleshooting, auditing, and improving overall system health.
- Understanding the distinction between logs and events and how they are used can help us better interpret system behaviors.

Understand logs and events



Understand logs and events

Aspect	Logs	Events
Definition	A record of system activities captured over time	A specific action or occurrence within the system
Content	Contains detailed information such as timestamps, messages, errors, etc.	Describes a significant change or state in the system, like an alert or notification
Granularity	Typically continuous and detailed, capturing all system activities	Typically discrete, representing a single instance of change
Purpose	Used for tracking, diagnosing, and understanding system behavior over time	Used to trigger alerts, notifications, or actions based on system state
Use Case	Analyzing system health, debugging, and root cause analysis	Monitoring system state, triggering automation, or handling real-time responses
Storage	Stored as log files or in log management systems	Stored in event logs, metrics, or monitoring systems
Example	"2024-10-11 10:45:32: Error connecting to database."	"User login successful" or "Server down at 10:45 AM"
Relation to Monitoring	Provides context for events, giving detailed system history	Indicates when something of importance occurs
Tool Examples	Logstash, Fluentd, Splunk	Prometheus, Nagios, AWS CloudWatch Events
Time Aspect	Often continuous and spans over a long period	Discrete and represents a point-in-time occurrence



Understand logs and events

- Logs:
 - A log is a detailed, time-stamped record of activity generated by systems, applications, devices, or users.
 - Logs capture various types of information and are used for debugging, auditing, monitoring, and analyzing the state of a system.

Understand logs and events

- **Characteristics of Logs:**

1. **Time-stamped:** Logs are usually recorded with the exact time and date when the action or event occurred.
2. **Detailed Information:** Logs typically contain detailed information such as user actions, system processes, requests, errors, and system messages.
3. **Continuous Generation:** Logs are continuously generated by systems, often creating large amounts of data over time.
4. **Text-based:** Most logs are stored as plain text or in structured formats like JSON or XML.
5. **Retained for Auditing:** Logs can be stored for long periods to serve as an audit trail of system activities.



Understand logs and events

- **Types of Logs:**

- **System Logs:** Captures operating system-related activities such as boot sequences, services starting or stopping, and system errors.
- **Application Logs:** Records events within an application, such as user requests, database queries, and application errors.
- **Security Logs:** Includes information about user logins, permissions, access control, and failed login attempts.
- **Network Logs:** Logs data about network traffic, firewall rules, and packet exchanges.



Understand logs and events

- **Examples of Logs:**

- Web server access logs capturing incoming HTTP requests.
- System logs capturing user logins, reboots, or file access.
- Error logs generated by applications when an error or crash occurs.

Understand logs and events

- **Events:**

- An event is a specific occurrence or action within a system or application, often defined by triggers or conditions.
- Events represent key moments that may require attention, such as a security alert, system crash, or significant change in state.

Understand logs and events

- **Characteristics of Events:**

1. **Discrete Occurrences:** An event is usually tied to a specific action or change, such as a system startup, security breach, or completed transaction.
2. **Higher-Level Information:** Events tend to summarize important activities, unlike logs which may capture more granular data.
3. **Trigger-Based:** Events are often triggered by specific conditions or thresholds, such as a system reaching a performance limit, or a user accessing restricted data.
4. **Alerting and Monitoring:** Events are often tied to monitoring systems that generate alerts, notifications, or reports for administrators.
5. **Structured Data:** Events are often captured in a more structured way, helping systems and humans process them efficiently.



Understand logs and events

- **Examples of Events:**

- A user successfully logs in to a system (authentication event).
- A network intrusion detection system identifies a potential attack (security event).
- A scheduled backup process successfully completes (system event).
- An application generates a "transaction completed" event when a purchase is made.

Understand logs and events

- **Differences Between Logs and Events:**
 - **Granularity:** Logs are more granular and detailed, capturing every action or change in a system, while events highlight more significant occurrences.
 - **Purpose:** Logs are used for detailed analysis, diagnostics, and audit trails, whereas events focus on triggering alerts, monitoring system health, and identifying critical issues.
 - **Data Structure:** Logs are often unstructured or semi-structured (e.g., text files), while events tend to be structured and can be more easily parsed by monitoring tools.
 - **Storage:** Logs are often stored for a long time to track past actions, while events are typically used in real-time for immediate responses or alerts.

Understand logs and events

- **Use Cases of Logs and Events:**
 - **Troubleshooting:** When a system issue occurs, logs help pinpoint the exact sequence of actions that led to the issue.
 - For example, an error log in a web server could reveal why a particular page failed to load.
 - **Security Monitoring:** Security logs and events help detect suspicious behavior such as failed login attempts, unauthorized access, or malware activity.
 - Event monitoring tools can raise an alert if certain thresholds are breached (e.g., multiple failed login attempts).

Understand logs and events

- Performance Analysis:
 - Logs can capture metrics such as CPU usage, memory consumption, and network traffic, helping administrators detect performance bottlenecks or inefficient processes.
 - Compliance Auditing: Logs serve as a record of system activity, which is often required for regulatory compliance.
 - Logs can show who accessed data, when, and what actions were taken.
 - Real-Time Monitoring and Alerting: Event management systems (like SIEM or APM tools) continuously monitor systems for events that may require immediate action, such as security breaches or system outages.

Tracing and Spans



Trace And Span

A span is one operation and can consist of multiple sub-operations, e.g. a database request, a HTTP request to another service etc.

Trace

A trace describes one "request" from start to finish, e.g. an incoming HTTP request (POST/users)

Span 200ms

Span 50ms

Span 35ms

Span 35ms

Span 100ms

Span 25ms

Span

Span

Tracing and Spans

Aspect	Tracing	Spans
Definition	A method used to track the entire journey of a request as it moves through different services in a distributed system.	A single unit of work within a trace, representing a specific operation or step in the request's journey.
Scope	Captures the flow of a request across multiple systems or services, providing a high-level overview.	Represents a specific operation, such as a function call or database query, within the trace.
Granularity	High-level, focused on the end-to-end lifecycle of the request.	Fine-grained, focused on individual components or steps within the trace.
Purpose	To give visibility into how requests traverse through services, helping to identify bottlenecks, failures, or latency issues.	To break down the trace into specific operations, allowing detailed performance analysis for each step.
Hierarchy	A trace consists of multiple spans that represent the steps involved in processing a request.	A span is a segment of a trace, and spans can be nested or linked to show relationships between operations.

Tracing and Spans

Data Captured	Collects metadata such as trace ID, timestamps, and relationships between spans.	Captures operation-specific details like start/end time, duration, and metadata (tags, logs).
Visualization	Typically visualized as a flow of connected spans, showing the end-to-end path of a request.	Visualized as individual nodes or blocks within the trace, often nested to show dependencies.
Use Case	Helps diagnose latency issues, track request flow, and identify problematic services in distributed architectures.	Helps to pinpoint delays or errors in specific operations within a service.
Example	A trace could show a user's request passing through a web server, an application server, and a database.	A span might represent the database query or the web server processing the request.
Tool Examples	Jaeger, OpenTelemetry, Zipkin	Spans are part of trace data in the same tools (Jaeger, Zipkin).

◆ What are “Multiple Statements on a Single Line”?



- 👉 Writing more than one statement in the same line using ;
- **Example**
- `x = 10; y = 20; print(x + y)`
- ✓ This is valid Python
 - ✓ It runs correctly
- ◆ **Another example (if statement)**
- `if x > 5: print("Greater")`



What are “Statement Groups as Suites”?

- 👉 In Python, a **suite** = a block of statements that belong together
- Examples:
 - inside if
 - inside for
 - inside while
 - inside def

◆ Normal (recommended) suite

- if $x > 0$:
 print("Positive")
 print("Valid number")
- 👉 This is a **multi-line suite**
 - ✓ Proper
 - ✓ Readable
 - ✓ Recommended



Command Line Arguments

- 👉 Values passed to a Python program when you run it from terminal.
- `python script.py hello 123`
- 👉 Here:
 - `script.py` → program
 - `hello, 123` → **command line arguments**

Command Line Arguments

✓ 1. Using `sys.argv` (basic way)

◆ Example

```
</> Python  
  
import sys  
  
print(sys.argv)
```

Run:

```
</> Bash  
  
python script.py apple 10
```

Output:

```
['script.py', 'apple', '10']
```

Command Line Arguments

-  **Problem with sys.argv**
- No validation 
- No help message 
- Hard to scale 

Command Line Arguments

✓ 2. Using `argparse` (recommended)

👉 Best practice for real projects

◆ Example

```
</> Python

import argparse

parser = argparse.ArgumentParser()

parser.add_argument("name")
parser.add_argument("age")

args = parser.parse_args()

print("Name:", args.name)
print("Age:", args.age)
```

Run:

```
</> Bash

python script.py Anita 25
```

Command Line Arguments

◆ Help command

`</> Bash`

```
python script.py --help
```

Output:

```
usage: script.py [-h] name age
```

Command Line Arguments

✓ 3. Optional arguments

Python

```
import argparse

parser = argparse.ArgumentParser()

parser.add_argument("--name", default="User")
parser.add_argument("--age", type=int)

args = parser.parse_args()

print(args.name, args.age)
```

Run:

Bash

```
python script.py --name Anita --age 25
```


Command Line Arguments

✓ 4. Boolean flags

⟨/⟩ Python

```
parser.add_argument("--debug", action="store_true")
```

Run:

⟨/⟩ Bash

```
python script.py --debug
```

◆ What is Variable Assignment?

- A **variable** is like a labeled box, and **assignment** means putting a value inside that box.
- 👉 General syntax (most languages):
- `variable_name = value`

◆ What is Variable Assignment?

- **1. Basic Assignment**

- `x = 10`
`name = "John"`
`price = 99.99`
- `x` stores an integer
 - `name` stores a string
 - `price` stores a float

◆ What is Variable Assignment?

- **2. Multiple Assignments**

- `a, b, c = 1, 2, 3`
- 👉 Assign same value:
- `x = y = z = 0`



◆ What is Variable Assignment?

- **3. Dynamic Typing (Python Special)**
- `x = 10`
`x = "Hello" # type changes`



◆ What is Variable Assignment?

- **4. Assigning Expressions**

- `total = 10 + 20`
`result = total * 2`

◆ What is Variable Assignment?

- **5. Swapping Values**

- `a = 5`
`b = 10`

`a, b = b, a`



◆ What is Variable Assignment?

- **6. Assigning from Input**
- `name = input("Enter your name: ")`

Waiting for the user input

- it usually means your Python code is using **input()** and is paused, waiting for you to type something.
- 🧠 **Why this happens**
- Example:
 - `name = input("Enter your name: ")`
`print(name)`
- 👉 Program stops and waits for you to type input.

Waiting for the user input



```
File Edit Selection View Go Run ... simple-project
```

EXPLORER

- SIMPLE-PROJECT
 - .pytest_cache
 - .venv
 - build
 - data
 - src
 - dsapp
 - __pypcache__
 - __init__.py
 - analysis.py
 - loader.py
 - main.py
 - model.py
 - dsapp.egg-info
 - tests
 - pyproject.toml
 - README.md

OUTPUT

Filter (e.g. text, !excludeText, text1,text2)

main.py

```
src > dsapp > main.py > run
3 from dsapp.model import top_selling_product
4
5
6 def run():
7     df = load_data("data/sales.csv")
8     name = input("Enter your name: ")
9     print(name)
10    print("Total Revenue:", total_revenue(df))
11    print("Average Price:", average_price(df))
```

TERMINAL

```
Successfully uninstalled dsapp-0.1.0
Successfully installed dsapp-0.1.0

[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) E:\Datascience2026\simple-project>pytest
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0
rootdir: E:\Datascience2026\simple-project
configfile: pyproject.toml
collected 1 item

tests\test_analysis.py . [100%]

===== 1 passed in 0.55s =====

(.venv) E:\Datascience2026\simple-project>python src/dsapp/main.py
Enter your name: Parameswari
Parameswari
Total Revenue: 182000
Average Price: 18750.0
Top Product: Shoes

(.venv) E:\Datascience2026\simple-project>
```

◆ What is Variable Assignment?

- ◆ Other Languages Comparison

- **Java**

- `int x = 10;`
`String name = "John";`

- **JavaScript**

- `let x = 10;`
`const name = "John";`

◆ What is Variable Assignment?

- ◆ Important Concepts
- ✓ 1. Mutable vs Immutable
- `x = 10` # immutable
`list1 = [1,2,3]` # mutable
- ✓ 2. Reference Assignment
- `a = [1,2,3]`
`b = a` # both point to same list

◆ What is Variable Assignment?

-  **3. Augmented Assignment**
- $x = 5$
 $x += 2 \quad \# \quad x = x + 2 \rightarrow 7$

◆ Main Standard Data Types in Python

- **① Numeric Types**
- Used to store numbers.
- `x = 10` `# int`
 `y = 3.14` `# float`
 `z = 2 + 3j` `# complex`
- **Types:**
- `int` → whole numbers
- `float` → decimal numbers
- `complex` → real + imaginary



◆ Main Standard Data Types in Python

- **2 String (str)**
- Used to store text.
- `name = "John"`
`message = 'Hello World'`
- **Features:**
- Immutable (cannot change directly)
- Supports indexing and slicing
- `print(name[0])` # J
`print(name[0:2])` # Jo

◆ Main Standard Data Types in Python

- **3 List (list)**
- Ordered, mutable collection.
- `numbers = [1, 2, 3, 4]`
`mixed = [1, "hello", 3.5]`
- **Features:**
- Mutable (can change)
- Allows duplicates
- `numbers.append(5)`
`numbers[0] = 10`



◆ Main Standard Data Types in Python

- **4 Tuple (tuple)**
- Ordered but **immutable** collection.
- `t = (1, 2, 3)`
- **Features:**
 - Faster than list
 - Cannot modify after creation

◆ Main Standard Data Types in Python

- **5 Set (set)**
- Unordered collection of unique values.
- $s = \{1, 2, 3, 3\}$
- 👉 Output:
- $\{1, 2, 3\}$
- **Features:**
- No duplicates
- Useful for mathematical operations

◆ Main Standard Data Types in Python

- **6 Dictionary (dict)**
- Key-value pairs.
- ```
student = {
 "name": "John",
 "age": 20
}
```
- **Access:**
- ```
print(student["name"])
```



◆ Main Standard Data Types in Python

- **7 Boolean (bool)**
- Represents True or False.
- `is_valid = True`



◆ Main Standard Data Types in Python

- ◆ **Type Checking**
- `x = 10`
`print(type(x))` # <class 'int'>



◆ Main Standard Data Types in Python

- ◆ **Type Conversion (Casting)**
- `x = int("10")`
`y = float(5)`
`z = str(100)`

◆ Main Standard Data Types in Python

Data Type	Example	Mutable
int	10	✗
float	3.14	✗
str	"Hello"	✗
list	[1,2,3]	✓
tuple	(1,2,3)	✗
set	{1,2,3}	✓
dict	{"a":1}	✓
bool	True	✗

Python Basic Operators



Operator	Description	Example
+	Addition - Adds values on either side of the operator	$a + b$ will give 30
-	Subtraction - Subtracts right hand operand from left hand operand	$a - b$ will give -10
*	Multiplication - Multiplies values on either side of the operator	$a * b$ will give 200
/	Division - Divides left hand operand by right hand operand	b / a will give 2
%	Modulus - Divides left hand operand by right hand operand and returns remainder	$b \% a$ will give 0
**	Exponent - Performs exponential (power) calculation on operators	$a ** b$ will give 10 to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed.	$9 // 2$ is equal to 4 and $9.0 // 2.0$ is equal to 4.



Python Comparison Operators

Operator	Description	Example
==	Checks if the value of two operands are equal or not, if yes then condition becomes true.	(a == b) is not true.
!=	Checks if the value of two operands are equal or not, if values are not equal then condition becomes true.	(a != b) is true.
<>	Checks if the value of two operands are equal or not, if values are not equal then condition becomes true.	(a <> b) is true. This is similar to != operator.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(a > b) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(a < b) is true.

Python Assignment Operators

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand	$c = a + b$ will assign value of $a + b$ into c
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand	$c -= a$ is equivalent to $c = c - a$
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand	$c *= a$ is equivalent to $c = c * a$
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand	$c /= a$ is equivalent to $c = c / a$

Python Assignment Operators

<code>>=</code>	Checks if the value of left operand is greater than or equal to the value of right operand, if (a <code>>=</code> b) is not true. yes then condition becomes true.
<code><=</code>	Checks if the value of left operand is less than or equal to the value of right operand, if (a <code><=</code> b) is true. yes then condition becomes true.
<code>%=</code>	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand <code>c %= a</code> is equivalent to <code>c = c % a</code>
<code>**=</code>	Exponent AND assignment operator, Performs exponential (power) calculation on operators and assign value to the left operand <code>c **= a</code> is equivalent to <code>c = c ** a</code>
<code>//=</code>	Floor Division and assigns a value, Performs floor division on operators and assign value to the left operand <code>c //= a</code> is equivalent to <code>c = c // a</code>

Python Bitwise Operators

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(a & b) will give 12 which is 0000 1100
	Binary OR Operator copies a bit if it exists in either operand.	(a b) will give 61 which is 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(a ^ b) will give 49 which is 0011 0001
~	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~a) will give -60 which is 1100 0011
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	a << 2 will give 240 which is 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 will give 15 which is 0000 1111

Python Logical Operators

Operator	Description	Example
and	Called Logical AND operator. If both the operands are true then then condition becomes true.	(a and b) is true.
or	Called Logical OR Operator. If any of the two operands are non zero then then condition becomes true.	(a or b) is true.
not	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	not(a and b) is false.



Python Membership Operators

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not find a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is a member of sequence y.

Python Identity Operators



Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y).

Python Operators Precedence



Operator	Description
**	Exponentiation (raise to the power)
~ + -	Ccomplement, unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction
>> <<	Right and left bitwise shift
&	Bitwise 'AND'

Python Operators Precedence



<code>^ </code>	Bitwise exclusive `OR' and regular `OR'
<code><= < > >=</code>	Comparison operators
<code><> == !=</code>	Equality operators
<code>= %= /= //= -= += *= **=</code>	Assignment operators
<code>is is not</code>	Identity operators
<code>in not in</code>	Membership operators
<code>not or and</code>	Logical operators

Python Decision Making



Statement

Description

if statements

An **if statement** consists of a boolean expression followed by one or more statements.

if...else statements

An **if statement** can be followed by an optional **else statement**, which executes when the boolean expression is false.

nested if statements

You can use one **if** or **else if** statement inside another **if** or **else if** statement(s).

Single Statement Suites

```
#!/usr/bin/python
```

```
var = 100
```

```
if ( var == 100 ) : print "Value of expression is 100"
```

```
    print "Good bye!"
```

Python IF...ELIF...ELSE Statements

```
#!/usr/bin/python
```

```
var = 100
```

```
if var == 200:
```

```
    print "1 - Got a true expression value"
```

```
    print var
```

```
elif var == 150:
```

```
    print "2 - Got a true expression value"
```

```
    print var2
```

Python IF...ELIF...ELSE Statements

```
elif var == 100:
```

```
    print "3 - Got a true expression value"
```

```
    print var
```

```
else:
```

```
    print "4 - Got a false expression value"
```

```
    print var
```

```
print "Good bye!"
```

Switch Case

- `match x:`
- `case n if n < 10:`
- `print("Less than 10")`
- `case n if 10 <= n < 50:`
- `print("Between 10 and 50")`
- `case _:`
- `print("50 or more")`

Python while Loop Statements

```
#!/usr/bin/python
```

```
count = 0
```

```
while (count < 9):
```

```
    print 'The count is:', count
```

```
    count = count + 1
```

```
print "Good bye!"
```

Python for Loop Statements

```
#!/usr/bin/python
```

```
for letter in 'Python':    # First Example  
    print 'Current Letter :', letter
```

```
fruits = ['banana', 'apple', 'mango']  
for fruit in fruits:      # Second Example  
    print 'Current fruit :', fruit  
print "Good bye!"
```


Python nested loops

```
#!/usr/bin/python
i = 2
while(i < 100):
    j = 2
    while(j <= (i/j)):
        if not(i%j): break
        j = j + 1
    if (j > i/j) : print i, " is prime"
    i = i + 1
print "Good bye!"
```

Python break statement

```
#!/usr/bin/python
for letter in 'Python': # First Example
    if letter == 'h':
        break
    print 'Current Letter :', letter
var = 10 # Second Example
while var > 0:
    print 'Current variable value :',
    var = var - 1
    if var == 5:
        break
print "Good bye!"
```

Python pass statement

- The **pass** statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.
- The **pass** statement is a *null* operation; nothing happens when it executes.
- The **pass** is also useful in places where your code will eventually go, but has not been written yet (e.g., in stubs for example):

Python pass statement



```
for letter in 'Python':  
    if letter == 'h':  
        pass  
        print 'This is pass block'  
    print 'Current Letter :', letter  
  
print "Good bye!"
```

Mathematical Functions



Function	Returns (description)
<u>abs(x)</u>	The absolute value of x: the (positive) distance between x and zero.
<u>ceil(x)</u>	The ceiling of x: the smallest integer not less than x
<u>cmp(x, y)</u>	-1 if $x < y$, 0 if $x == y$, or 1 if $x > y$
<u>exp(x)</u>	The exponential of x: e^x
<u>fabs(x)</u>	The absolute value of x.
<u>floor(x)</u>	The floor of x: the largest integer not greater than x

Mathematical Functions

[log\(x\)](#)

The natural logarithm of x , for $x > 0$

[log10\(x\)](#)

The base-10 logarithm of x for $x > 0$.

[max\(x1, x2,...\)](#)

The largest of its arguments: the value closest to positive infinity

[min\(x1, x2,...\)](#)

The smallest of its arguments: the value closest to negative infinity

[modf\(x\)](#)

The fractional and integer parts of x in a two-item tuple. Both parts have the same sign as x . The integer part is returned as a float.

[pow\(x, y\)](#)

The value of $x^{**}y$.

[round\(x \[,n\]\)](#)

x rounded to n digits from the decimal point. Python rounds away from zero as a tie-breaker: `round(0.5)` is 1.0 and `round(-0.5)` is -1.0.

[sqrt\(x\)](#)

The square root of x for $x > 0$

Random Number Functions

Function	Description
<u>choice(seq)</u>	A random item from a list, tuple, or string.
<u>randrange ([start,] stop [,step])</u>	A randomly selected element from range(start, stop, step)
<u>random()</u>	A random float r, such that 0 is less than or equal to r and r is less than 1
<u>seed([x])</u>	Sets the integer starting value used in generating random numbers. Call this function before calling any other random module function. Returns None.
<u>shuffle(lst)</u>	Randomizes the items of a list in place. Returns None.
<u>uniform(x, y)</u>	A random float r, such that x is less than or equal to r and r is less than y

Trigonometric Functions

Function	Description
<u>acos(x)</u>	Return the arc cosine of x, in radians.
<u>asin(x)</u>	Return the arc sine of x, in radians.
<u>atan(x)</u>	Return the arc tangent of x, in radians.
<u>atan2(y, x)</u>	Return $\text{atan}(y / x)$, in radians.
<u>cos(x)</u>	Return the cosine of x radians.
<u>hypot(x, y)</u>	Return the Euclidean norm, $\sqrt{x^2 + y^2}$.
<u>sin(x)</u>	Return the sine of x radians.
<u>tan(x)</u>	Return the tangent of x radians.
<u>degrees(x)</u>	Converts angle x from radians to degrees.
<u>radians(x)</u>	Converts angle x from degrees to radians.

Mathematical Constants



Constants	Description
π	The mathematical constant π .
e	The mathematical constant e .

- Strings in Python are identified as a contiguous set of characters in between quotation marks.
- Python allows for either pairs of single or double quotes.
- Subsets of strings can be taken using the slice operator (`[ ]` and `[ : ]`) with indexes starting at 0 in the beginning of the string and working their way from -1 at the end.



Python Strings

```
#!/usr/bin/python
```

```
str = 'Hello World!'
```

```
print str # Prints complete string
```

```
print str[0] # Prints first character of the string
```

```
print str[2:5] # Prints characters starting from 3rd to 5th
```

```
print str[2:] # Prints string starting from 3rd character
```

```
print str * 2 # Prints string two times
```

```
print str + "TEST" # Prints concatenated string
```



Python Strings

- **Accessing Values in Strings**

```
#!/usr/bin/python
```

```
var1 = 'Hello World!'
```

```
var2 = "Python Programming"
```

```
print "var1[0]: ", var1[0]
```

```
print "var2[1:5]: ", var2[1:5]
```



Updating Strings

```
#!/usr/bin/python
```

```
var1 = 'Hello World!'
```

```
print "Updated String :- ", var1[:6] + 'Python'
```

Escape Characters

Backslash notation	Hexadecimal character	Description
<code>\a</code>	<code>0x07</code>	Bell or alert
<code>\b</code>	<code>0x08</code>	Backspace
<code>\cx</code>		Control-x
<code>\C-x</code>		Control-x
<code>\e</code>	<code>0x1b</code>	Escape
<code>\f</code>	<code>0x0c</code>	Formfeed
<code>\M-\C-x</code>		Meta-Control-x
<code>\n</code>	<code>0x0a</code>	Newline

Escape Characters



\nnn		Octal notation, where n is in the range 0.7
\r	0x0d	Carriage return
\s	0x20	Space
\t	0x09	Tab
\v	0x0b	Vertical tab
\x		Character x
\xnn		Hexadecimal notation, where n is in the range 0.9, a.f, or A.F

String Special Operators



Operator	Description	Example
+	Concatenation - Adds values on either side of the operator	a + b will give HelloPython
*	Repetition - Creates new strings, concatenating multiple copies of the same string	a*2 will give -HelloHello
[]	Slice - Gives the character from the given index	a[1] will give e
[:]	Range Slice - Gives the characters from the given range	a[1:4] will give ell
in	Membership - Returns true if a character exists in the given string	H in a will give 1

String Special Operators



not in	Membership - Returns true if a character does not exist in the given string	M not in a will give 1
r/R	Raw String - Suppress actual meaning of Escape characters. The syntax for raw strings is exactly the same as for normal strings with the exception of the raw string operator, the letter "r," which precedes the quotation marks. The "r" can be lowercase (r) or uppercase (R) and must be placed immediately preceding the first quote mark.	print r'\n' prints \n and print R'\n' prints \n

String Formatting Operator

```
#!/usr/bin/python
```

```
print "My name is %s and weight is %d kg!" % ('Zara',  
21)
```

String Formatting Operator

Format Symbol	Conversion
<code>%c</code>	character
<code>%s</code>	string conversion via <code>str()</code> prior to formatting
<code>%i</code>	signed decimal integer
<code>%d</code>	signed decimal integer
<code>%u</code>	unsigned decimal integer
<code>%o</code>	octal integer
<code>%x</code>	hexadecimal integer (lowercase letters)

String Formatting Operator



<code>%X</code>	hexadecimal integer (UPPERcase letters)
<code>%e</code>	exponential notation (with lowercase 'e')
<code>%E</code>	exponential notation (with UPPERcase 'E')
<code>%f</code>	floating point real number
<code>%g</code>	the shorter of %f and %e
<code>%G</code>	the shorter of %f and %E



Triple Quotes

- Python's triple quotes comes to the rescue by allowing strings to span multiple lines, including verbatim NEWLINES, TABs, and any other special characters.



Triple Quotes

```
#!/usr/bin/python
```

```
para_str = """this is a long string that is made up of  
several lines and non-printable characters such as  
TAB ( \t ) and they will show up that way when displayed.  
NEWLINEs within the string, whether explicitly given like  
this within the brackets [ \n ], or just a NEWLINE within  
the variable assignment will also show up.
```

```
"""
```

```
print para_str;
```



Raw String

```
#!/usr/bin/python
```

```
print 'C:\\nowhere'
```

```
C:\\nowhere #output
```

```
#!/usr/bin/python
```

```
print r'C:\\nowhere'
```

```
C:\\nowhere #output
```



Unicode String

```
#!/usr/bin/python  
print u'Hello, world!'
```




Python String capitalize() Method

This method returns a copy of the string with only its first character capitalized. For 8-bit strings, this method is locale-dependent.

```
#!/usr/bin/python
```

```
str = "this is string example....wow!!!";
```

```
print "str.capitalize() : ", str.capitalize()
```



Python String center() Method

- This method returns centered in a string of length *width*.
- Padding is done using the specified *fillchar*.
- Default filler is a space.

```
#!/usr/bin/python
```

```
str = "this is string example....wow!!!";
```

```
print "str.center(40, 'a') : ", str.center(40, 'a')
```

SN	Methods with Description
1	<u>capitalize()</u> Capitalizes first letter of string
2	<u>center(width, fillchar)</u> Returns a space-padded string with the original string centered to a total of width columns
3	<u>count(str, beg= 0,end=len(string))</u> Counts how many times str occurs in string, or in a substring of string if starting index beg and ending index end are given
3	<u>decode(encoding='UTF-8',errors='strict')</u> Decodes the string using the codec registered for encoding. encoding defaults to the default string encoding.
4	<u>encode(encoding='UTF-8',errors='strict')</u> Returns encoded string version of string; on error, default is to raise a ValueError unless errors is given with 'ignore' or 'replace'.
5	<u>endswith(suffix, beg=0, end=len(string))</u> Determines if string or a substring of string (if starting index beg and ending index end are given) ends with suffix; Returns true if so, and false otherwise

- 6 `expandtabs(tabsize=8)`
Expands tabs in string to multiple spaces; defaults to 8 spaces per tab if tabsize not provided
- 7 `find(str, beg=0 end=len(string))`
Determine if str occurs in string, or in a substring of string if starting index beg and ending index end are given; returns index if found and -1 otherwise
- 8 `index(str, beg=0, end=len(string))`
Same as find(), but raises an exception if str not found
- 9 `isalnum()`
Returns true if string has at least 1 character and all characters are alphanumeric and false otherwise
- 10 `isalpha()`
Returns true if string has at least 1 character and all characters are alphabetic and false otherwise
- 11 `isdigit()`
Returns true if string contains only digits and false otherwise
- 12 `islower()`
Returns true if string has at least 1 cased character and all cased characters are in lowercase and false otherwise

- 13 isnumeric()
Returns true if a unicode string contains only numeric characters and false otherwise
- 14 isspace()
Returns true if string contains only whitespace characters and false otherwise
- 15 istitle()
Returns true if string is properly "titlecased" and false otherwise
- 16 isupper()
Returns true if string has at least one cased character and all cased characters are in uppercase and false otherwise
- 17 join(seq)
Merges (concatenates) the string representations of elements in sequence seq into a string, with separator string
- 18 len(string)
Returns the length of the string

- 19 ljust(width[, fillchar])
Returns a space-padded string with the original string left-justified to a total of width columns
- 20 lower()
Converts all uppercase letters in string to lowercase
- 21 lstrip()
Removes all leading whitespace in string
- 22 maketrans()
Returns a translation table to be used in translate function.
- 23 max(str)
Returns the max alphabetical character from the string str
- 24 min(str)
Returns the min alphabetical character from the string str
- 25 replace(old, new [, max])
Replaces all occurrences of old in string with new, or at most max occurrences if max given

-
- 26 `rfind(str, beg=0, end=len(string))`
Same as `find()`, but search backwards in string
- 27 `rindex( str, beg=0, end=len(string))`
Same as `index()`, but search backwards in string
- 28 `rjust(width,[, fillchar])`
Returns a space-padded string with the original string right-justified to a total of width columns.
- 29 `rstrip()`
Removes all trailing whitespace of string
- 30 `split(str="", num=string.count(str))`
Splits string according to delimiter `str` (space if not provided) and returns list of substrings; split into at most `num` substrings if given
- 31 `splitlines( num=string.count("\n"))`
Splits string at all (or `num`) NEWLINES and returns a list of each line with NEWLINES removed
- 32 `startswith(str, beg=0, end=len(string))`
Determines if string or a substring of string (if starting index `beg` and ending index `end` are given) starts with substring `str`; Returns true if so, and false otherwise

Built-in String Methods

- 33 [strip\(\[chars\]\)](#)
Performs both lstrip() and rstrip() on string
- 34 [swapcase\(\)](#)
Inverts case for all letters in string
- 35 [title\(\)](#)
Returns "titlecased" version of string, that is, all words begin with uppercase, and the rest are lowercase
- 36 [translate\(table, deletechars=""\)](#)
Translates string according to translation table str(256 chars), removing those in the del string
- 37 [upper\(\)](#)
Converts lowercase letters in string to uppercase
- 38 [zfill \(width\)](#)
Returns original string leftpadded with zeros to a total of width characters; intended for numbers, zfill() retains any sign given (less one zero)
- 39 [isdecimal\(\)](#)
Returns true if a unicode string contains only decimal characters and false otherwise

Encoding Standards



Codec	Aliases	Operand type	Purpose
base64_codec	base64, base-64	byte string	Convert operand to MIME base64
bz2_codec	bz2	byte string	Compress the operand using bz2
hex_codec	hex	byte string	Convert operand to hexadecimal representation, with two digits per byte
idna		Unicode string	Implements RFC 3490 , see also encodings.idna
mbcs	dbcs	Unicode string	Windows only: Encode operand according to the ANSI codepage (CP_ACP)
palmos		Unicode string	Encoding of PalmOS 3.5
punycode		Unicode string	Implements RFC 3492
quopri_codec	quopri, quoted-printable, quotedprintable	byte string	Convert operand to MIME quoted printable

Functions

- Function blocks begin with the keyword **def** followed by the function name and parentheses (()).
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The code block within every function starts with a colon (:) and is indented.
- The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return None`.

Functions

1. What is a Function in Python?

A **function** is a reusable block of code that performs a specific task.

```
python
```

```
def greet():  
    print("Hello!")
```

You call it with:

```
python
```

```
greet()  # Output: Hello!
```

Functions



2. Function with Parameters

python

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Alice")  # Output: Hello, Alice!
```



3. Function with Return Value

python

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  # result = 8
```

Functions



4. Default Parameters

python

```
def greet(name="Guest"):  
    print(f"Welcome, {name}!")  
  
greet()           # Welcome, Guest!  
greet("Ravi")     # Welcome, Ravi!
```



5. Keyword Arguments

python

```
def full_name(first, last):  
    print(f"{first} {last}")  
  
full_name(last="Sharma", first="Ankit") # Ankit Sharma
```



6. Variable-Length Arguments

`*args` (Non-keyword)

python

```
def total(*numbers):  
    return sum(numbers)  
  
print(total(1, 2, 3)) # 6
```

`**kwargs` (Keyword)

python

```
def print_info(**details):  
    for key, value in details.items():  
        print(f"{key}: {value}")  
  
print_info(name="Alice", age=30)
```

7. Lambda (Anonymous) Functions

python

```
square = lambda x: x * x  
print(square(4)) # 16
```

8. Recursion Example

python

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(5)) # 120
```

9. Real-World Example: Email Validation

python

```
def is_valid_email(email):  
    return "@" in email and "." in email  
  
print(is_valid_email("a@b.com")) # True
```


Module Basics

- What is Modular Programming?
 - Modular programming is the practice of segmenting a single, complicated coding task into multiple, simpler, easier-to-manage sub-tasks.
 - We call these subtasks modules.
 - Therefore, we can build a bigger program by assembling different modules that act like building blocks.



Module Basics

- A Python module is a file containing Python definitions and statements.
- A module can define functions, classes, and variables.
- A module can also include runnable code.
- Grouping related code into a module makes the code easier to understand and use.
- It also makes the code logically organized.



Types of Modules

- There are three different ways to define a module in Python:
 - A module can be written in Python itself.
 - A module can be written in C and loaded dynamically at run-time, like the re (regular expression) module.
 - A built-in module is intrinsically contained in the interpreter, like the itertools module.

Module Basics

1 usage

```
def fundTransfer(fromAccountNo, toAccountNo, amount):  
    currentTime = datetime.datetime.now().hour;  
    print(currentTime)  
    if (account['accountNo'] == fromAccountNo):  
        if (account['balance'] > int(amount)):  
            if (currentTime > 21):  
                print("Fund Transfer will happen in next working day");  
            else:  
                print("Fund transfer initiated to Account No %d"  
                    % (toAccountNo));
```

Module Basics



```
import Day3.functionsdemo as fd  
fd.fundTransfer( fromAccountNo: 23864585, toAccountNo: 59756, amount: "5000")  
|
```



Module

- A module is a file containing Python definitions and functions that you can reuse across projects.

Example: A file named `math_utils.py` is a module if it contains:

```
python
```

```
def add(a, b):  
    return a + b
```

2. Importing Modules

Import entire module:

```
python

import math
print(math.sqrt(16))  # 4.0
```

Import specific function:

```
python

from math import sqrt
print(sqrt(25))  # 5.0
```

Alias a module:

```
python

import math as m
print(m.pi)  # 3.1415...
```

3. Built-in Modules

Examples:

- `math` : mathematical operations
- `random` : random numbers
- `os` : file system operations
- `datetime` : date/time operations
- `sys` : command-line and interpreter info

```
python
```

```
import random  
print(random.randint(1, 10))
```




4. User-defined Modules

Create a file `greetings.py` :

```
python

def hello(name):
    print(f"Hello, {name}!")
```

Use it:

```
python

import greetings
greetings.hello("Alice")
```

5. The `__name__ == "__main__"` Trick

To make your module executable and import-safe:

```
python

def greet():
    print("Hi!")

if __name__ == "__main__":
    greet()
```

7. Real-World Example: Logging Module

```
python

import logging

logging.basicConfig(level=logging.INFO)
logging.info("Script started")
```

Common Third-party Modules

Install with `pip install module_name`

Module	Use Case
<code>requests</code>	HTTP requests
<code>pandas</code>	Data manipulation
<code>numpy</code>	Numerical operations
<code>flask</code>	Web development
<code>pytest</code>	Testing



Packages

- Python Packages are a way to organize and structure python code into reusable components.
- Like a folder that contains related Python files (modules) that work together to provide certain functionality.
- Packages help keep your code organized, make it easier to manage and maintain, and allow you to share your code with others.
- They're like a toolbox where you can store and organize your tools (functions and classes) for easy access and reuse in different projects.



How to Create Package in Python?

- Creating packages in Python allows you to organize your code into reusable and manageable modules.
- Create a Directory: Start by creating a directory (folder) for your package.
- This directory will serve as the root of your package structure.
- Add Modules: Within the package directory, you can add Python files (modules) containing your code.
- Each module should represent a distinct functionality or component of your package.
- Init File: Include an `__init__.py` file in the package directory. This file can be empty or can contain an initialization code for your package.
- It signals to Python that the directory should be treated as a package.



How to Create Package in Python?

- Subpackages: You can create sub-packages within your package by adding additional directories containing modules, along with their own `__init__.py` files.
- Importing: To use modules from your package, import them into your Python scripts using dot notation.
- For example, if you have a module named `module1.py` inside a package named `mypackage`, you would import its function like this: `from mypackage.module1 import greet`.



How to Create Package in Python?

- **Distribution:** If you want to distribute your package for others to use, you can create a `setup.py` file using Python's `setuptools` library. This file defines metadata about your package and specifies how it should be installed.



How to Create Package in Python?

- Create a directory named mypackage.
 - Inside mypackage, create two Python files: module1.py and module2.py.
 - Create an `__init__.py` file inside mypackage (it can be empty).
 - Add some code to the modules.
 - Finally, demonstrate how to import and use the modules from the package.

```
mypackage/  
|  
├── __init__.py  
├── module1.py  
└── module2.py
```

`__init__.py`

- The `__init__.py` file is a Python file that is executed when a package is imported.
- `__init__.py` is a special file used in Python to define packages and initialize their namespaces.
- It can contain an initialization code that runs when the package is imported.
- Without this file, Python won't recognize a directory as a package.
- It serves two main purposes:
 - It marks the directory as a Python Package so that the interpreter can find the modules inside it.
 - It can contain initialization code for the Package, such as importing submodules, defining variables, or executing other code.

`__init__.py`

`__all__` variable to specify which modules or names should be imported when using the wildcard (*) import.

```
# Define the __all__ variable
__all__ = ["functionsdemo", "mysqlcruddemo"]

# Import the submodules
# from . import functionsdemo
# from . import mysqlcruddemo
```

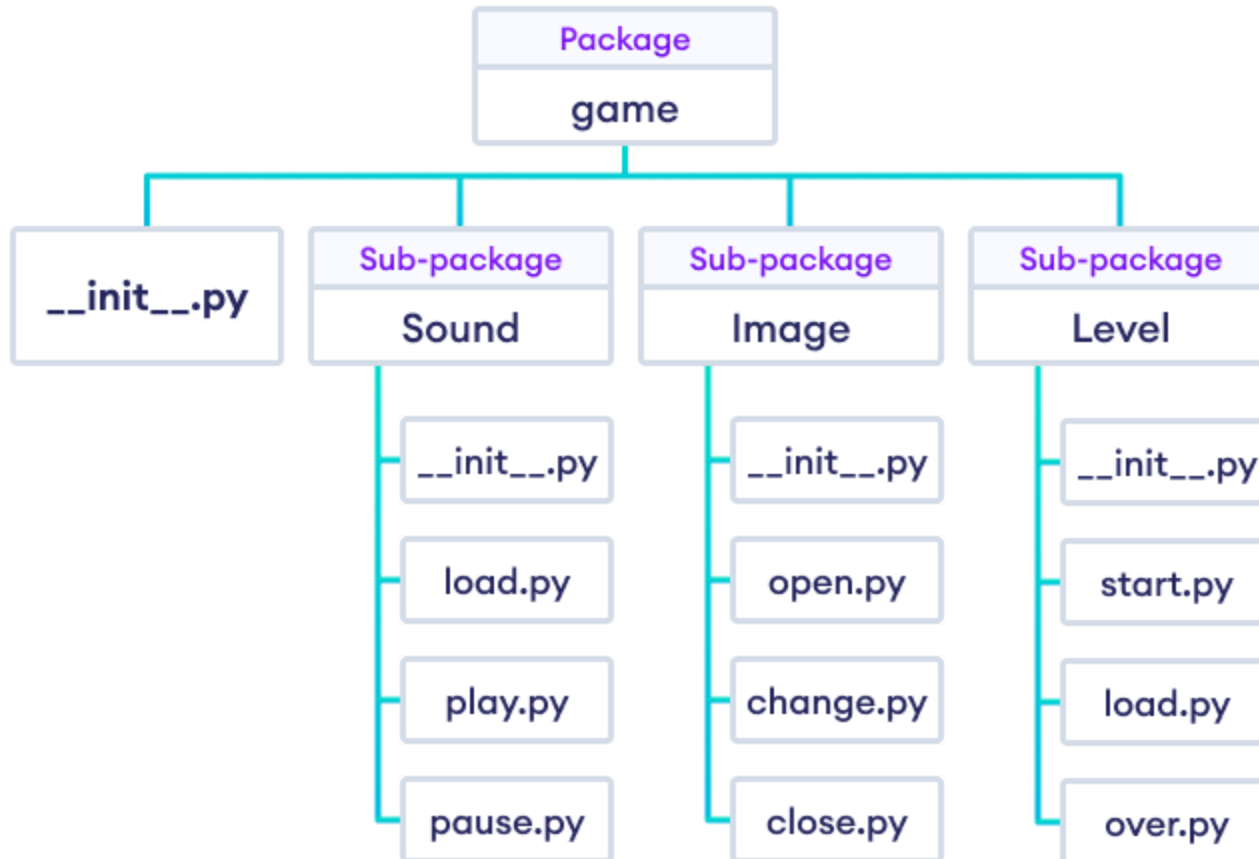
```
@author: Balasubramaniam
"""
# from Day3.functionsdemo import check,fundTransfer

# import Day3.functionsdemo as fd

from ecommerce import functionsdemo as fd

fd.fundTransfer( fromAccountNo: 23864585, toAccountNo: 59756, amount: "5000")
```

How to Create Package in Python?





Python Packages for Web frameworks

- Flask
- FastAPI
- Django
- Pyramid
- Tornado
- Falcon
- Cherrypy
- Bottle
- Web2py



Python Packages for Statistical Analysis

- NumPy
- Pandas
- SciPy
- XGBoost
- StatsModels
- Yellowbrick
- Arch
- Dask-ML



Python Packages for Data Visualization

- Matplotlib
- Seaborn
- Plotly
- Bokeh
- Altair
- Pygal
- Plotnine
- Dash



Python Packages for Deep Learning

- Scikit-learn
- TensorFlow
- torch
- Keras
- Keras-RL
- Lasagne
- Fastai



Python Packages for Natural Processing Language

- NLTK
- spaCy
- FastText
- Transformers
- fastText
- AllenNLP
- TextBlob



Python Packages for Gen AI

- Keras
- spaCy
- generative
- GPy
- Pillow
- ImageIO
- Fastai



Python Packages for Computer Vision

- OpenCV
- TensorFlow
- torch
- scikit-image
- SimpleCV
- ImageAI
- imageio
- Dlib
- Theano
- Mahotas

Package

6. Packages (Folders with `__init__.py`)

Directory structure:

markdown

mypackage/

|

|— `__init__.py`

|— `math_utils.py`

|— `string_utils.py`

Usage:

python

```
from mypackage.math_utils import add
```

Package

- 📦 **1. Why it was needed (important concept)**
- Earlier (before Python 3.3):
- 👉 Without `__init__.py`, Python would **NOT treat a folder as a package**
- Example:
- ```
project/
├── main.py
└── utils/
 ├── helper.py
```
- ❌ This would FAIL:
- `from utils.helper import my_function`

## Package

---

-  Add `__init__.py`:
- `utils/`
  - `└── __init__.py`
  - `└── helper.py`
- Now it works ✓

# Package

---

-  **3. Modern Python (IMPORTANT UPDATE)**
- From Python 3.3+:
- 📌 `__init__.py` is **NOT strictly required** (namespace packages)
- BUT...
- ⚠️ Best practice:  
✓ Always include it for:
- clarity
- compatibility
- tooling (flake8, pytest, IDEs)



# Final Summary

---



| Concept           | Explanation                    |
|-------------------|--------------------------------|
| Purpose           | Marks folder as Python package |
| Required?         | Not always (Python 3.3+)       |
| Best practice     | ✓ Always include               |
| Can contain code? | ✓ Yes                          |
| Common use        | Simplify imports               |



# Python Data Types Deep Dive

|                  |                                                                                       |
|------------------|---------------------------------------------------------------------------------------|
| Python topic     | BMW/automotive discussion context                                                     |
| Lists            | Sensor readings, vehicle inventories, diagnostic events, production-line measurements |
| Tuples           | GPS coordinates, fixed vehicle configuration, RGB/status codes                        |
| Sets             | Unique fault codes, unique vehicle features, supported protocols                      |
| Frozensets       | Immutable feature/configuration combinations, dictionary keys                         |
| Dictionaries     | Vehicle telemetry, VIN-based data, API/JSON responses                                 |
| NamedTuple       | Lightweight immutable sensor/diagnostic records                                       |
| Dataclasses      | Vehicle, Battery, ECU, DiagnosticEvent domain models                                  |
| Comprehensions   | Filtering and transforming telemetry/vehicle data                                     |
| Generators       | Processing large CAN/telemetry/log datasets efficiently                               |
| Decorators       | Logging, timing, authorization, retry, metrics                                        |
| Context managers | Files, database connections, network/resources                                        |
| Lambda           | Sorting/filtering vehicle and production data                                         |



# Python Data Types Deep Dive

---

- **1. Lists — Vehicle and Sensor Data**
- Discuss lists using collections of readings generated by a vehicle.
- `speed_readings = [72, 75, 81, 79, 92, 88]`
- `print(speed_readings[0])`
- `print(speed_readings[-1])`
- `speed_readings.append(95)`
- `average_speed = sum(speed_readings) / len(speed_readings)`
- `print(average_speed)`
- BMW-oriented discussions can include time-series sensor readings, battery temperature measurements, wheel-speed values, production inspection results and diagnostic events.



# Python Data Types Deep Dive

---

- A useful exercise is identifying abnormal temperatures:
- `battery_temperatures = [31.5, 33.2, 34.8, 47.5, 36.1, 51.2]`
- `high_temperature = [`
- `temp for temp in battery_temperatures`
- `if temp > 45`
- `]`
- `print(high_temperature)`
- This naturally introduces **mutability, indexing, slicing, sorting, copying, nested lists and performance.**



# Python Lists

---

- Lists are the most versatile of Python's compound data types.
- A list contains items separated by commas and enclosed within square brackets ([]).
- To some extent, lists are similar to arrays in C.
- One difference between them is that all the items belonging to a list can be of different data type.



# Python Lists

---

- The values stored in a list can be accessed using the slice operator ( `[ ]` and `[ : ]` ) with indexes starting at 0 in the beginning of the list and working their way to end-1.
- The plus ( `+` ) sign is the list concatenation operator, and the asterisk ( `*` ) is the repetition operator.



# Python Lists

---

```
#!/usr/bin/python
```

```
list = ['abcd', 786 , 2.23, 'john', 70.2]
```

```
tinylist = [123, 'john']
```

```
print list # Prints complete list
```

```
print list[0] # Prints first element of the list
```

```
print list[1:3] # Prints elements starting from 2nd till 3rd
```

```
print list[2:] # Prints elements starting from 3rd element
```

```
print tinylist * 2 # Prints list two times
```

```
print list + tinylist # Prints concatenated lists
```



## Updating Lists:

---

```
#!/usr/bin/python
```

```
list = ['physics', 'chemistry', 1997, 2000];
```

```
print "Value available at index 2 : "
```

```
print list[2];
```

```
list[2] = 2001;
```

```
print "New value available at index 2 : "
```

```
print list[2];
```





## Delete List Elements

---

```
#!/usr/bin/python
```

```
list1 = ['physics', 'chemistry', 1997, 2000];
```

```
print list1;
```

```
del list1[2];
```

```
print "After deleting value at index 2 : "
```

```
print list1;
```

# Basic List Operations

---



| Python Expression                         | Results                                   | Description   |
|-------------------------------------------|-------------------------------------------|---------------|
| <code>len([1, 2, 3])</code>               | 3                                         | Length        |
| <code>[1, 2, 3] + [4, 5, 6]</code>        | <code>[1, 2, 3, 4, 5, 6]</code>           | Concatenation |
| <code>['Hi!'] * 4</code>                  | <code>['Hi!', 'Hi!', 'Hi!', 'Hi!']</code> | Repetition    |
| <code>3 in [1, 2, 3]</code>               | True                                      | Membership    |
| <code>for x in [1, 2, 3]: print x,</code> | 1 2 3                                     | Iteration     |

# Indexing, Slicing, and Matrixes:

---



`L = ['spam', 'Spam', 'SPAM!']`

| Python Expression  | Results                        | Description                    |
|--------------------|--------------------------------|--------------------------------|
| <code>L[2]</code>  | <code>'SPAM!'</code>           | Offsets start at zero          |
| <code>L[-2]</code> | <code>'Spam'</code>            | Negative: count from the right |
| <code>L[1:]</code> | <code>['Spam', 'SPAM!']</code> | Slicing fetches sections       |

# Built-in List Functions & Methods

---

| SN | Function with Description                                                      |
|----|--------------------------------------------------------------------------------|
| 1  | <u><a href="#">cmp(list1, list2)</a></u><br>Compares elements of both lists.   |
| 2  | <u><a href="#">len(list)</a></u><br>Gives the total length of the list.        |
| 3  | <u><a href="#">max(list)</a></u><br>Returns item from the list with max value. |
| 4  | <u><a href="#">min(list)</a></u><br>Returns item from the list with min value. |
| 5  | <u><a href="#">list(seq)</a></u><br>Converts a tuple into list.                |

# Built-in List Functions & Methods

| SN | Methods with Description                                                                          |
|----|---------------------------------------------------------------------------------------------------|
| 1  | <u><a href="#">list.append(obj)</a></u><br>Appends object obj to list                             |
| 2  | <u><a href="#">list.count(obj)</a></u><br>Returns count of how many times obj occurs in list      |
| 3  | <u><a href="#">list.extend(seq)</a></u><br>Appends the contents of seq to list                    |
| 4  | <u><a href="#">list.index(obj)</a></u><br>Returns the lowest index in list that obj appears       |
| 5  | <u><a href="#">list.insert(index, obj)</a></u><br>Inserts object obj into list at offset index    |
| 6  | <u><a href="#">list.pop(obj=list[-1])</a></u><br>Removes and returns last object or obj from list |
| 7  | <u><a href="#">list.remove(obj)</a></u><br>Removes object obj from list                           |
| 8  | <u><a href="#">list.reverse()</a></u><br>Reverses objects of list in place                        |
| 9  | <u><a href="#">list.sort([func])</a></u><br>Sorts objects of list, use compare func if given      |

- A tuple is another sequence data type that is similar to the list.
- A tuple consists of a number of values separated by commas. Unlike lists, however, tuples are enclosed within parentheses.
- The main differences between lists and tuples are: Lists are enclosed in brackets ( [ ] ), and their elements and size can be changed, while tuples are enclosed in parentheses ( ( ) ) and cannot be updated.



# Python Tuples

---

```
#!/usr/bin/python
```

```
tuple = ('abcd', 786 , 2.23, 'john', 70.2)
```

```
tinytuple = (123, 'john')
```

```
print tuple # Prints complete list
```

```
print tuple[0] # Prints first element of the list
```

```
print tuple[1:3] # Prints elements starting from 2nd till 3rd
```

```
print tuple[2:] # Prints elements starting from 3rd element
```

```
print tinytuple * 2 # Prints list two times
```

```
print tuple + tinytuple # Prints concatenated lists
```



# Python Data Types Deep Dive

---

- **2. Tuples — Fixed Vehicle Information**
- Tuples are appropriate where a collection represents a fixed structure.
- `vehicle_position = (13.0827, 80.2707)`
- `latitude, longitude = vehicle_position`
- `print(latitude)`
- `print(longitude)`





# Python Data Types Deep Dive

---

- Another automotive example:

```
wheel_positions = (
 "front-left",
 "front-right",
 "rear-left",
 "rear-right"
)
```

- Discussion point: **Why use a tuple instead of a list?**
- It provides an opportunity to discuss immutability, packing/unpacking, hashability and communicating design intent.

# Python Data Types Deep Dive

---

- **3. Sets — Diagnostic Fault Codes**

- A vehicle may report the same diagnostic event repeatedly. A set is useful when only unique values matter.

```
fault_codes = [
 "P0300",
 "P0171",
 "P0300",
 "P0420",
 "P0171"
]
```

- `unique_faults = set(fault_codes)`
- `print(unique_faults)`



# Python Data Types Deep Dive

---

- Compare faults from two vehicles:
  - `vehicle_a = {"P0300", "P0171", "P0420"}`
  - `vehicle_b = {"P0171", "P0500"}`
  - `print(vehicle_a & vehicle_b) # common`
  - `print(vehicle_a | vehicle_b) # all`
  - `print(vehicle_a - vehicle_b) # only A`
- This makes **union**, **intersection**, **difference** and **symmetric difference** much easier to understand.



# Python Data Types Deep Dive

---

- **4. Frozensets — Vehicle Feature Configuration**
- A frozenset is an **immutable version of a set**.
- It is useful when a collection of values represents a configuration that **must not change after creation**.



# Python Data Types Deep Dive

---

- For example, a vehicle may have a fixed package of features:
- `features = frozenset({`
- `"Adaptive Cruise Control",`
- `"Parking Assistance",`
- `"Lane Assistance"`
- `})`
- `print(features)`
- Unlike a normal set, you cannot add or remove features:
- `features.add("Navigation")`
- This produces:
- `AttributeError: 'frozenset' object has no attribute 'add'`



# Python Data Types Deep Dive

---

- **What does "hashable" mean?**
- Python dictionaries use a **hash table** internally.
- When you write:
- `vehicle = {`
- `"model": "BMW iX",`
- `"price": 12000000`
- `}`
- Python calculates a hash value for each dictionary key:
- `print(hash("model"))`
- `print(hash("price"))`

# Python Data Types Deep Dive

---

- Conceptually:

"model"



hash("model")



some integer hash value



dictionary location

- Python uses this hash value to quickly locate the corresponding value.
- Therefore, dictionary keys must be **hashable**.

# Python Data Types Deep Dive

---

- **Why can't a normal set be a dictionary key?**
- Consider:
  - `features = {"Sport", "Navigation"}`
  - `configuration_price = {`
  - `features: 150000`
  - `}`
- Python raises:
  - `TypeError: unhashable type: 'set'`
- The reason is that a set is **mutable**.



# Python Data Types Deep Dive

---

- For example:
- `features = {"Sport", "Navigation"}`
- `features.add("Parking")`
- `features.remove("Sport")`
- The contents can change:
- Before:
- `{"Sport", "Navigation"}`
- After:
- `{"Sport", "Navigation", "Parking"}`



# Python Data Types Deep Dive

---

- If Python allowed this object to be a dictionary key, the key could change **after it had already been stored in the dictionary**.
- That would break the dictionary's hash-table lookup mechanism.
- For this reason:
- `hash({"Sport", "Navigation"})`
- raises:
- `TypeError: unhashable type: 'set'`



# Python Data Types Deep Dive

---

- **Why can frozenset be a dictionary key?**
- A frozenset cannot change after it is created:
- `package = frozenset({`
- `"Sport",`
- `"Navigation"`
- `})`
- `print(hash(package))`
- This works because the contents are fixed, so Python can calculate a stable hash value for the frozenset.



# Python Data Types Deep Dive

---

- Therefore, we can use it as a dictionary key:
- `configuration_price = {`
- `frozenset({"Sport", "Navigation"}): 150000,`
- `frozenset({"Sport", "Navigation", "Parking"}): 220000`
- `}`
- Now suppose a customer selects:
- `selected_features = frozenset({`
- `"Navigation",`
- `"Sport"`
- `})`



# Python Data Types Deep Dive

---

- `price = configuration_price[selected_features]`
- `print(price)`
- Output:
  - 150000
- An important point is that **set order does not matter.**



# Python Data Types Deep Dive

---

- These represent the same configuration:
  - `frozenset({"Sport", "Navigation"})`
  - `frozenset({"Navigation", "Sport"})`
- So:
  - `package1 = frozenset({"Sport", "Navigation"})`
  - `package2 = frozenset({"Navigation", "Sport"})`
  - `print(package1 == package2)`
- Output:
  - `True`
- That makes `frozenset` especially suitable for vehicle feature combinations.
- The customer can select the features in any order and still get the correct configuration price.

# Python Data Types Deep Dive

---

```
configuration_price = {
 frozenset({"Sport", "Navigation"}): 150000,
 frozenset({"Sport", "Navigation", "Parking"}): 220000,
 frozenset({"Comfort", "Parking"}): 175000
}

• customer_selection = ["Parking", "Navigation", "Sport"]
• key = frozenset(customer_selection)
• if key in configuration_price:
• print("Configuration price:", configuration_price[key])
• else:
• print("Configuration not available")
```

# Set vs Frozen Set

---

| Property                             | set   | frozenset |
|--------------------------------------|-------|-----------|
| Mutable                              | ✓ Yes | ✗ No      |
| Add/remove elements                  | ✓ Yes | ✗ No      |
| Unique elements                      | ✓ Yes | ✓ Yes     |
| Order matters                        | ✗ No  | ✗ No      |
| Hashable                             | ✗ No  | ✓ Yes*    |
| Dictionary key                       | ✗ No  | ✓ Yes*    |
| Good for changing feature selections | ✓     | ✗         |
| Good for fixed configuration keys    | ✗     | ✓         |



## Set vs Frozen Set

---

- `# Customer is still selecting features`
- `selected_features = set()`
- `selected_features.add("Sport")`
- `selected_features.add("Navigation")`
- `selected_features.add("Parking")`
- `# Selection is finalized`
- `configuration_key = frozenset(selected_features)`
- `price = configuration_price[configuration_key]`



# Python Dictionary

---

- Python's dictionaries are kind of hash table type.
- They work like associative arrays or hashes found in Perl and consist of key-value pairs.
- A dictionary keys can be almost any Python type, but are usually numbers or strings.
- Values, on the other hand, can be any arbitrary Python object.
- Dictionaries are enclosed by curly braces ( { } ) and values can be assigned and accessed using square braces ( [ ] ).
- Dictionaries have no concept of order among elements. It is incorrect to say that the elements are "out of order"; they are simply unordered.



# Python Dictionary

---

```
#!/usr/bin/python
```

```
dict = {}
```

```
dict['one'] = "This is one"
```

```
dict[2] = "This is two"
```

```
tinydict = {'name': 'john', 'code': 6734, 'dept': 'sales'}
```

```
print dict['one'] # Prints value for 'one' key
```

```
print dict[2] # Prints value for 2 key
```

```
print tinydict # Prints complete dictionary
```

```
print tinydict.keys() # Prints all the keys
```

```
print tinydict.values() # Prints all the values
```

# Built-in Dictionary functions

---

| SN | Function with Description                                                                                                                                  |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <u><a href="#">cmp(dict1, dict2)</a></u><br>Compares elements of both dict.                                                                                |
| 2  | <u><a href="#">len(dict)</a></u><br>Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.                |
| 3  | <u><a href="#">str(dict)</a></u><br>Produces a printable string representation of a dictionary                                                             |
| 4  | <u><a href="#">type(variable)</a></u><br>Returns the type of the passed variable. If passed variable is dictionary then it would return a dictionary type. |

# Built-in Dictionary functions

---

## SN      Methods with Description

- 1      [dict.clear\(\)](#)  
Removes all elements of dictionary *dict*
- 2      [dict.copy\(\)](#)  
Returns a shallow copy of dictionary *dict*
- 2      [dict.fromkeys\(\)](#)  
Create a new dictionary with keys from *seq* and values set to *value*.
- 3      [dict.get\(key, default=None\)](#)  
For *key* key, returns value or default if key not in dictionary
- 4      [dict.has\\_key\(key\)](#)  
Returns *true* if key in dictionary *dict*, *false* otherwise

# Built-in functions

---

- 5      [dict.items\(\)](#)  
Returns a list of *dict*'s (key, value) tuple pairs
- 6      [dict.keys\(\)](#)  
Returns list of dictionary *dict*'s keys
- 7      [dict.setdefault\(key, default=None\)](#)  
Similar to `get()`, but will set `dict[key]=default` if *key* is not already in *dict*
- 8      [dict.update\(dict2\)](#)  
Adds dictionary *dict2*'s key-values pairs to *dict*
- 9      [dict.values\(\)](#)  
Returns list of dictionary *dict2*'s values

## 5. Dictionaries — Vehicle Telemetry

---

- Dictionaries map naturally to JSON/API data.

```
vehicle = {
 "vin": "BMW001",
 "model": "iX",
 "speed": 82,
 "battery": 74,
 "temperature": 35.8
}
```

- • `print(vehicle["vin"])`  
• `print(vehicle["battery"])`

## 5. Dictionaries — Vehicle Telemetry

---

- Nested telemetry:

```
vehicle = {
 "vin": "BMW001",
 "telemetry": {
 "speed": 82,
 "battery": 74,
 "temperature": 35.8
 }
}
```

- `print(vehicle["telemetry"]["battery"])`
- This can lead into discussions about **REST APIs, JSON serialization, telemetry ingestion and defensive access with `get()`.**



## 6. NamedTuple — Sensor Reading

---

- ```
from typing import NamedTuple  
class SensorReading(NamedTuple):  
    sensor_id: str  
    value: float  
    unit: str
```

```
reading = SensorReading(  
    "BATTERY_TEMP",  
    38.5,  
    "C"  
)  
print(reading.sensor_id)  
print(reading.value)
```

7. Dataclasses — Automotive Domain Modelling

- from dataclasses import dataclass

```
@dataclass
```

```
class Vehicle:
```

```
    vin: str
```

```
    model: str
```

```
    battery_level: float
```

```
    speed: float = 0
```

```
vehicle = Vehicle(
```

```
    "BMW001",
```

```
    "BMW iX",
```

```
    82.5
```

```
)
```

```
print(vehicle)
```

7. Dataclasses — Automotive Domain Modelling

- Add validation:
- `from dataclasses import dataclass`
- `@dataclass`
- `class Vehicle:`
- `vin: str`
- `battery_level: float`
- `def __post_init__(self):`
- `if not 0 <= self.battery_level <= 100:`
- `raise ValueError("Invalid battery level")`



List Comprehension

- **8. List Comprehensions — Telemetry Filtering**
- `temperatures = [31, 36, 42, 48, 35, 51]`
- `critical = [`
 - `temp for temp in temperatures`
 - `if temp > 45`
 - `]`
- `print(critical)`

Dictionary Comprehension

• 9. Dictionary Comprehensions — Vehicle Lookup

- vehicles = [
 - ("VIN001", "BMW iX"),
 - ("VIN002", "BMW i4"),
 - ("VIN003", "BMW i5")
-]
- vehicle_lookup = {
 - vin: model
 - for vin, model in vehicles
- }

Dictionary Comprehension

- **9. Dictionary Comprehensions — Vehicle Lookup**
- `battery_levels = { "VIN001": 82, "VIN002": 31, "VIN003": 74, "VIN004": 18 }`
- `low_battery = {
 vin: level
 for vin, level in battery_levels.items()
 if level < 30
}`

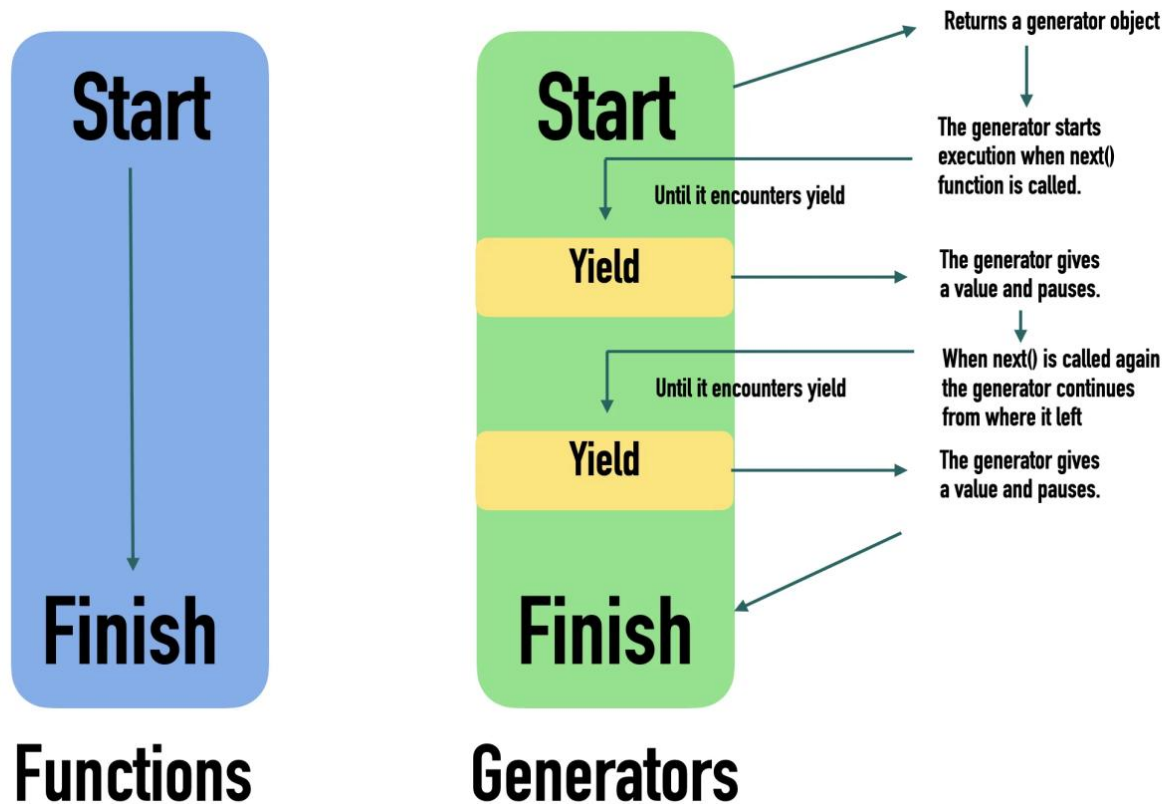
Generators

- Generator is a special routine that can be used to control the iteration behaviour of a loop.
- A generator is like a function returning an array.
- A generator has parameters, it can be called and it generates a sequence of numbers.
- But unlike functions, which return a whole array, a generator yields one value at a time.
- This requires less memory.

Generators

- Generators in Python:
 - Are defined with the def keyword
 - Use the yield keyword
 - May use several yield keywords
 - Return an iterator

Generators





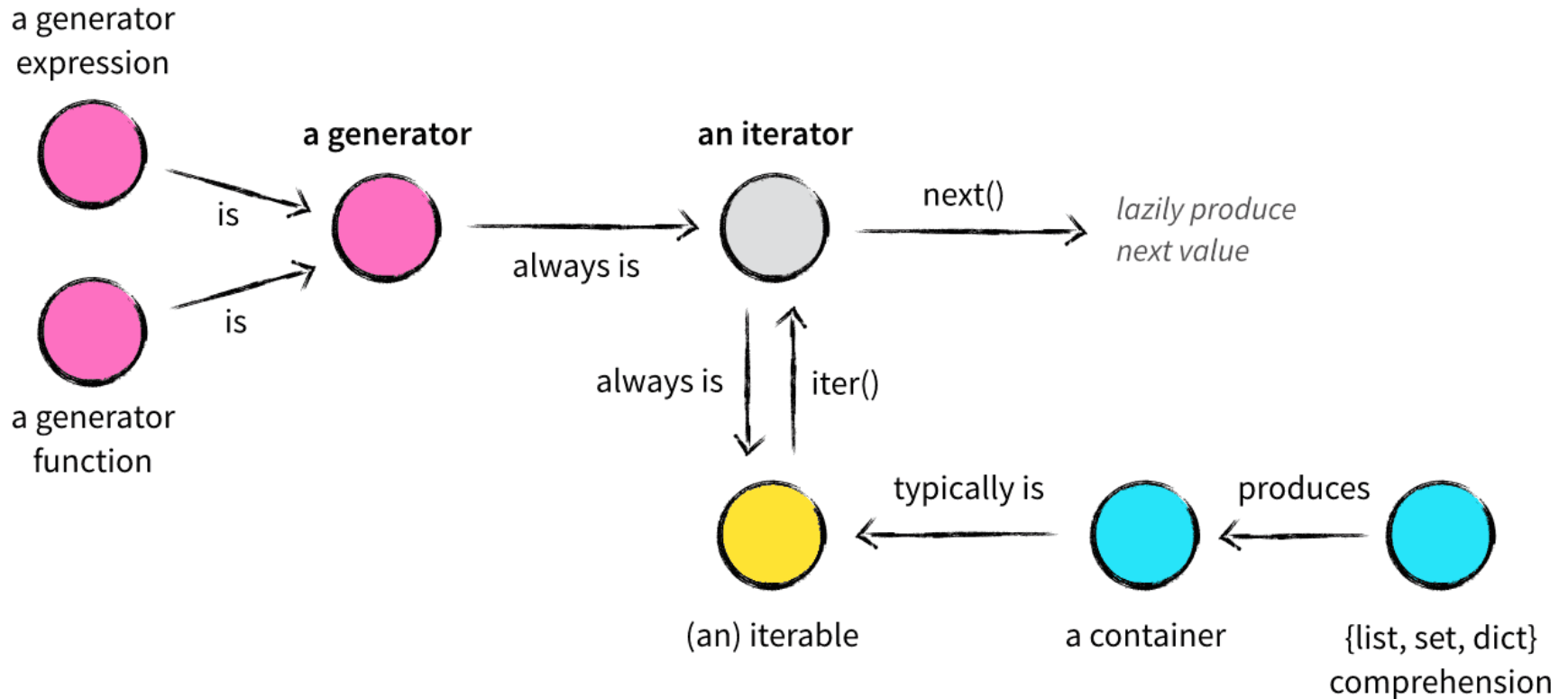
Generators vs Iterators

- **Generator is created using a function** containing the **yield** keyword.
- **Iterator works through iter() and next().**
- A generator uses **yield** to produce values one at a time.
- An iterator does **not require yield**.
- A generator **automatically saves its execution state**, including local variables, whenever **yield** is reached.
- When **next()** is called again, the generator **resumes from where it paused**.
- A generator function can contain **multiple yield statements**.
- A custom iterator can be implemented using a **class** with:
 - `__iter__()`
 - `__next__()`

Generators vs Iterators

- A generator **does not require a class**.
- Generators can be created using:
 - **Generator functions** → `yield`
 - **Generator expressions** → `(x for x in iterable)`
- Generators generally require **less code** than manually implementing an iterator class.
- Generators are **memory efficient** because they generate values **lazily, one at a time**.
- **Every generator is an iterator, but not every iterator is a generator.**
- Both generators and iterators can be consumed using `next()`.

Generators vs Iterators



Generators

- **10. Generators and yield — Large Vehicle Data**
- A **generator** is a special way of producing data **one item at a time, only when the item is requested**, instead of creating and storing all the items in memory first.
- This is especially useful in automotive systems where applications may process **large telemetry files, CAN-bus messages, sensor streams, diagnostic logs, GPS records, or production-line data**.

Generators

- **Start with the problem**
- Imagine a BMW connected vehicle sends millions of telemetry records:
- `def get_vehicle_data():`
 - `readings = []`
 - `for i in range(1_000_000):`
 - `readings.append(f"Telemetry-{i}")`
 - `return readings`
- When we call:
 - `data = get_vehicle_data()`



Generators

- Python creates:
- Telemetry-0
- Telemetry-1
- Telemetry-2
- ...
- Telemetry-999999
- **All 1 million values are created and stored in memory** before the function returns.

Generators

- **2. Generator solution**

Change return to yield:

```
def get_vehicle_data():  
    for i in range(1_000_000):  
        yield f"Telemetry-{i}"
```

- Now:
- `data = get_vehicle_data()`
 - does **not create one million telemetry records**.
- Instead, it creates a **generator object**.
 - `print(data)`
- Conceptually:
 - `<generator object get_vehicle_data ...>`
- The data is produced only when requested.

Generators

- **yield is the key**
- Consider this simple generator:

```
def vehicle_status():  
    yield "Vehicle Started"  
    yield "Vehicle Moving"  
    yield "Vehicle Stopped"
```
- Create the generator:
- `status = vehicle_status()`
- Nothing inside the function has fully executed yet.

Generators

- Now:
 - `print(next(status))`
- Output:
 - Vehicle Started
- Python reaches:
 - `yield "Vehicle Started"`
- and **pauses the function.**
- Call again:
 - `print(next(status))`
- Output:
 - Vehicle Moving
- The important point is that Python **continues from where it previously paused.**

Generators

- **4. yield vs return**

- This is the most important generator concept.

- **Normal function**

- `def get_status():`
 - `return "Vehicle Started"`
- `return` means:
 - Give the result to the caller and **finish the function**.

- **Generator**

```
def get_status():  
    yield "Vehicle Started"  
    yield "Vehicle Moving"  
    yield "Vehicle Stopped"
```

- `yield` means:
 - Give one value to the caller, **pause here**, remember the current state, and continue from here when another value is requested.



Decorators

- **Decorators** allow us to **extend or modify the behavior of a function** without changing the original function's code.
- A decorator **wraps another function** and adds additional functionality before or after it executes.
- Decorators are possible because **functions are first-class objects in Python**.
- A function is an **object** and can be treated like other Python values.



Decorators

- **Properties of First-Class Functions**
- A function can be **assigned to a variable**.
- A function can be **passed as an argument** to another function.
- A function can be **returned from another function**.
- Functions can be **stored in data structures**, such as:
 - Lists
 - Dictionaries
 - Tuples
 - Sets
- A function can be defined **inside another function** (nested function).
- Decorators commonly use the **@decorator_name syntax**.



Decorators

```
def log_action(func):  
    def wrapper():  
        print("Before function execution")  
        func()  
        print("After function execution")  
    return wrapper
```

```
@log_action  
def start_vehicle():  
    print("Vehicle started")
```

```
start_vehicle()
```



Decorators

- Output:
 - Before function execution
 - Vehicle started
 - After function execution

Decorators

- **Important Concept**
- `@log_action`
- `def start_vehicle():`
- `...`
- is essentially equivalent to:
- `start_vehicle = log_action(start_vehicle)`



Decorators

- # BMW example to illustrate functions
- # can be passed as arguments to other functions
- `def sport_mode(model):`
 - `return f"{model} activated SPORT mode - Performance enhanced!"`
- `def eco_mode(model):`
 - `return f"{model} activated ECO PRO mode - Efficiency enhanced!"`
- `def drive_mode(func):`
 - `# Calling the function passed as an argument`
 - `message = func("BMW iX")`
 - `print(message)`
- # Passing functions as arguments
- `drive_mode(sport_mode)`
- `drive_mode(eco_mode)`

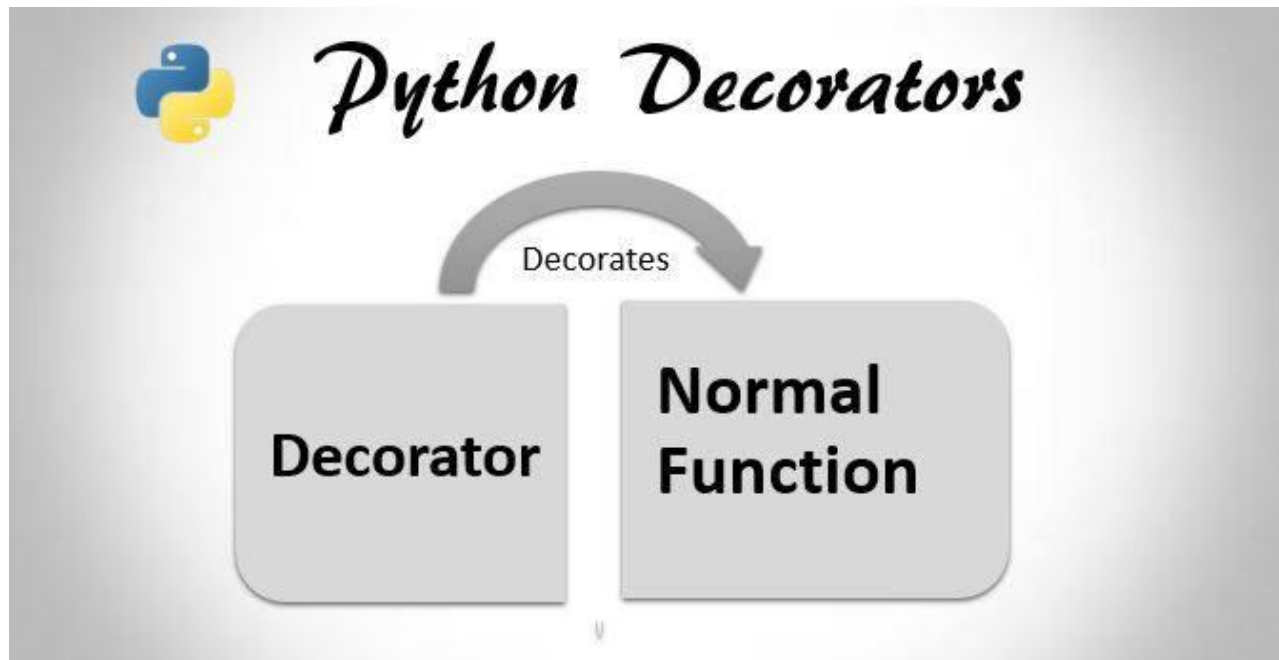
Decorators

- **Output**
- BMW iX activated SPORT mode - Performance enhanced!
- BMW iX activated ECO PRO mode - Efficiency enhanced!

Decorators

we have two different kinds of decorators in Python:

- Function decorators
- Class decorators



Decorators

Function Decorators

- A function decorator is applied to a function definition by placing it on the line before that function definition begins.

```
@myDecorator
```

```
def aFunction():
```

```
    print("inside aFunction")
```

- When the compiler passes over this code, aFunction() is compiled.
- The resulting function object is passed to the myDecorator code, which does something to produce a function-like object that is then substituted for the original aFunction().



Decorators

- The only constraint upon the object returned by the decorator is that it can be used as a function.
- Means it must be callable.
- Thus, any classes we use as decorators must implement `__call__`.

Decorators

```
def hello_decorator(func):
    def inner1(*args, **kwargs):

        print("before Execution")

        # getting the returned value
        returned_value = func(*args, **kwargs)
        print("after Execution")

        # returning the value to the original frame
        return returned_value

    return inner1

# adding decorator to the function
@hello_decorator
def sum_two_numbers(a, b):
    print("Inside the function")
    return a + b

a, b = 1, 2

# getting the value through return of the function
print("Sum =", sum_two_numbers(a, b))
```

Decorators

```
# code for testing decorator chaining
def decor1(func):
    def inner():
        x = func()
        return x * x
    return inner

def decor(func):
    def inner():
        x = func()
        return 2 * x
    return inner

@decor1
@decor
def num():
    return 10

@decor
@decor1
def num2():
    return 10

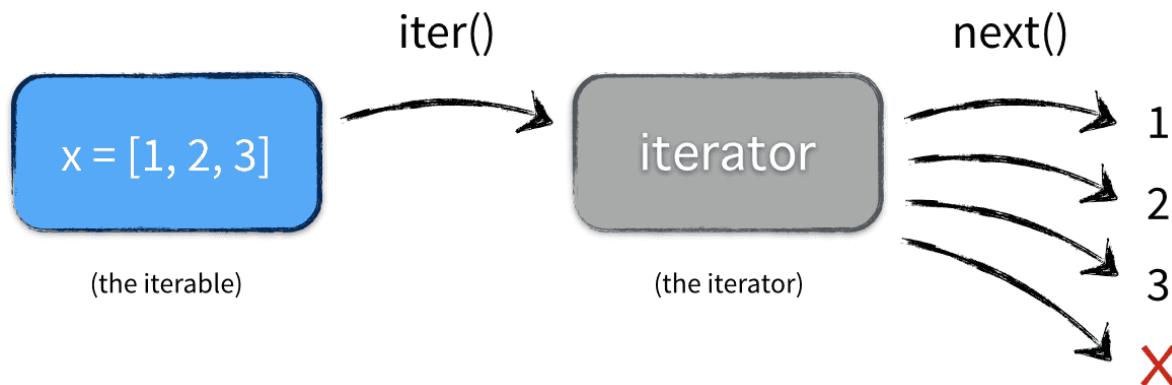
print(num())
print(num2())
```

Iterators

- An iterator in Python is an object that is used to iterate over iterable objects like lists, tuples, dicts, and sets.
- The Python iterators object is initialized using the `iter()` method. It uses the `next()` method for iteration.
- `__iter__()`: The `iter()` method is called for the initialization of an iterator.
- This returns an iterator object
- `__next__()`: The `next` method returns the next value for the iterable.
- When we use a for loop to traverse any iterable object, internally it uses the `iter()` method to get an iterator object, which further uses the `next()` method to iterate over.
- This method raises a `Stop Iteration` to signal the end of the iteration.

Iterators

- Iterators have several advantages:
- Cleaner code
- Iterators can work with infinite sequences
- Iterators save resources
- Python has several built-in objects, which implement the iterator protocol. For example lists, tuples, strings, dictionaries or files



Python iter() Example

Python3



```
string = "GFG"
```



```
ch_iterator = iter(string)
```



```
print(next(ch_iterator))
```

```
print(next(ch_iterator))
```



```
print(next(ch_iterator))
```

Context Managers with with Statement

- A **context manager** manages resources automatically.
- It handles **setup before an operation** and **cleanup after the operation**, even when an exception occurs.
- **BMW Example – Vehicle Log File**
- with open("bmw_vehicle_log.txt", "w") as log_file:
- log_file.write("BMW iX - Vehicle diagnostics started")

Context Managers with with Statement

- Without with:
- `log_file = open("bmw_vehicle_log.txt", "w")`
- try:
- `log_file.write("BMW iX - Vehicle diagnostics started")`
- finally:
- `log_file.close()`
- The with statement automatically closes the file.

Context Managers with with Statement

- Context managers are used for **automatic resource management**.
- The with statement makes code cleaner and safer.
- Resources are automatically released after the block finishes.
- Cleanup happens **even if an exception occurs**.
- Common uses include:
 - File handling
 - Database connections
 - Network connections
 - Locks
 - Transactions
- Context managers are based mainly on:
 - `__enter__()` → setup/acquire resource
 - `__exit__()` → cleanup/release resource

BMW Context Manager

- `class BMWDiagnosticSession:`
- `def __enter__(self):`
- `print("Connecting to BMW diagnostic system...")`
- `return self`
- `def run_diagnostics(self):`
- `print("Running BMW vehicle diagnostics...")`
- `def __exit__(self, exc_type, exc_value, traceback):`
- `print("Disconnecting from BMW diagnostic system...")`
- `with BMWDiagnosticSession() as session:`
- `session.run_diagnostics()`

BMW Context Manager

- Output:
 - Connecting to BMW diagnostic system...
 - Running BMW vehicle diagnostics...
 - Disconnecting from BMW diagnostic system...
- Think of it as:
- **Acquire → Use → Automatically Release**



Lambda Functions

- A **lambda function** is a small **anonymous function** written without using the `def` keyword.
- Normal function:
- `def calculate_tax(price):`
- `return price * 0.18`
- Equivalent lambda:
 - `calculate_tax = lambda price: price * 0.18`

Syntax

Python

```
lambda arguments: expression
```


Lambda Functions

- **BMW Example – Vehicle Prices**

- vehicles = [
 - ("BMW iX", 12000000),
 - ("BMW X1", 5000000),
 - ("BMW X5", 9500000),
 - ("BMW i4", 7500000)
-]
- Suppose we want to sort vehicles based on **price**:
- vehicles.sort(key=lambda vehicle: vehicle[1])
- print(vehicles)



Lambda Functions

- means:
- Take each vehicle and return its price.
- It is similar to:
- `def get_price(vehicle):`
 - `return vehicle[1]`
- `vehicles.sort(key=get_price)`



Lambda Functions

- **Lambda with filter()**
- Find BMW vehicles costing more than ₹70 lakh:

```
expensive_cars = filter(  
    lambda vehicle: vehicle[1] > 7000000,  
    vehicles  
)
```
- ```
print(list(expensive_cars))
```

# Lambda Functions

---

- **Lambda with map()**
- Apply a 5% price increase:
- `new_prices = map(`
- `lambda vehicle: (vehicle[0], vehicle[1] * 1.05),`
- `vehicles`
- `)`
- `print(list(new_prices))`

# Lambda Functions

---

- Lambda is an **anonymous function**.
- Defined using the **lambda keyword**.
- Can take **any number of arguments**.
- Contains a **single expression**.
- Automatically returns the result of that expression.
- Useful for **short, temporary operations**.
- Frequently used with:
  - `sorted()`
  - `map()`
  - `filter()`
- Avoid lambda when the logic becomes complicated; use a normal `def` function instead.



# What is Error Handling?

---

- Error handling allows you to manage unexpected events or exceptions that occur during the execution of your program — without crashing it.
- Exception handling allows a program to **detect runtime errors, respond appropriately, and continue or terminate gracefully** instead of crashing unexpectedly.

# What is Error Handling?

---

- The main concepts are:

try / except / else / finally



Custom Exceptions



Exception Chaining

# What is Error Handling?

---

- **1. try and except**

- Put code that may fail inside try. Handle the expected error in except.

- BMW example — processing vehicle sensor data:

```
def calculate_efficiency(distance, energy_used):
```

```
 try:
```

```
 efficiency = distance / energy_used
```

```
 print(f"Efficiency: {efficiency:.2f} km/kWh")
```

```
 except ZeroDivisionError:
```

```
 print("Energy consumption cannot be zero.")
```

- `calculate_efficiency(420, 60)`
- `calculate_efficiency(420, 0)`



# What is Error Handling?

---

- **2. Handling Multiple Exceptions**

- Different problems should normally have different handlers.

- `def process_battery_data(capacity, consumption):`

- `try:`

- `capacity = float(capacity)`

- `consumption = float(consumption)`

- `range_km = capacity / consumption`

- `print(f"Estimated range: {range_km:.2f} km")`

- `except ValueError:`

- `print("Battery data must contain numeric values.")`

- `except ZeroDivisionError:`

- `print("Consumption cannot be zero.")`



# What is Error Handling?

---

- **3. else**

- The else block executes **only when no exception occurs in the try block.**

try:

```
battery_percentage = int(input("Enter battery percentage: "))
```

except ValueError:

```
 print("Please enter a valid number.")
```

else:

```
 print(f"Battery level recorded: {battery_percentage}%")
```

- Think of it as:

try succeeds



else

- Use else for code that should run only after the risky operation succeeds.

# What is Error Handling?

---

- **4. finally**
- finally executes **whether an exception occurs or not.**
- This is particularly useful for cleanup operations such as closing files, database connections, network connections, or hardware resources.
- try:
  - `print("Connecting to BMW vehicle...")`
  - `value = 100 / 0`
- except `ZeroDivisionError`:
  - `print("Invalid sensor calculation.")`
- finally:
  - `print("Closing vehicle connection.")`



# What is Error Handling?

---

- **5. Complete try/except/else/finally Example**

- Suppose a BMW service application reads battery information from a vehicle.

```
def check_battery(capacity, used_energy):
 try:
 capacity = float(capacity)
 used_energy = float(used_energy)
 remaining = capacity - used_energy
 except ValueError:
 print("Invalid battery data.")
 else:
 print(f"Remaining battery: {remaining:.2f} kWh")
 finally:
 print("Battery diagnostic completed.")
```

# What is Error Handling?

---

- **6. Raising Exceptions with raise**

- Sometimes Python has not encountered a technical error, but **our business rules say the value is invalid**.
- Use raise to explicitly generate an exception.

```
def set_battery_level(level):
 if level < 0 or level > 100:
 raise ValueError("Battery level must be between 0 and
100.")
 print(f"Battery level: {level}%")
set_battery_level(120)
```

- Result:
- `ValueError: Battery level must be between 0 and 100.`



# What is Error Handling?

---

- **7. Custom Exceptions**
- Built-in exceptions such as `ValueError` and `TypeError` are not always descriptive enough for domain-specific problems.
- You can create your own exception by inheriting from `Exception`.
- `class BatteryLevelError(Exception):`
- `pass`

# What is Error Handling?

---

- Now use it:

```
def set_battery_level(level):
 if level < 0 or level > 100:
 raise BatteryLevelError(
 f"Invalid battery level: {level}%"
)
 print(f"Battery level: {level}%")
```

- Handle it:

```
try:
 set_battery_level(125)
except BatteryLevelError as e:
 print(f"Battery Error: {e}")
```

- Output:

- Battery Error: Invalid battery level: 125%

# What is Error Handling?

---

- **8. Exception Chaining**

- Exception chaining is useful when a **low-level technical exception causes a higher-level application/domain exception**.
- For example, suppose raw battery data cannot be converted into a number.
- **Without chaining:**

```
class BatteryDataError(Exception):
```

```
 pass
```

```
def read_battery(value):
```

```
 try:
```

```
 return float(value)
```

```
 except ValueError:
```

```
 raise BatteryDataError("Unable to process battery data.")
```





# What is Error Handling?

---

```
class BatteryDataError(Exception):
 pass

def read_battery(value):
 try:
 return float(value)
 except ValueError as e:
 raise BatteryDataError(
 "Unable to process battery data."
) from e
```



# What is Error Handling?

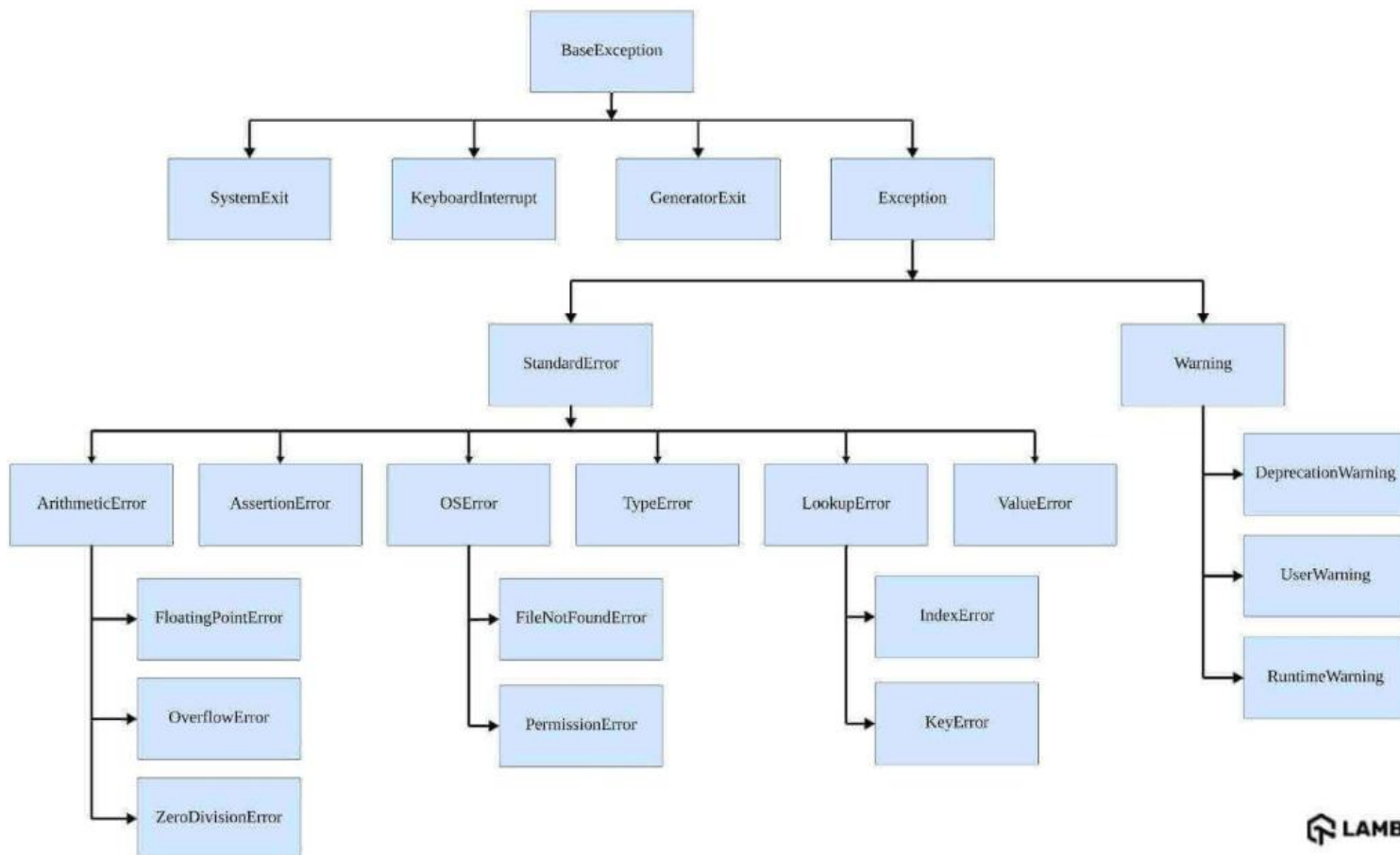
---

```
class VehicleDataError(Exception):
 pass

def get_vehicle_speed(raw_speed):
 try:
 speed = int(raw_speed)
 return speed
 except ValueError as e:
 raise VehicleDataError(
 f"Invalid vehicle speed data: {raw_speed}"
) from e
```

# What is Error Handling?

## Python Exception Hierarchy



# What is Error Handling?

---

## 2. Common Error Types

| Error Type                     | Meaning                         |
|--------------------------------|---------------------------------|
| <code>SyntaxError</code>       | Code is not written correctly   |
| <code>NameError</code>         | Variable not defined            |
| <code>TypeError</code>         | Invalid operation between types |
| <code>ValueError</code>        | Wrong value type                |
| <code>IndexError</code>        | Index out of range              |
| <code>KeyError</code>          | Missing key in dictionary       |
| <code>ZeroDivisionError</code> | Division by zero                |
| <code>FileNotFoundError</code> | File doesn't exist              |

# Exceptions



Microservices

| Class Name     | Description                                                        |
|----------------|--------------------------------------------------------------------|
| Exception      | The base class for all exceptions                                  |
| AttributeError | Raised when attribute reference or assignment fails                |
| IOError        | Raised when trying to open a nonexistent file (among other things) |
| IndexError     | Raised when using a nonexistent index on a sequence                |
| KeyError       | Raised when using a nonexistent key on a mapping                   |
| NameError      | Raised when a name (variable) is not found                         |

# Exceptions



Microservices

| Class Name        | Description                                                                                                                 |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------|
| SyntaxError       | Raised when the code is ill-formed                                                                                          |
| TypeError         | Raised when a built-in operation or function is applied to an object of the wrong type                                      |
| ValueError        | Raised when a built-in operation or function is applied to an object with the correct type, but with an inappropriate value |
| ZeroDivisionError | Raised when the second argument of a division or modulo operation is zero                                                   |

# What is Error Handling?

## 3. Basic try-except

python

```
try:
 x = 5 / 0
except ZeroDivisionError:
 print("You can't divide by zero!")
```

## 4. Multiple Exceptions

python

```
try:
 val = int("abc")
except ValueError:
 print("Invalid number")
except Exception as e:
 print("Something went wrong:", e)
```

# What is Error Handling?

---

## 5. try-except-else-finally

python

```
try:
 f = open("data.txt", "r")
except FileNotFoundError:
 print("File not found")
else:
 print("File read successfully")
 f.close()
finally:
 print("Execution completed")
```



# What is Error Handling?

---

## 6. Raising Custom Errors

python

```
def set_age(age):
 if age < 0:
 raise ValueError("Age cannot be negative")
 print("Age is valid")

set_age(-5)
```

# What is Error Handling?

---

## 7. Real-World Example: Login Check

```
python

def login(username):
 if username != "admin":
 raise PermissionError("Access Denied")
 print("Welcome!")

try:
 login("user")
except PermissionError as pe:
 print(pe)
```

# What is Error Handling?

---

## ✓ 8. Best Practices

- Catch only necessary exceptions
- Use specific exceptions ( `except ValueError` ) instead of general ones ( `except:` )
- Always close resources ( `finally` or `with open` )
- Avoid silent failures (don't use `except: pass` )

# What is Error Handling?

---

## 9. Debugging Tip

Use the `traceback` module to log the full error stack:

```
python

import traceback

try:
 1 / 0
except Exception:
 traceback.print_exc()
```

## 1. Opening a File

python

```
file = open("example.txt", "r") # "r" = read mode
```

### Common Modes:

| Mode | Description              |
|------|--------------------------|
| "r"  | Read (default)           |
| "w"  | Write (overwrite)        |
| "a"  | Append                   |
| "x"  | Create (fail if exists)  |
| "b"  | Binary mode (e.g., "rb") |
| "+"  | Read and write           |

## File I/O

---

- **2. Reading Text Files**
- Suppose vehicles.txt contains:
  - BMW iX
  - BMW i7
  - BMW X5
  - BMW X3
- **read()**
- Reads the entire file.
- with open("vehicles.txt", "r") as file:
- content = file.read()
- print(content)

## File I/O

---

- **readline()**
- Reads one line.
- with `open("vehicles.txt", "r")` as file:
  - `first_line = file.readline()`
  - `print(first_line)`
- **readlines()**
- Returns the lines as a list.
- with `open("vehicles.txt", "r")` as file:
  - `vehicles = file.readlines()`
  - `print(vehicles)`



- Result:
- [
  - "BMW iX\n",
  - "BMW i7\n",
  - "BMW X5\n",
  - "BMW X3\n"
- ]





## File I/O

---

- A memory-efficient approach for large files is iteration:
- with `open("vehicles.txt", "r")` as file:
  - for line in file:
  - `print(line.strip())`

## File I/O

---

- **3. Writing Text Files**
- Use "w" mode.
- with `open("vehicles.txt", "w")` as file:
  - `file.write("BMW iX\n")`
  - `file.write("BMW i7\n")`
  - `file.write("BMW X5\n")`
- Be careful: "w" **replaces existing content**.

## File I/O

---

- Existing file
- BMW iX
- BMW i7
- BMW X5
- ↓ append
- BMW iX
- BMW i7
- BMW X5
- BMW X3

## File I/O

---

- **5. Encoding**
- For text files, explicitly specifying the encoding is a good practice.
- with `open("vehicles.txt", "r", encoding="utf-8")` as file:
  - `content = file.read()`
- And when writing:
- with `open("vehicles.txt", "w", encoding="utf-8")` as file:
  - `file.write("BMW iX")`

# File I/O

---

- **6. Binary Files**
- Binary mode is used for files such as:
  - Images
  - PDFs
  - Audio
  - Videos
  - ZIP files
- Use "rb" for reading binary files.
- with `open("bmw.png", "rb")` as file:
- `data = file.read()`
- `print(type(data))`
- Result:
- `<class 'bytes'>`

## File I/O

---

- **6. Binary Files**
- Use "wb" to write binary data.
- with open("bmw.png", "rb") as source:
  - data = source.read()
- with open("bmw\_copy.png", "wb") as destination:
  - destination.write(data)

## File I/O

---

- **7. CSV Files**
- CSV stands for **Comma-Separated Values**.
- Example vehicles.csv:  
vin,model,battery\_kwh  
WBA101,BMW iX,111.5  
WBA102,BMW i4,83.9  
WBA103,BMW i7,105.7
- Python provides the built-in csv module.
- `import csv`

## File I/O

---

- **7. CSV Files**
- **Reading with `csv.reader()`**

```
import csv
```

```
with open("vehicles.csv", "r", newline="", encoding="utf-8")
as file:
```

```
 reader = csv.reader(file)
```

```
 for row in reader:
```

```
 print(row)
```

- Output:
- ['vin', 'model', 'battery\_kwh']
- ['WBA101', 'BMW iX', '111.5']
- ['WBA102', 'BMW i4', '83.9']



## File I/O

---

- **8. csv.DictReader**

- For business applications, DictReader is often easier because column names become dictionary keys.
- `import csv`
- `with open("vehicles.csv", "r", newline="", encoding="utf-8") as file:`
- `reader = csv.DictReader(file)`
- `for vehicle in reader:`
- `print(vehicle["model"], vehicle["battery_kwh"])`
- Output:
- BMW iX 111.5
- BMW i4 83.9
- BMW i7 105.7

## File I/O

---

- **9. Writing CSV Files**

- Using `csv.writer()`:

- `import csv`

- `vehicles = [`

- `["WBA101", "BMW iX", 111.5],`

- `["WBA102", "BMW i4", 83.9],`

- `["WBA103", "BMW i7", 105.7]`

- `]`

`with open("vehicles.csv", "w", newline="", encoding="utf-8") as file:`

`writer = csv.writer(file)`

`writer.writerow(["vin", "model", "battery_kwh"])`

`writer.writerows(vehicles)`

## File I/O

---

- **10. csv.DictWriter**

- This is convenient when your data is already represented by dictionaries.

```
import csv
```

```
vehicles = [
```

```
 {"vin": "WBA101", "model": "BMW iX", "battery": 111.5},
```

```
 {"vin": "WBA102", "model": "BMW i4", "battery": 83.9}
```

```
]
```

```
with open("vehicles.csv", "w", newline="", encoding="utf-8") as file:
```

```
 fields = ["vin", "model", "battery"]
```

```
 writer = csv.DictWriter(file, fieldnames=fields)
```

```
 writer.writeheader()
```

```
 writer.writerows(vehicles)
```

## File I/O

---

- **11. pathlib**
- Modern Python code often uses pathlib instead of manually manipulating file paths.
- `from pathlib import Path`
- `file_path = Path("vehicles.txt")`
- Check whether it exists:
- `if file_path.exists():`
- `print("File exists")`

## File I/O

---

- Check whether it is a file:
- `print(file_path.is_file())`
- Check whether something is a directory:
- `folder = Path("data")`
- `print(folder.is_dir())`

## File I/O

---

- **Creating Directories**
- `from pathlib import Path`
- `folder = Path("data")`
- `folder.mkdir(exist_ok=True)`
- For nested directories:
- `folder = Path("data/vehicles/2026")`
- `folder.mkdir(parents=True, exist_ok=True)`

## File I/O

---

- **12. Reading and Writing with pathlib**
- **Instead of:**
- **with open("vehicles.txt", "r") as file:**
- **content = file.read()**
- you can use:
- from pathlib import Path
- path = Path("vehicles.txt")
- content = path.read\_text(encoding="utf-8")
- print(content)

## File I/O

---

- **12. Reading and Writing with pathlib**
- Writing:
- `path.write_text(`
- `"BMW iX\nBMW i7\nBMW X5",`
- `encoding="utf-8"`
- `)`
- Binary data:
- `data = Path("bmw.png").read_bytes()`
- and:
- `Path("bmw_copy.png").write_bytes(data)`



## File I/O

---

- **13. Working with Paths**

- pathlib makes building paths cleaner.

```
from pathlib import Path
folder = Path("data")
file = folder / "vehicles.csv"
print(file)
```

- Windows:
- data\vehicles.csv
- Linux/macOS:
- data/vehicles.csv
- This makes the program more portable across operating systems.

## File I/O

---

- **14. JSON**
- JSON stands for **JavaScript Object Notation**.
- Example:
- {
  - "vin": "WBA101",
  - "model": "BMW iX",
  - "battery": 111.5,
  - "electric": true
- }
- Python provides the built-in:
- `import json`

# File I/O

---



| Function                  | Conversion                  |
|---------------------------|-----------------------------|
| <code>json.loads()</code> | JSON string → Python object |
| <code>json.dumps()</code> | Python object → JSON string |
| <code>json.load()</code>  | JSON file → Python object   |
| <code>json.dump()</code>  | Python object → JSON file   |

## File I/O

---

- **15. json.loads()**
- Suppose an API returns:

```
json_data = ""
{
 "vin": "WBA101",
 "model": "BMW iX",
 "battery": 111.5
}
• ""
```

## File I/O

---

- Convert it into a Python dictionary:

```
import json
vehicle = json.loads(json_data)
print(vehicle)
print(type(vehicle))
```

## File I/O

---

- Output:
- `{'vin': 'WBA101', 'model': 'BMW iX', 'battery': 111.5}`
- `<class 'dict'>`
- Now access individual values:
- `print(vehicle["model"])`
- `print(vehicle["battery"])`

## File I/O

---

- **16. json.dumps()**
- The reverse operation converts Python data into a JSON string.
- `import json`
- `vehicle = {`
- `"vin": "WBA101",`
- `"model": "BMW iX",`
- `"battery": 111.5`
- `}`
- `json_string = json.dumps(vehicle)`
- `print(json_string)`
- Output:
- `{"vin": "WBA101", "model": "BMW iX", "battery": 111.5}`

## File I/O

---

- **17. Nested JSON**

- Real API responses usually contain nested objects and arrays.

- Example:

- `json_data = "`

- `{`

- `"vin": "WBA101",`

- `"model": "BMW iX",`

- `"battery": {`

- `"capacity": 111.5,`

- `"level": 82,`

- `"temperature": 31`

- `},`





- **17. Nested JSON**
- "features": [
  - "Navigation",
  - "Parking Assistant",
  - "Adaptive Cruise Control"
- ]
- }
- ""

## File I/O

---

- Parse it:
- `import json`
- `vehicle = json.loads(json_data)`
- Access the model:
- `print(vehicle["model"])`
- Output:
  - BMW iX

## File I/O

---

- Parse it:
- `import json`
- `vehicle = json.loads(json_data)`
- Access the model:
- `print(vehicle["model"])`
- Output:
  - BMW iX

## File I/O

---

- **Reading a JSON File**
- Suppose vehicle.json contains JSON data.
- Use `json.load()`:
- `import json`
- `with open("vehicle.json", "r", encoding="utf-8") as file:`
- `vehicle = json.load(file)`
- `print(vehicle["model"])`
- Notice the difference:
- `json.loads(json_string)`
- `json.load(file)`

## File I/O

---

- **Writing JSON to a File**
- Use `json.dump()`:
- `import json`
- `vehicle = {`
- `"vin": "WBA101",`
- `"model": "BMW iX",`
- `"battery": 111.5`
- `}`
- `with open("vehicle.json", "w", encoding="utf-8") as file:`
- `json.dump(vehicle, file, indent=4)`

# File I/O

---

- **Handling Invalid JSON**

- JSON parsing can fail.

- `import json`

- 

- `data = '{"model": "BMW iX"},{'`

- 

- `try:`

- `vehicle = json.loads(data)`

- 

- `except json.JSONDecodeError as e:`

- `print("Invalid JSON:", e)`

- This connects directly with Python exception handling.



# Date abbreviation



## `strftime()` Date Abbreviations (Format Codes)

| Format Code     | Meaning                        | Example Output |
|-----------------|--------------------------------|----------------|
| <code>%Y</code> | Year (4 digits)                | 2025           |
| <code>%y</code> | Year (2 digits)                | 25             |
| <code>%B</code> | Full month name                | May            |
| <code>%b</code> | Abbreviated month name         | May            |
| <code>%m</code> | Month as number (01–12)        | 05             |
| <code>%d</code> | Day of the month (01–31)       | 28             |
| <code>%A</code> | Full weekday name              | Wednesday      |
| <code>%a</code> | Abbreviated weekday name       | Wed            |
| <code>%w</code> | Weekday (0=Sunday, 6=Saturday) | 3              |
| <code>%H</code> | Hour (24-hour clock, 00–23)    | 16             |
| <code>%I</code> | Hour (12-hour clock, 01–12)    | 04             |

# Date abbreviation



Microservices

|    |                             |                          |
|----|-----------------------------|--------------------------|
| %p | AM/PM                       | PM                       |
| %M | Minute (00–59)              | 30                       |
| %S | Second (00–59)              | 45                       |
| %f | Microsecond (000000–999999) | 123456                   |
| %z | UTC offset                  | +0530                    |
| %Z | Time zone name              | IST                      |
| %j | Day of the year (001–366)   | 149                      |
| %U | Week number (Sunday start)  | 21                       |
| %W | Week number (Monday start)  | 22                       |
| %c | Local date and time         | Wed May 28 16:30:45 2025 |
| %x | Local date                  | 05/28/25                 |
| %X | Local time                  | 16:30:45                 |



# Date abbreviation

---



## Example

```
python
```

```
from datetime import datetime
```

```
now = datetime.now()
```

```
print(now.strftime("%A, %d %B %Y %I:%M:%S %p"))
```

```
Output: Wednesday, 28 May 2025 04:30:45 PM
```



# What is a Virtual Environment?

---

- A virtual environment is an isolated environment for Python projects that allows you to:
  - Install specific dependencies without affecting the global Python setup.
  - Work on multiple projects with different package versions.



## 2. Why Use Virtual Environments?

---

- Avoid version conflicts between projects
- Keep your global Python clean
- Manage project-specific dependencies

## 2. Virtual Environments

---

### 3. Creating a Virtual Environment

```
bash
```

```
python -m venv myenv
```

Creates a folder `myenv/` with a self-contained Python installation.

## 3. Activating Environments

---

- **Windows:**

```
bash

myenv\Scripts\activate
```

- **Linux/Mac:**

```
bash

source myenv/bin/activate
```

After activation, your terminal shows `(myenv)` prefix.



## 3. Deactivating the Environment

---

```
deactivate
```

## 3. Packages in Environment

---



### 6. Installing Packages in the Environment

```
bash
```

```
pip install requests
```

Packages are installed **inside** `myenv/`, not globally.



### 7. Freezing Requirements

```
bash
```

```
pip freeze > requirements.txt
```

Generates a list of installed packages. Useful for sharing.

## 3. Packages in Environment

---

### 8. Installing from requirements.txt

```
bash

pip install -r requirements.txt
```

Perfect for **cloning environments** in another system or during CI/CD.

### 9. Real-World Example: Data Project Setup

```
bash

python -m venv dataenv
source dataenv/bin/activate
pip install pandas matplotlib scikit-learn
pip freeze > requirements.txt
```



## 3. Packages in Environment

---

### ✓ 10. **Best Practices**

- Use one virtual environment per project
- Name your env folder clearly ( `.venv` , `env` , `myproject_env` )
- Add `env/` to `.gitignore` in Git projects
- Always activate env before running or testing

## 3. Packages in Environment

---

### 11. Tools for Managing Environments

| Tool                    | Description                                    |
|-------------------------|------------------------------------------------|
| <code>venv</code>       | Standard library (recommended)                 |
| <code>virtualenv</code> | Older, more features                           |
| <code>pipenv</code>     | Combines pip + virtualenv + lockfile           |
| <code>conda</code>      | Package and environment manager (Data Science) |

# OOPS

---



( Object - Oriented  
Programming System)



# What is OOP?

A way of organizing code around objects, not just functions.

1

## Object

A real-world entity in code: it has data (attributes) and behavior (methods).

2

## Class

The blueprint that describes what its objects will look like.

3

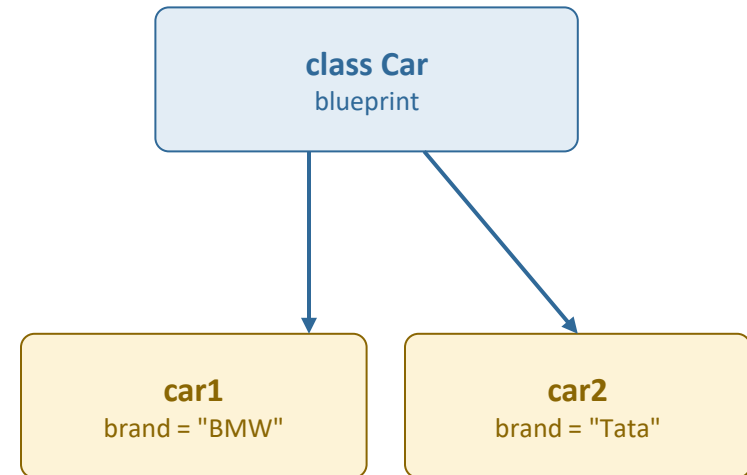
## Why OOP?

Reusable, organized, easy-to-extend code — model things as they exist.

4

## Everyday analogy

“Car” is a class; your BMW and your friend's Tata are objects.



*one blueprint → many objects*

# The four pillars of OOP

---

## Encapsulation

Bundle data + methods together; hide internal details (e.g. `_price` with `@property`).

## Abstraction

Expose a simple interface, hide the complex machinery (you call `car.drive()`, not the engine).

## Inheritance

A child class reuses and extends a parent class (Dog inherits from Animal).

## Polymorphism

The same method name behaves differently per class (`dog.speak()` vs `cat.speak()`).

*We will see all four in action in the next slides.*



## OOPs CONCEPTS IN BANKING

Using Object-Oriented Programming to Build Banking Systems

### 1 CLASS

A Class is a blueprint for creating banking objects.

Class: Account

- account\_number: String
- holder\_name: String
- balance: double
- + deposit(amount)
- + withdraw(amount)
- + get\_balance()

Defines what data and operations an Account will have.



### 2 OBJECT

An Object is an instance of a Class.  
Each customer has their own Account object.

acc1: Account

12345  
Rohan Sharma  
Balance: ₹ 50,000

acc2: Account

67890  
Priya Singh  
Balance: ₹ 75,000

### 3 ENCAPSULATION

Hiding sensitive data and allowing access through methods.

```
class Account:
 def __init__(self, acc_no, bal):
 self.__balance = bal # private
 def deposit(self, amount):
 if amount > 0:
 self.__balance += amount
 def get_balance(self):
 return self.__balance
```



Balance is private.  
Use methods to access and modify.

### 4 ABSTRACTION

Showing only essential features and hiding complex implementation.

```
from abc import ABC, abstractmethod
```

```
class Loan(ABC):
 @abstractmethod
 def calculate_interest(self):
 pass
```

Loan defines what must be done, subclasses provide how it is done.

### 5 INHERITANCE

Reusing code. SavingsAccount and CurrentAccount inherit from Account.



### 6 POLYMORPHISM

Same method name behaves differently.

```
def process_transaction(acc):
 acc.deposit(1000) # Works for both SavingsAccount and CurrentAccount
```

SavingsAccount

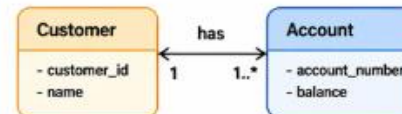
deposit()  
Calculates interest and then deposits.

CurrentAccount

deposit()  
Allows deposit with overdraft check.

### 7 ASSOCIATION

A relationship where one object uses another.

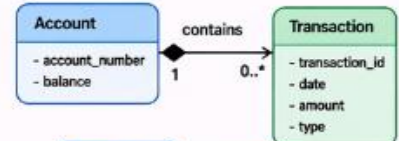


A Customer can have one or more Accounts.



### 8 COMPOSITION

A strong relationship where Account depends on Transaction. If Account is deleted, its Transactions are also deleted.



**KEY TAKEAWAY:** OOPs helps us build secure, modular, reusable, and easy-to-maintain banking applications by modeling real-world entities like Customers, Accounts, Loans, and Transactions.



## OOP CONCEPTS AT BMW TECHWORKS

Using object-oriented programming to build automotive software systems



### 1 CLASS

A class is a **blueprint** for creating vehicle-software objects.

```

Class: Vehicle
- vin: String
- model: String
- battery_kwh: float

+ start()
+ charge(kwh)
+ send_telemetry()

```

Defines what data and operations every Vehicle will have.

### 2 OBJECT

An object is an **instance** of a class. Every car rolling off the line gets its own Vehicle object.

```

i4: Vehicle
VIN WBA1234
BMW i4 eDrive40
Battery: 83.9 kWh

ix: Vehicle
VIN WBA5678
BMW ix xDrive50
Battery: 111.5 kWh

```

Same blueprint — each object keeps its own VIN, model and battery state.

### 3 ENCAPSULATION

Hiding sensitive vehicle data and allowing access only through methods.

```

class Vehicle:
 def __init__(self, vin):
 self.__soc = 100 # private

 def discharge(self, pct):
 if 0 < pct <= self.__soc:
 self.__soc -= pct

 def get_soc(self):
 return self.__soc

```

Battery state of charge is private — the BMS methods control every read and write.

### 4 ABSTRACTION

Showing only essential features and hiding the complex implementation.

```

from abc import ABC, abstractmethod

class ECU(ABC):
 @abstractmethod
 def run_diagnostics(self):
 pass

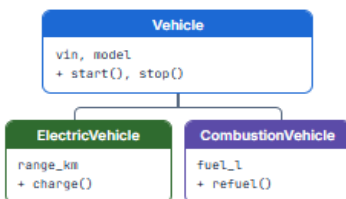
BrakeECU, BatteryECU each
implement it their own way

```

ECU defines what must be done; each control unit decides how it is done.

### 5 INHERITANCE

Reusing code: **ElectricVehicle** and **CombustionVehicle** inherit from **Vehicle**.



### 6 POLYMORPHISM

The same method name behaves differently for each vehicle type.

```

def service_check(vehicle):
 vehicle.run_diagnostics()
 # works for EV and
 # combustion alike

```

```

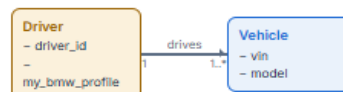
ElectricVehicle
run_diagnostics()
checks battery cells
and inverter.

CombustionVehicle
run_diagnostics()
checks emissions
and injectors.

```

### 7 ASSOCIATION

A relationship where one object **uses** another — both live independently.



One My BMW profile can be linked to one or more vehicles — deleting the profile never deletes the cars.

### 8 COMPOSITION

A strong "part-of" relationship: **TelemetryLog** cannot exist without its **Vehicle**.



If a Vehicle record is deleted, its telemetry logs are deleted with it.



**Key takeaway:** OOP helps BMW TechWorks build safe, modular, reusable and easy-to-maintain automotive software by modeling real-world entities like Vehicles, ECUs, Drivers and Telemetry — the same way it structures banking, retail or any other domain.

# OOPS – Class and Object

## ✓ Syntax:

python

```
class Person:
 def __init__(self, name, age):
 self.name = name
 self.age = age

 def greet(self):
 print(f"Hello, my name is {self.name} and I am {self.age} years old.")

Creating an object
p1 = Person("Alice", 30)
p1.greet()
```



# Class, object and `__init__`

```
class Car:
 def __init__(self, brand, price):
 self.brand = brand # attribute
 self.price = price

 def drive(self): # method
 print(self.brand, "is driving")

car1 = Car("BMW", 50) # __init__ runs
car1.drive() # BMW is driving
```

## `__init__`

The constructor — runs automatically the moment an object is created.

## `self`

Refers to the current object: “this particular car”.

## Attributes

Stored on `self`, so every object keeps its own copy of the data.

## Methods

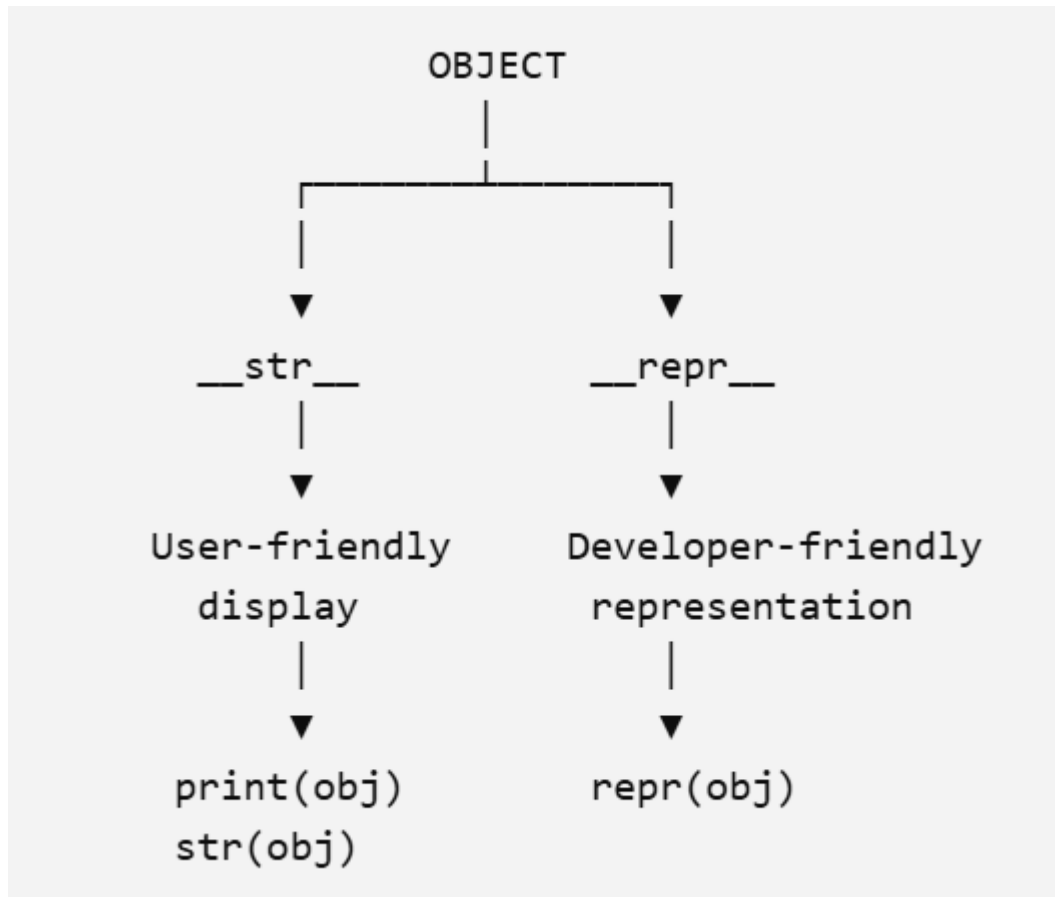
Functions inside a class; always take `self` as the first parameter.

## `__str__` vs `__repr__`

---

- Both `__str__` and `__repr__` are **magic (dunder) methods** used to represent an object as a string.
- The easiest way to remember:
- `__str__` → **for USERS**  
`__repr__` → **for DEVELOPERS**

# `__str__` vs `__repr__`



## \_\_str\_\_ vs \_\_repr\_\_

---

</> Python

```
class Car:
 def __init__(self, model, year):
 self.model = model
 self.year = year

 def __str__(self):
 return f"{self.model} ({self.year})"

 def __repr__(self):
 return f"Car(model={self.model!r}, year={self.year!r})"

car = Car("BMW X5", 2026)
```

# `__str__` vs `__repr__`

---

Using `__str__`

```
</> Python

print(car)
```

Python effectively calls:

```
</> Python

car.__str__()
```

Output:

```
BMW X5 (2026)
```

## \_\_str\_\_ vs \_\_repr\_\_

---

Using `__repr__`

```
</> Python
```

```
print(repr(car))
```

Python calls:

```
</> Python
```

```
car.__repr__()
```

Output:

```
Car(model='BMW X5', year=2026)
```

This gives the developer more information about the object.

# Magic methods: `__str__` vs `__repr__`

Dunder methods (double underscore) are hooks Python calls for you.

## `__str__` → for users

Friendly text. Used by `print(obj)` and `str(obj)`.

## `__repr__` → for developers

Precise/debug text. Used in the shell and inside lists.

```
class Car:
 def __str__(self):
 return f"A {self.brand} car"
 def __repr__(self):
 return f"Car (brand='{self.brand}')"

print(c) # A BMW car
print([c]) # [Car (brand='BMW')]
```

## Rule of thumb

`__repr__` should look like code that recreates the object. If `__str__` is missing, Python falls back to `__repr__`.

# Side By Side Comparison



| Feature                 | <code>__str__</code>              | <code>__repr__</code>                              |
|-------------------------|-----------------------------------|----------------------------------------------------|
| Audience                | End user                          | Developer                                          |
| Purpose                 | Readable output                   | Precise/debugging output                           |
| Called by               | <code>str(obj)</code>             | <code>repr(obj)</code>                             |
| <code>print(obj)</code> | Usually uses <code>__str__</code> | Used as fallback if <code>__str__</code> is absent |
| Typical output          | BMW X5 (2026)                     | <code>Car(model='BMW X5', year=2026)</code>        |
| Main goal               | Easy to read                      | Clear/unambiguous representation                   |



# OOPS - Encapsulation

---

- Encapsulation is bundling data and the methods that manage it together in one class, then hiding the data so it can only be changed through those methods.
- In fresher terms: the class guards its own data.

class Vehicle:

```
def __init__(self):
 self.speed = 0
```

```
v = Vehicle()
```

```
v.speed = -500 # Python happily accepts it
```

- Public attributes accept anything. Encapsulation puts a gatekeeper in front of the data.



## Three Levels of Hiding

---

- **public** — normal attribute, free access.
- **`_single`** — just a convention. Python doesn't stop you; it's a signal to other developers: "internal detail, use the methods instead."
- **`__double`** — Python renames it to `_Vehicle__soc` (name mangling), so `v.__soc` fails with `AttributeError`. Not true security, but accidental access is blocked.



## Three Levels of Hiding

---

class Vehicle:

    def \_\_init\_\_(self):

        self.model = "BMW i4"      # public: anyone can  
read/write

        self.\_speed = 0            # protected (convention):  
"internal, don't touch"

        self.\_\_soc = 100          # private: name-mangled by  
Python

## Two ways to encapsulate

---

- Getter/setter methods (Java style — works, but clunky in Python):

class Vehicle:

```
def __init__(self):
```

```
 self.__soc = 100 # battery state of charge
```

```
def discharge(self, pct):
```

```
 if 0 < pct <= self.__soc: # validation = the gatekeeper
```

```
 self.__soc -= pct
```

```
def get_soc(self):
```

```
 return self.__soc
```



# Two ways to encapsulate

---

- `@property` (the Pythonic way — this is what `vehicle.py` in your project uses):
- `python`

class Vehicle:

```
def __init__(self):
```

```
 self._speed = 0
```

```
@property
```

```
def speed(self): # read gate
```

```
 return self._speed
```

```
@speed.setter
```

```
def speed(self, value): # write gate with validation
```

```
 if value < 0:
```

```
 raise ValueError("Speed can't be negative")
```

```
 self._speed = value
```



## Two ways to encapsulate

---

```
v = Vehicle()
```

```
v.speed = 120 # looks like plain access, but the
setter runs
```

```
v.speed = -10 # ValueError — bad data never gets in
```

- The key insight to give freshers: encapsulation and `@property` aren't two separate topics — `@property` is *how* Python does encapsulation while keeping clean `v.speed` syntax.

# OOPS – Properties Getter and Setter

Python uses `@property` to create managed attributes.

## ✓ Example:

```
python

class Product:
 def __init__(self, price):
 self._price = price

 @property
 def price(self):
 return self._price

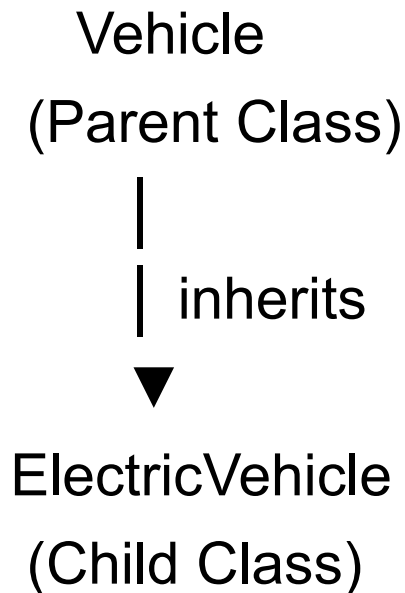
 @price.setter
 def price(self, value):
 if value >= 0:
 self._price = value

p = Product(100)
p.price = 120
print(p.price)
```

# OOPS - Inheritance

---

- **Inheritance** allows one class to **reuse the attributes and methods of another class**.
- Think of it as:
- **Parent provides common features → Child inherits and can add its own features.**





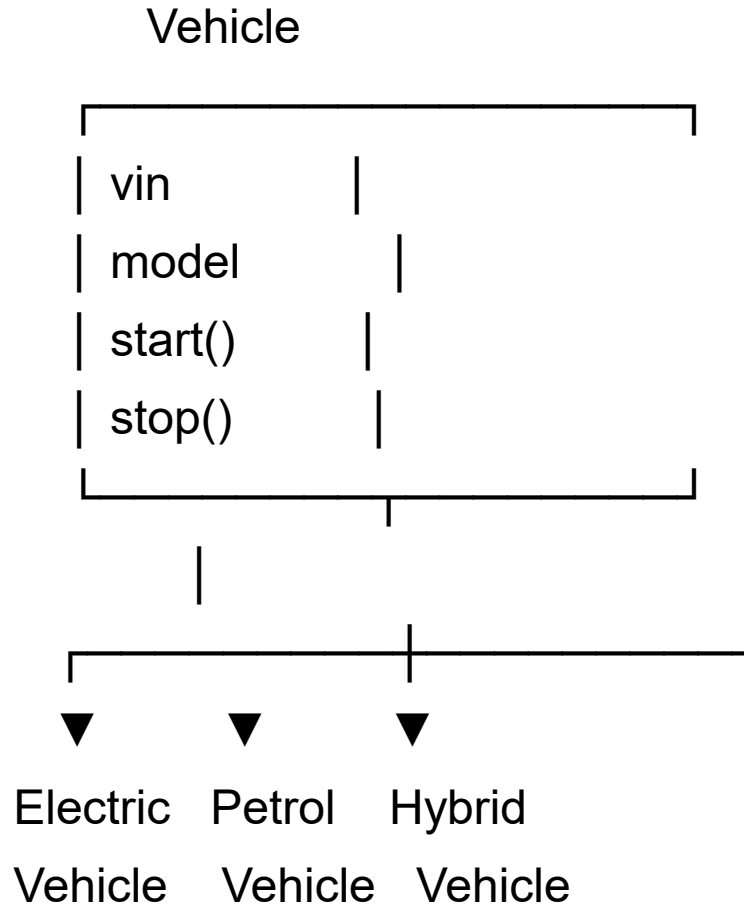
# Why Do We Need Inheritance?

---

- Suppose we have different BMW vehicle types:
- BMW Vehicle
  - |—— BMW Electric Vehicle
  - |—— BMW Petrol Vehicle
  - |—— BMW Hybrid Vehicle
- All vehicles have common information:
  - VIN
  - Model
  - Start
  - Stop

# Why Do We Need Inheritance?

- Instead of writing these repeatedly, put them in a parent Vehicle class.



- This promotes **code reuse**.

# Inheritance

---

```
class Parent:
```

```
 pass
```

```
class Child(Parent):
```

```
 pass
```

- The parent class goes inside ():

```
class ElectricVehicle(Vehicle):
```

```
 pass
```

- This means:
- ElectricVehicle IS-A Vehicle

# Inheritance

---

- **Parent Class**
- `class Vehicle:`
- `def start(self):`
- `print("Vehicle started")`
- **Child Class**
- `class ElectricVehicle(Vehicle):`
- `def charge(self):`
- `print("Battery charging")`

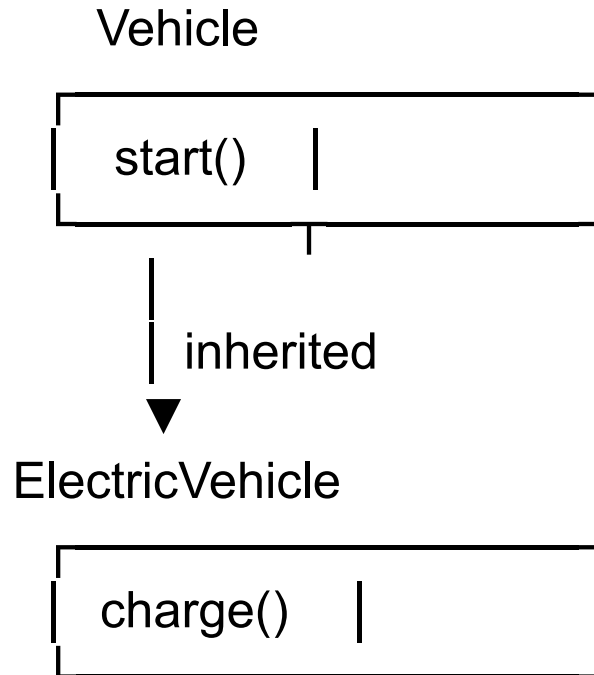
# Inheritance

---

- Create an object:
  - `ev = ElectricVehicle()`
  - `ev.start()`
  - `ev.charge()`
- Output:
  - Vehicle started
  - Battery charging
- Notice that `ElectricVehicle` doesn't define `start()`.

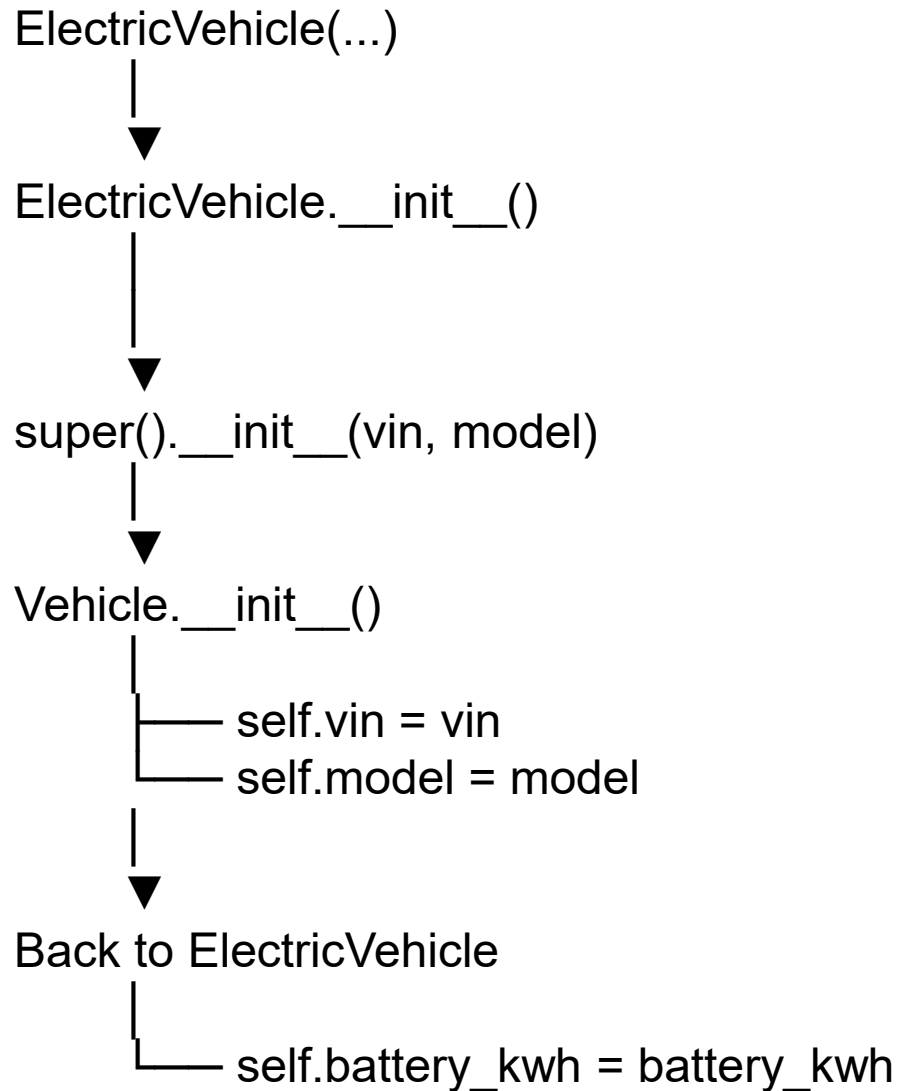
# Inheritance

- It gets start() from Vehicle.



- 
- `ev.start()` —► inherited from Vehicle
- `ev.charge()` —► ElectricVehicle's own method

# What Does `super()` Do?



# Method Overriding

---

- Inheritance isn't only about reusing methods.
- The child can **change the behavior** of an inherited method.

- Parent:

```
class Vehicle:
```

```
 def start(self):
 return "Vehicle started"
```

- Child:

```
class ElectricVehicle(Vehicle):
```

```
 def start(self):
 return "BMW EV started silently"
```





# Method Overriding

---

- Now:
- `ev = ElectricVehicle()`
- `print(ev.start())`
- Output:
  - BMW EV started silently

# Calling Parent Method While Overriding

---

- class Vehicle:
  - def start(self):
  - print("Basic vehicle checks")
- class ElectricVehicle(Vehicle):
  - def start(self):
  - super().start()
  - print("Electric motor started")

# OOPS - Polymorphism

---

Same method name behaves differently based on the object.

## ✓ Example:

python

```
class Bird:
 def fly(self):
 print("Bird can fly")

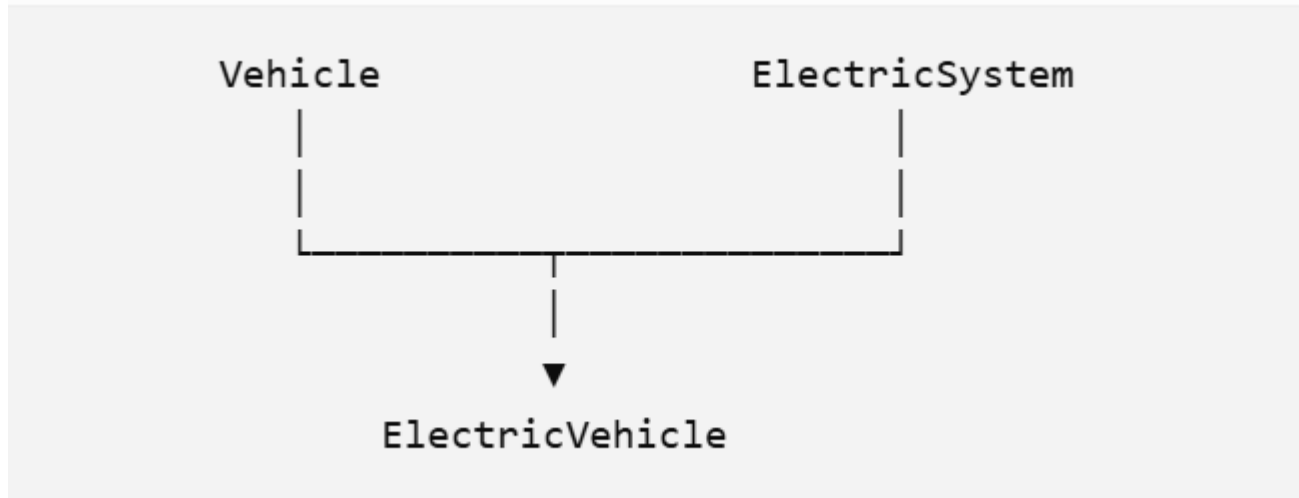
class Ostrich(Bird):
 def fly(self):
 print("Ostrich cannot fly")

def flying_test(bird):
 bird.fly()

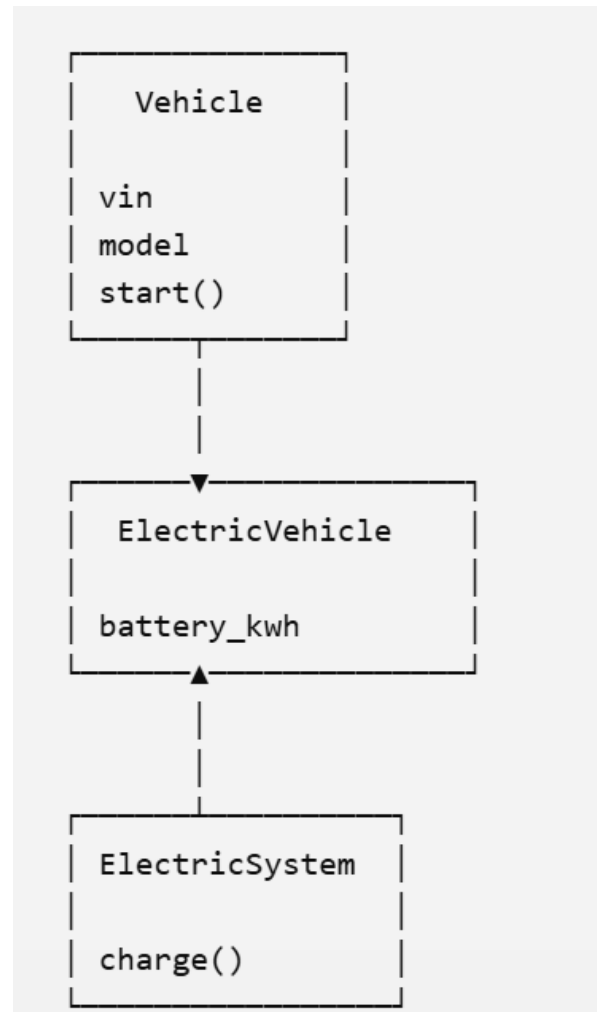
flying_test(Bird())
flying_test(Ostrich())
```

# Multiple Inheritance

- **Multiple inheritance** means a child class inherits features from **more than one parent class**.



# Multiple Inheritance





# Multiple Inheritance

---

```
class Vehicle:
 def __init__(self, vin, model):
 self.vin = vin
 self.model = model
 def start(self):
 return f"{self.model} started"

class ElectricSystem:
 def charge(self):
 return "Charging battery"

class ElectricVehicle(Vehicle, ElectricSystem):
 def __init__(self, vin, model, battery_kwh):
 Vehicle.__init__(self, vin, model)
 self.battery_kwh = battery_kwh
```

## Multiple Inheritance

---

- `ev = ElectricVehicle("WBA456", "BMW iX", 111.5)`
- `print(ev.start())`
- `print(ev.charge())`
  
- Output:
- BMW iX started
- Charging battery



## isinstance() with Multiple Inheritance

---

- `print(isinstance(ev, ElectricVehicle))`
- `print(isinstance(ev, Vehicle))`
- `print(isinstance(ev, ElectricSystem))`





## What if Both Parents Have the Same Method?

---

```
class Vehicle:
 def start(self):
 return "Vehicle started"

class ElectricSystem:
 def start(self):
 return "Electric system started"

class ElectricVehicle(Vehicle, ElectricSystem):
 pass
```



## What if Both Parents Have the Same Method?

---

- Now:
- `ev = ElectricVehicle()`
- `print(ev.start())`
- Which `start()` will Python execute?
  - Vehicle started
- Why?
- Because of **MRO — Method Resolution Order**.

# Method Resolution Order (MRO)

For:

</> Python

```
class ElectricVehicle(Vehicle, ElectricSystem):
 pass
```

Python searches:

```
ElectricVehicle
 |
 ▼
 Vehicle
 |
 ▼
ElectricSystem
 |
 ▼
 object
```

So Python finds `Vehicle.start()` first.

# Method Resolution Order (MRO)

---

- So Python finds `Vehicle.start()` first.
- Check the MRO:
- `print(ElectricVehicle.mro())`
- Conceptually:
  - [  
    ElectricVehicle,  
    Vehicle,  
    ElectricSystem,  
    object  
]
- **Easy rule for freshers**
- **When multiple parent classes have the same method, Python uses MRO to determine which implementation is selected.**

# Multiple Inheritance with super()

---

- class Vehicle:
  - def start(self):
  - print("Vehicle")
- class ElectricSystem:
  - def start(self):
  - print("Electric System")
- class ElectricVehicle(Vehicle, ElectricSystem):
  - def start(self):
  - super().start()
  - print("Electric Vehicle")

## Multiple Inheritance with super()

---

- Calling:
  - `ev = ElectricVehicle()`
  - `ev.start()`
- produces:
  - Vehicle
  - Electric Vehicle
- `super()` does **not simply mean "call my parent."**
- A better definition is:
  - `super()` calls the **next class in the MRO.**

# OOPS - Composition

---

- 👉 **Composition means “has-a” relationship**
- It is when **one class contains another class as a part of it.**
- 📌 **Example:**
  - **A Car has an Engine**
  - **A House has Rooms**

# OOPS - Composition

---

```
</> Python

class Engine:
 def start(self):
 print("Engine started")

class Car:
 def __init__(self):
 self.engine = Engine() # Composition

 def drive(self):
 self.engine.start()
 print("Car is moving")

Usage
car = Car()
car.drive()
```

## Output:

```
Engine started
Car is moving
```



# OOPS – Composition – Dependency Inversion Style

---

Python

```
class Database:
 def connect(self):
 print("DB Connected")

class Service:
 def __init__(self, db):
 self.db = db # Injected (composition)

 def process(self):
 self.db.connect()

db = Database()
service = Service(db)
service.process()
```

# OOPS - Interfaces

---

- 👉 An **interface** defines:
  - **What methods a class must implement**
  - **But not how they are implemented**
- 📌 Think of it like:
  - “Rules a class must follow”
- Python doesn't have a strict interface keyword like Java, but we use: Abstract classes

# OOPS - Abstraction

Hides complex logic and exposes only essential features.

✓ Using `abc` module:

```
python

from abc import ABC, abstractmethod

class Shape(ABC):
 @abstractmethod
 def area(self):
 pass

class Square(Shape):
 def __init__(self, side):
 self.side = side

 def area(self):
 return self.side * self.side

s = Square(4)
print(s.area()) # 16
```

## ◆ What is Dependency Inversion?

---

- 👉 **Definition (simple):**
- High-level modules should not depend on low-level modules.  
Both should depend on abstractions.
- 👉 And:
- Abstractions should not depend on details.  
Details should depend on abstractions.

## ◆ What is Dependency Inversion?

---

- ◆ Why is this important?
- Without DIP:
  - Code becomes **tightly coupled**
  - Hard to **test**
  - Hard to **change or extend**
- With DIP:
  - Code becomes **flexible**
  - Easy to **replace components**
  - Better for **microservices & scalable systems**

## ◆ What is Dependency Inversion?

---

### Step 1: Create Abstraction

```
</> Python

from abc import ABC, abstractmethod

class Database(ABC):
 @abstractmethod
 def connect(self):
 pass
```

### Step 2: Implement Concrete Classes

```
</> Python

class MySQLDatabase(Database):
 def connect(self):
 print("Connected to MySQL")

class PostgreSQLDatabase(Database):
 def connect(self):
 print("Connected to PostgreSQL")
```

## ◆ What is Dependency Inversion?

---

### Step 3: Inject Dependency

Python

```
class UserService:
 def __init__(self, db: Database): # Depends on abstraction
 self.db = db

 def process(self):
 self.db.connect()

Usage
db = MySQLDatabase()
service = UserService(db)
service.process()
```



# Destructor

---

- In Python, a **destructor** is typically represented by the special method `__del__()`.
- It may be called when an object is about to be garbage-collected.



# Destructor

---

```
class Employee:
 def __init__(self, name):
 self.name = name
 print(f"Employee {self.name} created")

 def __del__(self):
 print(f"Employee {self.name} destroyed")

emp = Employee("John")
del emp
```

- Possible output:
  - Employee John created
  - Employee John destroyed

# Constructor vs destructor

---

| Feature     | Constructor             | Destructor                       |
|-------------|-------------------------|----------------------------------|
| Method      | <code>__init__()</code> | <code>__del__()</code>           |
| Purpose     | Initialize an object    | Cleanup when object is destroyed |
| Called      | After object creation   | Around garbage collection        |
| Typical use | Set attributes          | Last-resort cleanup              |

## ◆ What is ConfigManager?

---

- 👉 A **centralized component** that:
  - Reads configuration from multiple sources
  - Provides a clean API to access config values
  - Avoids hardcoding values
- 📌 Used in:
  - Microservices
  - APIs
  - Data pipelines
  - CLI tools

## ◆ Why Use ConfigManager?

---

- ❌ Without it:
  - DB\_HOST = "localhost"
  - DB\_PORT = 5432
- 👉 Hard to change, duplicate everywhere
- ✅ With ConfigManager:
- `config.get("db.host")`
- 👉 Centralized & reusable

## ◆ Simple ConfigManager (Python)

```
import os

class ConfigManager:
 def __init__(self):
 self.config = {
 "db_host": os.getenv("DB_HOST", "localhost"),
 "db_port": os.getenv("DB_PORT", 5432),
 "debug": os.getenv("DEBUG", "false") == "true"
 }

 def get(self, key):
 return self.config.get(key)
```

```
config = ConfigManager()

print(config.get("db_host"))
print(config.get("debug"))
```

## ◆ Improved Version (File + Env Support)

---

```
import json
import os

class ConfigManager:
 def __init__(self, file_path=None):
 self.config = {}

 # Load from file
 if file_path:
 with open(file_path) as f:
 self.config.update(json.load(f))

 # Override with env variables
 for key in self.config:
 env_value = os.getenv(key.upper())
 if env_value:
 self.config[key] = env_value

 def get(self, key, default=None):
 return self.config.get(key, default)
```

## ◆ Improved Version (File + Env Support)

---

### ◆ Example config.json

```
{
 "db_host": "localhost",
 "db_port": 5432,
 "debug": true
}
```

# Data Loader

---

- 👉 **What is Data Loader?**
- A **Data Loader** is responsible for:
  - Reading data from sources
  - Returning data in a usable format
- 📌 **Sources:**
  - CSV / JSON files
  - Databases
  - APIs
  - Data lakes (S3, GCS)





## Data Loader using Pandas

---

- Pandas is Python's main library for working with **tabular data** like Excel sheets, CSV files, SQL results, and time-series datasets.
- Its two core structures are **Series** for 1D labeled data and **DataFrame** for 2D row-column data.
- The current official docs describe pandas as an open-source library for high-performance, easy-to-use data structures and analysis tools, and the latest docs are for pandas 3.0.2.

# Data Loader using Pandas

---

- **1. Why pandas is used**
  - read data from CSV, Excel, JSON, SQL
  - clean missing or messy data
  - filter rows and columns
  - sort and group records
  - calculate statistics
  - join multiple datasets
  - reshape data
  - work with dates and time
  - prepare data for ML or reporting

# Data Loader using Pandas

---

- Example use cases:
  - employee records
  - sales reports
  - healthcare data
  - test result analysis
  - log analysis
  - time-series trends



# Data Loader using Pandas

---

- **2. Install pandas**
  - pip install pandas
- Usually imported as:
  - import pandas as pd

# Data Loader using Pandas

---

- **3. Main pandas data structures**
- **Series**
- A **Series** is a one-dimensional labeled array.
- `import pandas as pd`

```
s = pd.Series([10, 20, 30, 40])
print(s)
```

# Data Loader using Pandas

---

- **3. Main pandas data structures**
- **Series**
- A **Series** is a one-dimensional labeled array.
- `import pandas as pd`
- ```
s = pd.Series([10, 20, 30, 40])  
print(s)
```
- With custom index:
- ```
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s)
```

# Data Loader using Pandas

---

- **4. Creating pandas objects**
  - **From list**
    - `s = pd.Series([1, 2, 3, 4])`
  - **From dictionary**
    - `s = pd.Series({"a": 100, "b": 200, "c": 300})`
  - A **DataFrame** is a two-dimensional table with rows and columns.
  - `data = {`
    - `"name": ["Anu", "Bala", "Chitra"],`
    - `"age": [25, 30, 28],`
    - `"city": ["Chennai", "Kochi", "Madurai"]``}`
- ```
df = pd.DataFrame(data)
print(df)
```



Data Loader using Pandas

- **5. Reading data**

- Pandas is commonly used to import data from many formats, including CSV, Excel, JSON, SQL, and more.
- The official “10 minutes” guide explicitly includes importing and exporting data as a core topic.

- **CSV**

- `df = pd.read_csv("employees.csv")`

- **Excel**

- `df = pd.read_excel("employees.xlsx")`

- **JSON**

- `df = pd.read_json("data.json")`

- **SQL**

- `import sqlite3`

- `conn = sqlite3.connect("company.db")`
 - `df = pd.read_sql("SELECT * FROM employees", conn)`

Data Loader using Pandas

- **6. Writing data**
- **To CSV**
 - `df.to_csv("output.csv", index=False)`
- **To Excel**
 - `df.to_excel("output.xlsx", index=False)`
- **To JSON**
 - `df.to_json("output.json", orient="records")`

Data Loader using Pandas

- **7. Viewing and understanding data**

- import pandas as pd

```
df = pd.DataFrame({  
    "id": [1, 2, 3, 4, 5],  
    "name": ["Asha", "Bala", "Charan", "Divya", "Eshan"],  
    "age": [23, 35, 29, 41, 26],  
    "salary": [25000, 50000, 42000, 65000, 30000],  
    "department": ["HR", "IT", "IT", "Finance", "HR"]  
})
```

- **First rows**

- df.head()

- **Last rows**

- df.tail()

- **Shape**

- df.shape

- **Result:**

- (5, 5)



Data Loader using Pandas

- **Column names**
 - `df.columns`
- **Data types**
 - `df.dtypes`
- **Summary info**
 - `df.info()`
- **Statistical summary**
 - `df.describe()`

Data Loader using Pandas

- **8. Selecting data**
- This is one of the most important pandas topics.
- **Select one column**
- `df["name"]`
- **Select multiple columns**
- `df[["name", "salary"]]`
- **Select row by position with `iloc`**
- `df.iloc[0]`
- **Select rows and columns by position**
- `df.iloc[0:3, 1:4]`

Data Loader using Pandas

- **8. Selecting data**
- **Select row by label with loc**
- `df.loc[0]`
- **Select specific rows and columns by label**
- `df.loc[0:2, ["name", "salary"]]`
- **Access single value**
- `df.at[0, "name"]`
`df.iat[0, 1]`

Data Loader using Pandas

- **9. Filtering data**
- **Simple condition**
 - `df[df["age"] > 30]`
- **Multiple conditions**
 - `df[(df["age"] > 25) & (df["department"] == "IT")]`
- **Using isin**
 - `df[df["department"].isin(["HR", "Finance"])]`
- **Using between**
 - `df[df["salary"].between(30000, 60000)]`
- **Null check**
 - `df[df["salary"].isnull()]`
- **Not null**
 - `df[df["salary"].notnull()]`

Data Loader using Pandas

- **10. Adding and modifying columns**
- **Add new column**
 - `df["bonus"] = df["salary"] * 0.10`
- **Create conditional column**
 - `df["senior"] = df["age"] > 30`
- **Update one column**
 - `df["salary"] = df["salary"] + 5000`
- **Rename columns**
 - `df.rename(columns={"name": "employee_name"}, inplace=True)`

Data Loader using Pandas

- **11. Deleting rows and columns**
- **Drop a column**
 - `df.drop("bonus", axis=1, inplace=True)`
- **Drop multiple columns**
 - `df.drop(["bonus", "senior"], axis=1, inplace=True)`
- **Drop rows by index**
 - `df.drop([0, 2], axis=0, inplace=True)`

Data Loader using Pandas

- **12. Sorting data**
- **Sort by one column**
 - `df.sort_values("salary")`
- **Descending**
 - `df.sort_values("salary", ascending=False)`
- **Multiple columns**
 - `df.sort_values(["department", "salary"], ascending=[True, False])`

Data Loader

◆ Simple DataLoader Example

Python

```
import pandas as pd

class DataLoader:
    def __init__(self, source):
        self.source = source

    def load(self):
        if self.source.endswith(".csv"):
            return pd.read_csv(self.source)
        elif self.source.endswith(".json"):
            return pd.read_json(self.source)
        else:
            raise ValueError("Unsupported format")
```

◆ 📌 What is Pipeline Runner?

- A Pipeline Runner usually:
 - takes input data
 - passes it through multiple stages
 - manages execution order
 - handles errors/logging
 - returns the result

◆ 📌 What is Pipeline Runner?

- **Example pipeline**
- For a data science or ETL project, the pipeline may look like this:
 - Load data
 - Clean data
 - Transform data
 - Train model
 - Evaluate model
 - Save output
- Here, the **PipelineRunner** is the class that runs these steps one by one.

◆ 2. Pipeline Runner

- ◆ **Simple PipelineRunner**

- class PipelineRunner:
 def __init__(self, steps):
 self.steps = steps

 def run(self, data=None):
 for step in self.steps:
 data = step(data)
 return data



◆ 2. Pipeline Runner

- ◆ **Example Steps**

- `def clean_data(data):
 return data.dropna()`

```
def transform_data(data):  
    return data * 2
```



◆ 2. Pipeline Runner

- ◆ **Example Steps**

- `def clean_data(data):
 return data.dropna()`

```
def transform_data(data):  
    return data * 2
```

◆ 2. Pipeline Runner

- ◆ **Usage**
- loader = DataLoader("data.csv")
data = loader.load()

```
pipeline = PipelineRunner([  
    clean_data,  
    transform_data  
])
```

```
result = pipeline.run(data)
```




Python Unit Testing

- A **unit** is the smallest testable part of your program.
- Example units:
 - `add(a, b)` function
 - `Customer.validate_email()` method
 - `PipelineRunner.run()` method
 - `ConfigManager.get()` method
- A **unit test** verifies:
 - correct output for valid input
 - correct behavior for invalid input
 - edge cases
 - exceptions



Why unit testing is important

- Unit testing helps you:
 - catch bugs early
 - improve confidence in code
 - refactor safely
 - document expected behavior
 - reduce manual testing effort
 - support CI/CD pipelines
- Without tests, even a small code change can break old functionality, and you may not notice immediately.

Without testing

- `def divide(a, b):`
 `return a / b`
- Looks simple but what about:
 - `divide(10, 2)` → should return 5
 - `divide(5, 0)` → should raise error
 - `divide(-4, 2)` → should return -2
- Unit tests verify all of that.



Python testing frameworks

- Python mainly uses two approaches:
- **A. unit test**
 - Built into Python standard library.
- Features:
 - class-based
 - similar to Java/JUnit style
 - no extra installation needed



Python testing frameworks

- Python mainly uses two approaches:
- **B. pytest**
- Most popular third-party framework.
- Features:
 - simpler syntax
 - powerful fixtures
 - easy assertions
 - better readability
- For learning, it is good to understand both. In real projects, many developers prefer pytest.

Subroutine



Async and CLI Automation

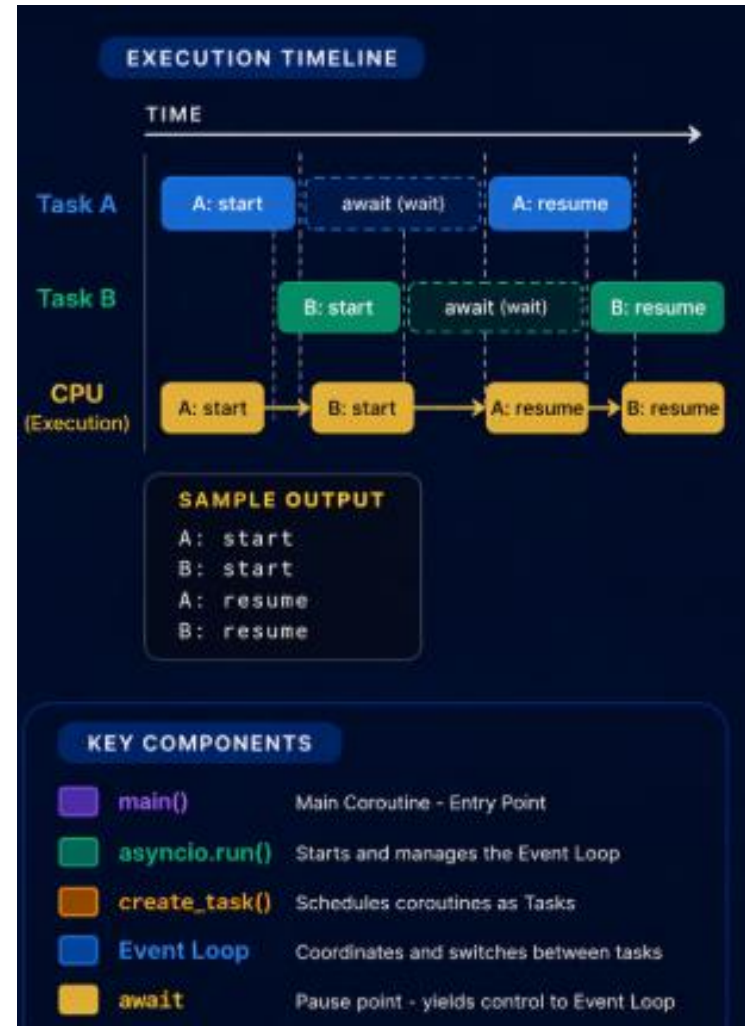
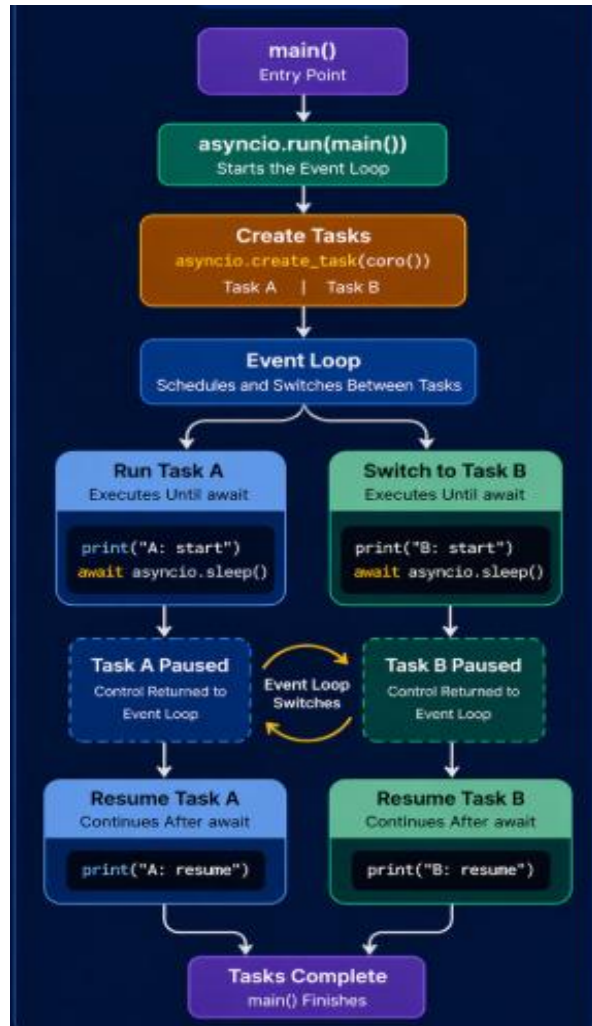
- ◆ Coroutine in Python (asyncio)
- A **coroutine** is the core building block of asynchronous programming in Python.
- It lets a function **pause (await) and resume later**, enabling efficient concurrency for I/O-bound work.



Async and CLI Automation

- **◆ 1. Formal Definition**
- A coroutine is:
 - A function defined with `async def`
 - Returns a **coroutine object**
- Executes only when:
 - awaited, or
 - scheduled as a **Task**
- `async def example():`
 `return "Hello"`

Coroutine



Async io



EVENT LOOP HANDLES COROUTINES FOR NON-BLOCKING I/O

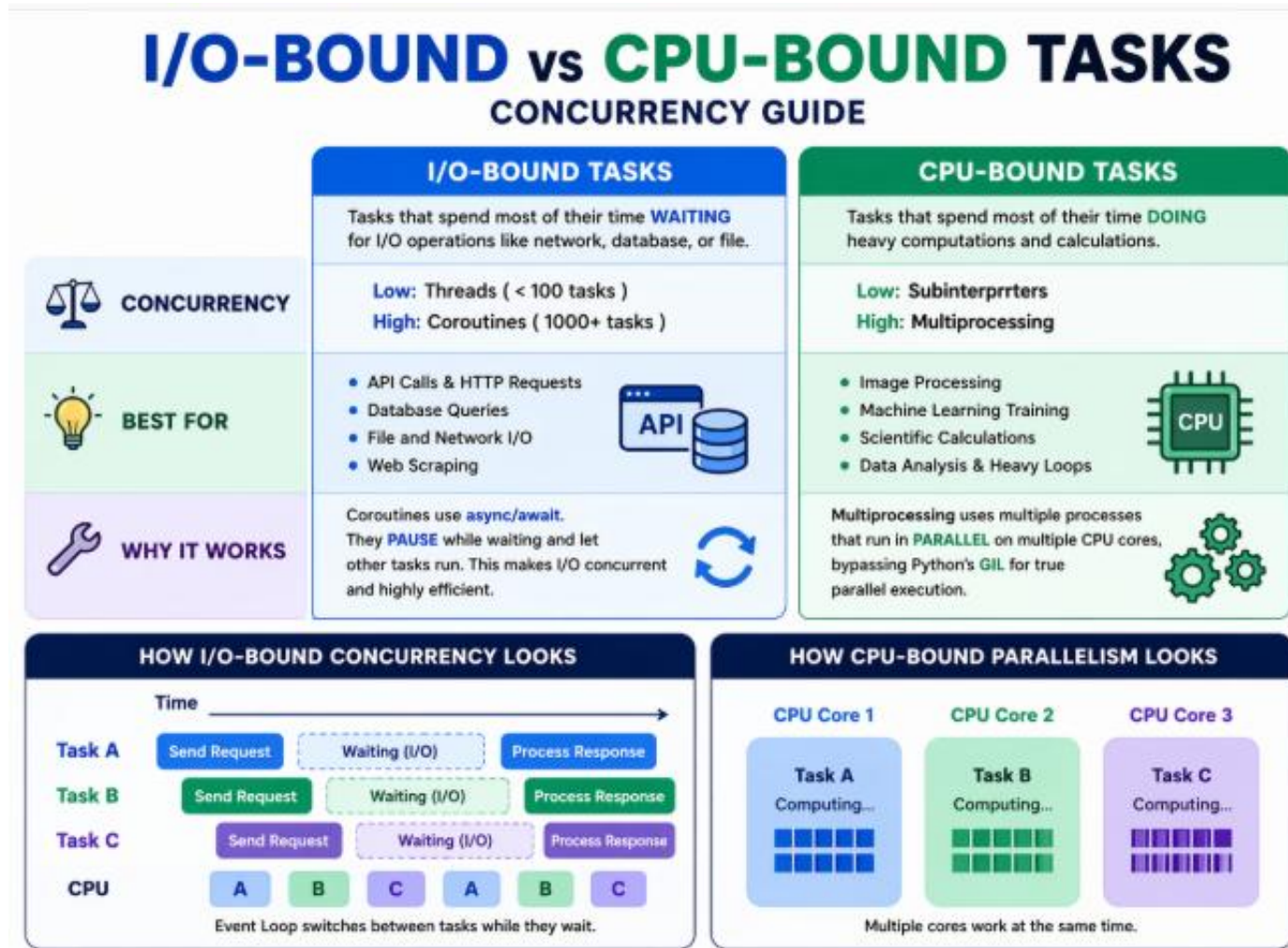


KEY IDEA: While one API call waits for a response, the event loop runs other tasks. This makes I/O concurrent and highly efficient!

2. Coroutine vs Normal Function

Feature	Normal Function	Coroutine
Keyword	<code>def</code>	<code>async def</code>
Execution	Immediate	Needs <code>await</code>
Blocking	Yes	No
Pausable	No	Yes (<code>await</code>)

3. I/O Bound vs CPU Bound



3. I/O Bound vs CPU Bound

REAL-WORLD EXAMPLES

I/O-BOUND - Use Coroutines (asyncio)



- Calling 1000 APIs simultaneously
- Making database queries
- Downloading multiple files
- Web scraping websites

```
async def fetch():  
    response = await  
    session.get(url)  
    return await response.json()
```

CPU-BOUND - Use Multiprocessing



- Image/video processing
- Training ML models
- Scientific simulations
- Heavy data computations

```
from multiprocessing import Pool  
def compute(data):  
    return heavy_calculation(data)  
  
with Pool() as pool:  
    results = pool.map(compute, data)
```



KEY TAKEAWAY

I/O-bound tasks wait, so we **switch between them**.
CPU-bound tasks compute, so we use **multiple cores**.

What “platform automation CLI” usually means



- This is the kind of internal tool that helps with tasks like:
 - deploy service
 - rotate secrets
 - sync config
 - run health checks
 - bootstrap environments
 - generate reports

What “platform automation CLI” usually means



- A good design is:
 - **CLI layer** for arguments/options/help
 - **service layer** for actual automation logic
 - **packaging** so teammates can install and run one command consistently

Why Typer is a good choice

- Typer makes commands from normal Python functions and supports multiple subcommands, arguments, options, and generated help text.

```
platform_tool/  
├─ pyproject.toml  
├─ README.md  
├─ src/  
│   └─ platform_tool/  
│       ├── __init__.py  
│       ├── cli.py  
│       └── commands/  
│           ├── deploy.py  
│           ├── health.py  
│           └── config.py  
│       └── services/  
│           ├── deployer.py  
│           ├── checks.py  
│           └── api_client.py  
└─ tests/  
    └─ test_cli.py
```




Why Typer is a good choice

- `pip install typer[all]`
- `import typer`
- ```
def main(name: str):
 print(f"Hello {name}")

if __name__ == "__main__":
 typer.run(main)
```
- `python app.py John`

# Why Typer is a good choice

```
← → sdkdemo | 🗨️ ▼
Welcome | ⚙️ pyproject.toml | 📄 typerdemo.py | 📄 typermultidemo.py X
typermultidemo.py > ...
1 import typer
2
3 app= typer.Typer()
4
5 @app.command()
6 def main(name: str, age: int):
7 print(f"Hello {name}, you are {age} years old!")
8 @app.command()
9 def greet(name: str):
10 print(f"Hello {name}!")
11
12 if __name__ == "__main__":
13 app()
```

PROBLEMS | TERMINAL | AZURE

```
(sdkenv) E:\Datascience2026\sdkdemo>python typermultidemo.py greet parameswari
Hello parameswari!

(sdkenv) E:\Datascience2026\sdkdemo>python typermultidemo.py main parameswari 56
Hello parameswari, you are 56 years old!

(sdkenv) E:\Datascience2026\sdkdemo>
```

# Python Click

---



- **Click** (Command Line Interface Creation Kit) is a popular Python library for building **robust and scalable CLI applications** using decorators.

# Python Click

---

-  **Why Use Click?**
-  Mature & widely used
-  Supports complex CLI apps
-  Nested commands (like git)
-  Argument & option parsing
-  Automatic help generation

# Python Click

---

-  **Install Click**

- pip install click
- import click

```
@click.command()
@click.argument("name")
def hello(name):
 click.echo(f"Hello {name}")
```

```
if __name__ == "__main__":
 hello()
```

# Python Click

---



- `python app.py John`

# Python Click



```
← → sdkdemo | [Icons]
Welcome | pyproject.toml | clickdemo.py X | typerdemo.py | typermultidemo.py
clickdemo.py > ...
1 import click
2
3 @click.command()
4 @click.option('--name', prompt='Your name', help='The person to greet.')
5 @click.option('--age', prompt='Your age', help='The age of the person to greet.', type=int)
6 def main(name, age):
7 print(f"Hello {name}, you are {age} years old!")
8
9 if __name__ == "__main__":
10 main()

PROBLEMS | TERMINAL | AZURE
(sdkenv) E:\Datascience2026\sdkdemo>python clickdemo.py
Your name: Parameswari
Your age: 56
Hello Parameswari, you are 56 years old!

(sdkenv) E:\Datascience2026\sdkdemo>
```

# Click vs Typer

---

| Feature        | Click      | Typer      |
|----------------|------------|------------|
| Syntax         | Decorators | Type hints |
| Complexity     | Powerful   | Simpler    |
| Learning Curve | Medium     | Easy       |
| Large Apps     | ✓ Best     | ✓ Good     |





# Packaging and Versioning

---

- Install build tools
  - `pip install build twine`
- Build distributions:
  - `python -m build`
- This creates:
  - Source distribution: `.tar.gz`
  - Wheel distribution: `.whl`



# Deployment

---

- Open new folder
- run
  - `pip install <filename>.tar.gz`
- File will be deployed under `venv\lib\site-packages`
- To test
- Activate venv, then
- `python -m async_dashboard.utils.main`

# Deployment



```
(day7env) E:\Datascience2026\cognizantaimlapr2026-master\cognizantaimlapr2026-master\day7>python -m
async_dashboard.utils.main
https://jsonplaceholder.typicode.com/
https://jsonplaceholder.typicode.com//users/1
https://jsonplaceholder.typicode.com//posts/1
https://jsonplaceholder.typicode.com//albums/1
https://jsonplaceholder.typicode.com//photos/1
{'user': {'id': 1, 'name': 'Leanne Graham', 'email': 'Sincere@april.biz'}, 'post': {'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et s
uscipit\nuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam
\nnostrum rerum est autem sunt rem eveniet architecto'}, 'album': {'userId': 1, 'id': 1, 'title': '
quidem molestiae enim'}, 'photos': {'albumId': 1, 'id': 1, 'title': 'accusamus beatae ad facilis cu
m similique qui sunt', 'url': 'https://via.placeholder.com/600/92c952', 'thumbnailUrl': 'https://vi
a.placeholder.com/150/92c952'}}

(day7env) E:\Datascience2026\cognizantaimlapr2026-master\cognizantaimlapr2026-master\day7>
```

# Mongodb Atlas Cluster



## Clusters

Find a database deployment...

Edit Config

+ Create

● cholacuster

Connect

View Monitoring

Browse Collections

...

### Visualize Your Data

Build dashboards and charts, and embed them in your apps with MongoDB Charts.

Explore Charts

Dismiss

● R 0

● W

Last 2 hours

0.01/s



i

Connections 8.0

Last 2 hours

8.0



i

● In 31.06 B/s

● Out 460.36 B/s

Last 2 hours

496.41 B/s



i

Data Si:

130.62

Last 2 ho

512.00 MB

# Mongodb Atlas connection

- A **Non-SRV connection string** is the **traditional MongoDB URI** that explicitly lists all cluster hosts instead of using DNS discovery.

| Type           | Format         | How it works                    |
|----------------|----------------|---------------------------------|
| <b>SRV</b>     | mongodb+srv:// | Uses DNS to auto-discover nodes |
| <b>Non-SRV</b> | mongodb://     | You manually specify all nodes  |

## 3. Add your connection string into your application code

Use this connection string in your application

☐ SRV Connection String ⓘ

☐ View full code sample

```
mongodb://vhebcompany:<db_password>@ac-iepkg00-shard-00-00.acakwxa.mongodb.net:27017,ac-iepkg00-shard-00-01.acakwxa.mongodb.net:27017,ac-iepkg00-shard-00-02.acakwxa.mongodb.net:27017/?ssl=true&replicaSet=atlas-wcg1od-shard-0&authSource=admin&appName=cholacluster
```



# Mongodb Atlas connection



Database & Network...

DATABASE ACCESS ▲  
Database Users  
Custom Roles

NETWORK ACCESS ▲  
**IP Access List**  
Peering  
Private Endpoint  
Advanced

ORGANIZATION  
Freelancer ▼

>

PROJECT  
Project 0 ▼

?

V

## IP Access List

+ADD IP ADDRESS

! You will only be able to connect to your cluster from the following list of IP Addresses:

| IP Address                                   | Comment                                   | Status   | Actions                           |
|----------------------------------------------|-------------------------------------------|----------|-----------------------------------|
| 223.187.118.16/32                            | Created as part of the Auto Setup process | ● Active | <div>EDIT</div> <div>DELETE</div> |
| 0.0.0.0/0 (includes your current IP address) |                                           | ● Active | <div>EDIT</div> <div>DELETE</div> |

# Edit Password



Database &

DATABASE AC

Database Use

Custom Roles

NETWORK AC

IP Access List

Peering

Private Endpoi

Advanced

MongoDB uses [SCRAM](#) as its default authentication method.

### Password Authentication

vhebcompany

Edit Password

### User Description

Add an optional description to your user.

### Database User Privileges

# VSCode Mongodb



The screenshot shows the VS Code interface with the MongoDB extension installed. The left sidebar contains the following sections:

- CONNECTIONS**
  - ac-iepkg00-shard-00-00.acakwxa....
    - admin
    - config
    - local
    - mongodbVSCodePlaygroundDB
    - sample\_mflix
- PLAYGROUNDS**

No MongoDB playground files found in the workspace.

Create New Playground
- HELP AND FEEDBACK**
  - What's New
  - Extension Documentation
  - MongoDB Documentation
  - Suggest a Feature
  - Report a Bug
  - Create Free Atlas Cluster

The main editor area displays the MongoDB playground interface. It includes the MongoDB logo, a navigation instruction, and a connection string: `ac-iepkg00-shard-00-00.acakwxa.mongodb.net:27017,ac-iepkg00-shard-00-01.acakwxa.mongodb.net:27017,ac-iepkg00-shard-00-02.acakwxa.mongodb.net:27017`. Below this, it says "All set. Ready to start?" and "Create a playground." with a "Create playground" button. At the bottom, there are two options: "Connect with Connection String" (with a "Connect" button) and "Advanced Connection Settings" (with an "Open form" button).

The terminal at the bottom shows the following commands and output:

```
(day8env) E:\Datascience2026\cognizantaimlapr2026-master\cognizantaimlapr2026-master\day8>python src\cli_automation\testmongo.py
['mongodbVSCodePlaygroundDB', 'sample_mflix', 'admin', 'local']

(day8env) E:\Datascience2026\cognizantaimlapr2026-master\cognizantaimlapr2026-master\day8>
```





# Relational + NoSQL Modeling – Entity Design

---

- **1. Identify business objects**
- Ask: what things does the system track?
- Example for ecommerce:
  - Customer
  - Product
  - Order
  - Payment



# Relational + NoSQL Modeling – Entity Design

---

- **2. Define attributes**
- For each entity, list the fields it needs.
- Example for Product:
  - ProductId
  - ProductName
  - Description
  - Price
  - Category
  - StockQuantity



# Relational + NoSQL Modeling – Entity Design

---

- **3. Choose primary key**
- Each entity should have a unique identifier.
- Examples:
  - CustomerId
  - ProductId
  - OrderId



# Relational + NoSQL Modeling – Entity Design

---

- **4. Define relationships**
- Understand how entities connect.
- Examples:
  - One customer can place many orders
  - One order can contain many products
  - One employee belongs to one department



# Relational + NoSQL Modeling – Entity Design

---

- **5. Avoid mixing unrelated data**
  - Each entity should represent one main concept.
- Bad example:
  - Customer entity containing product stock details
- Good example:
  - Customer data in Customer
  - Product stock in Product or Inventory



# Relational + NoSQL Modeling – Entity Design

---

- **Common relationship types**
- **One-to-One**
  - One record matches one other record.
- Example:
  - Person  $\leftrightarrow$  Passport
- **One-to-Many**
  - One record relates to many records.
- Example:
  - Customer  $\rightarrow$  Orders
- **Many-to-Many**
  - Many records relate to many records.
- Example:
  - Students  $\leftrightarrow$  Courses

# Good entity design principles

---

- **Keep entities focused**
  - Each entity should have one purpose.
- **Use meaningful names**
  - Use names that clearly represent business meaning.
- Good:
  - Customer
  - OrderItem
  - InvoicePayment
- Avoid vague names:
  - Data1
  - InfoTable
  - MasterRecord



# Good entity design principles

---

- **Store atomic values**
  - Each field should contain one value only.
- Bad:
  - PhoneNumbers = "12345,67890"
- Good:
  - One phone per field, or separate phone entity





# Good entity design principles

---

- **Reduce duplication**
  - Do not repeat the same data unnecessarily.
- **Bad:**
  - Customer name stored in many tables again and again without reason
- **Good:**
  - Store customer once and refer by ID where appropriate

# Good entity design principles

---

- **Design for change**
  - Think about future growth.
- **Example:**

Instead of one Address field, use:

  - Street
  - City
  - State
  - PostalCode
  - Country



# Good entity design principles

---

- **Design for change**
  - Think about future growth.
- **Example:**

Instead of one Address field, use:

  - Street
  - City
  - State
  - PostalCode
  - Country

# Good entity design principles

---

- **Example: student system**
- **Entity: Student**
  - StudentId
  - Name
  - DateOfBirth
  - Email
- **Entity: Course**
  - CourseId
  - CourseName
  - Duration



# Good entity design principles

---

- **Entity: Enrollment**

- EnrollmentId
- StudentId
- CourseId
- EnrollmentDate



## Entity vs table

---

- In relational databases:
  - entity is the logical business object
  - table is the physical storage structure
- Often one entity becomes one table, but not always.



# Entity design in SQL vs NoSQL

---

- **In SQL**

- Entities are usually separated and linked using foreign keys.
- Example:
  - Customer table
  - Order table
  - Product table

- **In NoSQL**

- Related data may be grouped together in one document.
- Example:  
An Order document may contain:
  - customer info
  - shipping address
  - items list

# Entity design in SQL vs NoSQL

---

- **Common mistakes**
  - putting too many unrelated fields in one entity
  - missing unique identifiers
  - repeating the same data everywhere
  - not defining relationships clearly
  - creating too many entities for very small concepts
  - creating one giant entity for the whole system





# Normalization

---

- DBMS Normalization is a systematic approach to decompose (break down) tables to eliminate data redundancy(repetition) and undesirable characteristics like Insertion anomaly in DBMS, Update anomaly in DBMS, and Delete anomaly in DBMS.

# Normalization



## DATABASE NORMALIZATION – STEP BY STEP

Transforming Unnormalized Data into Well Structured Relational Tables

| UNNORMALIZED<br>Single Table with Redundancy |            |            |              |                 |           |             |           |          |
|----------------------------------------------|------------|------------|--------------|-----------------|-----------|-------------|-----------|----------|
| ORDERS_UNNORM                                |            |            |              |                 |           |             |           |          |
| OrderID                                      | OrderDate  | CustomerID | CustomerName | CustomerEmail   | ProductID | ProductName | UnitPrice | Quantity |
| 1001                                         | 2024-05-01 | C001       | Alice Smith  | alice@email.com | P001      | Laptop      | 1200      | 1        |
| 1001                                         | 2024-05-01 | C001       | Alice Smith  | alice@email.com | P002      | Mouse       | 25        | 2        |
| 1002                                         | 2024-05-02 | C002       | Bob Jones    | bob@email.com   | P001      | Laptop      | 1200      | 1        |
| 1002                                         | 2024-05-02 | C002       | Bob Jones    | bob@email.com   | P003      | Keyboard    | 45        | 1        |



### Problems (Anomalies)

- **Update Anomaly** – Customer email stored multiple times
- **Insert Anomaly** – Cannot insert a new customer without an order
- **Delete Anomaly** – Deleting an order may remove customer or product info

### Functional Dependencies in Unnormalized Table

OrderID → OrderDate, CustomerID  
CustomerID → CustomerName, CustomerEmail  
ProductID → ProductName, UnitPrice  
(OrderID, ProductID) → Quantity

### Legend

- PK – Primary Key
- FD – Functional Dependency
- 1NF – First Normal Form
- 2NF – Second Normal Form
- 3NF – Third Normal Form
- BCNF – Boyce-Codd Normal Form

1NF

First Normal Form

Eliminate repeating groups.

Make each field atomic.

ORDERS\_1NF

| OrderID | OrderDate  | CustomerID |
|---------|------------|------------|
| 1001    | 2024-05-01 | C001       |
| 1001    | 2024-05-01 | C001       |
| 1002    | 2024-05-02 | C002       |
| 1002    | 2024-05-02 | C002       |

ORDER\_ITEMS\_1NF

| OrderID | ProductID | ProductName | UnitPrice | Quantity |
|---------|-----------|-------------|-----------|----------|
| 1001    | P001      | Laptop      | 1200      | 1        |
| 1001    | P002      | Mouse       | 25        | 2        |
| 1002    | P001      | Laptop      | 1200      | 1        |
| 1002    | P003      | Keyboard    | 45        | 1        |

 PKs: ORDERS\_1NF(OrderID)

ORDER\_ITEMS\_1NF(OrderID, ProductID)



PKs: ORDERS\_1NF(OrderID)  
ORDER\_ITEMS\_1NF(OrderID, ProductID)

## 2NF

### Second Normal Form

Remove partial dependencies.  
Every non-key attribute depends on the whole primary key.

#### ORDERS

| OrderID | OrderDate  | CustomerID |
|---------|------------|------------|
| 1001    | 2024-05-01 | C001       |
| 1002    | 2024-05-02 | C002       |

#### ORDER\_ITEMS\_2NF

| OrderID | ProductID | UnitPrice | Quantity |
|---------|-----------|-----------|----------|
| 1001    | P001      | 1200      | 1        |
| 1001    | P002      | 25        | 2        |
| 1002    | P001      | 1200      | 1        |
| 1002    | P003      | 45        | 1        |

#### PRODUCTS\_2NF

| ProductID | ProductName |
|-----------|-------------|
| P001      | Laptop      |
| P002      | Mouse       |
| P003      | Keyboard    |



**PKs:**  
ORDERS(OrderID)  
ORDER\_ITEMS\_2NF(OrderID, ProductID)  
PRODUCTS\_2NF(ProductID)



PKs:  
ORDERS(OrderID)  
ORDER\_ITEMS\_2NF(OrderID, ProductID)  
PRODUCTS\_2NF(ProductID)

### 3NF

#### Third Normal Form

Remove transitive dependencies.  
Non-key attributes should depend only on the primary key.

#### ORDERS

| OrderID | OrderDate  | CustomerID |
|---------|------------|------------|
| 1001    | 2024-05-01 | C001       |
| 1002    | 2024-05-02 | C002       |

#### ORDER\_ITEMS

| OrderID | ProductID | UnitPrice | Quantity |
|---------|-----------|-----------|----------|
| 1001    | P001      | 1200      | 1        |
| 1001    | P002      | 25        | 2        |
| 1002    | P001      | 1200      | 1        |
| 1002    | P003      | 45        | 1        |

#### CUSTOMERS

| CustomerID | CustomerName | CustomerEmail   |
|------------|--------------|-----------------|
| C001       | Alice Smith  | alice@email.com |
| C002       | Bob Jones    | bob@email.com   |

#### PRODUCTS

| ProductID | ProductName | UnitPrice |
|-----------|-------------|-----------|
| P001      | Laptop      | 1200      |
| P002      | Mouse       | 25        |
| P003      | Keyboard    | 45        |

**PKs:**  
ORDERS(OrderID)  
ORDER\_ITEMS(OrderID, ProductID)  
CUSTOMERS(CustomerID)  
PRODUCTS(ProductID)



PKs:  
ORDERS(OrderID)  
ORDER\_ITEMS\_3NF(OrderID, ProductID)  
CUSTOMERS(CustomerID)  
PRODUCTS(ProductID)

BCNF

Boyce-Codd Normal Form

For every determinant,  
it must be a candidate key.  
(Stronger than 3NF)

ORDERS

| OrderID | OrderDate  | CustomerID |
|---------|------------|------------|
| 1001    | 2024-05-01 | C001       |
| 1002    | 2024-05-02 | C002       |

ORDER\_ITEMS

| OrderID | ProductID | UnitPrice | Quantity |
|---------|-----------|-----------|----------|
| 1001    | P001      | 1200      | 1        |
| 1001    | P002      | 25        | 2        |
| 1002    | P001      | 1200      | 1        |
| 1002    | P003      | 45        | 1        |

CUSTOMERS

| CustomerID | CustomerName | CustomerEmail   |
|------------|--------------|-----------------|
| C001       | Alice Smith  | alice@email.com |
| C002       | Bob Jones    | bob@email.com   |

PRODUCTS

| ProductID | ProductName | UnitPrice |
|-----------|-------------|-----------|
| P001      | Laptop      | 1200      |
| P002      | Mouse       | 25        |
| P003      | Keyboard    | 45        |

PKs:

ORDERS(OrderID)

ORDER\_ITEMS(OrderID, ProductID)

CUSTOMERS(CustomerID)

PRODUCTS(ProductID)



PKs:  
ORDERS(OrderID)  
ORDER\_ITEMS\_3NF(OrderID, ProductID)  
CUSTOMERS(CustomerID)  
PRODUCTS(ProductID)

### Normal Forms Summary

- 1NF – Atomic values, no repeating groups
- 2NF – 1NF + no partial dependency
- 3NF – 2NF + no transitive dependency
- BCNF – Every determinant is a candidate key

### Benefits of Normalization

- ✓ Eliminates Redundancy
- ✓ Improves Data Integrity
- ✓ Reduces Anomalies
- ✓ Saves Storage Space
- ✓ Easier Maintenance
- ✓ Better Data Consistency



Redundant  
Unreliable  
Inconsistent



Normalized  
Reliable  
Consistent

### Best Practices

- Identify entities and their relationships.
- Choose meaningful primary keys.
- Remove partial and transitive dependencies.
- Normalize up to 3NF (BCNF when required).
- Denormalize only when performance demands and after careful analysis.



# Why we need Normalization in DBMS?

---

- Eliminating redundant(useless) data, therefore handling data integrity, because if data is repeated it increases the chances of inconsistent data.
- Normalization helps in keeping data consistent by storing the data in one table and referencing it everywhere else.
- Storage optimization although that is not an issue these days because Database storage is cheap.
- Breaking down large tables into smaller tables with relationships, so it makes the database structure more scalable and adaptable.
- Ensuring data dependencies make sense i.e. data is logically stored.

# Why we need Normalization in DBMS?

---

| rollno | name | branch | hod   | office_tel |
|--------|------|--------|-------|------------|
| 401    | Akon | CSE    | Mr. X | 53337      |
| 402    | Bkon | CSE    | Mr. X | 53337      |
| 403    | Ckon | CSE    | Mr. X | 53337      |
| 404    | Dkon | CSE    | Mr. X | 53337      |

# Why we need Normalization in DBMS?

---

- Insertion Anomaly in DBMS
  - Suppose for a new admission, until and unless a student opts for a branch, data of the student cannot be inserted, or else we will have to set the branch information as NULL.
  - Also, if we must insert data for 100 students of the same branch, then the branch information will be repeated for all those 100 students.
  - These scenarios are nothing but Insertion anomalies.
  - If you must repeat the same data in every row of data, it's better to keep the data separately and reference that data in each row.
  - So, in the above table, we can keep the branch information separately, and just use the branch\_id in the student table, where branch\_id can be used to get the branch information.



# Why we need Normalization in DBMS?

---

- Updating Anomaly in DBMS
  - What if Mr. X leaves the college? or Mr. X is no longer the HOD of the computer science department?
  - In that case, all the student records will have to be updated, and if by mistake we miss any record, it will lead to data inconsistency.
  - This is an Updating anomaly because you need to update all the records in your table just because one piece of information got changed.

# Why we need Normalization in DBMS?

---

- Deletion Anomaly in DBMS
  - In our Student table, two different pieces of information are kept together, the Student information and the Branch information.
  - So, if only a single student is enrolled in a branch, and that student leaves the college, or for some reason, the entry for the student is deleted, we will lose the branch information too.
  - So never in DBMS, we should keep two different entities together, which in the above example is Student and branch,

# Primary Key and Non-key attributes

Primary Key

Non-key attributes

| student_id | student_name | mobile     | gender |
|------------|--------------|------------|--------|
| 1          | John         | 9797979797 | Male   |
| 2          | Ron          | 7878787878 | Male   |
| 3          | Pom          | 8282828282 | Female |

The diagram illustrates a database table with four attributes: student\_id, student\_name, mobile, and gender. The student\_id attribute is identified as the Primary Key, indicated by an arrow pointing to it from the label 'Primary Key'. The other three attributes (student\_name, mobile, and gender) are collectively identified as Non-key attributes, indicated by an arrow pointing to them from the label 'Non-key attributes'.





# Types of DBMS Normal forms

---

- Normalization rules are divided into the following normal forms:
  - First Normal Form
  - Second Normal Form
  - Third Normal Form
  - BCNF
  - Fourth Normal Form
  - Fifth Normal Form



## First Normal Form (1NF)

---

- For a table to be in the First Normal Form, it should follow the following 4 rules:
  - It should only have single(atomic) valued attributes/columns.
  - Values stored in a column should be of the same domain.
  - All the columns in a table should have unique names.
  - And the order in which data is stored should not matter.

# First Normal Form (1NF)

---

If we have an Employee table in which we store the employee information along with the employee skillset, the table will look like this:

| emp_id | emp_name     | emp_mobile | emp_skills            |
|--------|--------------|------------|-----------------------|
| 1      | John Tick    | 9999957773 | Python, JavaScript    |
| 2      | Darth Trader | 8888853337 | HTML, CSS, JavaScript |
| 3      | Rony Shark   | 7777720008 | Java, Linux, C++      |



## First Normal Form (1NF)

---

- emp\_skills column holds multiple comma-separated values, while as per the First Normal form, each column should have a single value.
- Remove the emp\_skills column from the Employee table and keep it in some other table.
- Or add multiple rows for the employee and each row is linked with one skill

# Create Separate tables for Employee and Employee Skills

---



So the **Employee** table will look like this,

| emp_id | emp_name     | emp_mobile |
|--------|--------------|------------|
| 1      | John Tick    | 9999957773 |
| 2      | Darth Trader | 8888853337 |
| 3      | Rony Shark   | 7777720008 |

# Create Separate tables for Employee and Employee Skills



And the new **Employee\_Skill** table:

| emp_id | emp_skill  |
|--------|------------|
| 1      | Python     |
| 1      | JavaScript |
| 2      | HTML       |
| 2      | CSS        |
| 2      | JavaScript |
| 3      | Java       |
| 3      | Linux      |
| 3      | C++        |



# Add Multiple rows for Multiple skills

---

| emp_id | emp_name     | emp_mobile | emp_skill  |
|--------|--------------|------------|------------|
| 1      | John Tick    | 9999957773 | Python     |
| 1      | John Tick    | 9999957773 | JavaScript |
| 2      | Darth Trader | 8888853337 | HTML       |
| 2      | Darth Trader | 8888853337 | CSS        |
| 2      | Darth Trader | 8888853337 | JavaScript |
| 3      | Rony Shark   | 7777720008 | Java       |
| 3      | Rony Shark   | 7777720008 | Linux      |
| 3      | Rony Shark   | 7777720008 | C++        |



## Second Normal Form (2NF)

---

- For a table to be in the Second Normal Form,
  - It should be in the First Normal form.
  - And, it should not have Partial Dependency.





## What is Partial Dependency?

---

- When a table has a primary key that is made up of two or more columns, then all the columns(not included in the primary key) in that table should depend on the entire primary key and not on a part of it.
- If any column(which is not in the primary key) depends on a part of the primary key, then we say we have Partial dependency in the table.

# What is Partial Dependency?

---

**Student** table:

| student_id | student_name | branch     |
|------------|--------------|------------|
| 1          | Akon         | CSE        |
| 2          | Bkon         | Mechanical |

**Subject** Table:

| subject_id | subject_name     |
|------------|------------------|
| 1          | C Language       |
| 2          | DSA              |
| 3          | Operating System |

# What is Partial Dependency?

---

And we have another table **Score** to store the marks scored by students in any subject like this,

| student_id | subject_id | marks | teacher_name |
|------------|------------|-------|--------------|
| 1          | 1          | 70    | Miss. C      |
| 1          | 2          | 82    | Mr. D        |
| 2          | 1          | 65    | Mr. Op       |



## What is Partial Dependency?

---

- Now in the above table, the primary key is `student_id` + `subject_id`, because both these information are required to select any row of data.
- But in the Score table, we have a column `teacher_name`, which depends on the subject information or just the `subject_id`, so we should not keep that information in the Score table.
- The column `teacher_name` should be in the Subjects table. And then the entire system will be Normalized as per the Second Normal Form.

# What is Partial Dependency?

---

Updated **Subject** table:

| subject_id | subject_name     | teacher_name |
|------------|------------------|--------------|
| 1          | C Language       | Miss. C      |
| 2          | DSA              | Mr. D        |
| 3          | Operating System | Mr. Op       |

Updated **Score** table:

| student_id | subject_id | marks |
|------------|------------|-------|
| 1          | 1          | 70    |
| 1          | 2          | 82    |
| 2          | 1          | 65    |



## Third Normal Form (3NF)

---

- A table is said to be in the Third Normal Form when,
  - It satisfies the First Normal Form and the Second Normal form.
  - And, it doesn't have Transitive Dependency.



# What is Transitive Dependency?

---

- In a table we have some column that acts as the primary key and other columns depends on this column.
- But what if a column that is not the primary key depends on another column that is also not a primary key or part of it?
- Then we have Transitive dependency in our table.

# What is Transitive Dependency?

---

The **Score** table will look like this,

| student_id | subject_id | marks | exam_type | total_marks |
|------------|------------|-------|-----------|-------------|
| 1          | 1          | 70    | Theory    | 100         |
| 1          | 2          | 82    | Theory    | 100         |
| 2          | 1          | 42    | Practical | 50          |



## What is Transitive Dependency?

---

- In the table above, the column `exam_type` depends on both `student_id` and `subject_id`, because,
- a student can be in the CSE branch or the Mechanical branch and based on that they may have different exam types for different subjects.
- The CSE students may have both Practical and Theory for Compiler Design, whereas Mechanical branch students may only have Theory exams for Compiler Design.
- But the column `total_marks` just depends on the `exam_type` column.
- And the `exam_type` column is not a part of the primary key.
- Because the primary key is `student_id + subject_id`, hence we have a Transitive dependency here.

# What is Transitive Dependency?

---

New **ExamType** table,

| exam_type_id | exam_type | total_marks | duration |
|--------------|-----------|-------------|----------|
| 1            | Practical | 50          | 45       |
| 2            | Theory    | 100         | 180      |
| 3            | Workshop  | 150         | 300      |



## BCNF

---

- Boyce and Codd Normal Form is a higher version of the Third Normal Form.
- This form deals with a certain type of anomaly that is not handled by 3NF.
- A 3NF table that does not have multiple overlapping candidate keys is said to be in BCNF.

## BCNF

---

- For a table to be in BCNF, the following conditions must be satisfied:
- R must be in the 3rd Normal Form
- and, for each functional dependency (  $X \rightarrow Y$  ), X should be a Super Key.

# BCNF

---



Below we have a college enrolment table with columns `student_id`, `subject` and `professor`.

| <b>student_id</b> | <b>subject</b> | <b>professor</b> |
|-------------------|----------------|------------------|
| 101               | Java           | P.Java           |
| 101               | C++            | P.Cpp            |
| 102               | Java           | P.Java2          |
| 103               | C#             | P.Chash          |
| 104               | Java           | P.Java           |

- One student can enrol for multiple subjects. For example, student with student\_id 101, has opted for subjects - Java & C++
- For each subject, a professor is assigned to the student.
- And, there can be multiple professors teaching one subject like we have for Java.

# BCNF

---



- In the table above, `student_id`, `subject` form primary key, which means `subject` column is a prime attribute.
- But there is one more dependency, `professor`  $\rightarrow$  `subject`.
- And while `subject` is a prime attribute, `professor` is a non-prime attribute, which is not allowed by BCNF.

# How to satisfy BCNF?

---

## Student Table

| <b>student_id</b> | <b>p_id</b> |
|-------------------|-------------|
| 101               | 1           |
| 101               | 2           |
| and so on...      |             |

## And, **Professor Table**

| <b>p_id</b> | <b>professor</b> | <b>subject</b> |
|-------------|------------------|----------------|
| 1           | P.Java           | Java           |
| 2           | P.Cpp            | C++            |





## Fourth Normal Form (4NF)

---

- Fourth Normal Form comes into picture when Multi-valued Dependency occur in any relation.



## Rules for 4th Normal Form

---

- For a table to satisfy the Fourth Normal Form, it should satisfy the following two conditions:
  - It should be in the Boyce-Codd Normal Form.
  - And the table should not have any Multi-valued Dependency.

## What is Multi-valued Dependency?

---

- A table is said to have multi-valued dependency, if the following conditions are true,
  - For a dependency  $A \twoheadrightarrow B$ , if for a single value of A, multiple value of B exists, then the table may have multi-valued dependency.
  - Also, a table should have at-least 3 columns for it to have a multi-valued dependency.
  - And, for a relation  $R(A,B,C)$ , if there is a multi-valued dependency between, A and B, then B and C should be independent of each other.

# What is Multi-valued Dependency?

---

Below we have a college enrolment table with columns **s\_id**, **course** and **hobby**.

| <b>s_id</b> | <b>course</b> | <b>hobby</b> |
|-------------|---------------|--------------|
| 1           | Science       | Cricket      |
| 1           | Maths         | Hockey       |
| 2           | C#            | Cricket      |
| 2           | Php           | Hockey       |

# What is Multi-valued Dependency?

---

Well the two records for student with `s_id 1`, will give rise to two more records, as shown below, because for one student, two hobbies exists, hence along with both the courses, these hobbies should be specified.

| <b>s_id</b> | <b>course</b> | <b>hobby</b> |
|-------------|---------------|--------------|
| 1           | Science       | Cricket      |
| 1           | Maths         | Hockey       |
| 1           | Science       | Hockey       |
| 1           | Maths         | Cricket      |



# How to satisfy 4th Normal Form?

---

To make the above relation satisfy the 4th normal form, we can decompose the table into 2 tables.

## CourseOpted Table

| s_id | course  |
|------|---------|
| 1    | Science |
| 1    | Maths   |
| 2    | C#      |
| 2    | Php     |

# How to satisfy 4th Normal Form?

---

And, **Hobbies Table**,

| s_id | hobby   |
|------|---------|
| 1    | Cricket |
| 1    | Hockey  |
| 2    | Cricket |
| 2    | Hockey  |



## Fifth Normal Form (5NF)

---

- The fifth normal form is also called the PJNF - Project-Join Normal Form
- It is the most advanced level of Database Normalization.
- Using Fifth Normal Form you can fix Join dependency and reduce data redundancy.
- It also helps in fixing Update anomalies in DBMS design.



# Denormalization

---

- Denormalization means combining tables or duplicating data to improve read/query performance.

| OrderID | CustomerName | CustomerPhone | ProductName | Price |
|---------|--------------|---------------|-------------|-------|
| 1       | John         | 9999999999    | Laptop      | 50000 |
| 2       | John         | 9999999999    | Mouse       | 500   |

# Denormalization

---

- This is denormalized because customer data is repeated.
- Benefits of denormalization:
  - faster reads
  - fewer joins
  - simpler reporting queries
  - useful in analytics/data warehouse systems
- Disadvantages:
  - duplicate data
  - more storage
  - harder updates
  - risk of inconsistent data

# Comparison

---



| Feature                  | Normalization       | Denormalization        |
|--------------------------|---------------------|------------------------|
| Data duplication         | Low                 | High                   |
| Storage usage            | Less                | More                   |
| Read performance         | Slower due to joins | Faster                 |
| Write/update performance | Better consistency  | More difficult         |
| Best for                 | OLTP systems        | OLAP/reporting systems |

# Comparison

---

- Examples:
  - Banking systems usually prefer normalization
  - Reporting dashboards often prefer denormalization
  - Data warehouses commonly use denormalized star schemas
- A simple rule:
  - Normalize for transaction systems
  - Denormalize for reporting and analytics



## CosmosDB/MongoDB schema patterns;

---

- In Azure Cosmos DB and MongoDB, schema design is usually based on document patterns rather than strict normalization.
- Because these databases store JSON-like documents, the main question is whether to:
  - embed related data inside the same document
  - reference related data across documents
  - duplicate selected fields for performance

# CosmosDB/MongoDB schema patterns;



## Cosmos DB / MongoDB Schema Patterns

Different data modeling patterns for Cosmos DB (MongoDB API) to fit various access patterns and scalability needs.

🔑 PK = Partition Key    🌱 Good for Scale    🔍 Good for Queries    ⚡ High Performance    💰 Cost Effective

### 1. Hierarchical (Embedded)

Embed related data in a single document



```
{
 "_id": "order_123",
 "customerId": "cust_1",
 "customer": { "name": "John", "email": "john@mail.com" },
 "items": [
 { "productId": "p1", "name": "Laptop", "qty": 1, "price": 1000 },
 { "productId": "p2", "name": "Mouse", "qty": 2, "price": 25 }],
 "shipping": { "address": "1 Main St", "city": "NY", "zip": "10001" },
 "orderDate": ISODate("2024-05-01T00:00:00Z"),
 "total": 1050
}
```

- Good for one-to-few relationships
- 🔍 Read entire aggregate in one query
- 💰 Lower RU cost

Use when: Data is tightly related and read together.

### 2. Root-Level (Referenced)

Store related entities as separate documents



```
// Orders
{ "_id": "order_123",
 "customerId": "cust_1",
 "orderDate": ISODate("2024-05-01"),
 "total": 1050 }

// Customers
{ "_id": "cust_1",
 "name": "John",
 "email": "john@mail.com" }

// Products
{ "_id": "p1", "name": "Laptop", "price": 1000 }
```

- Good for many-to-many relationships
- 🔍 Query entities independently
- ⚡ High write flexibility

Use when: Entities are reused across many documents.

### 3. Bucket Pattern (Time Series)

Group time-series data into buckets



```
{
 "_id": "sensorA_2024-05-01",
 "deviceId": "sensorA",
 "date": ISODate("2024-05-01"),
 "readings": [
 { "ts": ISODate("2024-05-01T00:00:00Z"), "v": 23.4 },
 { "ts": ISODate("2024-05-01T00:01:00Z"), "v": 23.2 },
 ...
]
}
```

- Great for high-frequency time-series
- ⚡ Reduces storage & RU consumption
- 🔍 Efficient range queries

Use when: Ingesting large time-series data.

### 4. Computed Pattern

Pre-compute and store query results



```
// Orders
{ "_id": "order_123",
 "customerId": "cust_1",
 "total": 1050,
 "orderDate": ISODate("2024-05-01") }

// OrdersByCustomer (computed)
{ "_id": "cust_1",
 "customerId": "cust_1",
 "orders": [
 { "orderId": "order_123", "total": 1050,
 "orderDate": ISODate("2024-05-01") }],
 "orderCount": 1, "totalAmount": 1050 }
```

- Fast read performance
- 💰 Reduces complex joins in application
- ⚡ Extra writes for maintaining views

Use when: Read performance is critical.

### 5. Polymorphic Pattern

Store different entity types in one collection



```
{
 "_id": "p1",
 "type": "product", "name": "Laptop", "price": 1000 }

{
 "_id": "order_123", "type": "order",
 "customerId": "cust_1", "total": 1050 }

{
 "_id": "cust_1", "type": "customer",
 "name": "John", "email": "john@mail.com" }
```

- Flexible schema
- 🔍 Query by type
- 💰 Cost effective (single collection)

Use when: Different entity types share similar access patterns.

### Partition Key (PK) Best Practices



- ✓ Choose a PK with high cardinality
- ✓ Distribute data evenly across partitions
- ✓ Avoid monotonically increasing keys
- ✓ Keep partition size between 10 GB - 20 GB
- ✓ Include PK in your queries

#### Good Examples

- userid
- tenantid
- deviceId
- orderId

#### Avoid Examples

- status (e.g. Active/Inactive)
- country (low cardinality)
- timestamp (monotonic)

### Common Query Patterns



- 🔍 **Point Read** Get item by id and partition key
- 🔍 **Range Query** Query by date, range, or numeric fields
- 🔍 **Aggregations** Count, sum, min, max, avg
- 🔍 **Paging** Use continuation tokens



### Design Tips

- ✓ Model for your most important queries
- ✓ Embed when data is read together
- ✓ Reference when data is shared or large
- ✓ Use indexing policies to optimize RU
- ✓ Test with realistic data & workloads
- ✓ Monitor RU, latency, and partitioning



**Goal:** Right schema + right partition key = High performance | Low latency | Scalable | Cost optimized

# Common Schema Patterns

- **1. Embed Pattern**
- Store related data inside a single document.

```
{
 "orderId": 1001,
 "customer": {
 "customerId": 1,
 "name": "John",
 "email": "john@example.com"
 },
 "items": [
 {
 "productId": 10,
 "productName": "Laptop",
 "quantity": 1,
 "price": 50000
 },
 {
 "productId": 11,
 "productName": "Mouse",
 "quantity": 2,
 "price": 500
 }
]
}
```

# Common Schema Patterns

---

- Use embed when:
  - child data is always accessed with parent
  - one-to-few relationship
  - low update frequency
  - limited document size
- Advantages:
  - fast reads
  - fewer queries
  - no joins/lookups needed
- Disadvantages:
  - duplicated data
  - large document growth
  - harder updates across multiple documents



# Common Schema Patterns

---

- **2. Reference Pattern**
- Store related entities separately and link them using IDs.

```
{
 "orderId": 1001,
 "customerId": 1,
 "itemIds": [10, 11]
}
```

```
{
 "customerId": 1,
 "name": "John",
 "email": "john@example.com"
}
```



# Common Schema Patterns

---

- Use reference when:
  - many-to-many relationships
  - child data changes frequently
  - data reused in many places
  - large collections
- Advantages:
  - less duplication
  - easier updates
  - smaller documents
- Disadvantages:
  - more queries
  - joins/lookups required
  - slower reads

# Common Schema Patterns

---

- **3. Hybrid Pattern**
- Keep important read-heavy fields embedded while still storing references.

```
{
 "orderId": 1001,
 "customerId": 1,
 "customerName": "John",
 "customerPhone": "9999999999",
 "items": [
 {
 "productId": 10,
 "productName": "Laptop",
 "price": 50000
 }
]
}
```



# Common Schema Patterns

---

- **3. Hybrid Pattern**

- Use hybrid when:
  - some fields are needed frequently
  - full related entity is rarely needed
  - read performance matters
  - This is one of the most common patterns in MongoDB and Azure Cosmos DB.

# Common Schema Patterns

---

- **Specialized Patterns**
- **Bucket Pattern**
- Group time-series or event data into buckets.

```
{
 "deviceId": "D100",
 "month": "2026-04",
 "readings": [
 { "time": "2026-04-01T10:00:00Z", "temp": 28 },
 { "time": "2026-04-01T11:00:00Z", "temp": 29 }
]
}
```



# Common Schema Patterns

---

- **Specialized Patterns**
- Use for:
  - IoT
  - logs
  - sensor readings
  - time-series workloads



# Common Schema Patterns

---

- **Specialized Patterns**
- **Outlier Pattern**
- Store most related items embedded but move unusually large collections to separate documents.
- Example:
  - most customers have 5 orders embedded
  - customers with 10,000 orders store them separately
  - Useful for preventing very large documents.

# Common Schema Patterns

---

- **Specialized Patterns**
- **Subset Pattern**
- Keep frequently used fields in the main document and rarely used fields in a separate document.

```
{
 "productId": 1,
 "name": "Laptop",
 "price": 50000,
 "shortDescription": "Gaming laptop"
}
```

```
{
 "productId": 1,
 "specifications": {
 "cpu": "i9",
 "ram": "32GB",
 "storage": "1TB SSD"
 },
 "reviews": [...]
}
```



# Common Schema Patterns

---

- **Specialized Patterns**
- **Computed Pattern**
- Store precomputed values to improve read speed.

```
{
 "customerId": 1,
 "totalOrders": 50,
 "totalSpent": 250000
}
```



# Partition Key Considerations in Azure Cosmos DB

---

- Partition key selection is critical.
- Good partition keys:
  - high cardinality
  - evenly distributed
  - commonly used in queries
- Examples:
  - customerId
  - tenantId
  - region
  - deviceId
- Bad partition keys:
  - country with only few values
  - status with low uniqueness
  - current month only

# When to embed vs Reference

---

| Scenario                    | Preferred Pattern   |
|-----------------------------|---------------------|
| Customer and address        | Embed               |
| Customer and orders         | Reference or Hybrid |
| Product and product reviews | Reference           |
| Small order items           | Embed               |
| User profile and settings   | Embed               |
| Large event history         | Bucket              |
| Dashboard totals            | Computed            |



## Azure SQL design

---

- **1) First choose the right Azure SQL deployment model**
- **Azure SQL Database** is the default choice for most new cloud applications.
  - It is a fully managed PaaS database with built-in HA, backups, patching, and scaling options.
  - It supports single databases, elastic pools, serverless, Business Critical, and Hyperscale models.



## Azure SQL design

---

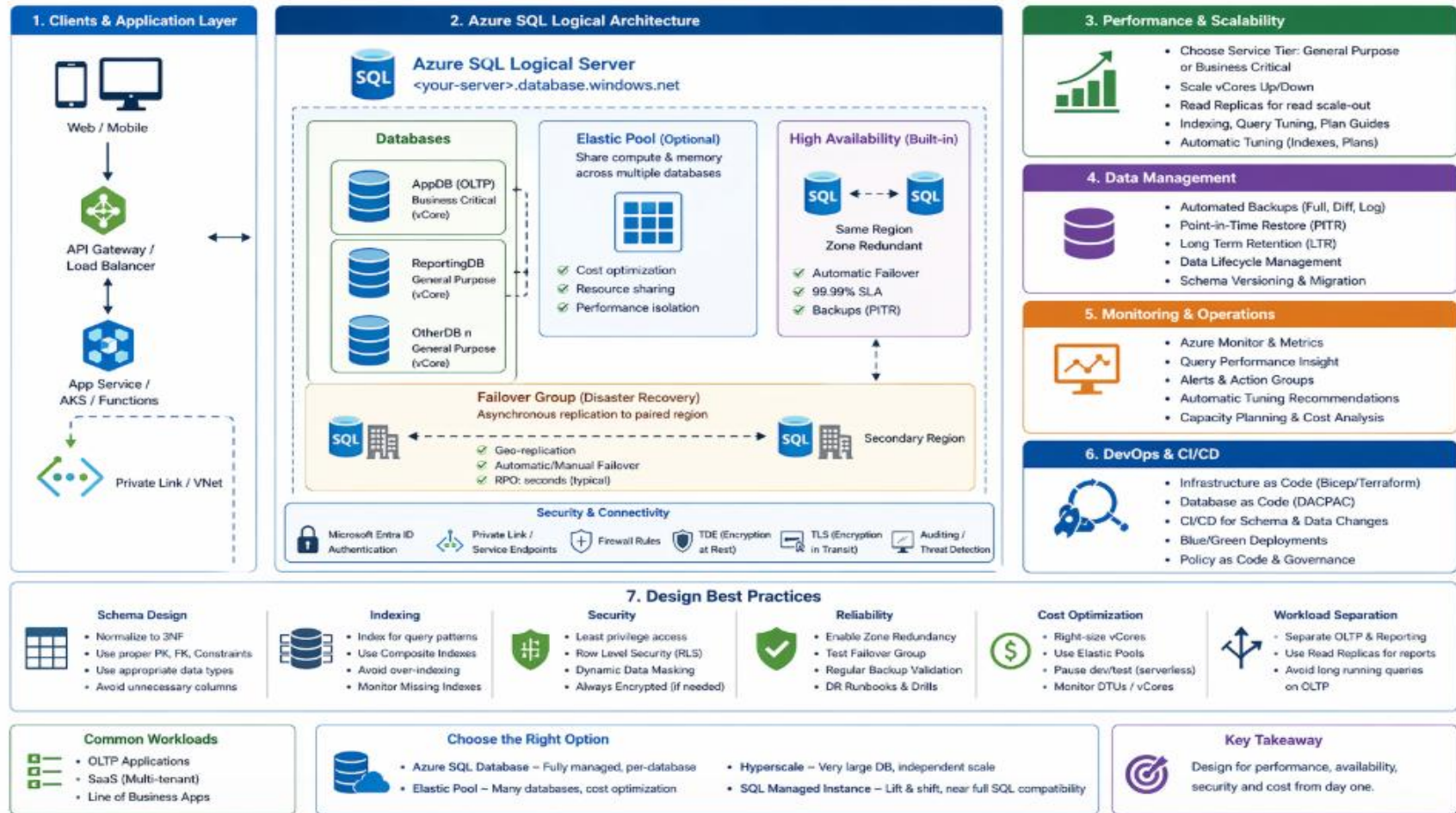
- **Azure SQL Managed Instance** is better when you need higher SQL Server compatibility or a migration path with fewer application and operational changes.
- Microsoft describes it as offering close to full SQL Server feature compatibility and operating inside a customer VNet-oriented architecture.
- A simple rule:
  - **Cloud-native new app** → Azure SQL Database.
  - **Lift-and-shift from SQL Server with compatibility needs** → Managed Instance.

# Azure SQL design



## Azure SQL Design – Reference Architecture

Highly Available | Secure | Scalable | Cost Optimized





# Azure SQL design

---

- **2) Choose the right scaling model**
- For **one important application database**, use a **single Azure SQL Database**.
- For **many small or medium tenant databases with bursty usage**, use **elastic pools**.
- Microsoft explicitly positions elastic pools as cost-effective for multiple databases with varying and unpredictable usage and notes they help deliver performance elasticity while sharing resources.
- For **very large OLTP databases** or where you want **independent compute and storage scaling**, evaluate **Hyperscale**.
- Microsoft's current service-tier guidance highlights architectural separation and large-scale design benefits in the vCore model, and Hyperscale remains the large-scale Azure SQL Database option.



# Azure SQL design

---

- **3) Pick the service tier by workload**
- A good design mapping is:
- **General Purpose** for typical business applications. Microsoft notes this tier uses separated compute and storage with storage-backed resilience.
- **Business Critical** when you need lower latency and stronger transactional performance characteristics. This is part of the current Azure SQL Database tier model.
- **Serverless** for intermittent workloads where automatic scaling and pause/resume economics fit the usage pattern. Azure SQL Database supports serverless as part of its managed performance models.
- **Hyperscale** for very large databases and rapid growth.





# Azure SQL design

---

- **4) Design the schema like a normal relational OLTP system first**
- For transactional systems:
  - normalize core tables first
  - use proper **primary keys, foreign keys, unique constraints**
  - index for real access patterns, not every column
  - keep reporting or ETL-heavy workloads separate from the transactional store where possible.
- Azure's architecture guidance recommends choosing data-store patterns based on workload rather than forcing one store to do everything.
- For Azure SQL, Microsoft's Well-Architected guidance centers performance, reliability, and operational design around workload-driven decisions.



- **5) For SaaS, choose a tenant pattern deliberately**
- Microsoft documents several multitenant SaaS patterns for Azure SQL Database.
- The main ones are shared database/shared schema, shared database/separate schemas, and database-per-tenant.
- In practice:
- **Database per tenant** is usually best for stronger isolation, easier tenant restore, and cleaner noisy-neighbor control. It pairs well with **elastic pools**.
- **Shared database** works when cost efficiency is more important and you can enforce tenant isolation carefully in schema, indexes, and application queries.



# Azure SQL design

---

- **6) Plan network design early**
- Azure SQL Database and Managed Instance differ a lot in connectivity architecture.
- Azure SQL Database uses logical server connectivity policies and has documented architecture for traffic routing and client connection behavior.
- Managed Instance has a more isolated architecture and runs in its own networking model.
- A practical design is:
  - keep app and database in Azure when possible
  - use private connectivity where your security model requires it
  - avoid treating network/security as an afterthought because it affects migration design, latency, and operations.



# Azure SQL design

---

- **7) Build HA and DR into the design**
- Azure SQL Database already has built-in high availability as a managed service.
- For stronger resilience, Microsoft recommends **zone redundancy** where available.
- For cross-region disaster recovery, use **failover groups**.
- Microsoft states failover groups manage replication and failover of some or all databases on a logical server to another server in another region;
- for Managed Instance, failover groups handle coordinated geo-replication of user databases to another instance in another region.
- So, a strong production design is:
- **same-region HA** → built-in HA + zone redundancy if supported and required.
- **cross-region DR** → failover group.



- **8) Security baseline for Azure SQL design**
- Your minimum production baseline should include:
  - Entra ID-based authentication where possible
  - least-privilege database roles
  - encryption at rest and in transit
  - auditing and threat monitoring
  - data classification and masking or row-level controls when needed.
- Microsoft's Well-Architected and Azure SQL security guidance emphasize secure identity, network isolation, monitoring, and platform-native protection features as core design decisions rather than add-ons.



# Azure SQL design

---

- **9) Recommended reference patterns**
- **Typical enterprise app**
  - App Service / AKS / VM application layer
  - Azure SQL Database single database
  - General Purpose or Business Critical depending on latency needs
  - zone redundancy if required
  - failover group for cross-region DR.



# Azure SQL design

---

- **9) Recommended reference patterns**
- **SaaS platform**
  - tenant catalog
  - database-per-tenant
  - elastic pools for cost control
  - automation for provisioning, schema rollout, and monitoring
  - failover group for business continuity.
- **SQL Server migration**
  - choose Managed Instance if you depend on instance-level compatibility and want fewer changes
  - choose Azure SQL Database if you can modernize and want simpler cloud-native operations.



- **10) Common design mistakes**
- Avoid these:
- choosing Managed Instance when a simpler Azure SQL Database design is enough
- putting many noisy tenants into one shared database without careful isolation
- using serverless for consistently busy workloads
- mixing analytics/reporting workloads directly into OLTP
- skipping zone redundancy and failover design until late in the project.



# Simple Design Decision Table

---

| Need                            | Best fit                          |
|---------------------------------|-----------------------------------|
| New cloud app                   | Azure SQL Database                |
| Many tenant DBs                 | Azure SQL Database + Elastic Pool |
| Very large fast-growing DB      | Azure SQL Database Hyperscale     |
| Higher SQL Server compatibility | Azure SQL Managed Instance        |
| Cross-region DR                 | Failover Group                    |
| Stronger zonal resilience       | Zone redundancy                   |

# Choosing stores for workloads

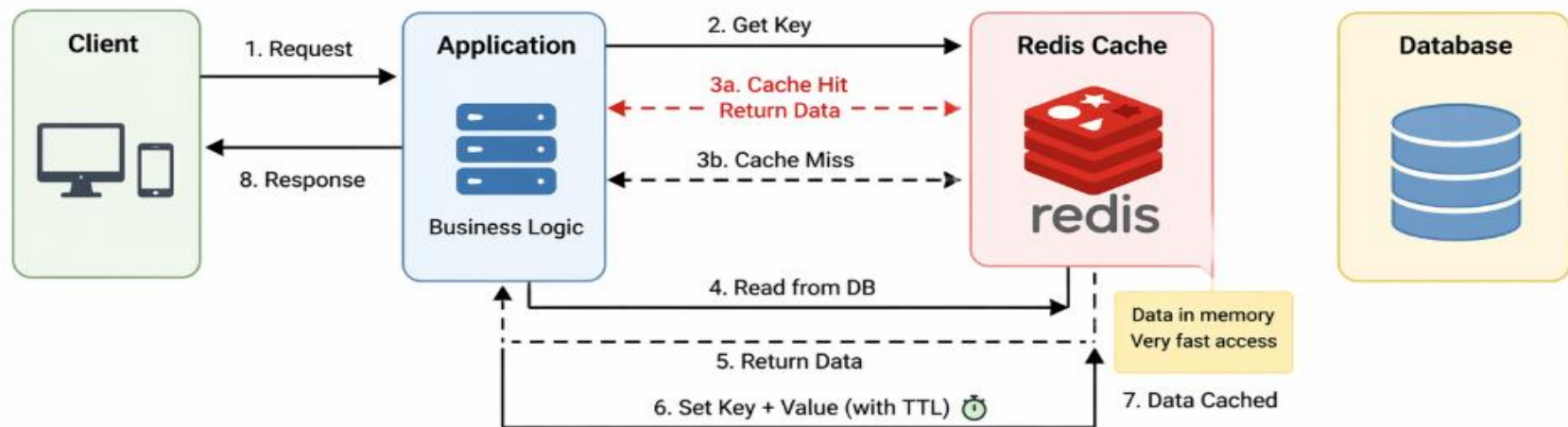
| Workload                      | Best store type  | Common Azure fit                 |
|-------------------------------|------------------|----------------------------------|
| Orders, billing, payments     | Relational       | Azure SQL Database               |
| Flexible app JSON             | Document         | Cosmos DB for NoSQL              |
| Session/cart/token lookup     | Key-value        | Azure Managed Redis              |
| Telemetry/logs/metrics        | Time-series      | Azure Data Explorer              |
| Fraud/social relationships    | Graph            | Cosmos DB Gremlin                |
| BI and historical reporting   | Analytical       | Fabric / ADX / Databricks        |
| Files, parquet, raw ingestion | Object/Data lake | Blob / ADLS                      |
| Full-text product/site search | Search           | Azure AI Search                  |
| Semantic retrieval / RAG      | Vector           | Azure AI Search / Cosmos options |

# Caching with Redis



## Caching with Redis – Cache-Aside Pattern

Application first checks Redis cache. On miss, it reads from the database and stores the result in Redis for future requests.



### Key Concepts

- Cache-Aside Pattern
- Read-through on demand
- Write/Invalidate on updates
- TTL for automatic expiration

### When to Use

- Read-heavy workloads
- Expensive DB queries
- Frequently accessed data
- Low latency requirements

### Benefits

- Reduced DB load
- Faster response times
- Improved scalability
- Cost effective

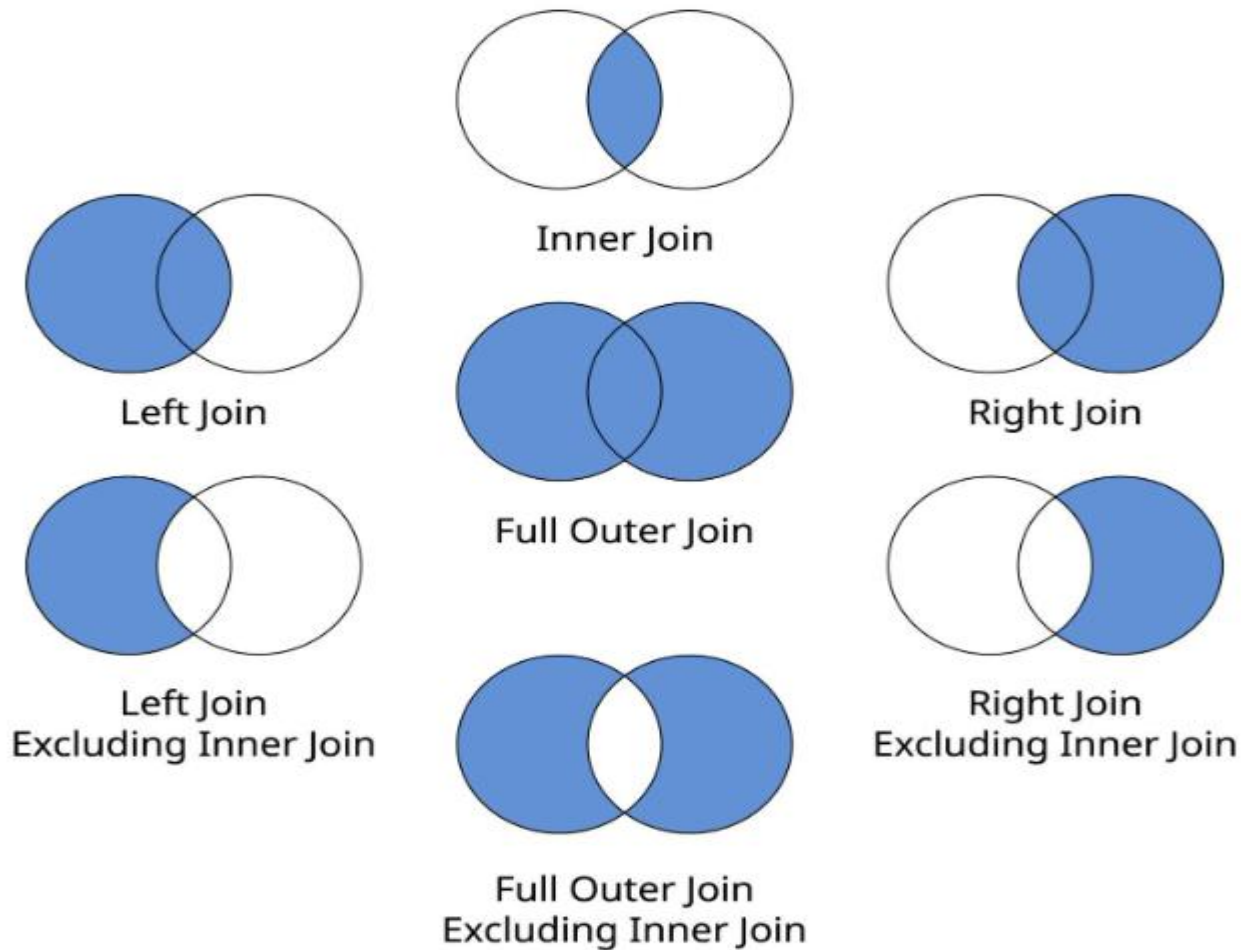
### Best Practices

- Use meaningful key names
- Set appropriate TTL
- Invalidate on data updates
- Handle cache failures gracefully

Legend: Solid line - Data Flow | Dashed line - Conditional Flow (Hit/Miss) | TTL - Time To Live (Expiration Time)

- Use joins based on the relationship between tables.
- **INNER JOIN** returns only matching rows from both tables.
- **LEFT JOIN** returns all rows from the left table and matching rows from the right table.
- **RIGHT JOIN** is the opposite of left join.
- **FULL OUTER JOIN** returns all rows from both sides.
- **CROSS JOIN** returns every possible combination.
- **SELF JOIN** joins a table to itself.

# Joins



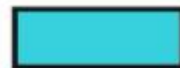
# Self Join



| ID | Name          | Phone_Number | Supervisor_ID |
|----|---------------|--------------|---------------|
| 1  | Raj Mehta     | 98540XXXXX   | 4             |
| 2  | Sanjay Mishra | 88888XXXXX   | 3             |
| 3  | Aditi Gupta   | 67809XXXXX   | 4             |
| 4  | Manish Chopra | 12345XXXXX   | 7             |



**Primary key**



**Foreign key**

**Result Table :**

| Supervisors   |
|---------------|
| Aditi Gupta   |
| Manish Chopra |

SCALER  
Topics

## Join

---

- ```
SELECT c.customer_name, o.order_id  
FROM Customers c  
INNER JOIN Orders o  
ON c.customer_id = o.customer_id;
```
- For performance:
 - Join on indexed columns
 - Prefer joining on numeric surrogate keys
 - Avoid functions on join columns
 - Filter early before joining large tables

Window Functions

- Window functions let you calculate values across related rows without collapsing the result set.
- Common functions:
 - ROW_NUMBER()
 - RANK()
 - DENSE_RANK()
 - LAG()
 - LEAD()
 - SUM() OVER()
 - AVG() OVER()

Aggregations

- Aggregations reduce many rows into summarized values.
- Common aggregation functions:
 - COUNT()
 - SUM()
 - AVG()
 - MIN()
 - MAX()
- Usually combined with GROUP BY.

Aggregations



Table: Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2

```
SELECT customer_id, AVG(amount) AS average_spends  
FROM Orders  
GROUP BY customer_id;
```

customer_id	average_spends
1	400
2	250
3	12000
4	350

Aggregations

```
SELECT Department, SUM(Salary) AS TotalSalary  
FROM Employee  
GROUP BY Department  
HAVING SUM(Salary) > 80000;
```

Employees

EmpID	Name	Department	Salary
1	Anni	HR	40000
2	Pooja	IT	55000
3	Raj	HR	45000
4	Sujan	IT	60000

Departments with total salary > 80000

Indexing Strategies

- **Indexing Strategies**
- Indexes improve read performance, but increase write cost.
- Main index types:
- **Clustered Index**
 - Determines physical row order
 - Usually on primary key
 - Only one per table
- **Non clustered Index**
 - Separate structure pointing to table rows
 - Can have many per table
 - Best for search, filter, sort, and join columns

Indexing Strategies

- **Composite Index**
 - Multiple columns in one index
 - Order matters
 - Useful for queries filtering on multiple columns
- **Covering Index**
 - Includes all columns needed by the query
 - Avoids extra table lookups

Indexing Strategies

- **Filtered Index**
 - Indexes only subset of rows
 - Useful for active records, status flags, recent data
- **Column store Index**
 - Best for analytics and aggregations on large tables
 - Very effective in data warehouses

Indexing Strategies

- A few practical rules:
 - Index columns used in WHERE, JOIN, ORDER BY, and GROUP BY
 - Avoid indexing very small tables
 - Avoid too many indexes on write-heavy tables
 - Put the most selective column first in a composite index
 - Review execution plans regularly
 - Remove unused or duplicate indexes
 - Use column store for reporting-heavy workloads

Execution plans

- **1. Execution Plans**
- Execution plan shows **how database executes a SQL query**.
- Example:
 - `EXPLAIN SELECT *
FROM customers
WHERE email = 'john@example.com';`
- For MySQL:
 - `EXPLAIN ANALYZE
SELECT *
FROM customers
WHERE email = 'john@example.com';`

Execution plans

Column	Meaning
type	Access type
key	Index used
rows	Estimated rows scanned
Extra	Extra operation details

Good signs:

key = idx_email
rows = small number

Bad signs:

type = ALL
key = NULL
type = ALL means full table scan.

2. Avoiding Full Table Scans

- Bad query:
 - `SELECT * FROM customers WHERE email = 'john@example.com';`
- If email has no index, DB scans all rows.
- Create index:
 - `CREATE INDEX idx_customers_email
ON customers(email);`
- Better query:
 - `SELECT customer_id, first_name, last_name, email
FROM customers
WHERE email = 'john@example.com';`
- Avoid:
 - `SELECT *`
- Use only required columns.

3. Avoid Functions on Indexed Columns

- Bad:
 - `SELECT *`
 `FROM orders`
 `WHERE YEAR(order_date) = 2026;`
- This may ignore index.
- Better:
 - `SELECT *`
 `FROM orders`
 `WHERE order_date >= '2026-01-01'`
 `AND order_date < '2027-01-01';`
- Index:
 - `CREATE INDEX idx_orders_order_date`
 `ON orders(order_date);`

4. Pagination

- Bad for large data:
- ```
SELECT *
FROM orders
ORDER BY order_id
LIMIT 10 OFFSET 100000;
```
- DB skips 100000 rows.
- Better: keyset pagination
- ```
SELECT *  
FROM orders  
WHERE order_id > 100000  
ORDER BY order_id  
LIMIT 10;
```
- For next page, pass last received order_id.

5. Parameterized Queries

- Bad:
- `email = input("Email: ")`
`query = f"SELECT * FROM customers WHERE email = '{email}'"`
- This can cause SQL injection.
- Good:
- `from sqlalchemy import text`

```
query = text("""  
SELECT *  
FROM customers  
WHERE email = :email  
""")
```

```
result = conn.execute(query, {"email": "john@example.com"})
```

- Parameterized queries are safer and reusable.

6. Connection Pooling

- Without pooling, every request creates a new DB connection.
- Better:
- `from sqlalchemy import create_engine`

```
engine = create_engine(
```

```
"mysql+pymysql://root:password@localhost:3306/customer  
db",  
    pool_size=5,  
    max_overflow=10,  
    pool_recycle=1800,  
    pool_pre_ping=True  
)
```

6. Connection Pooling

Option	Purpose
<code>pool_size</code>	Normal number of open connections
<code>max_overflow</code>	Extra temporary connections
<code>pool_recycle</code>	Recreate old connections
<code>pool_pre_ping</code>	Checks dead connection before use

Python Capabilities



Dynamically builds
web applications

01

Grants ease to build
multi-protocol
networks

02

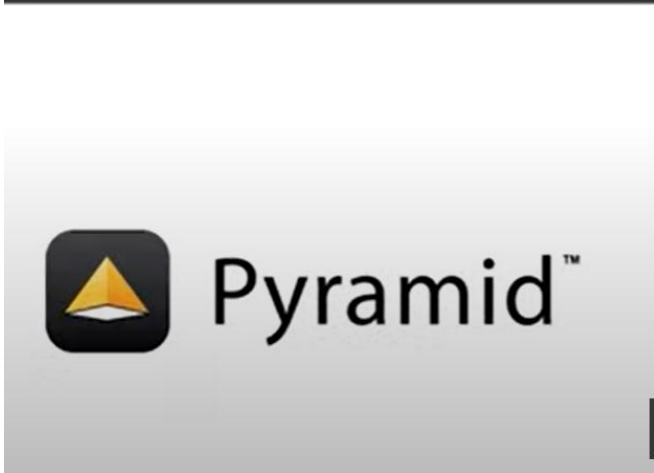
Provides quick
Processing

03

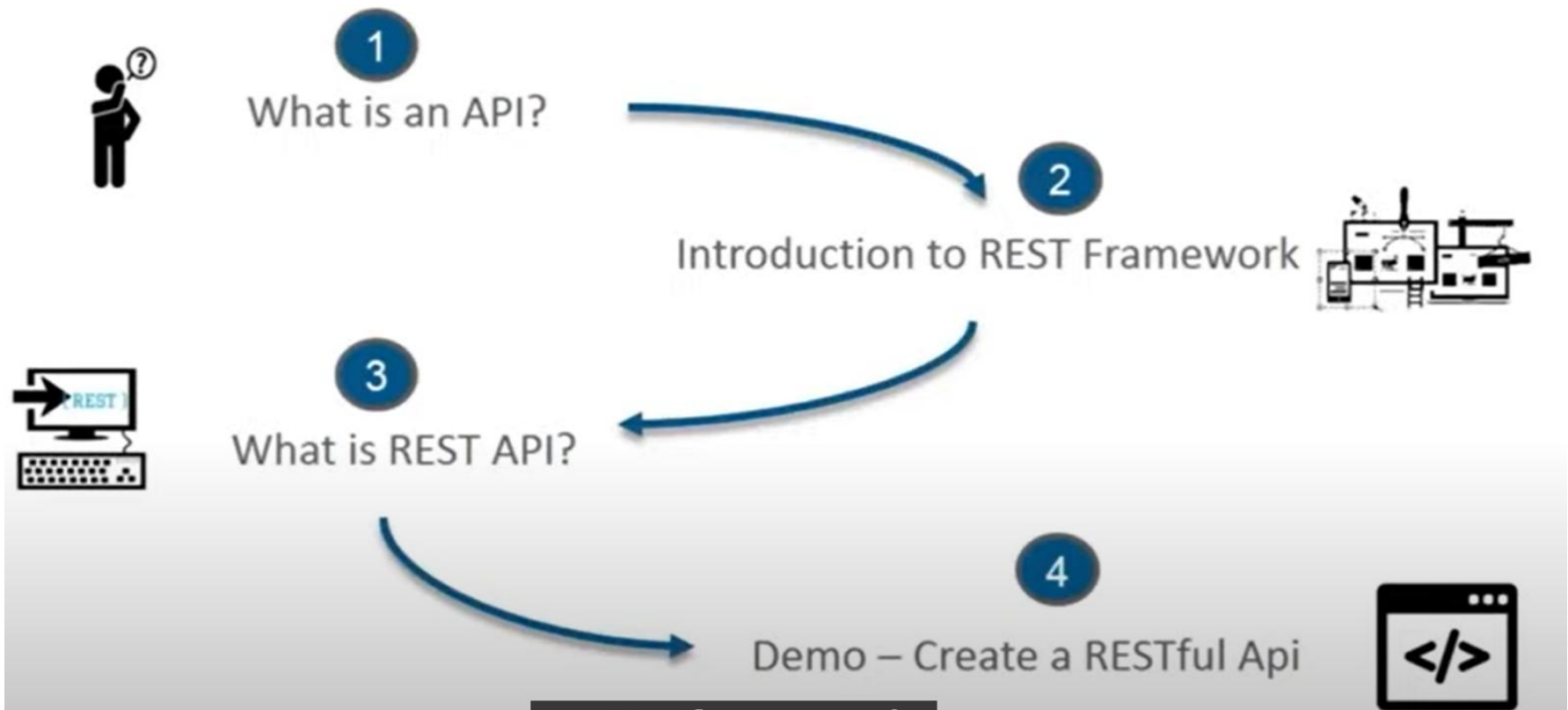
Python is an all in
one tool

04

Python Libraries For Web Development



REST API





-  **1. What is HTTP?**
- **HTTP = HyperText Transfer Protocol**
It's the language your browser, backend, and apps use to talk to each other.
- You send a request → you get a response.

2. HTTP Request Structure

- When the client wants something, it sends a **request** that looks like:
- ✓ **Request Line**
- GET /products HTTP/1.1
- ✓ **Headers**
- Content-Type: application/json
- Authorization: Bearer <token>
- User-Agent: Chrome
- ✓ **Body (optional)**
- Used in POST/PUT:
- {
- "name": "Laptop",
- "price": 50000
- }



3. HTTP Response Structure

- Server replies with:
- **✓ Status Line**
- HTTP/1.1 200 OK
- **✓ Headers**
- Content-Type: application/json
- **✓ Body**
- {
- "message": "Product created successfully"
- }

4. HTTP Methods (Very Important)

Method	Purpose	Example
GET	Fetch data	Get all users
POST	Create new data	Add user
PUT	Update entire record	Replace user
PATCH	Update partial	Change user email
DELETE	Remove resource	Delete user

12 5. HTTP Status Codes

- **2xx → Success**
- 200 OK
- 201 Created
- 204 No Content
- **4xx → Client Errors**
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found

5. HTTP Status Codes

- **5xx → Server Errors**
- 500 Internal Server Error
- 503 Service Unavailable



6. What is REST?

- **REST = Representational State Transfer**
It's an architecture style for designing APIs.
- REST APIs follow rules for:
 - ✓ Resources
 - ✓ URLs
 - ✓ HTTP methods
 - ✓ Stateless communication
 - ✓ JSON responses
 - ✓ Layered system

7. REST Important Principles (Simple Explanation)



- **1 Stateless**
 - Server doesn't remember anything between requests. Every request contains all information needed.
- **2 Resource-Based URLs**
 - URLs represent **nouns**, not verbs.
 -  BAD
 - /getAllUsers
 - /addNewUser

7. REST Important Principles (Simple Explanation)



- ✓ GOOD
- /users
- /users/1

7. REST Important Principles (Simple Explanation)



- **③ Use HTTP Methods Properly**
 - GET /users
 - POST /users
 - PUT /users/10
 - PATCH /users/10
 - DELETE /users/10
- **④ Return Proper Status Codes**
 - REST APIs must clearly tell if action succeeded or failed.

7. REST Important Principles (Simple Explanation)



- **5 Use JSON**
- REST APIs usually use JSON format.



Path Parameters, Query and Body Parameters



Param Type	Comes From	Example URL	FastAPI Usage
Path	Part of the route	/users/5	@app.get("/users/{id}")
Query	After ?	/users?active=true	def fn(active: bool)
Body	Request body (JSON)	POST/PUT	def fn(obj: Model)

🌟 What is Pydantic?

- **Pydantic = Data validation + Data parsing using Python type hints**
- FastAPI **automatically** uses Pydantic to:
- Validate request bodies
- Convert data types
- Generate error messages
- Auto-generate Swagger documentation
- Create clean, structured JSON
- It's basically magic ✨

Fast API vs Django

Feature / Aspect	FastAPI	Django
Type	API framework (lightweight)	Full-stack web framework
Best For	REST APIs, microservices, async workloads	Websites, dashboards, admin apps, monoliths
Performance	★ Extremely fast (async native)	Good, but not FastAPI-level
Architecture	ASGI (modern, async)	WSGI (sync) + optional ASGI
Learning Curve	Easy if you know Python typing	Medium; has “Django way”
Built-in Features	Minimal (add your own ORM, admin, auth)	★ ORM, Auth, Admin, Forms, Templates, Sessions
Automatic Docs	★ Yes (Swagger / ReDoc auto)	No (unless using Django REST Framework)
ORM	No built-in (use SQLAlchemy / SQLModel)	★ Built-in ORM

Fast API vs Django

Feature / Aspect	FastAPI	Django
Admin Panel	No	★ Auto-generated Admin Panel
Template System	No	Yes (Jinja-like templates)
Ecosystem	Growing but smaller	Huge (Django + DRF + plugins)
Use Case Fit	Mobile backends, async IO apps, ML model serving	Portals, CMS, ERP-like apps, dashboards
Code Style	Modern, type-hinted, concise	Traditional MVC/MTV
Async Support	Built-in native async	Partial (some async, ecosystem still catching up)
Community	Smaller but active	Very large, mature



What is FastAPI?

- FastAPI is a modern Python web framework for building **REST APIs** (and async apps) with:
- **Super high performance** (on par with Node & Go thanks to Uvicorn/Starlette)
- **Auto-generated Swagger UI** ✨
- **Pydantic models** for validation
- **Async support**
- **Dependency Injection system**
- **Zero boilerplate**

1. Project Structure (Recommended)

```
fastapi-project/  
|  
├─ app/  
|   ├── main.py  
|   ├── config.py  
|   ├── routers/  
|   |   ├── users.py  
|   |   └─ items.py  
|   ├── schemas/  
|   |   ├── user_schema.py  
|   |   └─ item_schema.py  
|   ├── models/  
|   |   └─ user_model.py  
|   └─ db/  
|       ├── database.py  
|       └─ connection.py  
|  
├─ requirements.txt  
└─ README.md
```

2. Installing FastAPI & Uvicorn

- 👉 **Step 1: Create a virtual environment**
- Windows:
 - `python -m venv env`
 - `env\Scripts\activate`
- 👉 **Step 2: Install FastAPI**
 - `pip install fastapi`
- 👉 **Step 3: Install Uvicorn server**
 - `pip install uvicorn[standard]`

2. Installing FastAPI & Uvicorn

- 👉 **Step 4: Create requirements.txt (optional but good practice)**
- `pip freeze > requirements.txt`



Why this structure?

- `main.py` → Entry point
- `routers/` → API endpoints
- `schemas/` → Pydantic models (request/response validation)
- `models/` → DB models (SQLAlchemy etc.)
- `db/` → Database connection
- `config.py` → App settings (env vars, secrets)
- Clean. Reusable. Easy to scale.

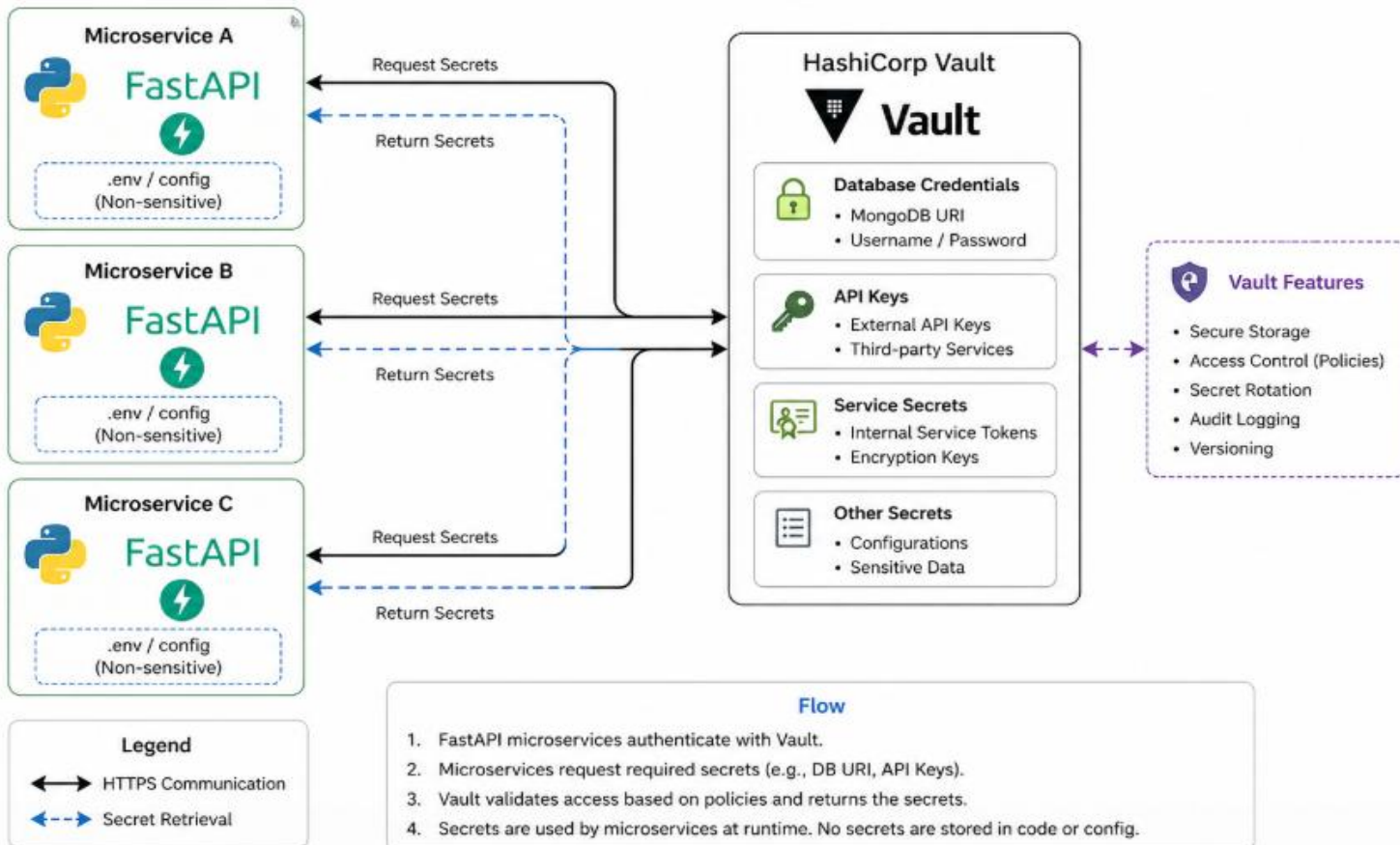


Run Project

- Go to src folder
- Run
- `uvicorn ecommerce.main:app --reload`
- #open in browser using
- `http://127.0.0.1:8000/docs`

FastAPI Microservices with HashiCorp Vault

Centralized Secrets Management



Fast API JWT



127.0.0.1:8000/docs#/default/login_login_post Summarize

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "admin",
    "password": "admin123"
  }'
```

Request URL

```
http://127.0.0.1:8000/login
```

Server response

Code	Details
200	Response body <pre>{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbGlzImV4cCI6MTc2MzQ4OTgwMX0.4KBv6Qhnt-XcrSxJ09VVgZTt84VXj7NUZrKYHDBd1Fk", "token_type": "bearer" }</pre> Response headers

Fast API JWT



HTTP http://localhost:8000/locations

Save

GET

http://localhost:8000/locations

Send

Params

Authorization ●

Headers (9)

Body ●

Pre-request Script

Tests

Settings

Cookies

Type

Bearer ...

Token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...

The authorization header will be automatically generated when you send the request.

[Learn more about authorization](#)



Store your secrets with end-to-end encryption locally using Vault.

Store in Vault

Body Cookies Headers (4) Test Results



200 OK

10 ms

214 B

Save Response

Pretty

Raw

Preview

Visualize

JSON



```
1 [
2   {
3     "name": "chennai",
4     "latitude": 104.0,
5     "longitude": 82.0,
6     "created_on": "2025-18-11",
7     "code": 1
8   }
9 ]
```

Logger Middleware



The screenshot shows a VS Code editor window with the following components:

- EXPLORER:** A file tree on the left showing the project structure. The file `request_logger.py` is selected.
- EDITOR:** The main area showing the code for `request_logger.py`. The code defines a `RequestLoggingMiddleware` class that inherits from `BaseHTTPMiddleware` and implements an `async def dispatch` method. The method logs the request URL and headers, reads the request body, and then calls `call_next()`.
- TERMINAL:** The bottom panel shows the output of the application. It displays two HTTP requests and their corresponding responses, along with log messages from SQLAlchemy.

```
locationapi > middleware > request_logger.py > ...
6 class RequestLoggingMiddleware(BaseHTTPMiddleware):
8     async def dispatch(self, request: Request, call_next):
14         print(f"URL      : {request.url}")
15         print(f"Headers   : {dict(request.headers)}")
16
17         # Read request body (must read & replace stream)
18         body_bytes = await request.body()
19         if body_bytes:
20             print(f"Body      : {body_bytes.decode('utf-8').rstrip('\n')}
```

python - locationapi

```
INFO:      127.0.0.1:52730 - "GET /locations HTTP/1.1" 200 OK
2025-11-18 23:22:34,632 INFO sqlalchemy.engine.Engine ROLLBACK
INFO:      Shutting down
INFO:      Waiting for application shutdown.
INFO:      Application shutdown complete. ...
2025-11-18 23:25:18,212 INFO sqlalchemy.engine.Engine [generated in 0.00098s] {}

===== RESPONSE =====
Status Code: 200
Time Taken : 28.64 ms
=====

INFO:      127.0.0.1:55740 - "GET /locations HTTP/1.1" 200 OK
2025-11-18 23:25:18,218 INFO sqlalchemy.engine.Engine ROLLBACK
[]
```



Audit Middleware With Background Tasks

 **http://localhost:8000/locations**

POST  http://localhost:8000/locations

Params Authorization ● Headers (9) **Body ●** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** 

```
1  {
2      "name": "bangalore",
3      "latitude": 103.0,
4      "longitude": 82.0,
5      "created_on": "2025-19-11"
6  }
```

Body Cookies Headers (4) Test Results  201 Created 75 ms 219 B

Pretty Raw Preview Visualize **JSON**  

```
1  {
2      "name": "bangalore",
3      "latitude": 103.0,
4      "longitude": 82.0,
5      "created_on": "2025-19-11",
6      "code": 4
7  }
```

Audit Middleware With Background Tasks



The screenshot shows a VS Code editor with a project named 'day8' containing a Flask application. The application uses SQLAlchemy for database operations and Celery for background tasks. A red bracket highlights the background task scheduling logic in the `create_location` function.

```
locationapi > main.py > create_location
42 def login(body: LoginRequest):
43     access_token = create_access_token({"sub": body.username})
44     return {"access_token": access_token, "token_type": "bearer"}
45
46 #location post api
47 @app.post("/locations", response_model=LocationOut, status_code=status.HTTP_201_CREATED)
48 def create_location(location: LocationCreate, background_tasks: BackgroundTasks, db: Session = Depends(get_db)):
49     db_location = Location(name=location.name, latitude=location.latitude, longitude=location.longitude,
50                             code=location.code)
51     db.add(db_location)
52     db.commit()
53     db.refresh(db_location)
54     # Schedule background job
55     background_tasks.add_task(
56         audit_location_created, db_location.code
57     )
58     return db_location
59
60 #location get api
61 @app.get("/locations/{location_id}", dependencies=[Depends(get_current_user)], tags=["locations"], response_model=LocationOut)
62 def get_location(location_id: int, db: Session = Depends(get_db)):
63     db_location = db.query(Location).filter(Location.id == location_id).first()
64     if db_location is None:
65         return {"error": "Location not found"}
66     return db_location
```

The terminal output shows the application running successfully:

```
(day8env) PS E:\MTSTraningmay2025\day8\locationapi> python main.py
===== RESPONSE =====
Status Code: 201
Time Taken : 51.33 ms
=====
INFO: 127.0.0.1:50164 - "POST /locations HTTP/1.1" 201 Created
2025-11-18 23:39:22,180 INFO sqlalchemy.engine.Engine ROLLBACK
```

Flask App



Folder Structure

```
pgsql  
  
flaskapp/  
|  
├─ app.py  
├─ requirements.txt  
└─ venv/ (after you create it)
```

Flask App



app.py

python

```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy
from werkzeug.security import generate_password_hash, check_password_hash

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///customers.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

class Customer(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(100), unique=True, nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False)
    password = db.Column(db.String(200), nullable=False)

@app.before_first_request
def create_tables():
    db.create_all()
```

Flask App

```
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    if Customer.query.filter_by(username=data['username']).first():
        return jsonify({"message": "Username already exists"}), 400
    hashed_password = generate_password_hash(data['password'], method='pbkdf2:sha256')
    user = Customer(username=data['username'], email=data['email'], password=hashed_password)
    db.session.add(user)
    db.session.commit()
    return jsonify({"message": "User registered successfully"}), 201

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    user = Customer.query.filter_by(username=data['username']).first()
    if user and check_password_hash(user.password, data['password']):
        return jsonify({"message": "Login successful", "user": user.username})
    return jsonify({"message": "Invalid credentials"}), 401
```




Flask App

```
@app.route('/')  
def index():  
    return "Flask API is running"  
  
if __name__ == '__main__':  
    app.run(debug=True)
```

Flask App

1. Open terminal in the `flaskapp` directory
2. Create & activate a virtual environment:

```
bash

python -m venv venv
.\venv\Scripts\activate    # Windows
```

3. Install packages:

```
bash

pip install -r requirements.txt
```

4. Run the app:

```
bash

python app.py
```

Flask App

Register

```
http

POST http://127.0.0.1:5000/register
Content-Type: application/json
{
  "username": "john",
  "email": "john@example.com",
  "password": "secret"
}
```

Login

```
http

POST http://127.0.0.1:5000/login
Content-Type: application/json
{
  "username": "john",
  "password": "secret"
}
```

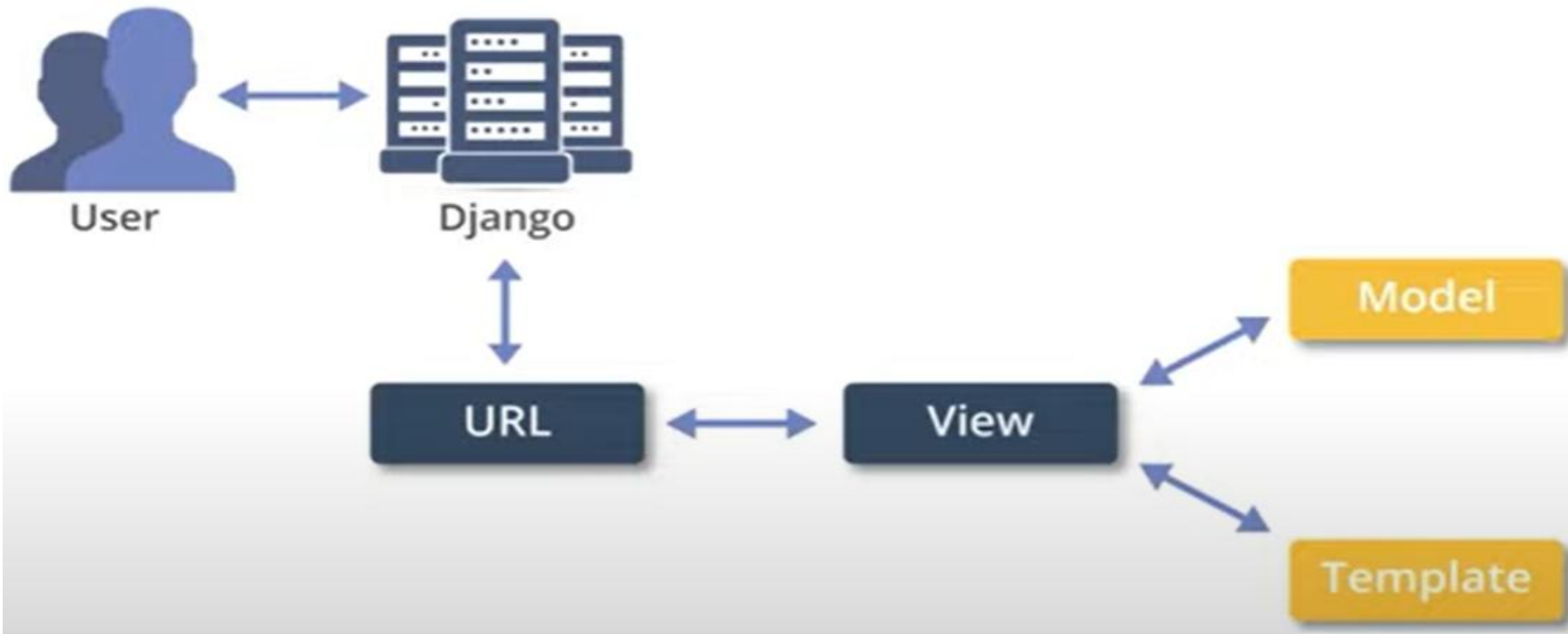
- **High-level** Python Web framework
- Allows **rapid** development and a **clean design**
- Takes care of the hassle of Web development
- Free and open source
- Extremely **fast**
- Excellent Documentation

Django Features



Django Architecture

Model View Template



Django Installation



1

Download Python from official Python website

2

Install Python on your PC

3

Set up the path variable

4

Download PyCharm from official website

5

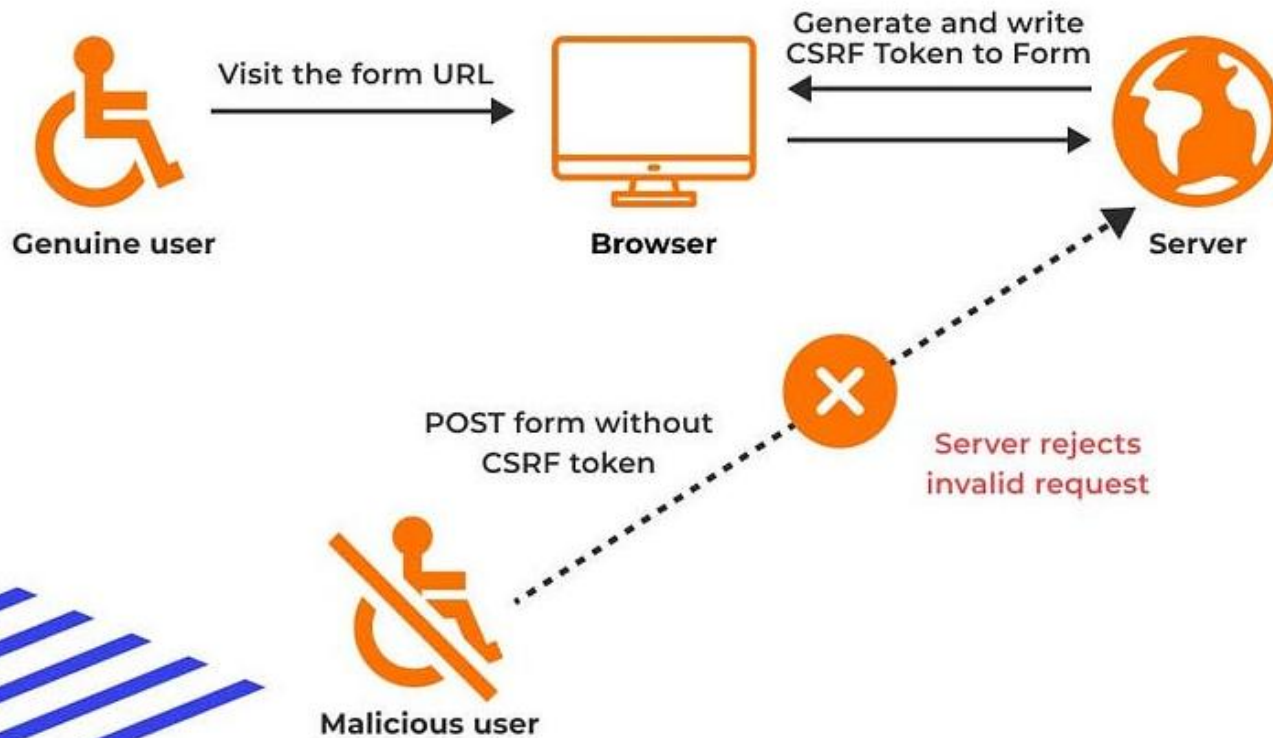
Install PyCharm on your PC

6

In the terminal type the command: `pip install Django`

Django MVC Post,Put and Delete

What is CSRF token?





Django MVC Post,Put and Delete

- Cross-Site Request Forgery (CSRF) is a type of attack that tricks a user into executing an unwanted action in a web application where they are authenticated.
- For example, a user might be logged into their bank account, and a malicious site could trick them into transferring money without their consent.



Django MVC Post,Put and Delete

1. A user logs into a secure website (e.g., their banking app) with valid credentials.
2. The user then visits a malicious website in another tab or window.
3. That website contains hidden forms or malicious scripts that trick the user's browser into sending a request to the banking website, performing actions like transferring funds or changing account details.



Django MVC Post,Put and Delete

Django's CSRF protection works by:

- **Generating a token:** A CSRF token is generated for each user session.
- **Embedding the token in forms:** Every form in Django is automatically protected by embedding the CSRF token in hidden fields.
- **Verifying the token:** When the form is submitted, Django checks if the submitted token matches the one stored in the user session.

Implementing Django CSRF Protection

1. Enabling CSRF Middleware

Django enables CSRF protection by default through its middleware. In your `settings.py`, ensure the `CsrfViewMiddleware` is listed under the `MIDDLEWARE` configuration:

```
MIDDLEWARE = [  
    'django.middleware.csrf.CsrfViewMiddleware',  
    # other middleware...  
]
```

Implementing Django CSRF Protection

2. Using the CSRF Token in Forms

In Django, forms automatically include the CSRF token when using the `{% csrf_token %}` template tag. Here's how you can include the CSRF token in a typical form:

```
<form method="post" action="/submit-data/">
    {% csrf_token %}
    <label for="name">Name:</label>
    <input type="text" id="name" name="name">
    <button type="submit">Submit</button>
</form>
```



Implementing Django CSRF Protection

- When the form is submitted, Django verifies that the CSRF token included in the form matches the one stored in the session, ensuring the request is legitimate

Best Practices for Securing Your Django App



- Keep CSRF Protection Enabled: Never disable CSRF protection unless absolutely necessary.
- Use HTTPS: Always serve your application over HTTPS to prevent man-in-the-middle attacks that could compromise your CSRF tokens.
- Audit Your Views: Ensure that all forms and sensitive operations are protected by CSRF tokens. Always use `{% csrf_token %}` in templates or the `X-CSRFToken` header in AJAX requests.
- Be Cautious with `@csrf_exempt`: If you must use `@csrf_exempt`, do so with caution and only for specific views where CSRF protection isn't necessary.
- Regularly Update Django: Security patches and improvements are frequently released, so always keep your Django version up-to-date.

What Python Version You can use for Django



Django Version	Python Versions
1.11	2.7, 3.4, 3.5, 3.6, 3.7 (added in 1.11.17)
2.0	3.4, 3.5, 3.6, 3.7
2.1, 2.2	3.5, 3.6, 3.7

of the Python versions
starting from 2.7

Regular Expression

- `re.match(pattern, string, flags=0)`

Parameter	Description
pattern	This is the regular expression to be matched.
string	This is the string which would be searched to match the pattern
flags	You can specify different flags using bitwise OR (). These are modifiers which are listed in the table below.

Regular Expression



Match Object Methods	Description
<code>group(num=0)</code>	This methods returns entire match (or specific subgroup num)
<code>groups()</code>	This method return all matching subgroups in a tuple (empty if there weren't any)

Regular-expression Modifiers - Option Flags



Modifier	Description
re.I	Performs case-insensitive matching.
re.L	Interprets words according to the current locale. This interpretation affects the alphabetic group (<code>\w</code> and <code>\W</code>), as well as word boundary behavior (<code>\b</code> and <code>\B</code>).
re.M	Makes <code>\$</code> match the end of a line (not just the end of the string) and makes <code>^</code> match the start of any line (not just the start of the string).
re.S	Makes a period (dot) match any character, including a newline.
re.U	Interprets letters according to the Unicode character set. This flag affects the behavior of <code>\w</code> , <code>\W</code> , <code>\b</code> , <code>\B</code> .
re.X	Permits "cuter" regular expression syntax. It ignores whitespace (except inside a set <code>[]</code> or when escaped by a backslash), and treats unescaped <code>#</code> as a comment marker.

Regular-expression Modifiers - Option Flags



Modifier	Description
re.I	Performs case-insensitive matching.
re.L	Interprets words according to the current locale. This interpretation affects the alphabetic group (<code>\w</code> and <code>\W</code>), as well as word boundary behavior (<code>\b</code> and <code>\B</code>).
re.M	Makes <code>\$</code> match the end of a line (not just the end of the string) and makes <code>^</code> match the start of any line (not just the start of the string).
re.S	Makes a period (dot) match any character, including a newline.
re.U	Interprets letters according to the Unicode character set. This flag affects the behavior of <code>\w</code> , <code>\W</code> , <code>\b</code> , <code>\B</code> .
re.X	Permits "cuter" regular expression syntax. It ignores whitespace (except inside a set <code>[]</code> or when escaped by a backslash), and treats unescaped <code>#</code> as a comment marker.

Python Debugging



- `Python -m pdb filename`

Python Debugging



Command	Short form	What it does
args	a	Print the argument list of the current function
break	b	Creates a breakpoint (requires parameters) in the program execution
continue	c or cont	Continues program execution
help	h	Provides list of commands or help for a specified command
jump	j	Set the next line to be executed
list	l	Print the source code around the current line
next	n	Continue execution until the next line in the current function is reached or returns
step	s	Execute the current line, stopping at first possible occasion
pp	pp	Pretty-prints the value of the expression
quit or exit	q	Aborts the program
return	r	Continue execution until the current function returns

PyInstaller



- `pyinstaller.exe --onefile --windowed --icon=app.ico app.py`

Creating the Application Structure



/MyApplication

|-- run.py

|-- config.py

|__ /app

|-- __init__.py

|-- /module_one

|-- __init__.py

|-- controllers.py

|-- models.py

|__ /templates

|-- module_one

|-- hello.html

|__ /static

|__ ..

|__ .

How to Create The Distribution File



- `python setup.py sdist`

How to Install The Application



- `python setup.py install`
- For development
- `python setup.py develop`

SNMP



- <https://pynet.twb-tech.com/blog/snmp/python-snmp-intro.html>

SNMP



```
snmpget -v 2c -c <community> <ip_addr>  
.1.3.6.1.2.1.1.1.0
```



Django Framework

- `pip install Django`
- `python -c "import django; print(django.get_version())"`
- `python -m django --version`
- `django-admin startproject myproject`
- `cd myproject`
- `python manage.py runserver`
- `python manage.py startapp myapp`

Django Framework

- `INSTALLED_APPS = (`
- `'django.contrib.admin',`
- `'django.contrib.auth',`
- `'django.contrib.contenttypes',`
- `'django.contrib.sessions',`
- `'django.contrib.messages',`
- `'django.contrib.staticfiles',`
- `'myapp',`
- `)`

Django Framework



- `python manage.py migrate`
- `python manage.py makemigrations polls`
- `python manage.py sqlmigrate polls 0001`
- `python manage.py migrate`

- `python manage.py createsuperuser`
- `python manage.py runserver`
- <http://localhost:8000/admin/>
- `http://localhost:8000/hello/12/`



Python Web Server

- `Python -m http.server 8080`

Higher Order Functions

MAP,FILTER AND REDUCE

MAP, FILTER AND REDUCE

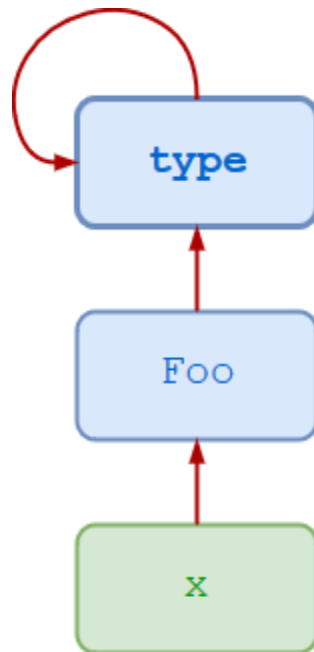
Meta class

- A metaclass is a class whose instances are classes. Like an "ordinary" class defines the behavior of the instances of the class, a metaclass defines the behavior of classes and their instances.
- Metaclasses are not supported by every object oriented programming language. Those programming language, which support metaclasses, considerably vary in way the implement them. Python is supporting them

Meta Class

- There are numerous use cases for metaclasses. Just to name a few:
- logging and profiling
- interface checking
- registering classes at creation time
- automatically adding new methods
- automatic property creation
- proxies
- automatic resource locking/synchronization.

Meta Class



Dynamic Meta class

- `>>> type(3)`
- `<class 'int'>`
- `>>> type(['foo', 'bar', 'baz'])`
- `<class 'list'>`
- `>>> t = (1, 2, 3, 4, 5)`
- `>>> type(t)`
- `<class 'tuple'>`

Dynamic Meta class

- `>>> class Foo:`
- `... pass`
- `...`
- `>>> type(Foo())`
- `<class '__main__.Foo'>`

Dynamic Meta class

- You can also call `type()` with three arguments—`type(<name>, <bases>, <dct>)`:
- `<name>` specifies the class name. This becomes the `__name__` attribute of the class.
- `<bases>` specifies a tuple of the base classes from which the class inherits. This becomes the `__bases__` attribute of the class.
- `<dct>` specifies a namespace dictionary containing definitions for the class body. This becomes the `__dict__` attribute of the class.

Dynamic Meta class

- `>>> Bar = type('Bar', (Foo,), dict(attr=100))`
- `>>> x = Bar()`
- `>>> x.attr`
- `100`
- `>>> x.__class__`
- `<class '__main__.Bar'>`
- `>>> x.__class__.__bases__`
- `(<class '__main__.Foo'>,)`

Decorators

- A decorator in Python is any callable Python object that is used to modify a function or a class.
- A reference to a function "func" or a class "C" is passed to a decorator and the decorator returns a modified function or class.
- The modified functions or classes usually contain calls to the original function "func" or class "C".

Pandas

- Pandas is used for data manipulation, analysis and cleaning. Python pandas is well suited for different kinds of data, such as:
- Tabular data with heterogeneously-typed columns
- Ordered and unordered time series data
- Arbitrary matrix data with row & column labels
- Unlabelled data
- Any other form of observational or statistical data sets

Pandas



Lambdas

- In Python, anonymous function is a function that is defined without a name.
- While normal functions are defined using the `def` keyword, in Python anonymous functions are defined using the `lambda` keyword.

How to use lambda Functions in Python?

- A lambda function in python has the following syntax.
- Syntax of Lambda Function in python
- **lambda arguments: expression**
- Lambda functions can have any number of arguments but only one expression.
- The expression is evaluated and returned. Lambda functions can be used wherever function objects are required.

How to use lambda Functions in Python?

- Lambda functions can have any number of arguments but only one expression.
- The expression is evaluated and returned.
- Lambda functions can be used wherever function objects are required.

Regular Expression

- A *regular expression* is a special sequence of characters that helps you match or find other strings or sets of strings, using a specialized syntax held in a pattern.
- Regular expressions are widely used in UNIX world.
- The module **re** provides full support for Perl-like regular expressions in Python.
- The re module raises the exception re.error if an error occurs while compiling or using a regular expression

Regular Expression

- In regular expressions, there are twelve metacharacters that should be escaped if they are to be used with their literal meaning:
- Backslash \
- Caret ^
- Dollar sign \$
- Dot .

Regular Expression

Pipe symbol |

- Question mark ?
- Asterisk *
- Plus sign +
- Opening parenthesis (
- Closing parenthesis)
- Opening square bracket [
- The opening curly brace {

Regular Expression

Element	Description
[	Matches the following set of characters
0-9	Matches anything between 0 and 9 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9).
	Or
a-z	Matches anything between a and z (a, b, c, d, ..., z)
	Or
A-Z	Matches anything between A and Z (A, B, C, D, ..., Z)
]	End of character set

Predefined character classes

Element	Description (for regex with default flags)
.	This element matches any character except newline <code>\n</code>
<code>\d</code>	This matches any decimal digit; this is equivalent to the class <code>[0-9]</code>
<code>\D</code>	This matches any non-digit character; this is equivalent to the class <code>[^0-9]</code>
<code>\s</code>	This matches any whitespace character; this is equivalent to the class <code>[\t\n\r\f\v]</code>
<code>\S</code>	This matches any non-whitespace character; this is equivalent to the class <code>[^\t\n\r\f\v]</code>
<code>\w</code>	This matches any alphanumeric character; this is equivalent to the class <code>[a-zA-Z0-9_]</code>
<code>\W</code>	This matches any non-alphanumeric character; this is equivalent to the class <code>[^a-zA-Z0-9_]</code>

Predefined character classes

Element	Description
.	Matches any character
.	Matches any character followed by the previous one
.	Matches any character followed by the previous one

Element	Description
[	Matches a set of characters
^	Not matching this symbol's following characters
\/	Matches a / character
\	Matches a \ character
]	End of the set

Quantifiers

Symbol	Name	Quantification of previous character
?	Question mark	Optional (0 or 1 repetitions)
*	Asterisk	Zero or more times
+	Plus sign	One or more times
{ <i>n</i> , <i>m</i> }	Curly braces	Between <i>n</i> and <i>m</i> times

Boundary Matchers

Element	Description
<code>\b</code>	Matches a word boundary.
<code>h</code>	Matches the followed by character h.
<code>e</code>	Matches the followed by character e.
<code>l</code>	Matches the followed by character l.
<code>l</code>	Matches the followed by character l.
<code>o</code>	Matches the followed by character o.
<code>\b</code>	Then matches another followed by word boundary.

Boundary Matchers

Element	Description
^	Matches the beginning of the line
N	Matches the followed by character N
a	Matches the followed by character a
m	Matches the followed by character m
e	Matches the followed by character e
:	Matches the followed by symbol colon

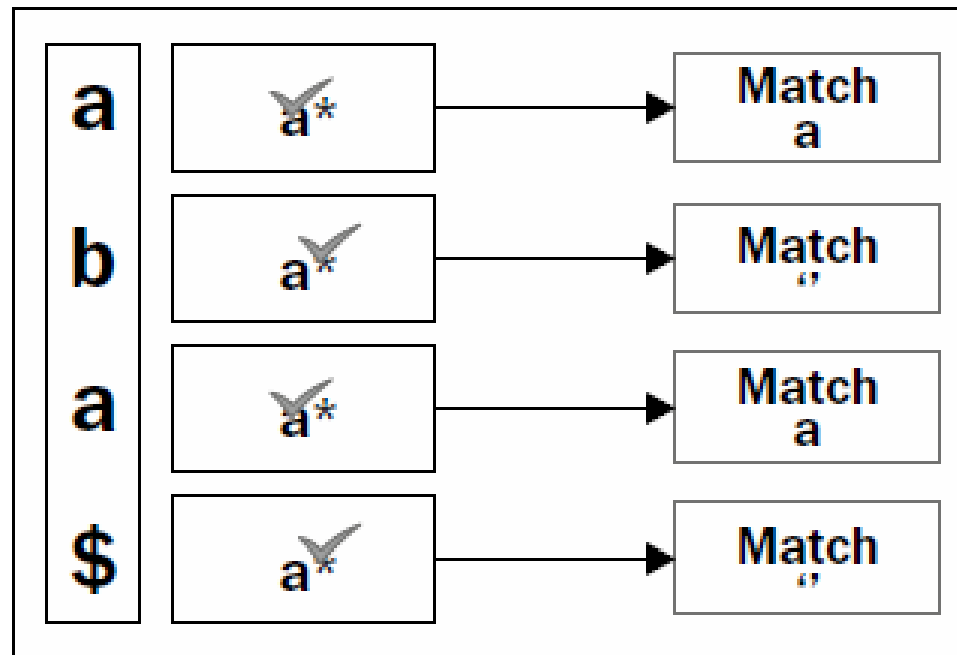
Boundary Matchers

Element	Description
<code>^</code>	Matches the beginning of the line.
<code>N</code>	Matches the followed by character N.
<code>a</code>	Matches the followed by character a.
<code>m</code>	Matches the followed by character m.
<code>e</code>	Matches the followed by character e.
<code>:</code>	Matches the followed by colon symbol.
<code>[\sa-zA-Z]</code>	Then matches the followed by whitespace, or any alphabetic lowercase or uppercase character.
<code>+</code>	The character can be repeated one or more times.
<code>\$</code>	Until the end of the line.

Boundary Matchers

Element	Description
<code>\b</code>	Matches a word boundary.
<code>h</code>	Matches the followed by character h.
<code>e</code>	Matches the followed by character e.
<code>l</code>	Matches the followed by character l.
<code>1</code>	Matches the followed by character 1.
<code>o</code>	Matches the followed by character o.
<code>\b</code>	Then matches another followed by word boundary.

Boundary Matchers



`findall` matching process

Functions

Function	Description
<code>compile(pattern[, flags])</code>	Creates a pattern object from a string with a regular expression
<code>search(pattern, string[, flags])</code>	Searches for pattern in string
<code>match(pattern, string[, flags])</code>	Matches pattern at the beginning of string
<code>split(pattern, string[, maxsplit=0])</code>	Splits a string by occurrences of pattern
<code>findall(pattern, string)</code>	Returns a list of all occurrences of pattern in string
<code>sub(pat, repl, string[, count=0])</code>	Substitutes occurrences of pat in string with repl
<code>escape(string)</code>	Escapes all special regular expression characters in string

Regular Expressions

[]	A set of characters	"[a-m]"
\	Signals a special sequence (can also be used to escape special characters)	"\d"
.	Any character (except newline character)	"he..o"
^	Starts with	"^hello"
\$	Ends with	"world\$"
*	Zero or more occurrences	"aix*"
+	One or more occurrences	"aix+"
{}	Exactly the specified number of occurrences	"al{2}"
	Either or	"falls stays"
()	Capture and group	

Regular Expressions

Character	Description	Example
\A	Returns a match if the specified characters are at the beginning of the string	"\AThe"
\b	Returns a match where the specified characters are at the beginning or at the end of a word	r"\bain" r"ain\b"
\B	Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word	r"\Bain" r"ain\B"
\d	Returns a match where the string contains digits (numbers from 0-9)	"\d"
\D	Returns a match where the string DOES NOT contain digits	"\D"
\s	Returns a match where the string contains a white space character	"\s"
\S	Returns a match where the string DOES NOT contain a white space character	"\S"
\w	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore _ character)	"\w"
\W	Returns a match where the string DOES NOT contain any word characters	"\W"
\Z	Returns a match if the specified characters are at the end of the string	"Spain\Z"

Regular Expressions

Set	Description
[arn]	Returns a match where one of the specified characters (a, r, or n) are present
[a-n]	Returns a match for any lower case character, alphabetically between a and n
[^arn]	Returns a match for any character EXCEPT a, r, and n
[0123]	Returns a match where any of the specified digits (0, 1, 2, or 3) are present
[0-9]	Returns a match for any digit between 0 and 9
[0-5][0-9]	Returns a match for any two-digit numbers from 00 and 59
[a-zA-Z]	Returns a match for any character alphabetically between a and z, lower case OR upper case
[+]	In sets, +, *, ., , (), \$, {} has no special meaning, so [+] means: return a match for any +character in the string

Sub Process

- The subprocess module allows you to spawn new processes, connect to their input/output/error pipes, and obtain their return codes.
- This module intends to replace several older modules and functions:
 - `os.system`
 - `os.spawn*`

Using Sub Process Module

- The recommended approach to invoking subprocesses is to use the `run()` function for all use cases it can handle. For more advanced use cases, the underlying `Popen` interface can be used directly.
- The `run()` function was added in Python 3.5; if you need to retain compatibility

Using Sub Process Module

- `subprocess.run(args, *, stdin=None, input=None, stdout=None, stderr=None, capture_output=False, shell=False, cwd=None, timeout=None, check=False, encoding=None, errors=None, text=None, env=None, universal_newlines=None)`

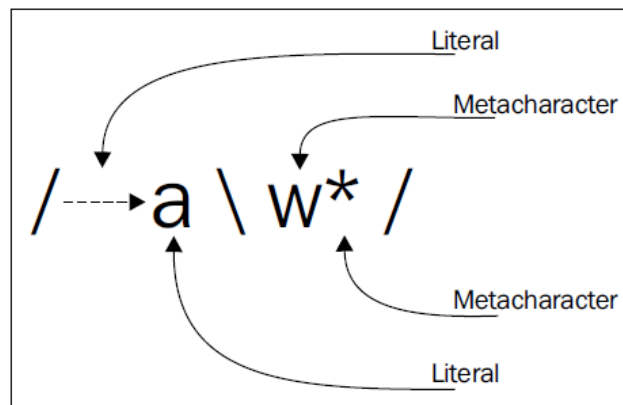


Regular Expressions

- A regular expression is a special sequence of characters that helps you match or find other strings or sets of strings, using a specialized syntax held in a pattern.
- Regular expressions are widely used in UNIX world.
- The module `re` provides full support for Perl-like regular expressions in Python.
- The `re` module raises the exception `re.error` if an error occurs while compiling or using a regular expression

Regular Expressions

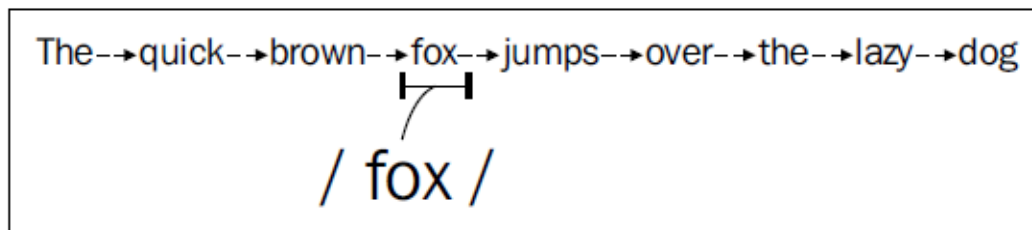
- A regular expression is a pattern of text that consists of ordinary characters (for example, letters a through z or numbers 0 through 9) and special characters known as meta characters.
- This pattern describes the strings that would match when applied to a text.



Regex using literals and metacharacters

Regular Expressions - Literals

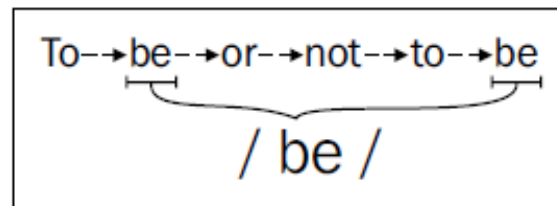
- Literals are the simplest form of pattern matching in regular expressions.
- They will simply succeed whenever that literal is found.
- If we apply the regular expression `/fox/` to search the phrase `The quick brown fox jumps over the lazy dog`, we will find one match:



Searching using a regular expression

Regular Expressions - Literals

- However, we can also obtain several results instead of just one, if we apply the
- regular expression `/be/` to the following phrase To be, or not to be:



Multiple results searching with regex

Regular Expressions - Literals

- Using the backslash method, we can convert the previous expression to `^(this is inside\)/` and apply it again to the same text to have the parentheses included in the result:

```
this-->is-->outside-->(this-->is-->inside)
/ \ (this -->is -->inside \) /
```

Escaped metacharacters in regex



Regular Expression

- In regular expressions, there are twelve meta characters that should be escaped if they are to be used with their literal meaning:
- Backslash \
- Caret ^
- Dollar sign \$
- Dot .

Regular Expression

Pipe symbol |

- Question mark ?
- Asterisk *
- Plus sign +
- Opening parenthesis (
- Closing parenthesis)
- Opening square bracket [
- The opening curly brace {



Regular Expressions Character classes

- The character classes (also known as character sets) allow us to define a character that will match if any of the defined characters on the set is present.
- To define a character class, we should use the opening square bracket meta character `[`, then any accepted characters, and finally close with a closing square bracket `]`.
- For instance, let's define a regular expression that can match the word "license" in British and American English written form:

Regular Expressions Character classes

licence → and → license → are → valid
/ licen[cs]e /

Searching using a character class

- It is possible to also use the range of a character.
- This is done by leveraging the hyphen symbol (-) between two related characters; for example, to match any lowercase letter we can use [a-z].
- Likewise, to match any single digit we can define the character set [0-9].

Regular Expression

The character classes' ranges can be combined to be able to match a character against many ranges by just putting one range after the other—no special separation is required.

For instance, if we want to match any lowercase or uppercase alphanumeric character, we can use `[0-9a-zA-Z]` (see next table for a more detailed explanation). This can be alternatively written using the union mechanism: `[0-9[a-zA-Z]]`.

Element	Description
<code>[</code>	Matches the following set of characters
<code>0-9</code>	Matches anything between 0 and 9 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9).
	Or
<code>a-z</code>	Matches anything between a and z (a, b, c, d, ..., z)
	Or
<code>A-Z</code>	Matches anything between A and Z (A, B, C, D, ..., Z)
<code>]</code>	End of character set

Predefined character classes

Element	Description (for regex with default flags)
.	This element matches any character except newline <code>\n</code>
<code>\d</code>	This matches any decimal digit; this is equivalent to the class <code>[0-9]</code>
<code>\D</code>	This matches any non-digit character; this is equivalent to the class <code>[^0-9]</code>
<code>\s</code>	This matches any whitespace character; this is equivalent to the class <code>[\t\n\r\f\v]</code>
<code>\S</code>	This matches any non-whitespace character; this is equivalent to the class <code>[^\t\n\r\f\v]</code>
<code>\w</code>	This matches any alphanumeric character; this is equivalent to the class <code>[a-zA-Z0-9_]</code>
<code>\W</code>	This matches any non-alphanumeric character; this is equivalent to the class <code>[^a-zA-Z0-9_]</code>

Predefined character classes

Element	Description
.	Matches any character
.	Matches any character followed by the previous one
.	Matches any character followed by the previous one

Element	Description
[	Matches a set of characters
^	Not matching this symbol's following characters
\/	Matches a / character
\	Matches a \ character
]	End of the set

Quantifiers

Symbol	Name	Quantification of previous character
?	Question mark	Optional (0 or 1 repetitions)
*	Asterisk	Zero or more times
+	Plus sign	One or more times
{ <i>n</i> , <i>m</i> }	Curly braces	Between <i>n</i> and <i>m</i> times

Boundary Matchers

Element	Description
<code>\b</code>	Matches a word boundary.
<code>h</code>	Matches the followed by character h.
<code>e</code>	Matches the followed by character e.
<code>l</code>	Matches the followed by character l.
<code>l</code>	Matches the followed by character l.
<code>o</code>	Matches the followed by character o.
<code>\b</code>	Then matches another followed by word boundary.

Boundary Matchers

Element	Description
^	Matches the beginning of the line
N	Matches the followed by character N
a	Matches the followed by character a
m	Matches the followed by character m
e	Matches the followed by character e
:	Matches the followed by symbol colon

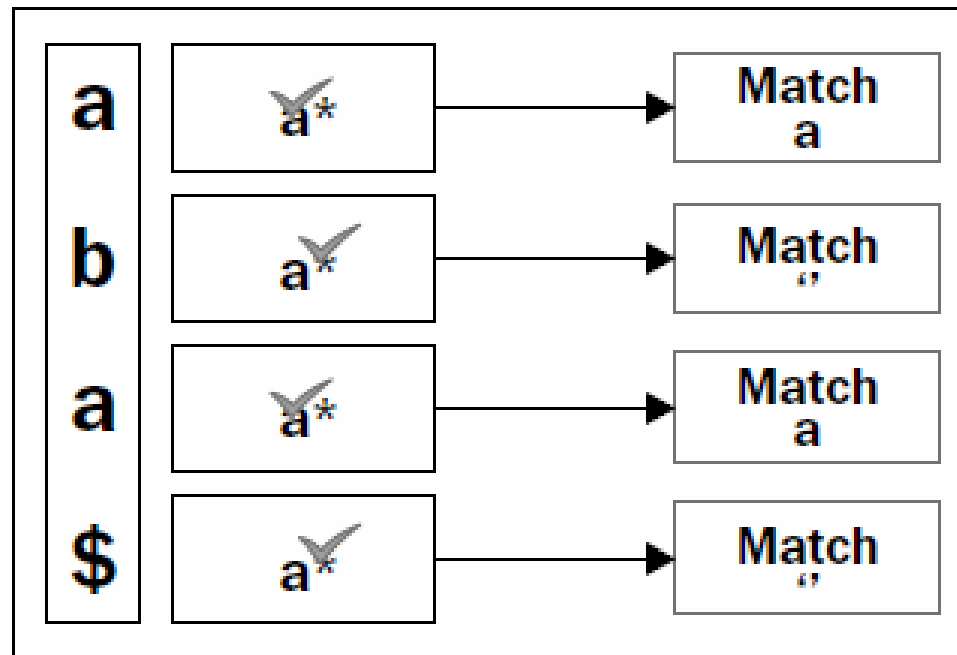
Boundary Matchers

Element	Description
<code>^</code>	Matches the beginning of the line.
<code>N</code>	Matches the followed by character N.
<code>a</code>	Matches the followed by character a.
<code>m</code>	Matches the followed by character m.
<code>e</code>	Matches the followed by character e.
<code>:</code>	Matches the followed by colon symbol.
<code>[\sa-zA-Z]</code>	Then matches the followed by whitespace, or any alphabetic lowercase or uppercase character.
<code>+</code>	The character can be repeated one or more times.
<code>\$</code>	Until the end of the line.

Boundary Matchers

Element	Description
<code>\b</code>	Matches a word boundary.
<code>h</code>	Matches the followed by character h.
<code>e</code>	Matches the followed by character e.
<code>l</code>	Matches the followed by character l.
<code>l</code>	Matches the followed by character l.
<code>o</code>	Matches the followed by character o.
<code>\b</code>	Then matches another followed by word boundary.

Boundary Matchers



`findall` matching process

Functions



Function	Description
<code>compile(pattern[, flags])</code>	Creates a pattern object from a string with a regular expression
<code>search(pattern, string[, flags])</code>	Searches for pattern in string
<code>match(pattern, string[, flags])</code>	Matches pattern at the beginning of string
<code>split(pattern, string[, maxsplit=0])</code>	Splits a string by occurrences of pattern
<code>findall(pattern, string)</code>	Returns a list of all occurrences of pattern in string
<code>sub(pat, repl, string[, count=0])</code>	Substitutes occurrences of pat in string with repl
<code>escape(string)</code>	Escapes all special regular expression characters in string

Regular Expressions

[]	A set of characters	"[a-m]"
\	Signals a special sequence (can also be used to escape special characters)	"\d"
.	Any character (except newline character)	"he..o"
^	Starts with	"^hello"
\$	Ends with	"world\$"
*	Zero or more occurrences	"aix*"
+	One or more occurrences	"aix+"
{}	Exactly the specified number of occurrences	"al{2}"
	Either or	"falls stays"
()	Capture and group	

Regular Expressions

Character	Description	Example
\A	Returns a match if the specified characters are at the beginning of the string	"\AThe"
\b	Returns a match where the specified characters are at the beginning or at the end of a word	r"\bain" r"ain\b"
\B	Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word	r"\Bain" r"ain\B"
\d	Returns a match where the string contains digits (numbers from 0-9)	"\d"
\D	Returns a match where the string DOES NOT contain digits	"\D"
\s	Returns a match where the string contains a white space character	"\s"
\S	Returns a match where the string DOES NOT contain a white space character	"\S"
\w	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore _ character)	"\w"
\W	Returns a match where the string DOES NOT contain any word characters	"\W"
\Z	Returns a match if the specified characters are at the end of the string	"Spain\Z"

Regular Expressions

Set	Description
[arn]	Returns a match where one of the specified characters (a, r, or n) are present
[a-n]	Returns a match for any lower case character, alphabetically between a and n
[^arn]	Returns a match for any character EXCEPT a, r, and n
[0123]	Returns a match where any of the specified digits (0, 1, 2, or 3) are present
[0-9]	Returns a match for any digit between 0 and 9
[0-5][0-9]	Returns a match for any two-digit numbers from 00 and 59
[a-zA-Z]	Returns a match for any character alphabetically between a and z, lower case OR upper case
[+]	In sets, +, *, ., , (), \$, {} has no special meaning, so [+] means: return a match for any +character in the string

Regular Expression

Expression	String	Matched?
^a...s\$	abs	No match
	alias	Match
	abyss	Match
	Alias	No match
	An abacus	No match

Regular Expression Meta Characters



[] - Square brackets

Square brackets specifies a set of characters you wish to match.

Expression	String	Matched?
[abc]	a	1 match
	ac	2 matches
	Hey Jude	No match
	abc de ca	5 matches

Regular Expression Meta Characters



. - Period

A period matches any single character (except newline '\n').

Expression	String	Matched?
..	a	No match
	ac	1 match
	acd	1 match
	acde	2 matches (contains 4 characters)

Regular Expression Meta Characters



^ - Caret

The caret symbol ^ is used to check if a string starts with a certain character.

Expression	String	Matched?
^a	a	1 match
	abc	1 match
	bac	No match
^ab	abc	1 match
	acb	No match (starts with a but not followed by b)

Regular Expression Meta Characters



\$ - Dollar

The dollar symbol \$ is used to check if a string ends with a certain character..

Expression	String	Matched?
a\$	a	1 match
	formula	1 match
	cab	No match

Regular Expression Meta Characters



* - Star

The star symbol * matches zero or more occurrences of the pattern left to it..

Expression	String	Matched?
ma*n	mn	1 match
	man	1 match
	maaan	1 match
	main	No match (a is not followed by n)
	woman	1 match

Regular Expression Meta Characters



+ - Plus

The plus symbol + matches one or more occurrences of the pattern left to it.

Expression	String	Matched?
ma+n	mn	No match (no a character)
	man	1 match
	maaan	1 match
	main	No match (a is not followed by n)
	woman	1 match

Regular Expression Meta Characters



? - Question Mark

The question mark symbol ? matches zero or one occurrence of the pattern left to it.

Expression	String	Matched?
ma?n	mn	1 match
	man	1 match
	maaan	No match (more than one a character)
	main	No match (a is not followed by n)
	woman	1 match

Regular Expression Meta Characters



{ } - Braces

Consider this code: `{n,m}`. This means at least `n`, and at most `m` repetitions of the pattern left to it.

Expression	String	Matched?
<code>a{2,3}</code>	abc dat	No match
	abc daat	1 match (at <u>da</u> at)
	aabc daaat	2 matches (at <u>aa</u> bc and <u>daa</u> at)
	aabc daaaat	2 matches (at <u>aa</u> bc and <u>daaaa</u> at)

Regular Expression Meta Characters



Special Sequences

Special sequences make commonly used patterns easier to write. Here's a list of special sequences:

`\A` - Matches if the specified characters are at the start of a string.

Expression	String	Matched?
\Athe	the sun	Match
	In the sun	No match

Regular Expression Meta Characters



`\b` - Matches if the specified characters are at the beginning or end of a word.

Expression	String	Matched?
\bfoo	football	Match
	a football	Match
	afootball	No match
foo\b	the foo	Match
	the afoo test	Match
	the afootest	No match

Regular Expression Meta Characters



`\B` - Opposite of `\b`. Matches if the specified characters are not at the beginning or end of a word.

Expression	String	Matched?
<code>\Bfoo</code>	football	No match
	a football	No match
	afootball	Match
<code>foo\B</code>	the foo	No match
	the afoo test	No match
	the afootest	Match

Regular Expression Meta Characters



`\d` - Matches any decimal digit. Equivalent to `[0-9]`

Expression	String	Matched?
<code>\d</code>	12abc3	3 matches (at <u>1</u> 2abc <u>3</u>)
	Python	No match

Regular Expression Meta Characters



`\D` - Matches any non-decimal digit. Equivalent to `[^0-9]`

Expression	String	Matched?
<code>\D</code>	1ab34"50	3 matches (at 1 <u>a</u> b34"50)
	1345	No match

Regular Expression Meta Characters



`\s` - Matches where a string contains any whitespace character. Equivalent to `[\t\n\r\f\v]`.

Expression	String	Matched?
<code>\s</code>	Python RegEx	1 match
	PythonRegEx	No match

Regular Expression Meta Characters



`\w` - Matches any alphanumeric character (digits and alphabets). Equivalent to `[a-zA-Z0-9_]`. By the way, underscore `_` is also considered an alphanumeric character.

Expression	String	Matched?
<code>\w</code>	<code>12&": ;c</code>	3 matches (at <u>1</u> <u>2</u> &": ; <u>c</u>)
	<code>%"> !</code>	No match

Regular Expression Meta Characters



`\Z` - Matches if the specified characters are at the end of a string.

Expression	String	Matched?
\ZPython	I like Python	1 match
	I like Python	No match
	Python is fun.	No match



Regular Expression

`re.findall()`

The `re.findall()` method returns a list of strings containing all matches.

```
string = 'hello 12 hi 89. Howdy 34'  
pattern = '\d+'
```

```
result = re.findall(pattern, string)  
print(result)
```

```
# Output: ['12', '89', '34']
```



Regular Expression

`re.split()`

The `re.split` method splits the string where there is a match and returns a list of strings where the splits have occurred.

```
string = 'Twelve:12 Eighty nine:89.'  
pattern = '\d+'
```

```
result = re.split(pattern, string)  
print(result)
```

```
# Output: ['Twelve:', ' Eighty nine:', '.']
```



Regular Expression

`re.split()`

The `re.split` method splits the string where there is a match and returns a list of strings where the splits have occurred.

```
string = 'Twelve:12 Eighty nine:89.'  
pattern = '\d+'
```

```
result = re.split(pattern, string)  
print(result)
```

```
# Output: ['Twelve:', ' Eighty nine:', '.']
```



Regular Expression

`re.split()`

The `re.split` method splits the string where there is a match and returns a list of strings where the splits have occurred.

```
string = 'Twelve:12 Eighty nine:89 Nine:9.'  
pattern = '\d+'
```

```
# maxsplit = 1  
# split only at the first occurrence  
result = re.split(pattern, string, 1)  
print(result)
```

```
# Output: ['Twelve:', ' Eighty nine:89 Nine:9.']
```



Regular Expression

`re.sub()`

The syntax of `re.sub()` is:

`re.sub(pattern, replace, string)`

The method returns a string where matched occurrences are replaced with the content of replace variable.

multiline string

`string = 'abc 12\`

`de 23 \n f45 6'`

`# matches all whitespace characters`

`pattern = '\s+'`

`# empty string`

`replace = "`

`new_string = re.sub(pattern, replace, string)`

`print(new_string)`

`# Output: abc12de23f456`



Regular Expression

`re.sub()`

The syntax of `re.sub()` is:

`re.sub(pattern, replace, string)`

The method returns a string where matched occurrences are replaced with the content of replace variable.

```
# multiline string
```

```
string = 'abc 12\
```

```
de 23 \n f45 6'
```

```
# matches all whitespace characters
```

```
pattern = '\s+'
```

```
replace = ""
```

```
new_string = re.sub(r'\s+', replace, string, 1)
```

```
print(new_string)
```

```
# Output:
```

```
# abc12de 23
```

```
# f45 6
```



Regular Expression

`re.subn()`

The `re.subn()` is similar to `re.sub()` expect it returns a tuple of 2 items containing the new string and the number of substitutions made..

```
# multiline string
string = 'abc 12\
de 23 \n f45 6'
```

```
# matches all whitespace characters
pattern = '\s+'
```

```
# empty string
replace = ''
```

```
new_string = re.subn(pattern, replace, string)
print(new_string)
```

```
# Output: ('abc12de23f456', 4)
```




Regular Expression

`re.search()`

The `re.search()` method takes two arguments: a pattern and a string. The method looks for the first location where the RegEx pattern produces a match with the string.

If the search is successful, `re.search()` returns a match object; if not, it returns `None`.

```
match = re.search(pattern, str)
```



Regular Expression

```
string = "Python is fun"
```

```
# check if 'Python' is at the beginning  
match = re.search('\APython', string)
```

```
if match:  
    print("pattern found inside the string")  
else:  
    print("pattern not found")
```

```
# Output: pattern found inside the string
```



Regular Expression

Match object

You can get methods and attributes of a match object using `dir()` function.

Some of the commonly used methods and attributes of match objects are:

`match.group()`

The `group()` method returns the part of the string where there is a match.



Regular Expression

```
string = '39801 356, 2102 1111'
```

```
# Three digit number followed by space followed by two digit number  
pattern = '(\d{3}) (\d{2})'
```

```
# match variable contains a Match object.  
match = re.search(pattern, string)
```

```
if match:  
    print(match.group())  
else:  
    print("pattern not found")
```

```
# Output: 801 35
```



Subroutines

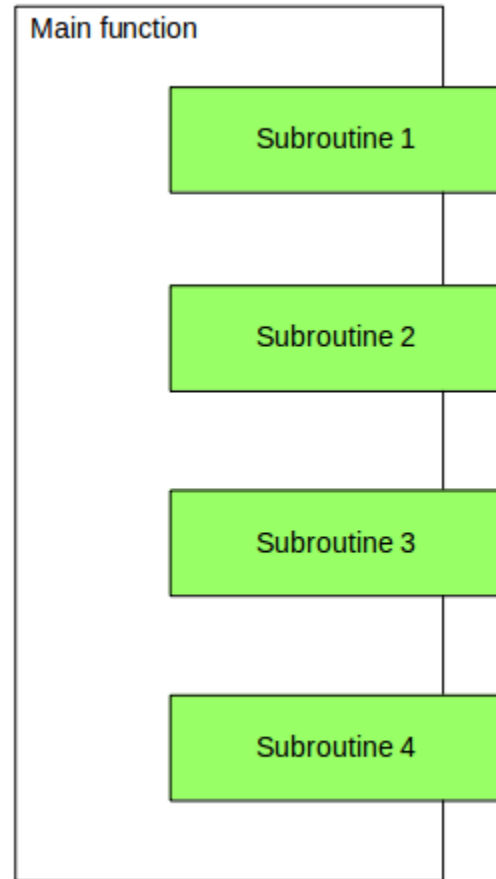
- A function is a sequence of instructions packed as a unit to perform a certain task.
- When the logic of a complex function is divided into several self-contained steps that are themselves functions, then these functions are called helper functions or subroutines.



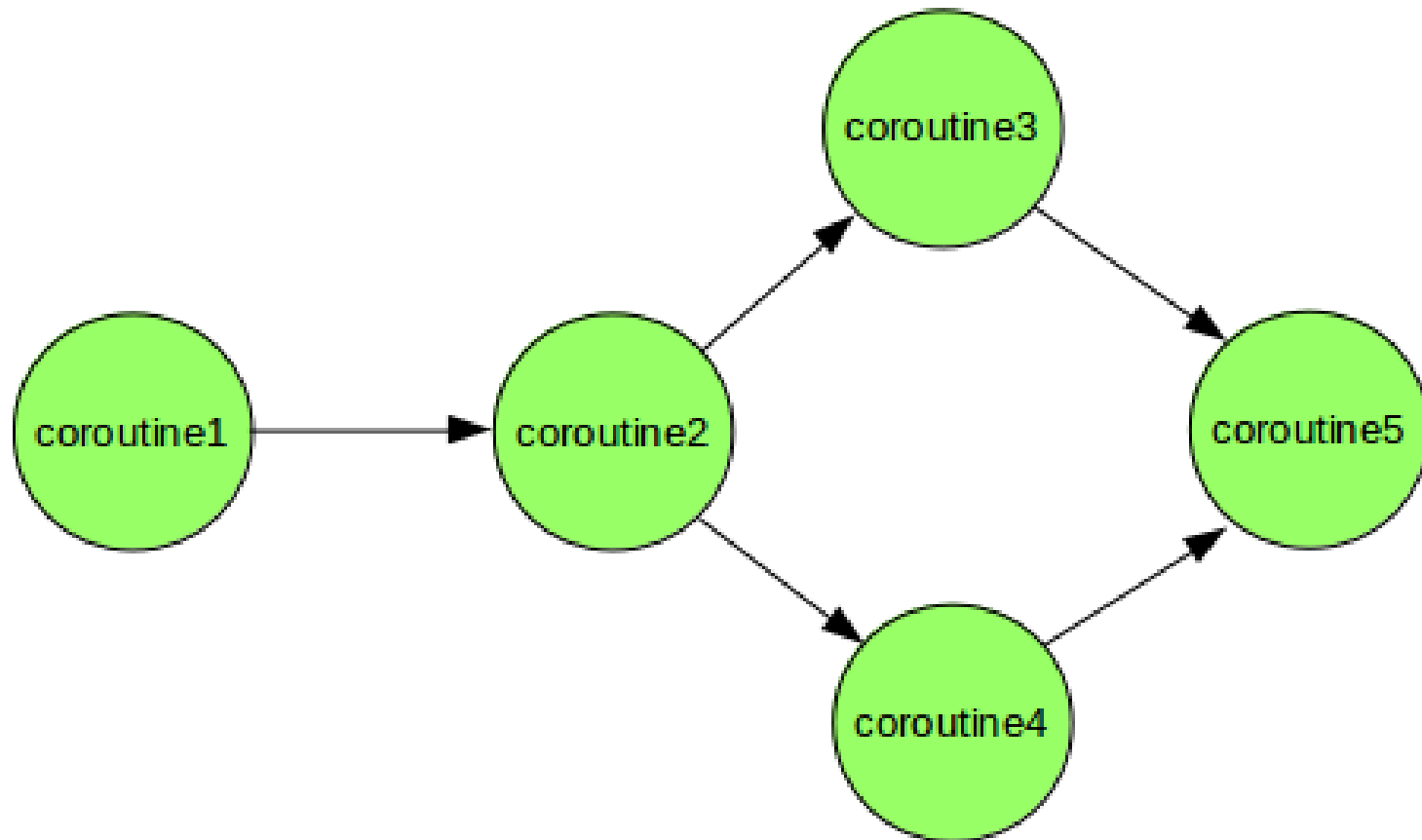
Subroutines

- Subroutines in Python are called by the main function which is responsible for coordinating the use of these subroutines.
- Subroutines have a single-entry point.
- Coroutines are generalizations of subroutines.
- They are used for cooperative multitasking where a process voluntarily yield (give away) control periodically or when idle to enable multiple applications to be run simultaneously.

Subroutines



Coroutines



Coroutines

- Unlike subroutines, coroutines have many entry points for suspending and resuming execution.
- Coroutine can suspend its execution and transfer control to other coroutine and can resume execution from the point it left off.
- Unlike subroutines, there is no main function to call coroutines in a particular order and coordinate the results.
- Coroutines are cooperative that means they link together to form a pipeline.
- One coroutine may consume input data and send it to other that process it.
- Finally, there may be a coroutine to display the result.



Chaining coroutines for creating pipeline

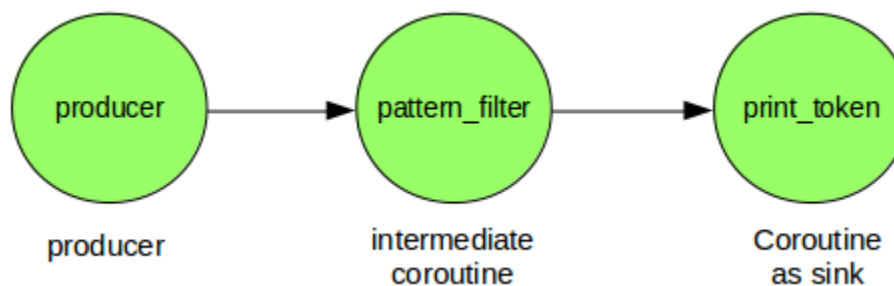
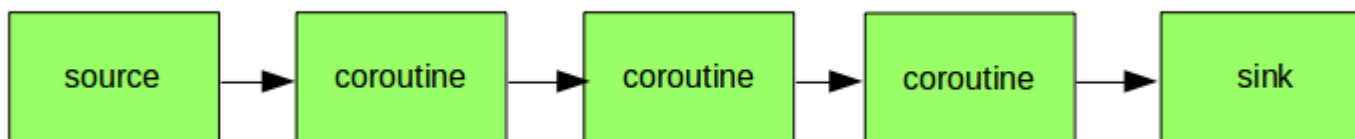
- Coroutines can be used to set pipes.
- We can chain together coroutines and push data through the pipe using `send()` method. A pipe needs :
- An initial source(producer) derives the whole pipeline. The producer is usually not a coroutine, it's just a simple method.
- A sink, which is the endpoint of the pipe. A sink might collect all data and display it.



Chaining coroutines for creating pipeline

- Coroutines are basically functions whose execution can be paused/suspended at a particular point.
- We can resume the execution from that same point later whenever we want.

Chaining coroutines for creating pipeline





Collections Framework

- Collections in Python are containers that are used to store collections of data, for example, list, dict, set, tuple etc.
- These are built-in collections.
- Several modules have been developed that provide additional data structures to store collections of data.
- One such module is the Python collections module.



- Counter
- defaultdict
- OrderedDict
- deque
- ChainMap
- namedtuple()



- The Counter
- Counter is a subclass of dictionary object.
- The Counter() function in collections module takes an iterable or a mapping as the argument and returns a Dictionary.
- In this dictionary, a key is an element in the iterable or the mapping and value is the number of times that element exists in the iterable or the mapping.
- You have to import the Counter class before you can create a counter instance.



- The Counter
- Counter is a subclass of dictionary object.
- The Counter() function in collections module takes an iterable or a mapping as the argument and returns a Dictionary.
- In this dictionary, a key is an element in the iterable or the mapping and value is the number of times that element exists in the iterable or the mapping.
- You have to import the Counter class before you can create a counter instance.



- The defaultdict
- The defaultdict works exactly like a python dictionary, except for it does not throw KeyError when you try to access a non-existent key.
- Instead, it initializes the key with the element of the data type that you pass as an argument at the creation of defaultdict. The data type is called default_factory.



- The OrderedDict
- OrderedDict is a dictionary where keys maintain the order in which they are inserted, which means if you change the value of a key later, it will not change the position of the key.
- The deque (Double Ended Queue)
- The deque is a list optimized for inserting and removing items.



- The ChainMap
- ChainMap is used to combine several dictionaries or mappings. It returns a list of dictionaries.
- The namedtuple()
- The namedtuple() returns a tuple with names for each position in the tuple.
- One of the biggest problems with ordinary tuples is that you have to remember the index of each field of a tuple object.
- This is obviously difficult. The namedtuple was introduced to solve this problem.

Meta class

- A metaclass is a class whose instances are classes. Like an "ordinary" class defines the behavior of the instances of the class, a metaclass defines the behavior of classes and their instances.
- Metaclasses are not supported by every object oriented programming language. Those programming language, which support metaclasses, considerably vary in way the implement them. Python is supporting them



Meta Class

- There are numerous use cases for metaclasses. Just to name a few:
- logging and profiling
- interface checking
- registering classes at creation time
- automatically adding new methods
- automatic property creation
- proxies
- automatic resource locking/synchronization.

Meta Class





Dynamic Meta class

- `>>> type(3)`
- `<class 'int'>`

- `>>> type(['foo', 'bar', 'baz'])`
- `<class 'list'>`

- `>>> t = (1, 2, 3, 4, 5)`
- `>>> type(t)`
- `<class 'tuple'>`



Dynamic Meta class

- `>>> class Foo:`
- `... pass`
- `...`
- `>>> type(Foo())`
- `<class '__main__.Foo'>`

Dynamic Meta class

- You can also call `type()` with three arguments—`type(<name>, <bases>, <dct>)`:
- `<name>` specifies the class name. This becomes the `__name__` attribute of the class.
- `<bases>` specifies a tuple of the base classes from which the class inherits. This becomes the `__bases__` attribute of the class.
- `<dct>` specifies a namespace dictionary containing definitions for the class body. This becomes the `__dict__` attribute of the class.

Dynamic Meta class

- `>>> Bar = type('Bar', (Foo,), dict(attr=100))`
- `>>> x = Bar()`
- `>>> x.attr`
- `100`
- `>>> x.__class__`
- `<class '__main__.Bar'>`
- `>>> x.__class__.__bases__`
- `(<class '__main__.Foo'>,)`

Threads

- In computing, a process is an instance of a computer program that is being executed.
- Any process has 3 basic components:
- An executable program.
- The associated data needed by the program (variables, work space, buffers, etc.)
- The execution context of the program (State of process)

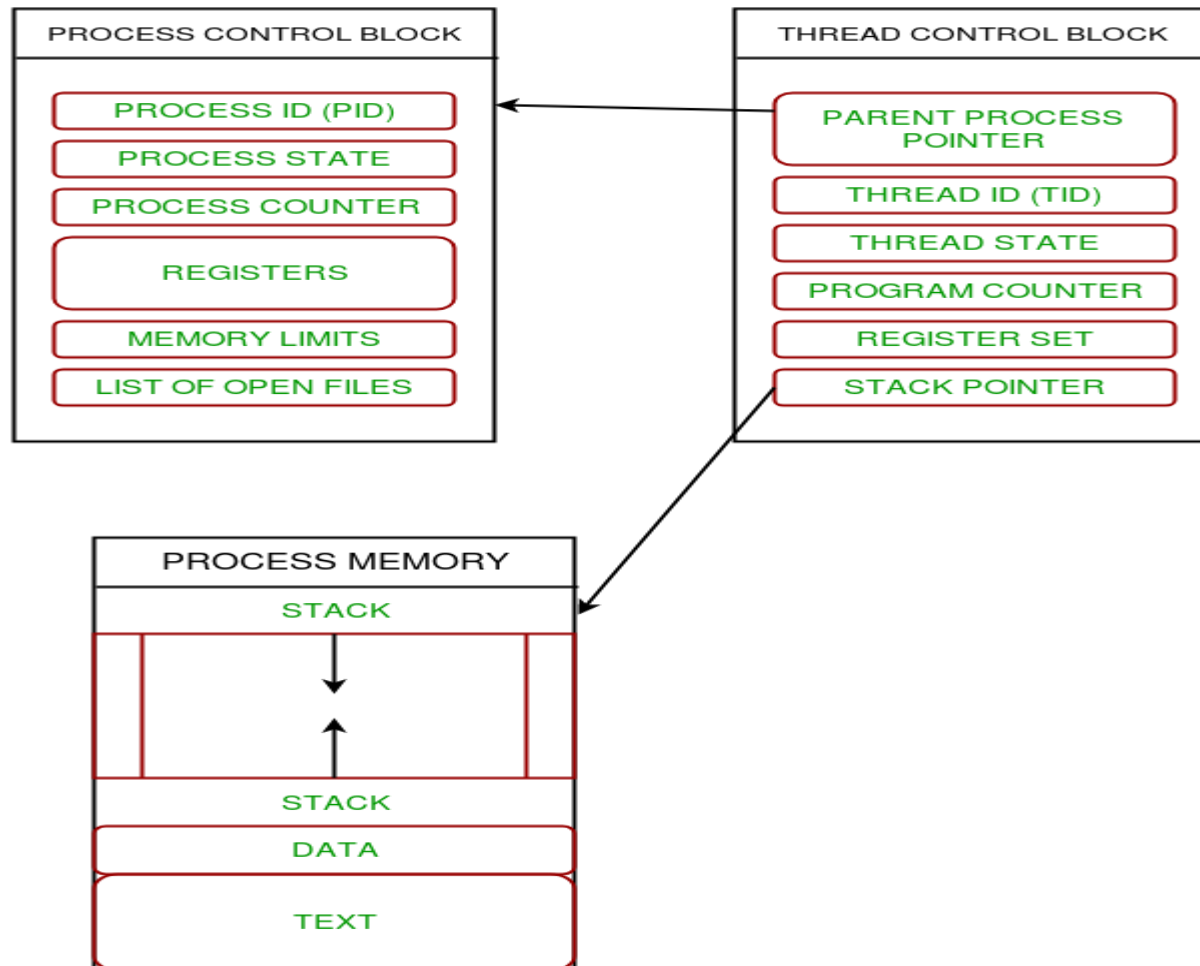
Threads

- A thread is an entity within a process that can be scheduled for execution. Also, it is the smallest unit of processing that can be performed in an OS (Operating System).
- In simple words, a thread is a sequence of such instructions within a program that can be executed independently of other code. For simplicity, you can assume that a thread is simply a subset of a process!

Threads

- A thread contains all this information in a Thread Control Block (TCB):
- Thread Identifier: Unique id (TID) is assigned to every new thread
- Stack pointer: Points to thread's stack in the process. Stack contains the local variables under thread's scope.
- Program counter: a register which stores the address of the instruction currently being executed by thread.
- Thread state: can be running, ready, waiting, start or done.
- Thread's register set: registers assigned to thread for computations.
- Parent process Pointer: A pointer to the Process control block (PCB) of the process that the thread lives on.

Threads

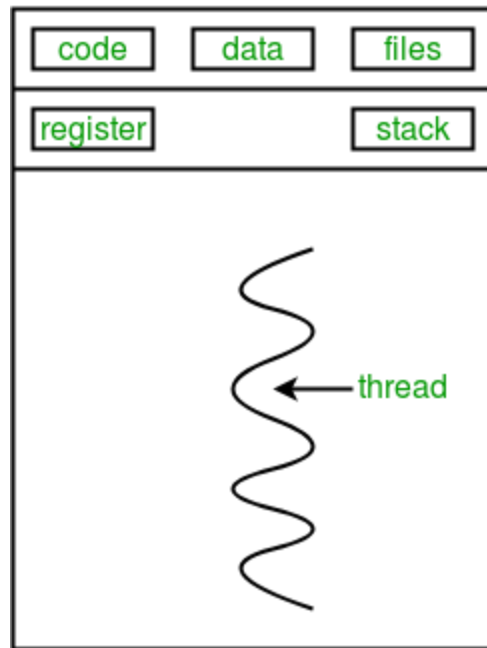




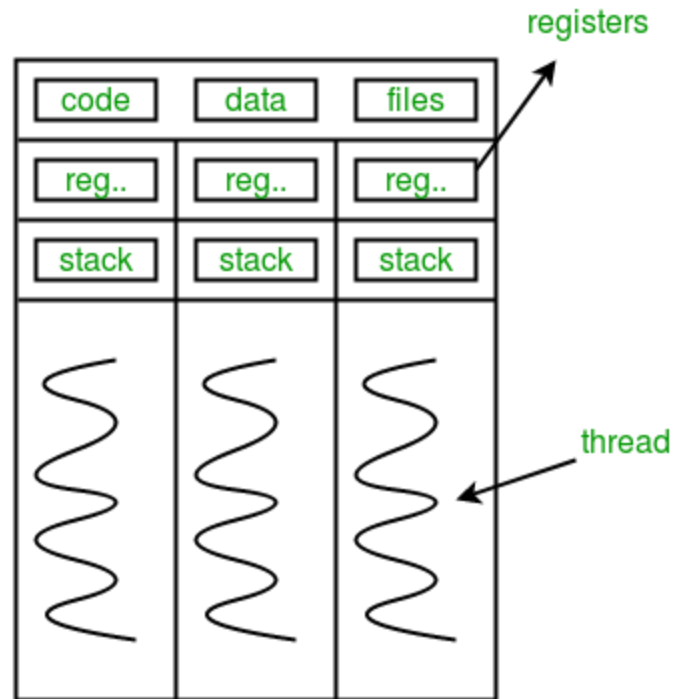
Threads

- Multiple threads can exist within one process where:
- Each thread contains its own register set and local variables (stored in stack).
- All thread of a process share global variables (stored in heap) and the program code.
- Consider the diagram below to understand how multiple threads exist in memory:

Threads

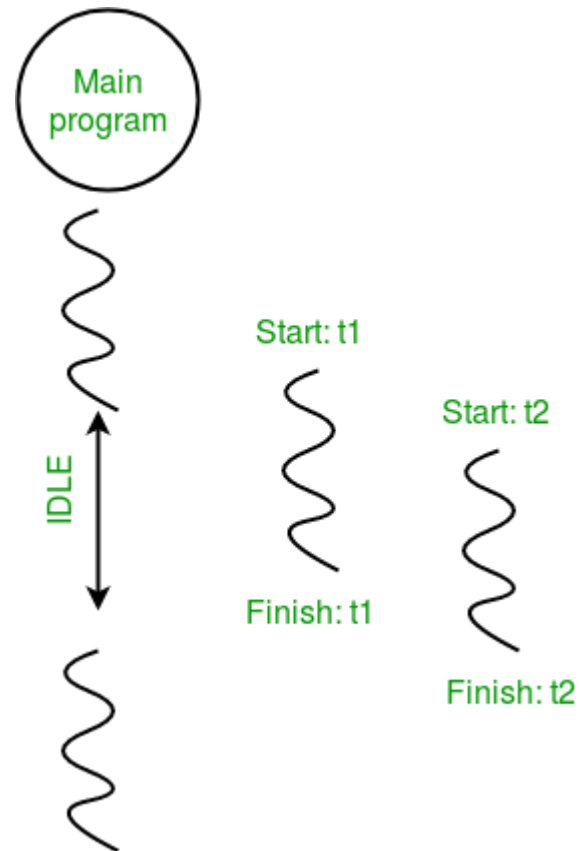


single-threaded process



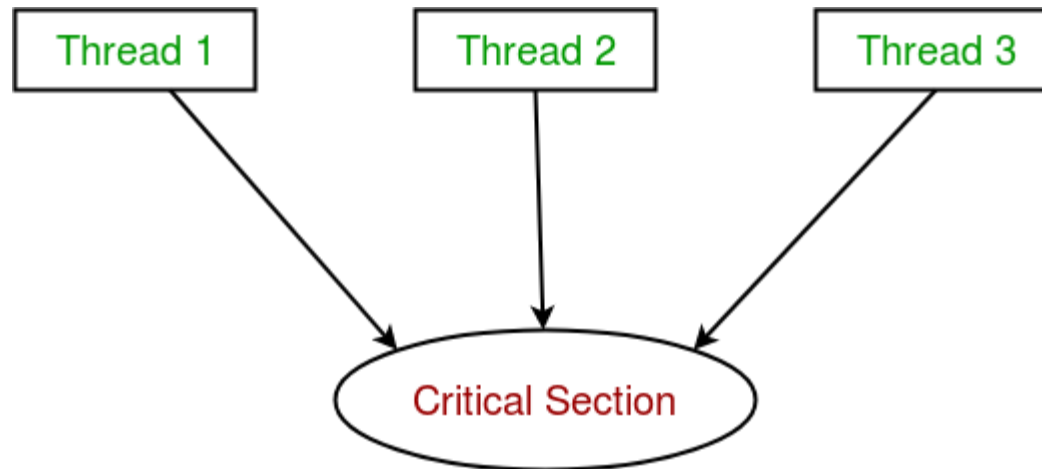
multithreaded process

Threads





- Thread synchronization is defined as a mechanism which ensures that two or more concurrent threads do not simultaneously execute some particular program segment known as critical section.



Multithreading

- Advantages:
 - It doesn't block the user. This is because threads are independent of each other.
 - Better use of system resources is possible since threads execute tasks parallelly.
 - Enhanced performance on multi-processor machines.
 - Multi-threaded servers and interactive GUIs use multithreading exclusively.
- Disadvantages:
 - As number of threads increase, complexity increases.
 - Synchronization of shared resources (objects, data) is necessary.
 - It is difficult to debug, result is sometimes unpredictable.
 - Potential deadlocks which leads to starvation, i.e. some threads may not be served with a bad design
 - Constructing and synchronizing threads is CPU/memory intensive.

Synchronization By using RLock concept in Python

- The standard lock object does not care which thread is currently holding that lock.
- If the lock is being held by one thread, and if any other thread tries to acquire the lock, then it will be blocked, even if it's the same thread that is already holding the lock.
- if the Thread calls recursive functions or nested access to resources, then the thread may try to acquire the same lock again and again, which may result in blocking of our thread.
- Hence Traditional Locking mechanism won't work for executing recursive functions.



Synchronization By using RLock concept in Python

- To overcome the above problem of Simple Lock, we should go for RLock(Reentrant Lock).
- Reentrant means the thread can acquire the same lock again and again.
- This will block the thread only if the lock is held by any other thread.
- Reentrant facility is available only for owner thread but not for other threads.



Synchronization By using RLock concept in Python

- This RLock keeps track of recursion level and hence for every `acquire()` there should be a `release()` call available.
- The number of `acquire()` calls and `release()` calls should be matched then for the lock to be released i.e if there are two `acquire` calls then there should be two `release` calls for the lock to be released.
- If there is only one `release` call for two `acquire` calls then the lock won't be released.

Lock vs RLock

Lock	RLock
A Lock object can be acquired by only one thread at a time. Even the owner thread also cannot acquire multiple times.	A RLock object can be acquired by only one thread at a time, but the owner thread can acquire the same lock object multiple times.
Not suitable to execute recursive functions and nested access calls	Best suitable to execute recursive functions and nested access calls
In this case the Lock object will only see whether its Locked or unlocked and it will never hold or take care of owner thread and recursion level.	In this case RLock object will see whether its Locked or unlocked and also about owner thread information and recursion level.



Semaphore

- Semaphore is advanced Synchronization Mechanism.
- Sometimes there might be requirements where at a time a particular number of threads should be allowed to access the resources.
- Like, at a time 10 members are allowed to access the database server or at a time 4 members are allowed to access Network connection.



Time class

- Python has a module named time to handle time-related tasks.
- To use functions defined in the module, we need to import the module first.
 - `time.time()`
 - `time.ctime()`
 - `time.sleep()`
 - `time.struct_time` Class
 - `time.localtime()`
 - `time.gmtime()`
 - `time.mktime()`
 - `time.asctime()`
 - `time.strftime()`
 - `time.strptime()`
-

Shell Commands from Cygwin and Python

```
/cygdrive/f/Python_Virtusa/day4/Threads

alasubramaniam@DESKTOP-55AGI0I /cygdrive/f
cd Python_Virtusa

alasubramaniam@DESKTOP-55AGI0I /cygdrive/f/Python_Virtusa
cd day4/Threads

alasubramaniam@DESKTOP-55AGI0I /cygdrive/f/Python_Virtusa/day4/Threads
ls
hellcommandsdemo.py  threaddemo.py  threaddemo1.py  threadsync.py  timedemo.py

alasubramaniam@DESKTOP-55AGI0I /cygdrive/f/Python_Virtusa/day4/Threads
python shellcommandsdemo.py
total 25
-rwxrwx---+ 1 Balasubramaniam None    0 Apr  7 19:51 .
-rwxrwx---+ 1 Balasubramaniam None    0 Apr  7 15:48 ..
-rwxrwx---+ 1 Balasubramaniam None   45 Apr  7 19:51 shellcommandsdemo.py
-rwxrwx---+ 1 Balasubramaniam None  758 Apr  7 05:11 threaddemo.py
-rwxrwx---+ 1 Balasubramaniam None  877 Apr  7 05:12 threaddemo1.py
-rwxrwx---+ 1 Balasubramaniam None  803 Apr  7 05:16 threadsync.py
-rwxrwx---+ 1 Balasubramaniam None 1476 Apr  7 19:49 timedemo.py

alasubramaniam@DESKTOP-55AGI0I /cygdrive/f/Python_Virtusa/day4/Threads
```



Parallel Processing

- Python offers three possible ways to handle that.
- First, you can execute functions in parallel using the multiprocessing module.
- Second, an alternative to processes are threads.
- Third, you can call external programs using the `system()` method of the `os` module, or methods provided by the `subprocess` module, and collect the results afterwards.

Parallel Processing

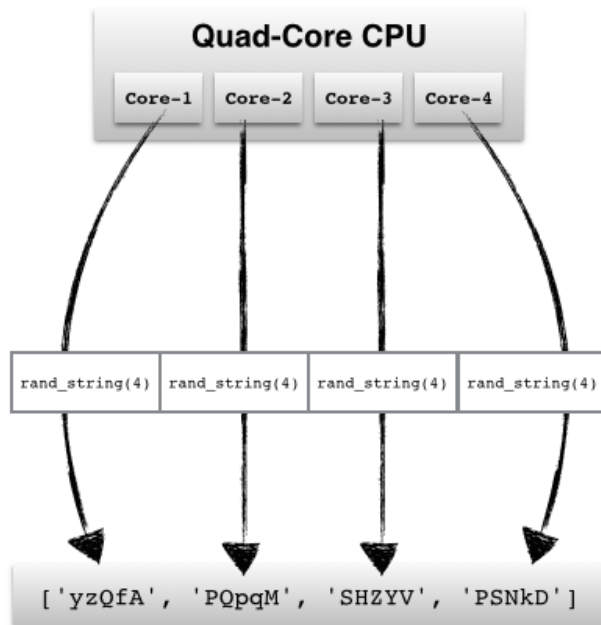
- The multiprocessing module covers a nice selection of methods to handle the parallel execution of routines.
- This includes processes, pools of agents, queues, and pipes.
- What is Synchronous and Asynchronous execution?
- In parallel processing, there are two types of execution: Synchronous and Asynchronous.
- A synchronous execution is one the processes are completed in the same order in which it was started. This is achieved by locking the main program until the respective processes are finished.
- Asynchronous, on the other hand, doesn't involve locking. As a result, the order of results can get mixed up but usually gets done quicker.

Parallel Processing

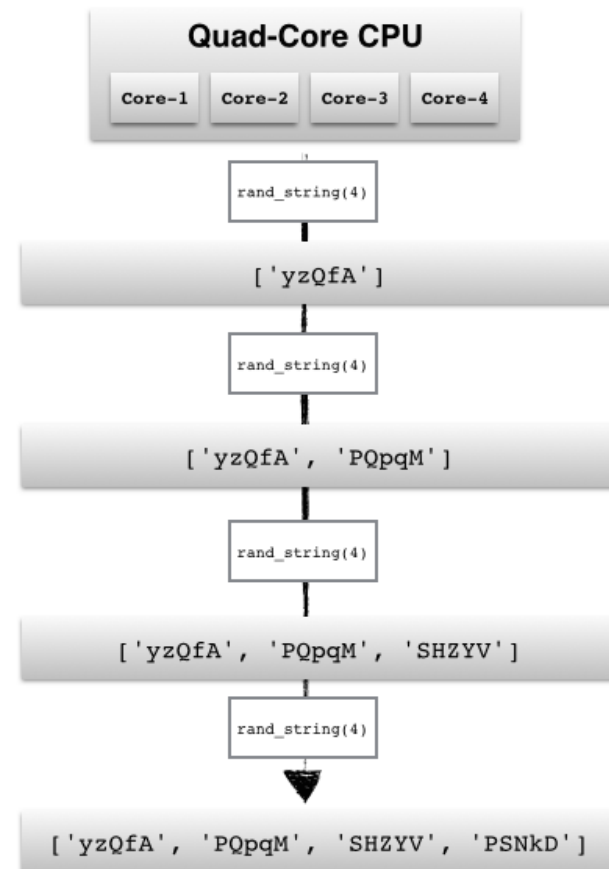
- There are 2 main objects in multiprocessing to implement parallel execution of a function: The Pool Class and the Process Class.
- Pool Class
 - Synchronous execution
 - Pool.map() and Pool.starmap()
 - Pool.apply()
- Asynchronous execution
 - Pool.map_async() and Pool.starmap_async()
 - Pool.apply_async()
- Process Class

Parallel Processing

[parallel processing]



[serial processing]



Parallel Processing

Steps to parallelize a function:

- To parallelize a function, you need to execute it in various processors.
- Start with initializing a pool with x number of processors.
- Now, pass that function to any one of the parallelization methods of a pool.
- The `apply()`, `starmap()` and `map()` methods of the `multiprocessing.Pool()` will allow you to execute functions parallelly.
- Here, both `map` and `apply` functions are used to take the functions for parallelization as the main argument.
- The only difference is that `map()` will only bring one iterable as an argument while the `apply()` collects args argument which takes the parameters of the parallelized function as an argument.



- Normal Lock objects cannot be acquired more than once, even by the same thread.
- This can introduce undesirable side-effects if a lock is accessed by more than one function in the same call chain.

Limiting Concurrent Access to Resources



- Sometimes it is useful to allow more than one worker access to a resource at a time, while still limiting the overall number.
- For example, a connection pool might support a fixed number of simultaneous connections, or a network application might support a fixed number of concurrent downloads.
- A Semaphore is one way to manage those connections.

Limiting Concurrent Access to Resources



- A semaphore is based on an internal counter which is decremented each time `acquire()` is called and incremented each time `release()` is called.
- If the counter is equal to 0 then `acquire()` blocks. It is the Python implementation of the Dijkstra semaphore concept: `P()` and `V()`.
- Using a semaphore makes sense when you want to control access to a resource with limited capacity like a server..

Reentrant Lock

- RLock is the reentrant lock provided by python.
- A reentrant lock or simply RLock allows itself to be locked several times.
- The RLock is released only after equal number `release()` calls as the `acquire()` calls made to it.
- Unlike the primitive lock given by the `Lock` class the RLock can only be released by the thread holding the RLock.



Sockets, IPv4, and Simple Client/Server Programming

- Python's socket module has both class-based and instances-based utilities.
- The difference between a class-based and instance-based method is that the former doesn't need an instance of a socket object.
- For example, in order to print your machine's IP address, you don't need a socket object. Instead, you can just call the socket's class-based methods.
- On the other hand, if you need to send some data to a server application, it is more intuitive that you create a socket object to perform that explicit operation.



Sockets, IPv4, and Simple Client/Server Programming

- The following is a list of those third-party libraries with their download URLs:
- ff ntplib: <https://pypi.python.org/pypi/ntplib/>
- ff diesel: <https://pypi.python.org/pypi/diesel/>
- ff nmap: <https://pypi.python.org/pypi/python-nmap>
- ff scapy: <https://pypi.python.org/pypi/scapy>
- ff netifaces: <https://pypi.python.org/pypi/netifaces/>
- ff netaddr: <https://pypi.python.org/pypi/netaddr>
- ff pyopenssl: <https://pypi.python.org/pypi/pyOpenSSL>



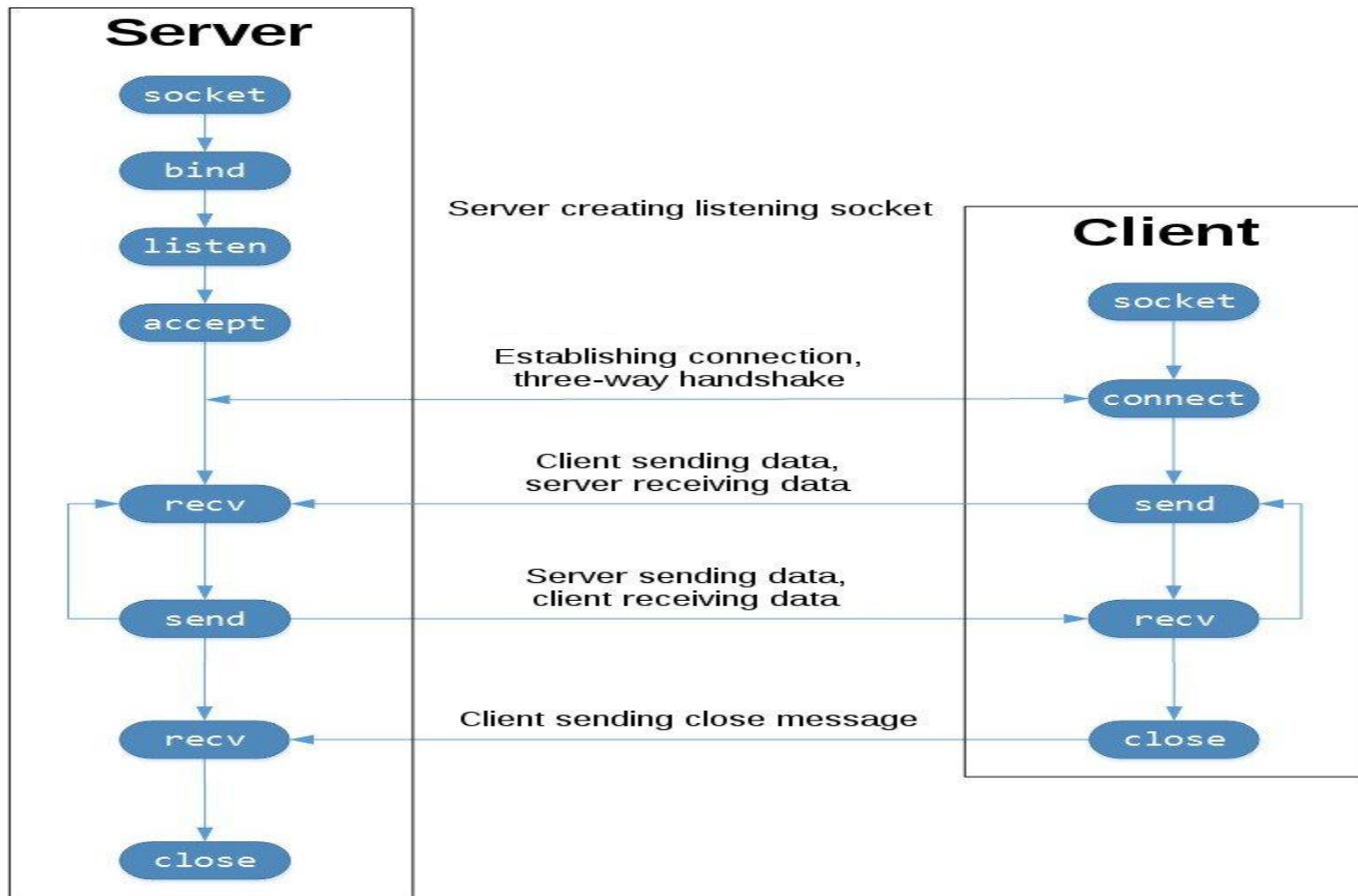
Sockets, IPv4, and Simple Client/Server Programming

- pygeocoder: <https://pypi.python.org/pypi/pygocoder>
- ff pyyaml: <https://pypi.python.org/pypi/PyYAML>
- ff requests: <https://pypi.python.org/pypi/requests>
- ff feedparser: <https://pypi.python.org/pypi/feedparser>
- ff paramiko: <https://pypi.python.org/pypi/paramiko/>
- ff fabric: <https://pypi.python.org/pypi/Fabric>
- ff supervisor: <https://pypi.python.org/pypi/supervisor>
- ff xmlrpclib: <https://pypi.python.org/pypi/xmlrpclib>
- ff SOAPpy: <https://pypi.python.org/pypi/SOAPpy>
- ff bottlenose: <https://pypi.python.org/pypi/bottlenose>
- ff construct: <https://pypi.python.org/pypi/construct>

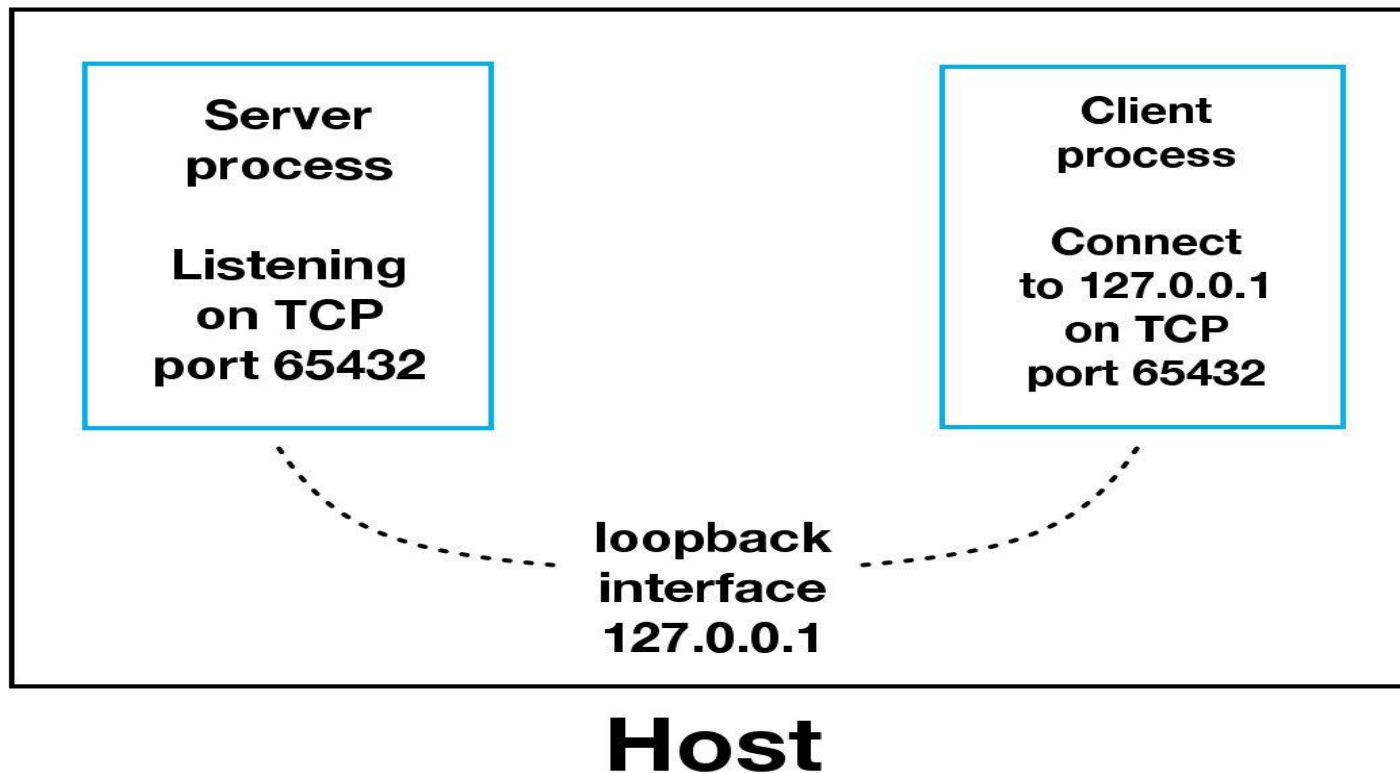
Sockets and Networking

- Socket programming is a way of connecting two nodes on a network to communicate with each other.
- One socket(node) listens on a particular port at an IP, while other socket reaches out to the other to form a connection.
- Server forms the listener socket while client reaches out to the server.
- They are the real backbones behind web browsing. In simpler terms there is a server and a client.

Sockets and Networking



Sockets and Networking



Signal Handling

- 1 (SIGHUP): terminate a connection, or reload the configuration for daemons
- 2 (SIGINT): interrupt the session from the dialogue station
- 3 (SIGQUIT): terminate the session from the dialogue station
- 4 (SIGILL): illegal instruction was executed
- 5 (SIGTRAP): do a single instruction (trap)
- 6 (SIGABRT): abnormal termination
- 7 (SIGBUS): error on the system bus
- 8 (SIGFPE): floating point error
- 9 (SIGKILL): immediately terminate the process
- 10 (SIGUSR1): user-defined signal
- 11 (SIGSEGV): segmentation fault due to illegal access of a memory segment
- 12 (SIGUSR2): user-defined signal
- 13 (SIGPIPE): writing into a pipe, and nobody is reading from it
- 14 (SIGALRM): the timer terminated (alarm)
- 15 (SIGTERM): terminate the process in a soft way



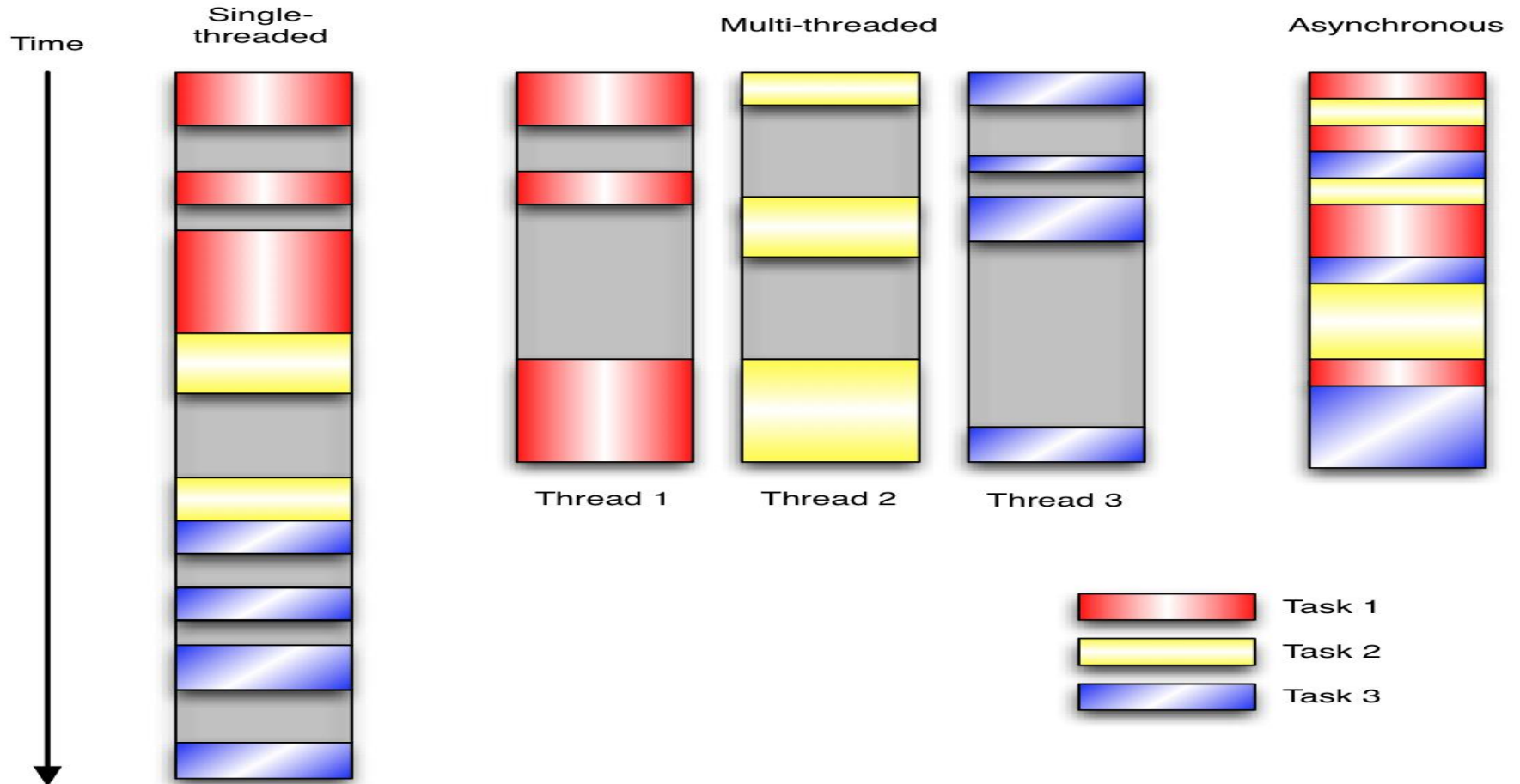
Twisted

- The main idea behind Twisted is that it gives us the parallelism of multithreading programming with the ease of reasoning of single threaded programming.

- In 2000, glyph, the creator of Twisted, was working on a text-based multiplayer game called Twisted Reality.
- It was a big mess of threads, 3 per connection, in Java.
- There was a thread for input that would block on reads, a thread for output that would block on some kind of write, and a "logic" thread that would sleep while waiting for timers to expire or events to queue.
- As players moved through the virtual landscape and interacted, threads were deadlocking, caches were getting corrupted, and the locking logic was never quite right—the use of threads had made the software complicated, buggy, and hard to scale.

- Over time, Twisted Reality the game became Twisted the networking platform, which would do what existing networking platforms in Python didn't:
- Use event-driven programming instead of multi-threaded programming.
- Be cross-platform: provide a uniform interface to the event notification systems exposed by major operating systems.
- Be "batteries-included": provide implementations of popular application-layer protocols out of the box, so that Twisted is immediately useful to developers.
- Conform to RFCs, and prove conformance with a robust test suite.
- Make it easy to use multiple networking protocols together.
- Be extensible.

Twisted





Twisted Architecture

- Twisted is an event-driven networking engine. Event-driven programming is so integral to Twisted's design philosophy that it is worth taking a moment to review what exactly event-driven programming means.
- Event-driven programming is a programming paradigm in which program flow is determined by external events. It is characterized by an event loop and the use of callbacks to trigger actions when events happen. Two other common programming paradigms are (single-threaded) synchronous and multi-threaded programming.
- Let's compare and contrast single-threaded, multi-threaded, and event-driven programming models with an example. Figure shows the work done by a program over time under these three models. The program has three tasks to complete, each of which blocks while waiting for I/O to finish. Time spent blocking on I/O is greyed out.



Twisted Architecture

- In the single-threaded synchronous version of the program, tasks are performed serially.
- If one task blocks for a while on I/O, all of the other tasks have to wait until it finishes and they are executed in turn.
- This definite order and serial processing are easy to reason about, but the program is unnecessarily slow if the tasks don't depend on each other, yet still have to wait for each other.
- In the threaded version of the program, the three tasks that block while doing work are performed in separate threads of control.
- These threads are managed by the operating system and may run concurrently on multiple processors or interleaved on a single processor.
- This allows progress to be made by some threads while others are blocking on resources. This is often more time-efficient than the analogous synchronous program,

Twisted Architecture

- but one has to write code to protect shared resources that could be accessed concurrently from multiple threads.
- Multi-threaded programs can be harder to reason about because one now has to worry about thread safety via process serialization (locking), reentrancy, thread-local storage, or other mechanisms, which when implemented improperly can lead to subtle and painful bugs.

Twisted Architecture

- The event-driven version of the program interleaves the execution of the three tasks, but in a single thread of control.
- When performing I/O or other expensive operations, a callback is registered with an event loop, and then execution continues while the I/O completes.
- The callback describes how to handle an event once it has completed.
- The event loop polls for events and dispatches them as they arrive, to the callbacks that are waiting for them.
- This allows the program to make progress when it can without the use of additional threads.
- Event-driven programs can be easier to reason about than multi-threaded programs because the programmer doesn't have to worry about thread safety.



Twisted Architecture

- The event-driven model is often a good choice when there are:
- many tasks, that are...
- largely independent (so they don't have to communicate with or wait on each other), and...
- some of these tasks block while waiting on events.

Twisted Architecture

- Twisted implements the reactor design pattern, which describes demultiplexing and dispatching events from multiple sources to their handlers in a single-threaded environment.
- The core of Twisted is the reactor event loop.
- The reactor knows about network, file system, and timer events.
- It waits on and then handles these events, abstracting away platform-specific behavior and presenting interfaces to make responding to events anywhere in the network stack easy.

Reactor

- Reactor Basics
- The reactor is the core of the event loop within Twisted – the loop which drives applications using Twisted.
- The event loop is a programming construct that waits for and dispatches events or messages in a program.
- It works by calling some internal or external “event provider”, which generally blocks until an event has arrived, and then calls the relevant event handler (“dispatches the event”).
- The reactor provides basic interfaces to a number of services, including network communications, threading, and event dispatching.



Reactor

- For information about using the reactor and the Twisted event loop, see:
- the event dispatching how tos: Scheduling and Using Deferreds ;
- the communication how tos: TCP servers , TCP clients , UDP networking and Using processes ; and
- Using threads .

- The reactor usually implements a set of interfaces, but depending on the chosen reactor and the platform, some of the interfaces may not be implemented:
- IReactorCore : Core (required) functionality.
- IReactorFDSet : Use FileDescriptor objects.
- IReactorProcess : Process management. Read the Using Processes document for more information.
- IReactorSSL : SSL networking support.
- IReactorTCP : TCP networking support. More information can be found in the Writing Servers and Writing Clients documents.
- IReactorThreads : Threading use and management. More information can be found within Threading In Twisted .
- IReactorTime : Scheduling interface. More information can be found within Scheduling Tasks .
- IReactorUDP : UDP networking support. More information can be found within UDP Networking .
- IReactorUNIX : UNIX socket support.
- IReactorSocket : Third-party socket support..

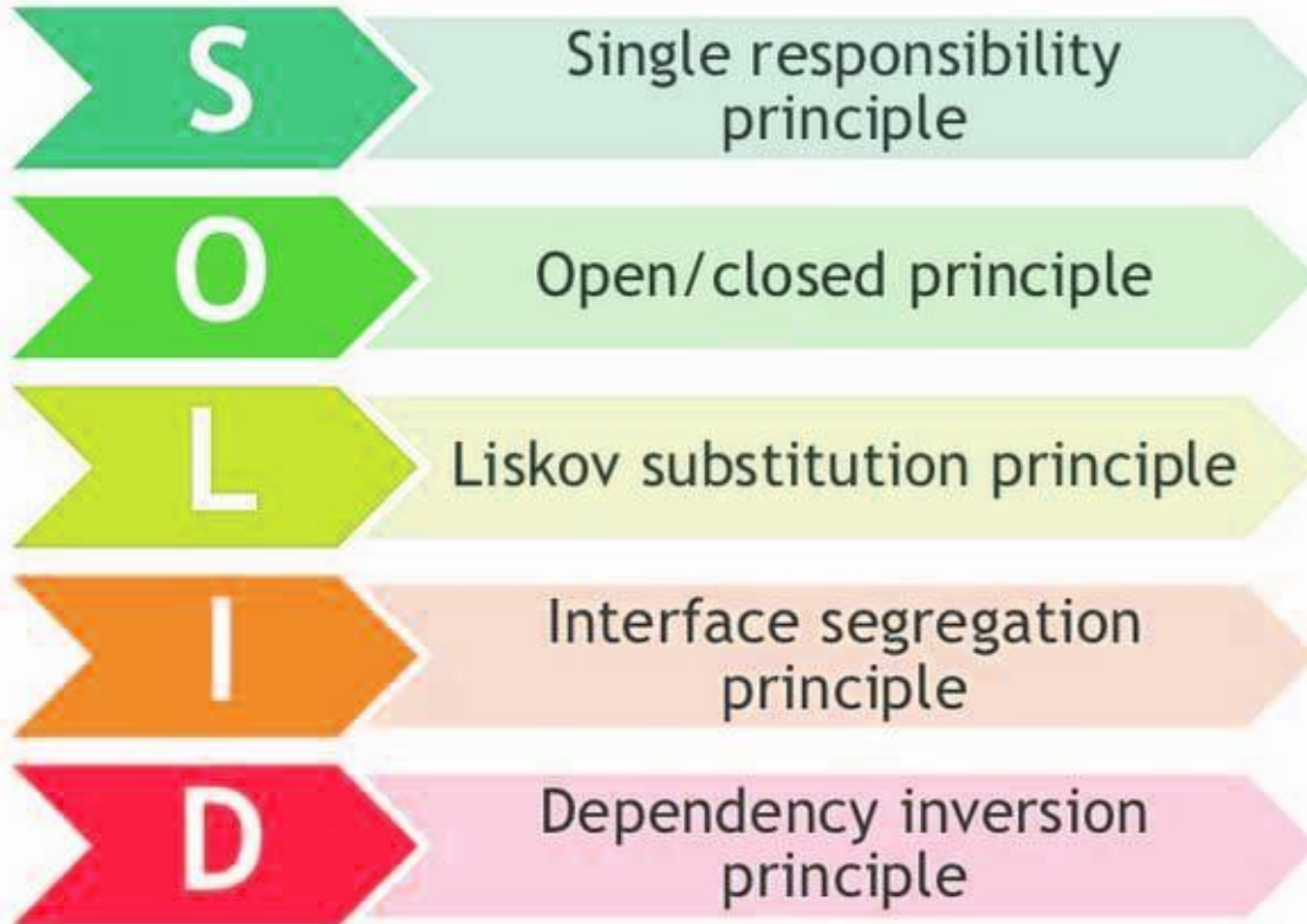
- Python SciPy has modules for the following tasks:
- Optimization
- Linear algebra
- Integration
- Interpolation
- Special functions
- FFT
- Signal and Image processing
- ODE solvers



Scipy – sub packages

- cluster- Hierarchical clustering.
- constants- Physical constants and factors of conversion.
- fftpack- Algorithms for Discrete Fourier Transform.
- integrate- Routines for numerical integration.
- interpolate- Tools for interpolation.
- io- Input and Output tools for data.
- lib- Wrappers to external libraries.
- linalg- Routines for linear algebra.
- misc- Miscellaneous utilities like image reading and writing.
- ndimage- Functions for processing multidimensional images.
- optimize- Algorithms for optimization.
- signal- Tools for processing signal.
- sparse- Algorithms for sparse matrices.
- spatial- KD-trees, distance functions, nearest neighbors.
- special- Special functions.
- stats- Functions to perform statistics.
- weave- Functionality that lets you write C++/C code in the form of multiline strings.

SOLID Design Pattern



Single Responsibility Principle



- As the single responsibility principle states, a class should have only one reason to change.
- This principle says that when we develop classes, it should cater to the given functionality well.
- If a class is taking care of two functionalities, it is better to split them. It refers to functionality as a reason to change.

Single Responsibility Principle



- Let's assume we need an object to keep an email message.
- We are going to use the IEmail.
- At the first sight everything looks just fine.
- IEmail and Email class have 2 responsibilities (reasons to change).
- One would be the use of the class in some email protocols such as pop3 or imap.
- If other protocols must be supported the objects should be serialized in another manner and code should be added to support new protocols.
- Another one would be for the Content field. Even if content is a string maybe we want in the future to support HTML or other formats.
- If we keep only one class, each change for a responsibility might affect the other one:
 - Adding a new protocol will create the need to add code for parsing and serializing the content for each type of field.
 - Adding a new content type (like html) make us to add code for each protocol implemented.



The open/close principle

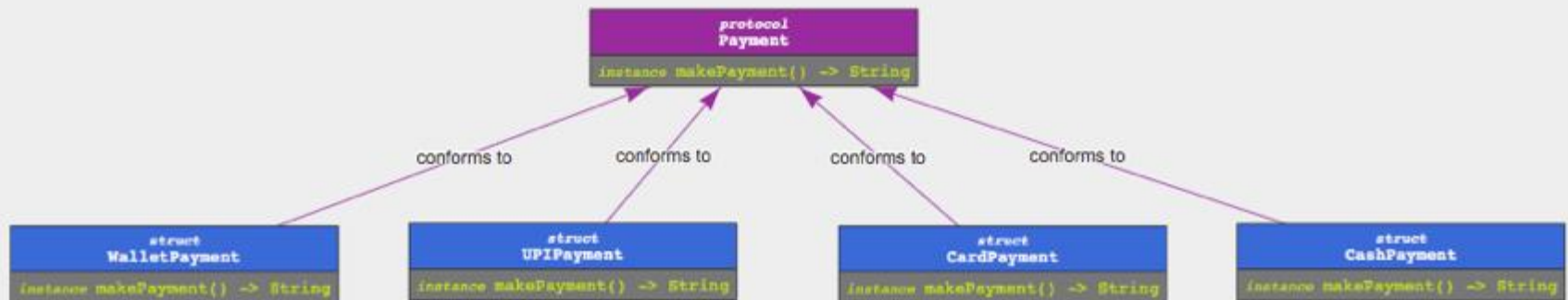
- The open/close principle states that *classes or objects and methods should be open for extension but closed for modifications*.
- When we develop our software application, make sure that we write our classes or modules in a generic way.
- Whenever we need to extend the behavior of the class or object, then we shouldn't have to change the class itself.
- Rather, a simple extension of the class should help us to build the new behavior.

The open/close principle

Not applying open/close principle



Open close principle

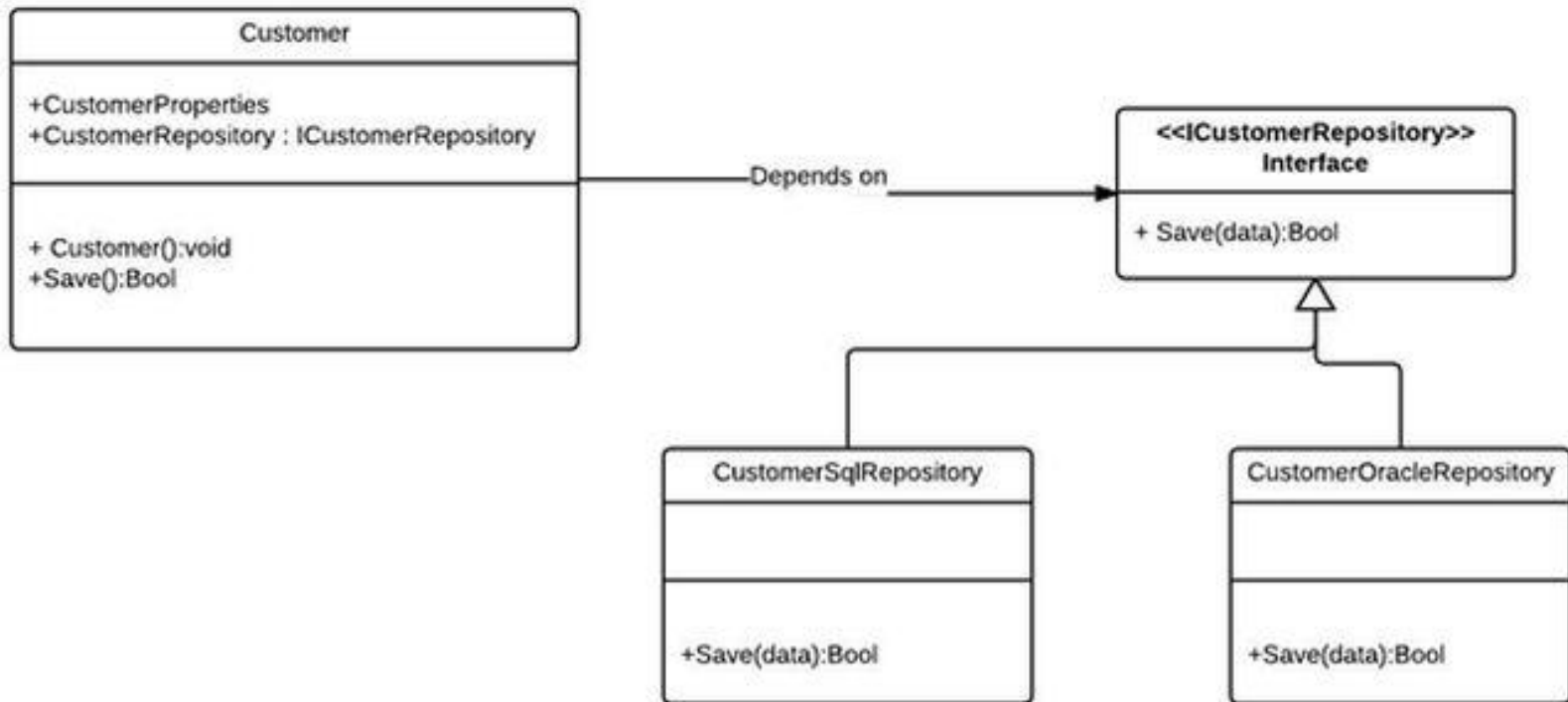




The inversion of control principle

- The inversion of control principle states that high-level modules shouldn't be dependent on low-level modules;
- They should both be dependent on abstractions. Details should depend on abstractions and not the other way round.

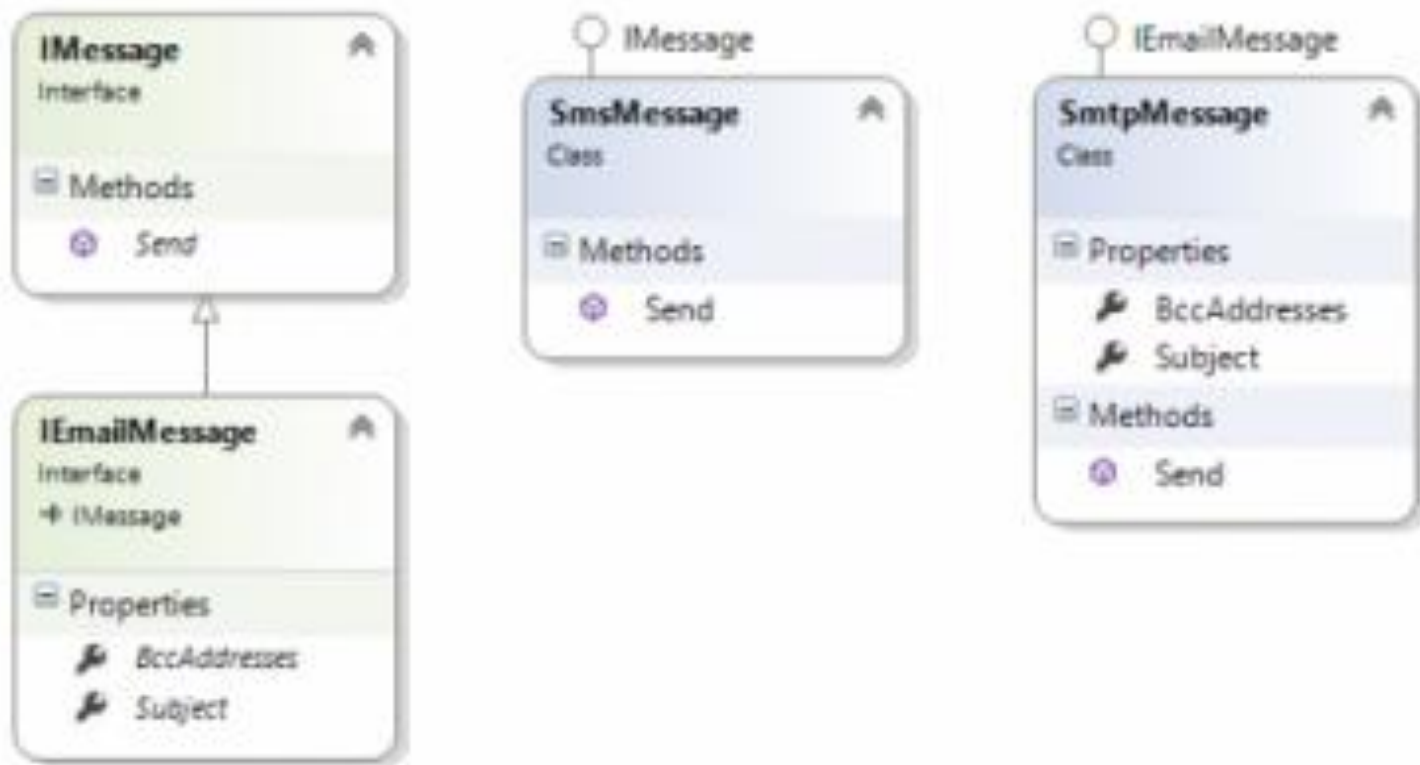
The inversion of control principle



The interface segregation principle

- As the interface segregation principle states, clients should not be forced to depend on interfaces they don't use.
- This principle talks about software developers writing their interfaces well.
- For instance, it reminds the developers/architects to develop methods that relate to the functionality.
- If there is any method that is not related to the interface, the class dependent on the interface has to implement it unnecessarily.

The interface segregation principle



Singleton

- Singleton provides you with a mechanism to have one, and only one, object of a given type and provides a global point of access.
- Hence, Singletons are typically used in cases such as logging or database operations, printer spoolers, and many others.
- Where there is a need to have only one instance that is available across the application to avoid conflicting requests on the same resource.
- For example, we may want to use one database object to perform operations on the DB to maintain data consistency or one object of the logging class across multiple services to dump log messages in a particular log file sequentially.
- In brief, the intentions of the Singleton design pattern are as follows:
 - Ensuring that one and only one object of the class gets created.
 - Providing an access point for an object that is global to the program.
 - Controlling concurrent access to resources that are shared.



Design Pattern

- 1) Creational Pattern
 - Factory Method Pattern
 - Abstract Factory Pattern
 - Singleton Pattern
 - Prototype Pattern
 - Builder Pattern
 - Object Pool Pattern

Design Pattern

- 2) Structural Pattern
 - Adapter Pattern
 - Bridge Pattern
 - Composite Pattern
 - Decorator Pattern
 - Facade Pattern
 - Flyweight Pattern
 - proxy Pattern

Design Pattern

- 3) Behavioral Pattern
 - Chain of Responsibility
 - Command Pattern
 - Interpreter Pattern
 - Iterator Pattern
 - Mediator Pattern
 - Memento Pattern
 - Observer Pattern
 - State Pattern
 - Strategy Pattern
 - Template Pattern

Singleton



Singleton
-instance : Singleton
- Singleton () +instance () : Singleton

Singleton

- A simple way of implementing Singleton is by making the constructor private and creating a static method that does the object initialization.
- This way, one object gets created on the first call and the class returns the same object thereafter.
- In Python, we will implement it in a different way as there's no option to create private constructors.

Singleton Disadvantages

- Global variables can be changed by mistake at one place and, as the developer may think that they have remained unchanged, the variables get used elsewhere in the application.
- Multiple references may get created to the same object. As Singleton creates only one object, multiple references can get created at this point to the same object.
- All classes that are dependent on global variables get tightly coupled as a change to the global data by one class can inadvertently impact the sother class.



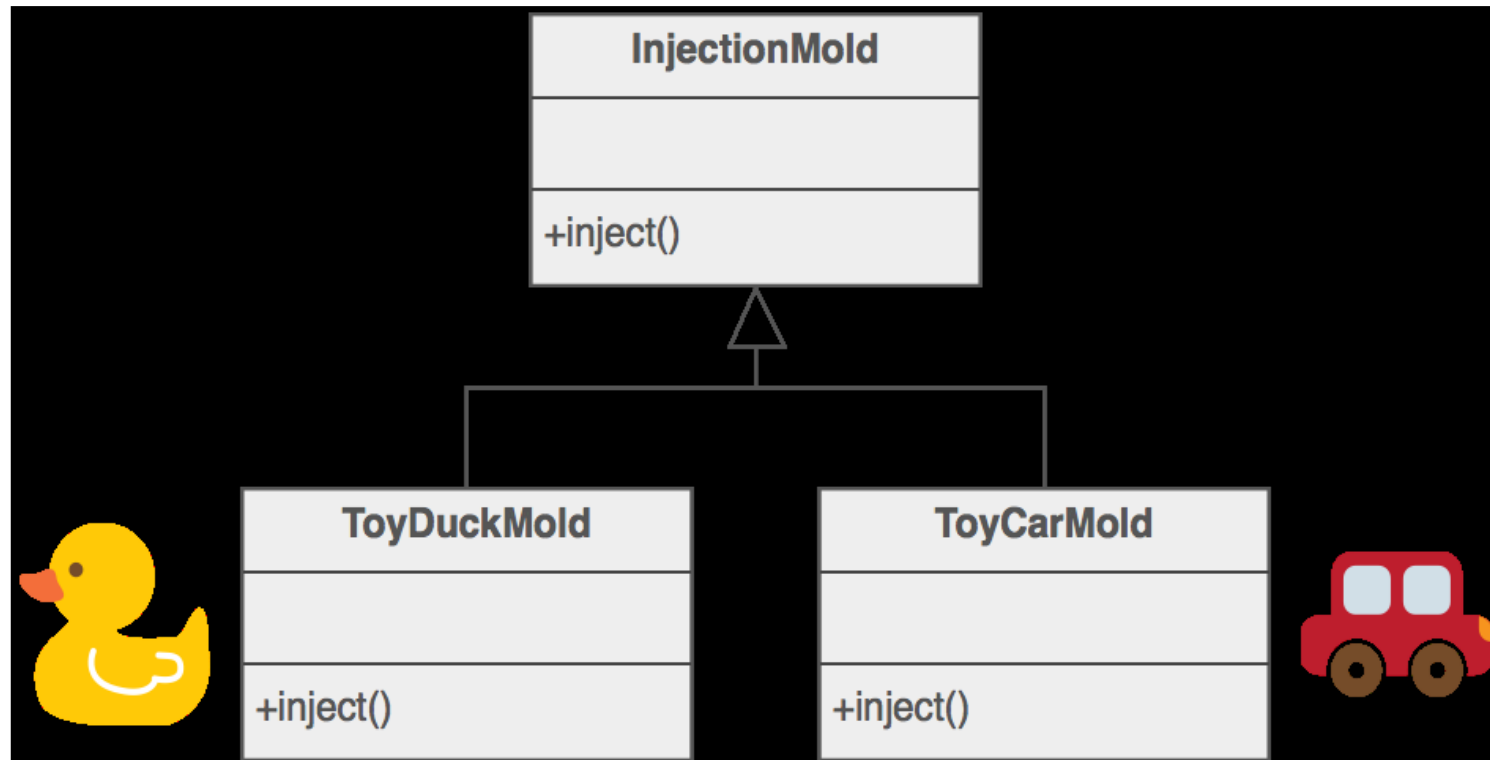
Factory Pattern

- The term factory means a class that is responsible for creating objects of other types.
- Typically, the class that acts as a factory has an object and methods associated with it.
- The client calls this method with certain parameters; objects of desired types are created in turn and returned to the client by the factory.

Factory Pattern

- A factory provides certain advantages that are listed here:
- The first advantage is loose coupling in which object creation can be independent of the class implementation.
- The client need not be aware of the class that creates the object which, in turn, is utilized by the client.
- It is only necessary to know the interface, methods, and parameters that need to be passed to create objects of the desired type.
- This simplifies implementations for the client.
- Adding another class to the factory to create objects of another type can be easily done without the client changing the code.
- At a minimum, the client needs to pass just another parameter.
- The factory can also reuse the existing objects. However, when the client does direct object creation, this always creates a new object.

Factory Pattern



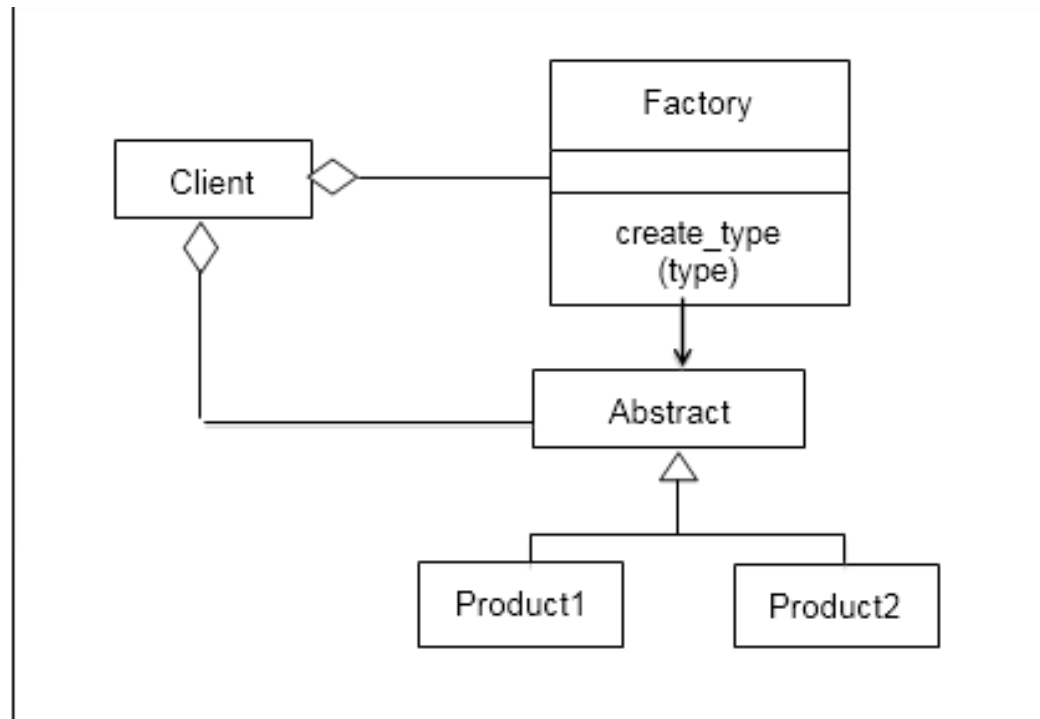
Factory Pattern

- **Simple Factory pattern:** This allows interfaces to create objects without exposing the object creation logic.
- **Factory method pattern:** This allows interfaces to create objects but defers the decision to the subclasses to determine the class for object creation.
- **Abstract Factory pattern:** An Abstract Factory is an interface to create related objects without specifying/exposing their classes.
- The pattern provides objects of another factory, which internally creates other objects.

The Simple Factory pattern

- The Factory helps create objects of different types rather than direct object instantiation.
- The client class uses the Factory class, which has the `create_type()` method.
- When the client calls the `create_type()` method with the type parameters, based on the parameters passed, the Factory returns **Product1** or **Product2**:

Factory Pattern



Factory Method Pattern

- We define an interface to create objects, but instead of the factory being responsible for the object creation, the responsibility is deferred to the subclass that decides the class to be instantiated.
- The Factory method creation is through inheritance and not through instantiation.
- The Factory method makes the design more customizable.
- It can return the same instance or subclass rather than an object of a certain type (as in the simple factory method).



Advantages of the Factory method pattern

- It brings in a lot of flexibility and makes the code generic, not being tied to a certain class for instantiation.
- There's loose coupling, as the code that creates the object is separate from the code that uses it.
- The client need not bother about what argument to pass and which class to instantiate.
- The addition of new classes is easy and involves low maintenance.



The Abstract Factory pattern

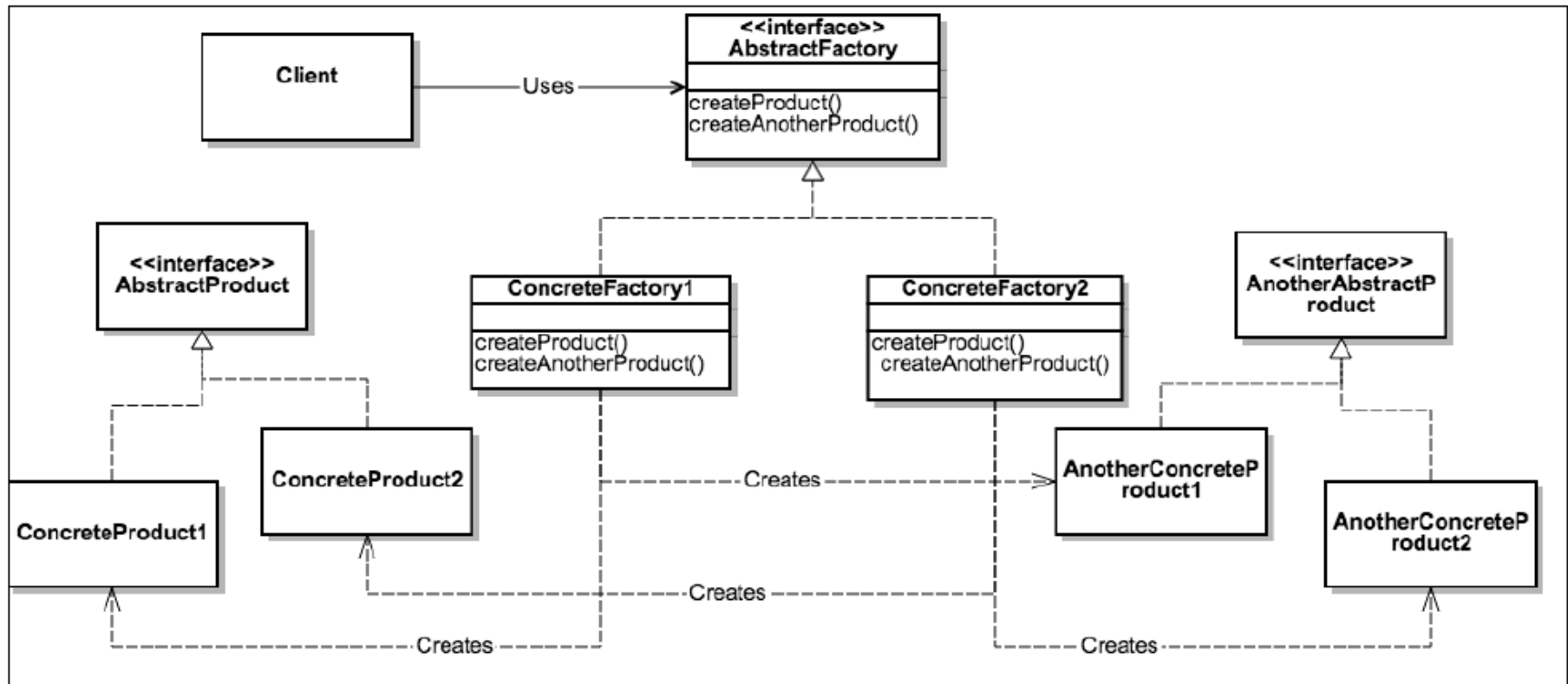
- The main objective of the Abstract Factory pattern is to provide an interface to create families of related objects without specifying the concrete class.
- While the factory method defers the creation of the instance to the subclasses, the goal of Abstract Factory method is to create families of related objects.

The Factory method versus Abstract Factory method

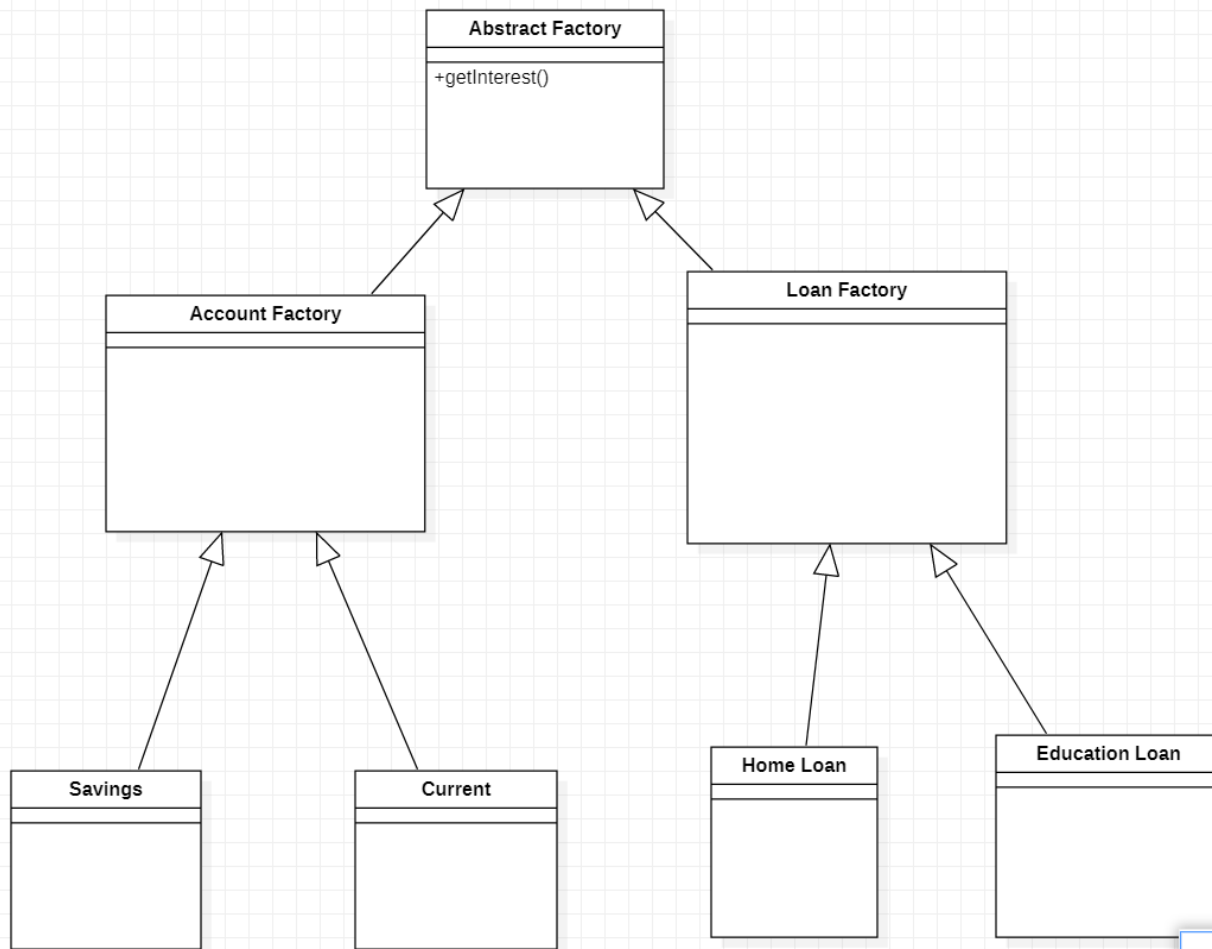


Factory method	Abstract Factory method
This exposes a method to the client to create the objects	Abstract Factory method contains one or more factory methods to create a family of related objects
This uses inheritance and subclasses to decide which object to create	This uses composition to delegate responsibility to create objects of another class
The factory method is used to create one product	Abstract Factory method is about creating families of related products

The Abstract Factory pattern



The Abstract Factory pattern





Prototype Pattern

- The **Prototype design pattern** helps us with creating object clones.
- In its simplest version, the Prototype pattern is just a `clone()` function that accepts an object as an input parameter and returns a clone of it.
- "A *shallow copy* constructs a new compound object and then (to the extent possible) inserts *references* into it to the objects found in the original.
- • A *deep copy* constructs a new compound object and then, recursively, inserts *copies* into it of the objects found in the original."
- In Python, this can be done using the `copy.deepcopy()` function.



Façade Design Pattern

- The façade is generally referred to as the face of the building, especially an attractive one.
- It can be also referred to as a behavior or appearance that gives a false idea of someone's true feelings or situation.
- When people walk past a façade, they can appreciate the exterior face but aren't aware of the complexities of the structure within. This is how a façade pattern is used.
- Façade hides the complexities of the internal system and provides an interface to the client that can access the system in a very simplified way.

Façade Design Pattern

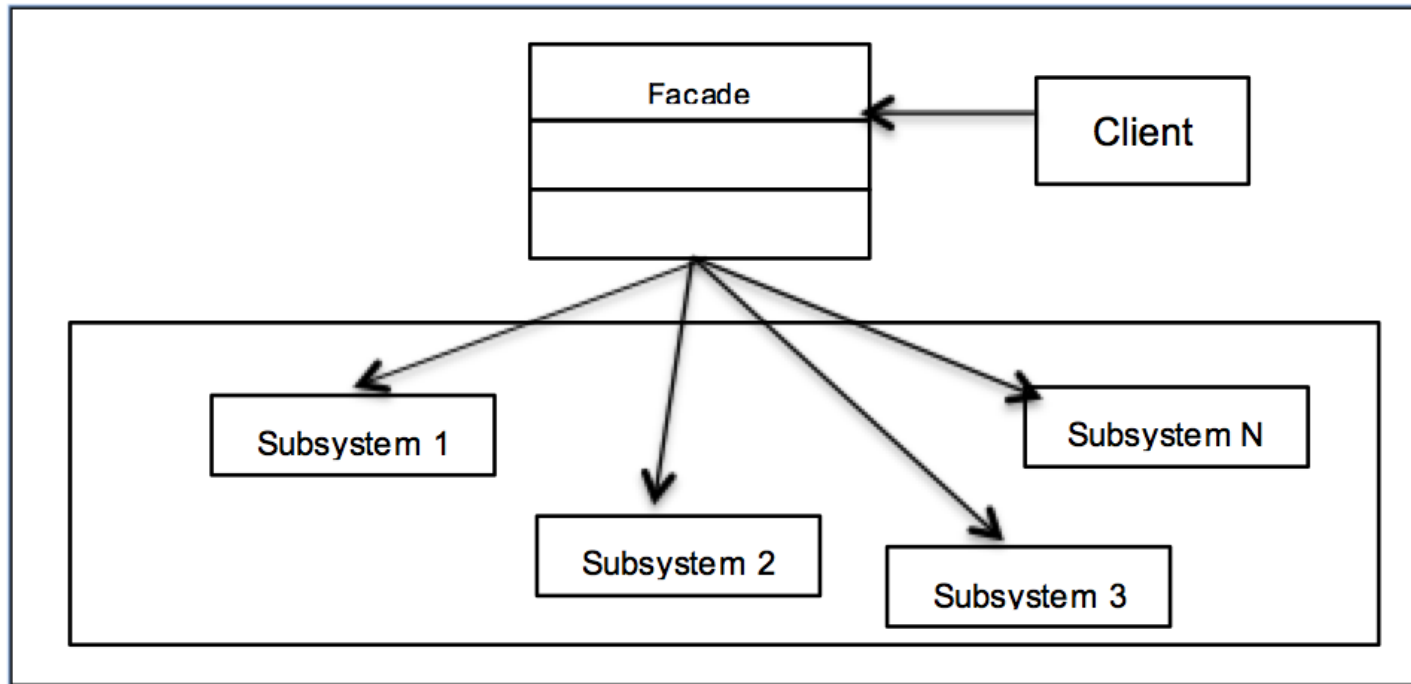
- Consider the example of a storekeeper. Now, when you, as a customer, visit a store to buy certain items, you're not aware of the layout of the store.
- You typically approach the storekeeper, who is well aware of the store system. Based on your requirements, the storekeeper picks up items and hands them over to you.
- Isn't this easy? The customer need not know how the store looks and s/he gets the stuff done through a simple interface, the storekeeper.

Façade Design Pattern

The Façade design pattern essentially does the following:

- It provides a unified interface to a set of interfaces in a subsystem and defines a high-level interface that helps the client use the subsystem in an easy way.
- Façade discusses representing a complex subsystem with a single interface object.
- It doesn't encapsulate the subsystem but combines the underlying subsystems.
- It promotes the decoupling of the implementation with multiple clients.

Façade Design Pattern



Facade

- It is an interface that knows which subsystems are responsible for a request
- It delegates the client's requests to the appropriate subsystem objects using composition
- For example, if the client is looking for some work to be accomplished, it need not have to go to individual subsystems but can simply contact the interface (Façade) that gets the work done.



Implementing the Façade pattern in the real world

- Consider that you have a marriage in your family and you are in charge of all the arrangements.
- That's a tough job on your hands. You have to book a hotel or place for marriage, talk to a caterer for food arrangements, organize a florist for all the decorations, and finally handle the musical arrangements expected for the event.



Implementing the Façade pattern in the real world

- In yesteryears, you'd have done all this by yourself, for example by talking to the relevant folks, coordinating with them, negotiating on the pricing, but now life is simpler.
- You go and talk to an event manager who handles this for you. S/he will make sure that they talk to the individual service providers and get the best deal for you.



Implementing the Façade pattern in the real world

- **Client:** It's you who need all the marriage preparations to be completed in time before the wedding. They should be top class and guests should love the celebrations.
- **Façade:** The event manager who's responsible for talking to all the folks that need to work on specific arrangements such as food, and flower decorations, among others
- **Subsystems:** They represent the systems that provide services such as catering, hotel management, and flower decorations

Proxy Pattern

- Proxy, in general terms, is a system that intermediates between the seeker and provider.
- Seeker is the one that makes the request, and provider delivers the resources in response to the request.
- In the web world, we can relate this to a proxy server. The clients (users in the World Wide Web), when they make a request to the website, first connect to a proxy server asking for resources such as a web page.
- The proxy server internally evaluates this request, sends it to an appropriate server, and gets back the response, which is then delivered to the client.
- Thus, a proxy server encapsulates requests, enables privacy, and works well in distributed architectures.

Proxy Pattern

- In the context of design patterns, Proxy is a class that acts as an interface to real objects.
- Objects can be of several types such as network connections, large objects in memory and file, among others.
- In short, Proxy is a wrapper or agent object that wraps the real serving object.
- Proxy could provide additional functionality to the object that it wraps and doesn't change the object's code.
- The main intention of the Proxy pattern is to provide a surrogate or placeholder for another object in order to control access to a real object.

Proxy Pattern Scenarios

- It represents a complex system in a simpler way. For example, a system that involves multiple complex calculations or procedures should have a simpler interface that can act as a proxy for the benefit of the client.
- It adds security to the existing real objects. In many cases, the client is not allowed to access the real object directly. This is because the real object can get compromised with malicious activities. This way proxies act as a shield against malicious intentions and protect the real object.

Proxy Pattern Scenarios

- It provides a local interface for remote objects on different servers.
- A clear example of this is with the distributed systems where the client wants to run certain commands on the remote system, but the client may not have direct permissions to make this happen. So it contacts a local object (proxy) with the request, which is then executed by the proxy on the remote machine.

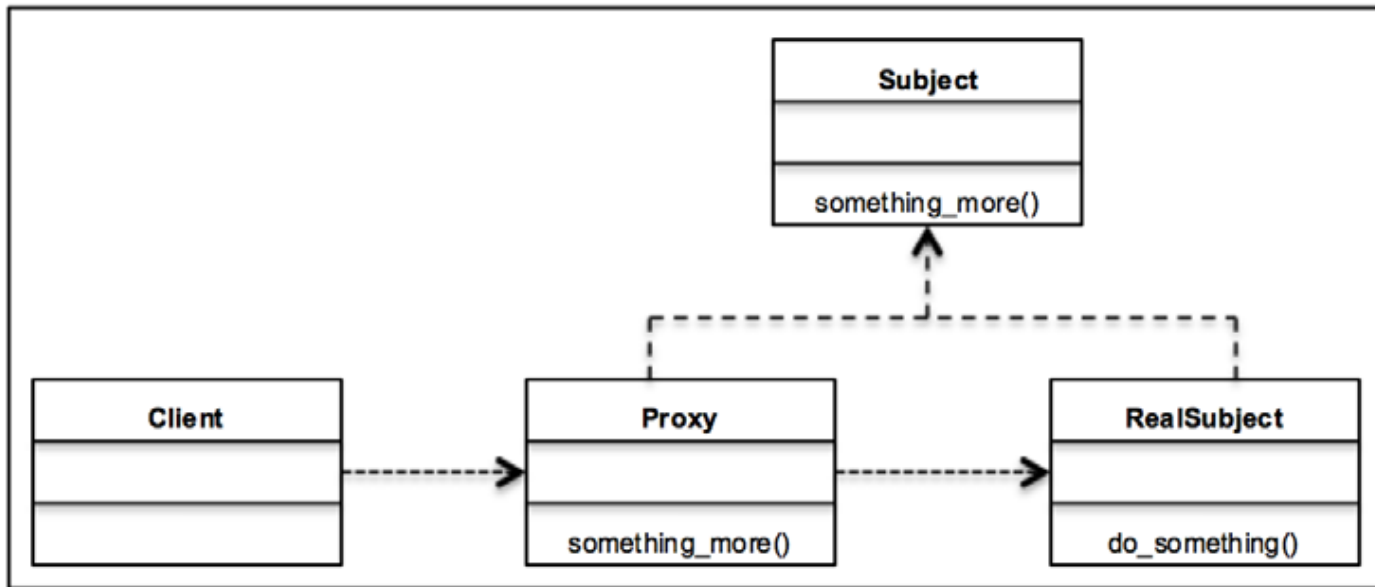
Proxy Pattern Scenarios

- It provides a light handle for a higher memory-consuming object. Sometimes, you may not want to load the main objects unless they're really necessary.
- This is because real objects are really heavy and may need high resource utilization.
- A classic example is that of profile pictures of users on a website. You're much better off showing smaller profile images in the list view, but of course, you'll need to load the actual image to show the detailed view of the user profile.
- .

Proxy Pattern Application

- Consider the example of an Actor and his Agent.
- When production houses want to approach an Actor for a movie, typically, they talk to the Agent and not to the Actor directly.
- Based on the schedule of the Actor and other engagements, the Agent gets back to the production house on the availability and interest in working in the movie.
- Now, in this scenario, instead of production houses directly talking to the Actor, the Agent acts as a Proxy that handles all the scheduling & payments for the Actor.

Proxy Pattern



Proxy Pattern

- Proxy: This maintains a reference that lets the Proxy access the real object. It provides an interface identical to the Subject so that Proxy can substitute the real subject. Proxies are also responsible for creating and deleting the RealSubject.
- Subject: It provides a representation for both, the RealSubject and Proxy. As Proxy and RealSubject implement Subject, Proxy can be used wherever RealSubject is expected.
- RealSubject: It defines the real object that the Proxy represents.

Proxy Pattern

- Proxy: It is a class that controls access to the RealSubject class. It handles the client's requests and is responsible for creating or deleting RealSubject.
- Subject/RealSubject: Subject is an interface that defines what RealSubject and Proxy should look like. RealSubject is an actual implementation of the Subject interface. It provides the real functionality that is then used by the client.
- Client: It accesses the Proxy class for the work to be accomplished. The Proxy class internally controls access to RealSubject and directs the work requested by Client.

Understanding different types of Proxies

- we can categorize them as virtual proxy, remote proxy, protective proxy, and smart proxy.
- **A virtual proxy**
- Here, you'll learn in detail about the virtual proxy. It is a placeholder for objects that are very heavy to instantiate.
- For example, you want to load a large image on your website. Now this request will take a long time to load.
- Typically, developers will create a placeholder icon on the web page suggesting that there's an image. However, the image will only be loaded when the user actually clicks on the icon thus saving the cost of loading a heavy image in memory.
- Thus, in virtual proxies, the real object is created when the client first requests or accesses the object.

Understanding different types of Proxies

- **A remote proxy**
- A remote proxy can be defined in the following terms. It provides a local representation of a real object that resides on a remote server or different address space.
- For example, you want to build a monitoring system for your application that has multiple web servers, DB servers, celery task servers, caching servers, among others.
- If we want to monitor the CPU and disk utilization of these servers, we need to have an object that is available in the context of where the monitoring application runs but can perform remote commands to get the actual parameter values. In such cases, having a remote proxy object that is a local representation of the remote object would help.

Understanding different types of Proxies

- **A protective proxy**
- This proxy controls access to the sensitive matter object of RealSubject.
- For example, in today's world of distributed systems, web applications have multiple services that work together to provide functionality.
- Now, in such systems, an authentication service acts as a protective proxy server that is responsible for authentication and authorization. I
- In this case, Proxy internally helps in protecting the core functionality of the website for unrecognized or unauthorized agents. Thus, the surrogate object checks that the caller has access permissions required to forward the request.

Understanding different types of Proxies

- **A smart proxy**
- Smart proxies interpose additional actions when an object is accessed. For example, consider that there's a core component in the system that stores states in a centralized location.
- Typically, such a component gets called by multiple different services to complete their tasks and can result in issues with shared resources. Instead of services directly invoking the core component, a smart proxy is built-in and checks whether the real object is locked before it is accessed in order to ensure that no other object can change it.

Proxy Scenario

- Let's say that you go to shop at a mall and like a nice denim shirt there. You would like to purchase the shirt but you don't have enough cash to do so.
- In yesteryears, you'd go to an ATM, take out the money, then come to the mall, and pay for it. Even earlier, you had a bank check for which you had to go to the bank, withdraw money, and then come back to pay for your expense.
- Thanks to the banks, we now have something called a debit card. So now, when you want to purchase something, you present your debit card to the merchant. When you punch in your card details, the money is debited in the merchant's account for your expense.

Proxy Scenario

- Your behavior is represented by the You class—the client
- To buy the shirt, the `make_payment()` method is provided by the class
- The special `__init__()` method calls the Proxy and instantiates it
- The `make_payment()` method invokes the Proxy's method internally to make the payment
- The `__del__()` method returns in case the payment is successful

Advantages

- Proxies can help improve the performance of the application by caching heavy objects or, typically, the frequently accessed objects.
- Proxies also authorize the access to RealSubject; thus, this pattern helps in delegation only if the permissions are right
- Remote proxies also facilitate interaction with remote servers that can work as network connections and database connections and can be used to monitor systems

Comparing the Façade and Proxy patterns

Proxy pattern	Façade pattern
It provides you with a surrogate or placeholder for another object to control access to it	It provides you with an interface to large subsystems of classes
A Proxy object has the same interface as that of the target object and holds references to target objects	It minimizes the communication and dependencies between subsystems
It acts as an intermediary between the client and object that is wrapped	A Façade object provides a single, simplified interface

Observer Pattern

- In the Observer design pattern, an object (Subject) maintains a list of dependents (Observers) so that the Subject can notify all the Observers about the changes that it undergoes using any of the methods defined by the Observer.
- In the world of distributed applications, multiple services interact with each other to perform a larger operation that a user wants to achieve.
- Services can perform multiple operations, but the operation they perform is directly or heavily dependent on the state of the objects of the service that it interacts with.



Observer Pattern

- Consider a use case for user registration where the user service is responsible for user operations on the website.
- Let's say that we have another service called e-mail service that observes the state of the user and sends e-mails to the user.
- For example, if the user has just signed up, the user service will call a method of the e-mail service that will send an e-mail to the user for account verification.
- If the account is verified but has fewer credits, the e-mail service will monitor the user service and send an e-mail alert for low credits to the user.

Observer Pattern

- Thus, if there's a core service in the application on which many other services are dependent, the core service becomes the Subject that has to be observed/monitored by the Observer for changes.
- The Observer should, in turn, make changes to the state of its own objects or take certain actions based on the changes that happen in the Subject.
- The above scenario, where the dependent service monitor's state changes in the core service, presents a classical case for the Observer design pattern.



Observer Pattern

- Consider the example of a blog. Let's suppose that you're a tech enthusiast who loves to read about the latest articles on Python on this blog.
- You subscribe to the blog. Like you, there would be multiple subscribers that are also registered with the blog. So, whenever there is a new blog, you get notified, or if there is a change on the published blog, you are also made aware of the edits.
- The way in which you're notified of the change can be an e-mail. Now if you apply this scenario to the Observer pattern, the blog is the Subject that maintains the list of subscribers or Observers.
- So when a new entry is added to the blog, all Observers are notified via e-mail or any other notification mechanism as defined by the Observer.



Observer Pattern

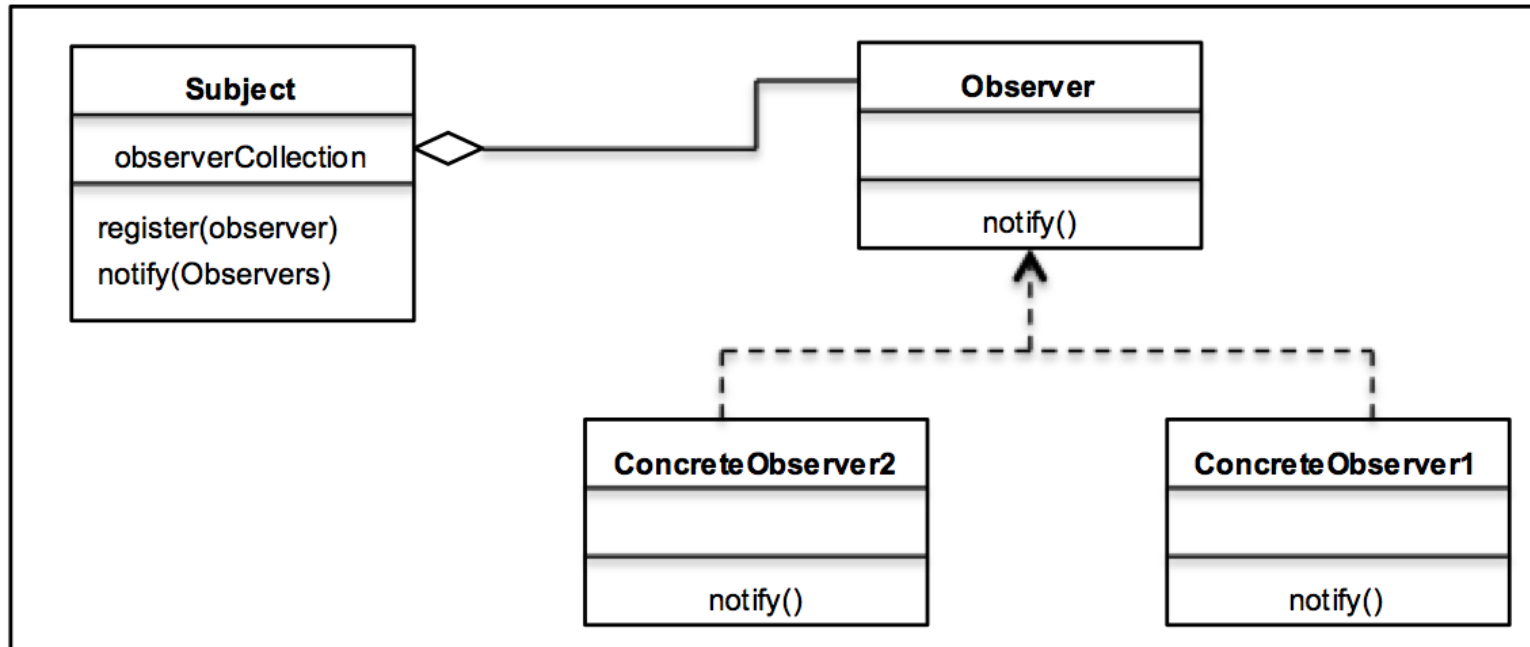
- The main intentions of the Observer pattern are as follows:
- It defines a one-to-many dependency between objects so that any change in one object will be notified to the other dependent objects automatically
- It encapsulates the core component of the Subject



Observer Pattern

- The Observer pattern is used in the following multiple scenarios:
- Implementation of the Event service in distributed systems
- A framework for a news agency
- The stock market also represents a great case for the Observer pattern

Observer Pattern





Observer Pattern

- News agencies typically gather news from various locations and publish them to the subscribers. Let's look at the design considerations for this use case.
- With information being sent/received in real time, a news agency should be able to publish the news as soon as possible to its subscribers.
- Additionally, because of the advancements in the technology industry, it's not just the newspapers, but also the subscribers that can be of different types such as an e-mail, mobile, SMS, or voice call. We should also be able to add any other type of subscriber in the future and budgeting for any new technology.

Observer Pattern

- **The Observer pattern methods**
- **The pull model**
- In the pull model, Observers play an active role as follows:
- The Subject broadcasts to all the registered Observers when there is any change
- The Observer is responsible for getting the changes or pulling data from the subscriber when there is an amendment
- The pull model is ineffective as it involves two steps—the first step where the Subject notifies the Observer and the second step where the Observer pulls the required data from the Subject



Observer Pattern

- **The push model**
- In the push model, the Subject is the one that plays a dominant role as follows:
- Unlike the pull model, the changes are pushed by the Subject to the Observer.
- In this model, the Subject can send detailed information to the Observer (even though it may not be needed). This can result in sluggish response times when a large amount of data is sent by the Subject but is never actually used by the Observer.
- Only the required data is sent from the Subject so that the performance is better.

The Observer pattern – advantages and disadvantages



- The Observer pattern provides you with the following advantages:
- It supports the principle of loose coupling between objects that interact with each other
- It allows sending data to other objects effectively without any change in the Subject or Observer classes
- Observers can be added/removed at any point in time

The Observer pattern – advantages and disadvantages



- The Observer interface has to be implemented by ConcreteObserver, which involves inheritance. There is no option for composition, as the Observer interface can be instantiated.
- If not correctly implemented, the Observer can add complexity and lead to inadvertent performance issues.
- In software application, notifications can, at times, be undependable and result in race conditions or inconsistency.



Builder Pattern

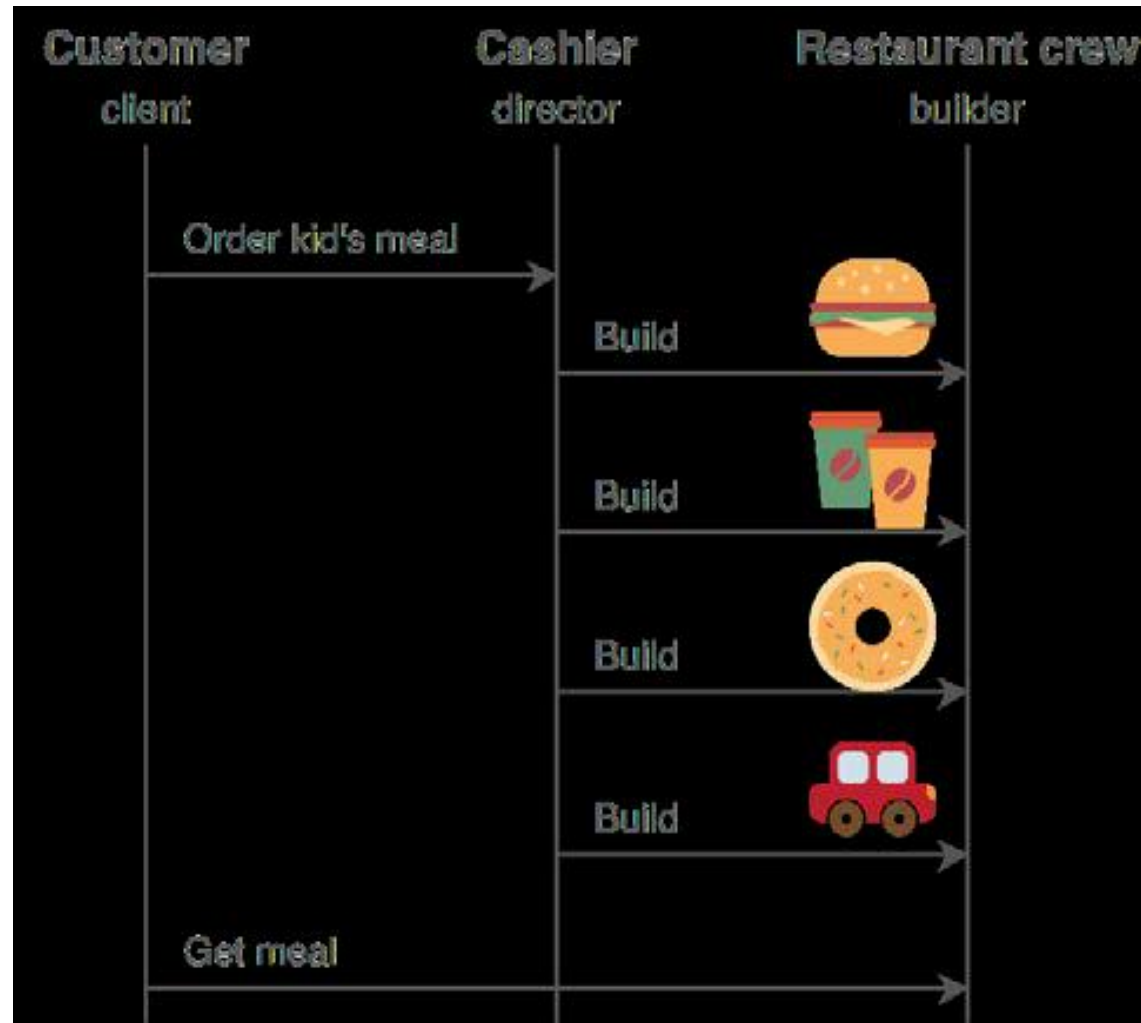
- The builder is responsible for creating the various parts of the complex object.
- In the HTML example, these parts are the title, heading, body, and the footer of the page.



Builder Pattern

- The Builder design pattern is used in fast-food restaurants. The same procedure is always used to prepare a burger and the packaging (box and paper bag), even if there are many different kinds of burgers (classic, cheeseburger, and more) and different packages (small-sized box, medium-sized box, and so forth).
- The difference between a classic burger and a cheeseburger is in the representation, and not in the construction procedure.
- The director is the cashier who gives instructions about what needs to be prepared to the crew, and the builder is the person from the crew that takes care of the specific order.

Builder Pattern





Adapter Pattern

- **Adapter** is a structural design pattern that helps us make two incompatible interfaces compatible.
- In such cases, we can write an extra layer that makes all the required modifications for enabling the communication between the two interfaces. This layer is called the Adapter.
- E-commerce systems are known examples. Assume that we use an e-commerce system that contains a `calculate_total(order)` function. The function calculates the total amount of an order, but only in **Danish Kroner (DKK)**.



Adapter Pattern

- It is reasonable for our customers to ask us to add support for more popular currencies, such as **United States Dollars (USD)** and **Euros (EUR)**.
- If we own the source code of the system we can extend it by adding new functions for doing the conversions from DKK to USD and from DKK to EUR.
- But what if we don't have access to the source code of the application because it is provided to us only as an external library? In this case, we can still use the library (for example, call its methods), but we cannot modify/extend it.
- The solution is to write a wrapper (also known as Adapter) that converts the data from the given DKK format to the expected USD or EUR format.



Adapter Pattern A real-life example

- **Grok** is a Python framework that runs on top of Zope 3 and focuses on agile development.
- The Grok framework uses Adapters for making it possible for existing objects to conform to specific APIs without the need to modify them [j.mp/grokada].
- The Python **Traits** package also uses the Adapter pattern for transforming an object that does not implement of a specific interface (or set of interfaces) to an object that does [j.mp/pytraitsad].



The Decorator Pattern

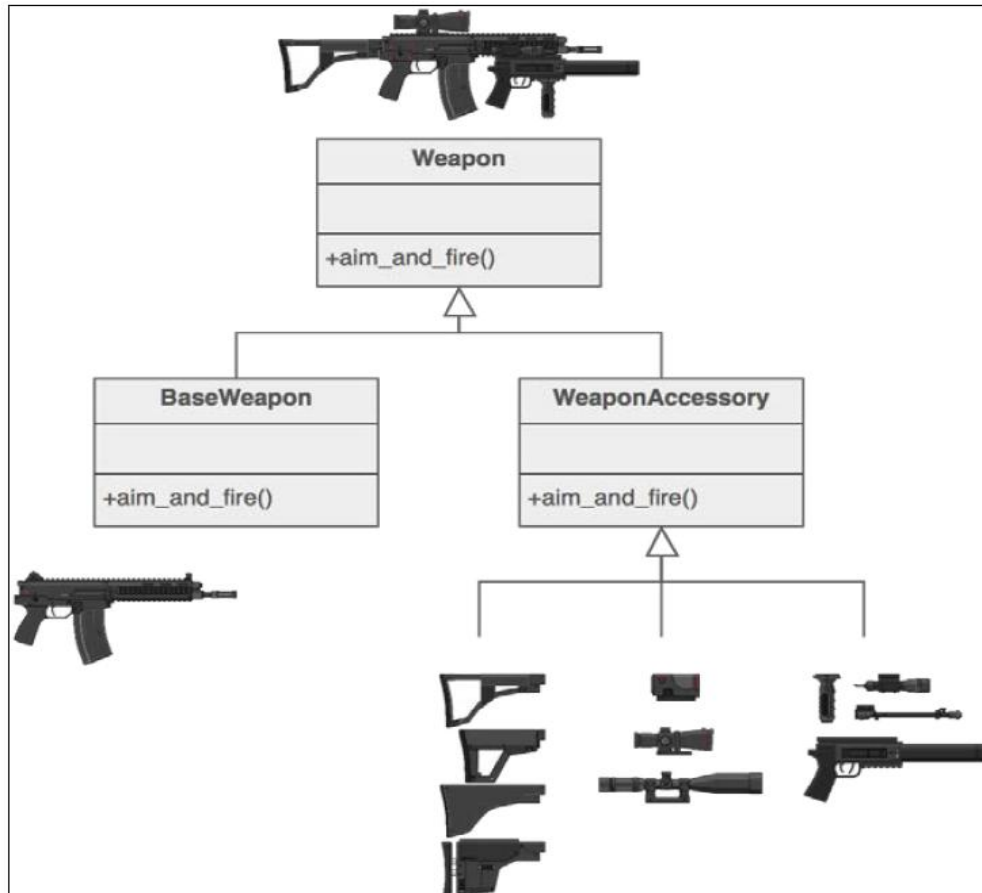
- A **Decorator** pattern can add responsibilities to an object dynamically, and in a transparent manner.
- This defines additional responsibilities for an object at runtime or dynamically. We add certain attributes to objects with an interface.
- Real examples of such extensions are: adding a silencer to a gun, using different camera lenses (in cameras with removable lenses), and so on.
- The Decorator pattern shines when used for implementing **cross-cutting concerns**



The Decorator Pattern

- Data validation
- Transaction processing (A transaction in this case is similar to a database transaction, in the sense that either all steps should be completed successfully, or the transaction should fail.)
- Caching
- Logging
- Monitoring
- Debugging
- Business rules
- Compression
- Encryption

The Decorator Pattern





The Decorator Pattern

- The Django framework uses decorators to a great extent. An example is the View decorator. Django's **View** decorators can be used for [j.mp/djangodec]:
- Restricting access to views based on the HTTP request
- Controlling the caching behavior on specific views
- Controlling compression on a per-view basis
- Controlling caching based on specific HTTP request headers



The Decorator Pattern

- The Grok framework also uses decorators for achieving different goals such as [\[j.mp/grokdeco\]](http://j.mp/grokdeco):
- Registering a function as an event subscriber
- Protecting a method with a specific permission
- Implementing the Adapter pattern



The Flyweight Pattern

- Flyweight design pattern is a technique used to minimize memory usage and improve performance by introducing data sharing between similar objects.
- A **Flyweight** is a shared object that contains state-independent, immutable (also known as intrinsic) data.
- The state-dependent, mutable (also known as extrinsic) data should not be part of Flyweight because this is information that cannot be shared since it differs per object.



The Flyweight Pattern

- We can think of Flyweight as caching in real life. For example, many bookstores have dedicated shelves with the newest and most popular publications.
- This is a cache. First, you can take a look at the dedicated shelves for the book you are looking for, and if you cannot find it, you can ask the librarian to assist you.



The Flyweight Pattern

- Memoization is an optimization technique that uses a cache to avoid recomputing results that were already computed in an earlier execution step.
- Memoization does not focus on a specific programming paradigm such as **object-oriented programming (OOP)**.
- In Python, memoization can be applied on both methods and simple functions.
- Flyweight is an OOP-specific optimization design pattern that focuses on sharing object data.

The Model-View-Controller Pattern

- One of the design principles related to software engineering is the **Separation of Concerns (SoC)** principle.
- The idea behind the SoC principle is to split an application into distinct sections, where each section addresses a separate concern.
- The **Model-View-Controller (MVC)** pattern is nothing more than the SoC principle applied to OOP.
- The name of the pattern comes from the three main components used to split a software application: the model, the view, and the controller.
- MVC is considered an architectural pattern rather than a design pattern



The Model-View-Controller Pattern

- The difference between an architectural and a design pattern is that the former has a broader scope than the latter.
- A typical use of an application that uses MVC after the initial screen is rendered to the user is as follows:
- The user triggers a view by clicking (typing, touching, and so on) a button
- The view informs the controller about the user's action
- The controller processes user input and interacts with the model
- The model performs all the necessary validation and state changes, and informs the controller about what should be done
- The controller instructs the view to update and display the output appropriately, following the instructions given by the model



The Model-View-Controller Pattern

- The **web2py** web framework [j.mp/webtopy] is a lightweight Python framework that embraces the MVC pattern
- Django is also an MVC framework, although it uses different naming conventions. The controller is called view, and the view is called template. Django uses the name **Model-Template-View (MTV)**.



The Model-View-Controller Pattern

- A model is considered smart because it:
- Contains all the validation/business rules/logic
- Handles the state of the application
- Has access to application data (database, cloud, and so on)
- Does not depend on the UI



The Model-View-Controller Pattern

- A controller is considered thin because it:
- Updates the model when the user interacts with the view
- Updates the view when the model changes
- Processes the data before delivering it to the model/view, if necessary



The Model-View-Controller Pattern

- A controller is considered thin because it:
- Updates the model when the user interacts with the view
- Updates the view when the model changes
- Processes the data before delivering it to the model/view, if necessary
- Does not display the data
- Does not access the application data directly
- Does not contain validation/business rules/logic



The Model-View-Controller Pattern

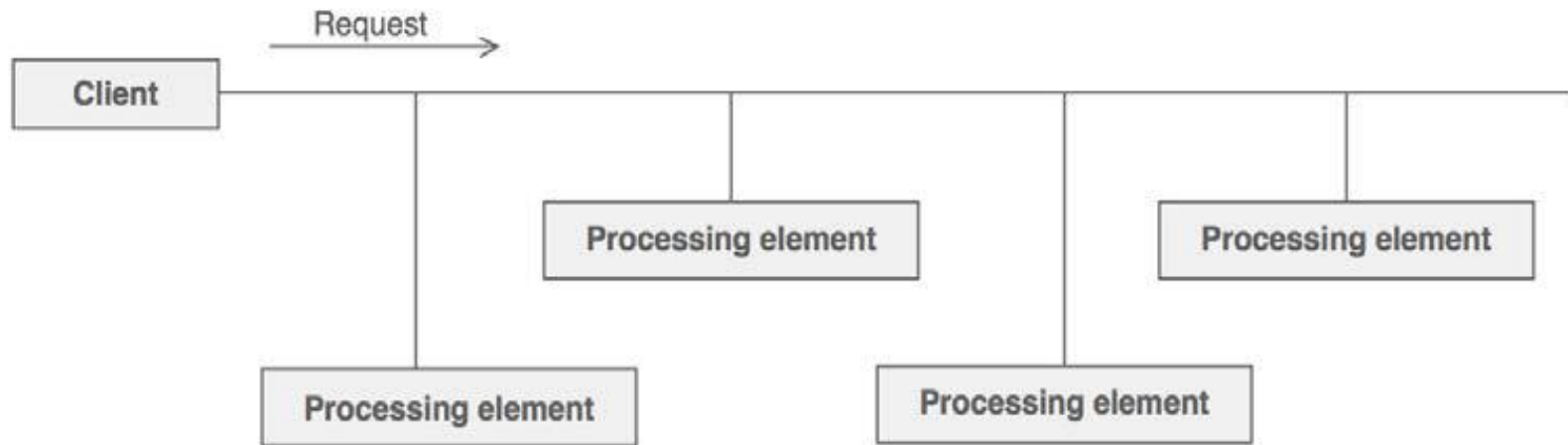
- A view is considered dumb because it:
- Displays the data
- Allows the user to interact with it
- Does only minimal processing, usually provided by a template language (for example, using simple variables and loop controls)
- Does not store any data
- Does not access the application data directly
- Does not contain validation/business rules/logic



The Chain of Responsibility Pattern

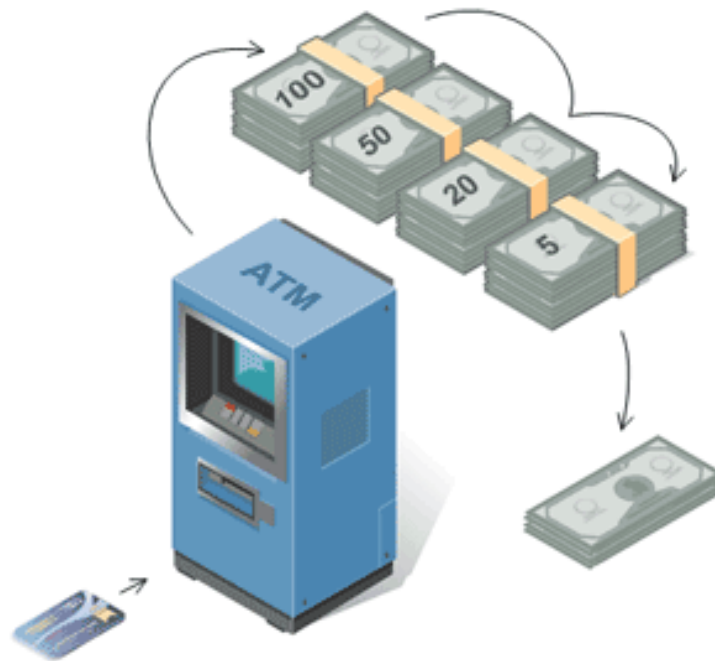
- The **Chain of Responsibility** pattern is used when we want to give a chance to multiple objects to satisfy a single request, or when we don't know which object (from a chain of objects) should process a specific request in advance.
- The principle is the same as the following:
- There is a chain (linked list, tree, or any other convenient data structure) of objects.
- We start by sending a request to the first object in the chain.
- The object decides whether it should satisfy the request or not.
- The object forwards the request to the next object.
- This procedure is repeated until we reach the end of the chain.

The Chain of Responsibility Pattern



The Chain of Responsibility Pattern

ATMs and, in general, any kind of machine that accepts/returns banknotes or coins (for example, a snack vending machine) use the chain of responsibility pattern. There is always a single slot for all banknotes, as shown in the following figure





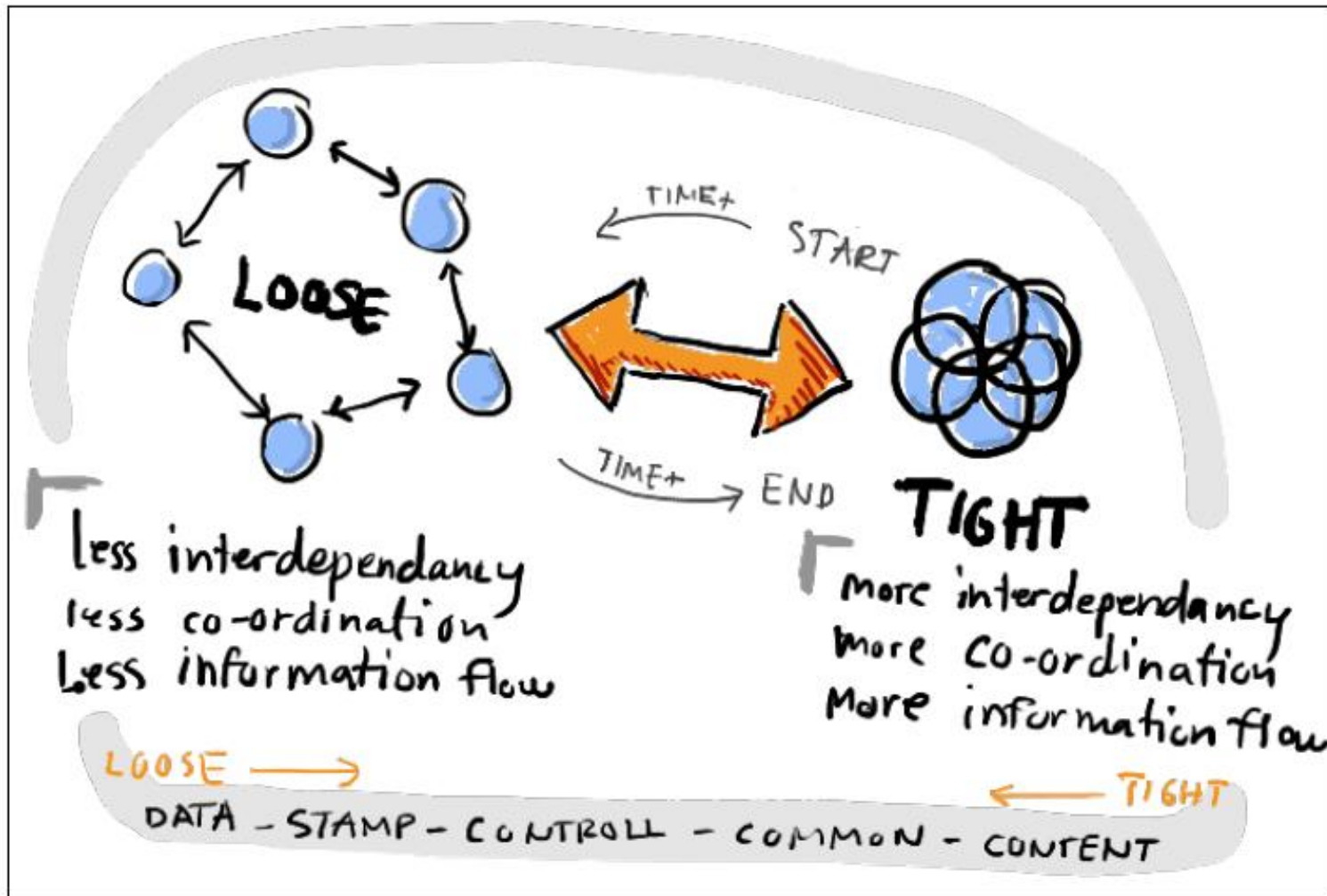
The Chain of Responsibility Pattern usecase

In purchase systems, there are many approval authorities. One approval authority might be able to approve orders up to a certain value, let's say \$100.

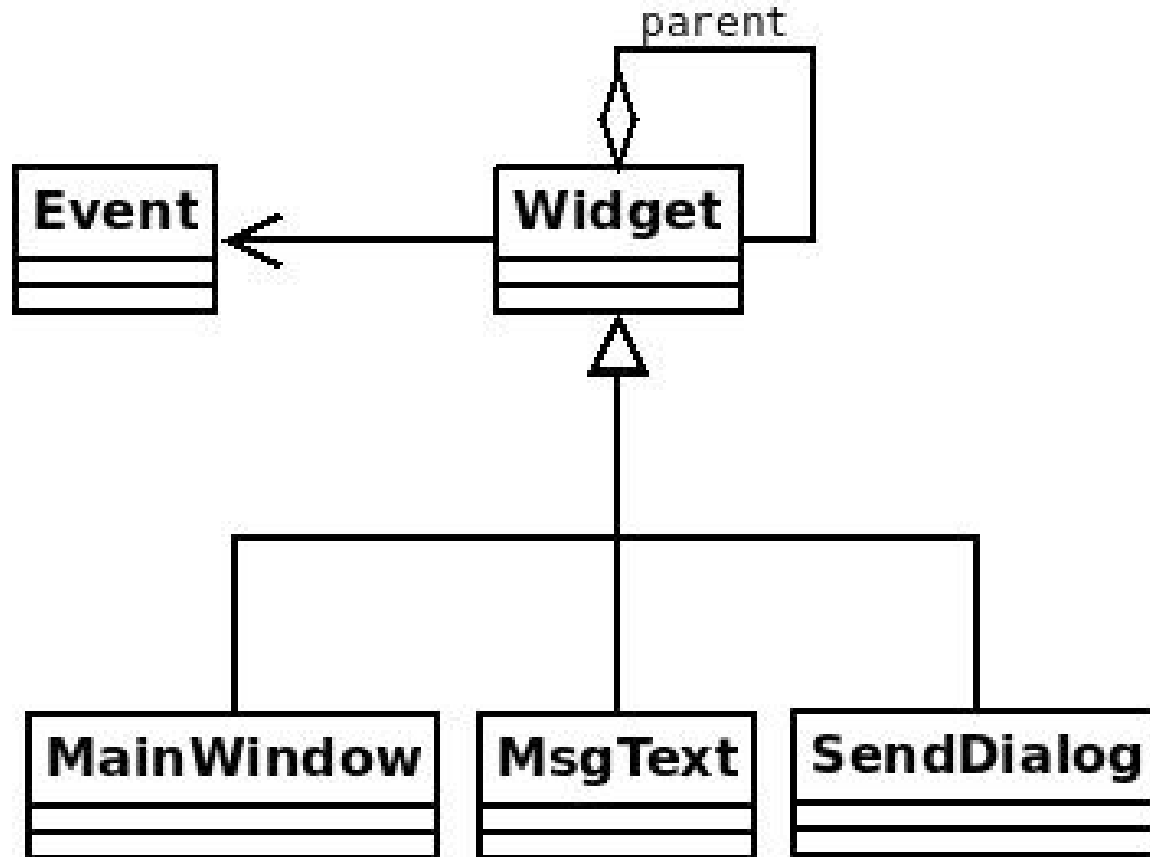
If the order is more than \$100, the order is sent to the next approval authority in the chain that can approve orders up to \$200, and so forth.

Another case where Chain of Responsibility is useful is when we know that more than one object might need to process a single request. This is what happens in an event-based programming. A single event such as a left mouse click can be caught by more than one listener.

The Chain of Responsibility Pattern usecase



The Chain of Responsibility Pattern usecase





Command Pattern

- The Command design pattern helps us encapsulate an operation (undo, redo, copy, paste, and so forth) as an object.
- What this simply means is that we create a class that contains all the logic and the methods required to implement the operation.



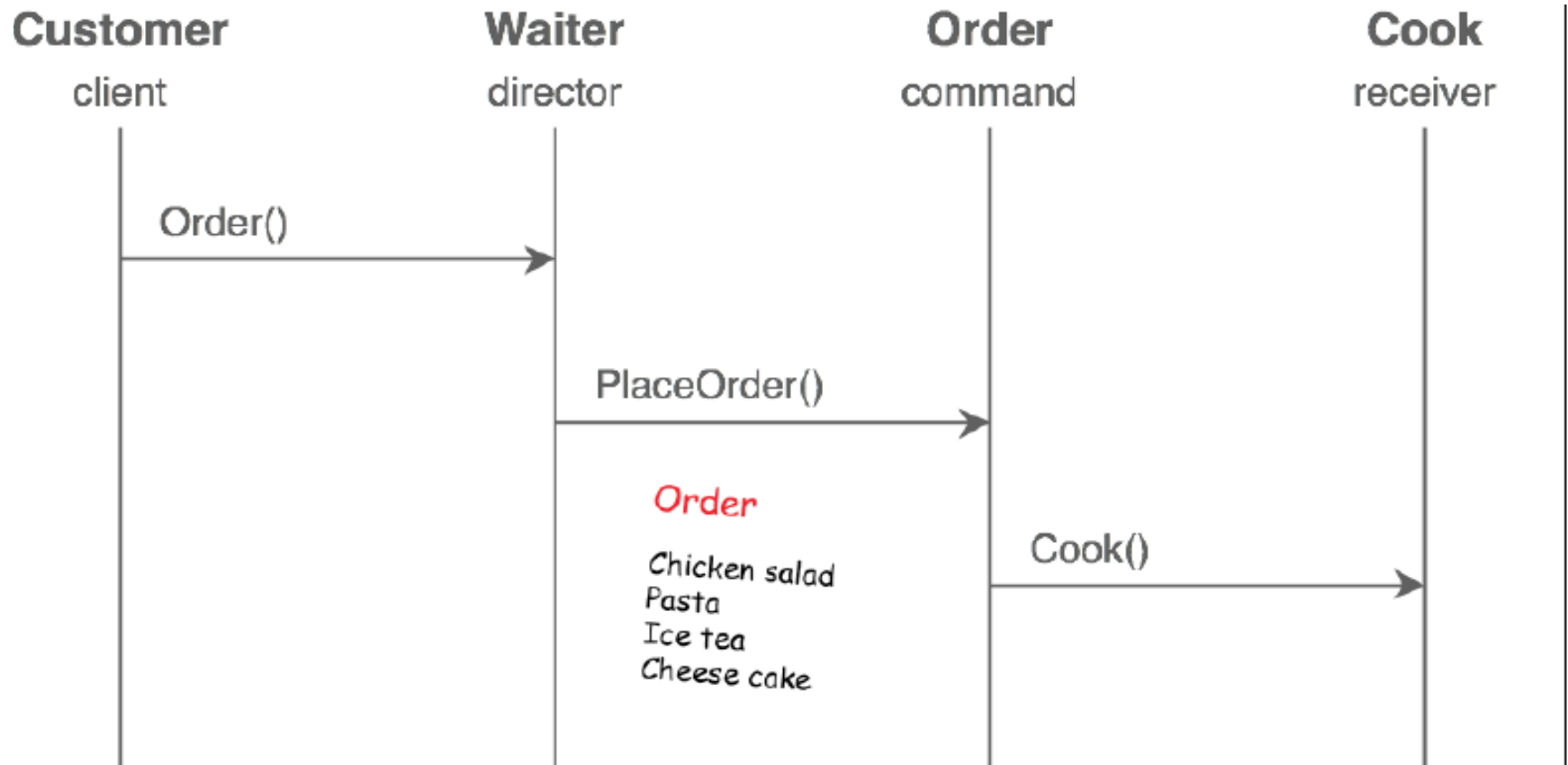
Command Pattern

- We don't have to execute a command directly. It can be executed on will.
- The object that invokes the command is decoupled from the object that knows how to perform it. The invoker does not need to know any implementation details about the command.
- If it makes sense, multiple commands can be grouped to allow the invoker to execute them in order. This is useful, for instance, when implementing a multilevel undo command.

Command Pattern Real Time

- When we go to the restaurant for dinner, we give the order to the waiter.
- The check (usually paper) they use to write the order on is an example of Command.
- After writing the order, the waiter places it in the check queue that is executed by the cook.
- Each check is independent and can be used to execute many and different commands, for example, one command for each item that will be cooked.

Command Pattern Real Time



Command Pattern Real Time – Use Case

- **GUI buttons and menu items:** The PyQt example that was already mentioned uses the Command pattern to implement actions on buttons and menu items.
- **Other operations:** Apart from undo, Command can be used to implement any operation. A few examples are cut, copy, paste, redo, and capitalize text.
- **Transactional behavior and logging:** Transactional behavior and logging are important to keep a persistent log of changes. They are used by operating systems to recover from system crashes, relational databases to implement transactions, filesystems to implement snapshots, and installers (wizards) to revert cancelled installations.
- **Macros:** By macros, in this case, we mean a sequence of actions that can be recorded and executed on demand at any point in time. Popular editors such as Emacs and Vim support macros.



Interpreter Pattern

- The **Interpreter** pattern is interesting only for the advanced users of an application.
- That's because the main idea behind Interpreter is to give the ability to non-beginner users and domain experts to use a simple language to express their ideas.[DSL]
- A DSL is a computer language of limited expressiveness targeting a particular domain.
- DSLs are used for different things, such as combat simulation, billing, visualization, configuration, communication protocols, and so on.
- DSLs are divided into internal DSLs and external DSLs



Interpreter Pattern

- Internal DSLs are built on top of a host programming language.
- An example of an internal DSL is a language that solves linear equations using Python.
- The advantages of using an internal DSL are that we don't have to worry about creating, compiling, and parsing grammar because these are already taken care of by the host language.
- The disadvantage is that we are constrained by the features of the host language. It is very challenging to create an expressive, concise, and fluent internal DSL if the host language does not have these features.



Interpreter Pattern

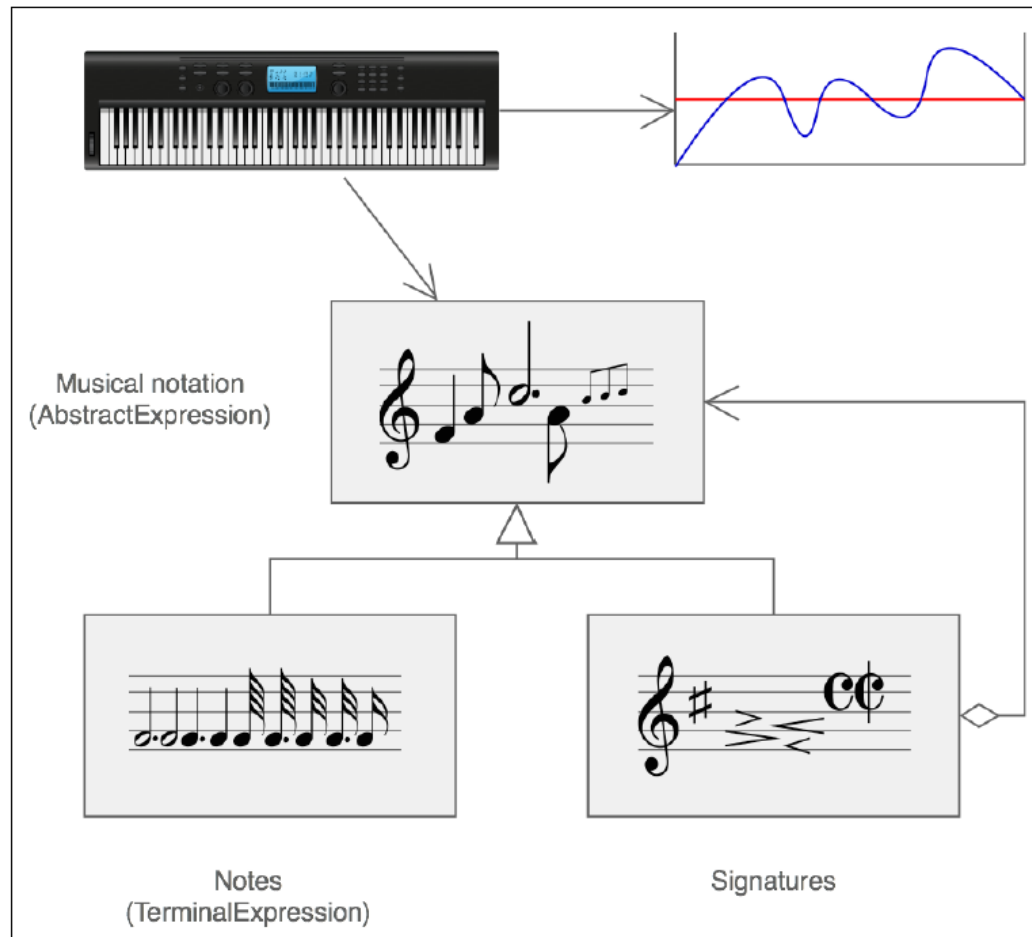
- External DSLs do not depend on host languages.
- The creator of the DSL can decide all aspects of the language (grammar, syntax, and so forth), but they are also responsible for creating a parser and compiler for it.
- Creating a parser and compiler for a new language can be a very complex, long, and painful procedure



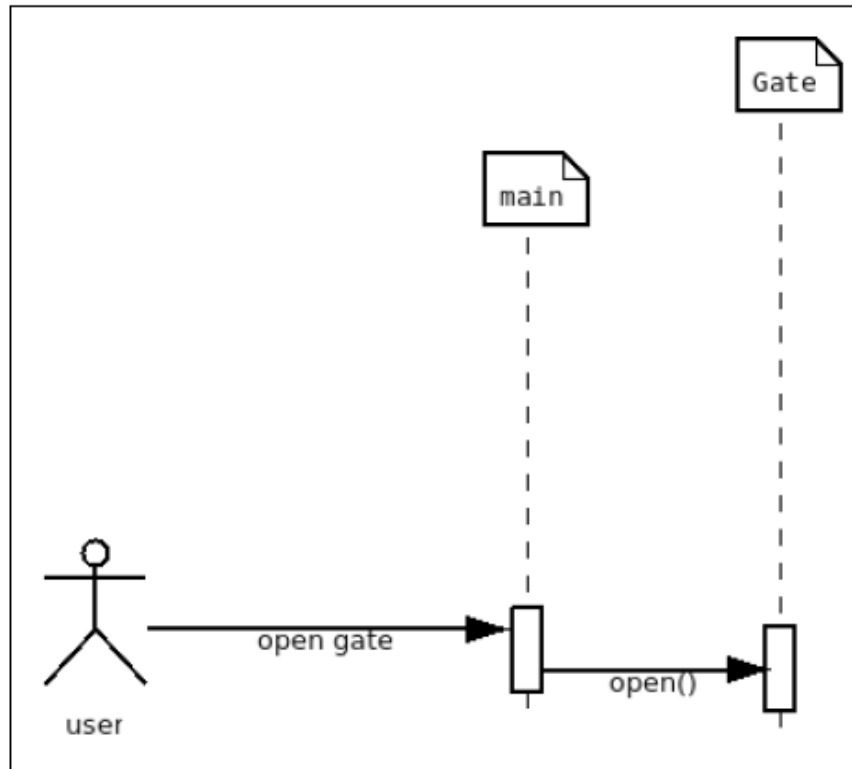
Interpreter Pattern

- A musician is an example of the Interpreter pattern in reality.
- Musical notation represents the pitch and duration of a sound graphically.
- The musician is able to reproduce a sound precisely based on its notation.
- In a sense, musical notation is the language of music, and the musician is the interpreter of that language.

Interpreter Pattern



Interpreter Pattern



State Pattern

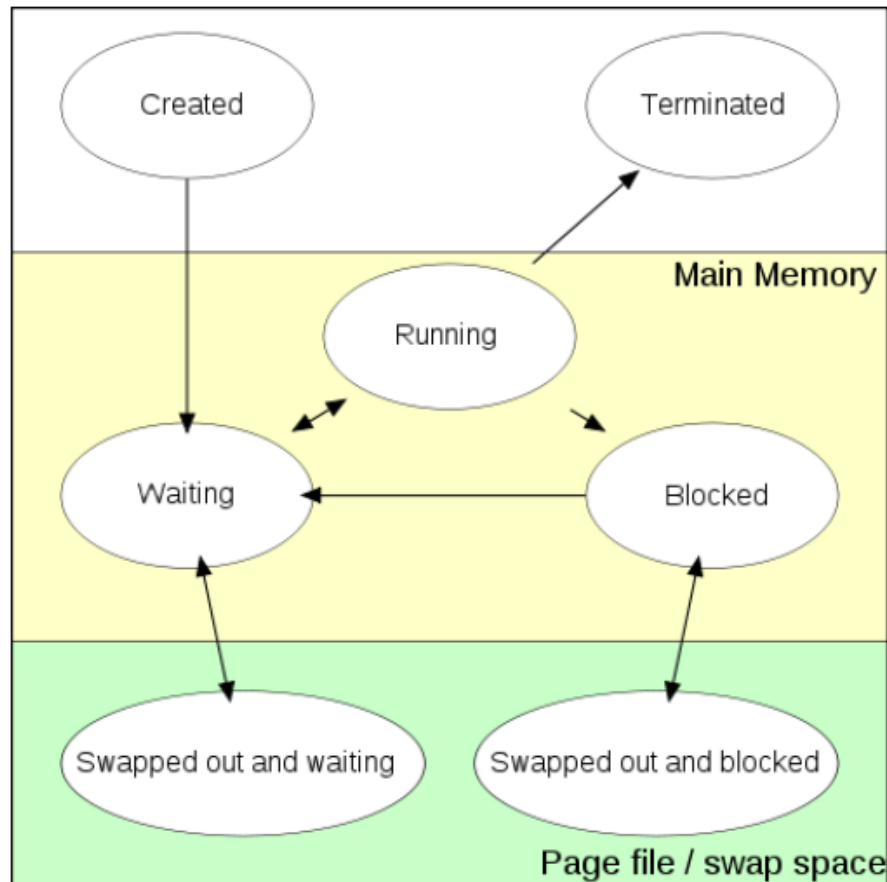
- A state machine is an abstract machine that has two key components: states and transitions.
- When a process is initially created by a user, it goes into the *created/new* state.
- From this state, the only transition is to go into the *waiting* state, which happens when the scheduler loads the process in memory and adds it to the queue of the processes that are *waiting/ready for execution*.
- A *waiting* process has two possible transitions: it can either be picked for execution (transition to *running*), or it can be replaced with a process that has higher priority (transition to *swapped out and waiting*).



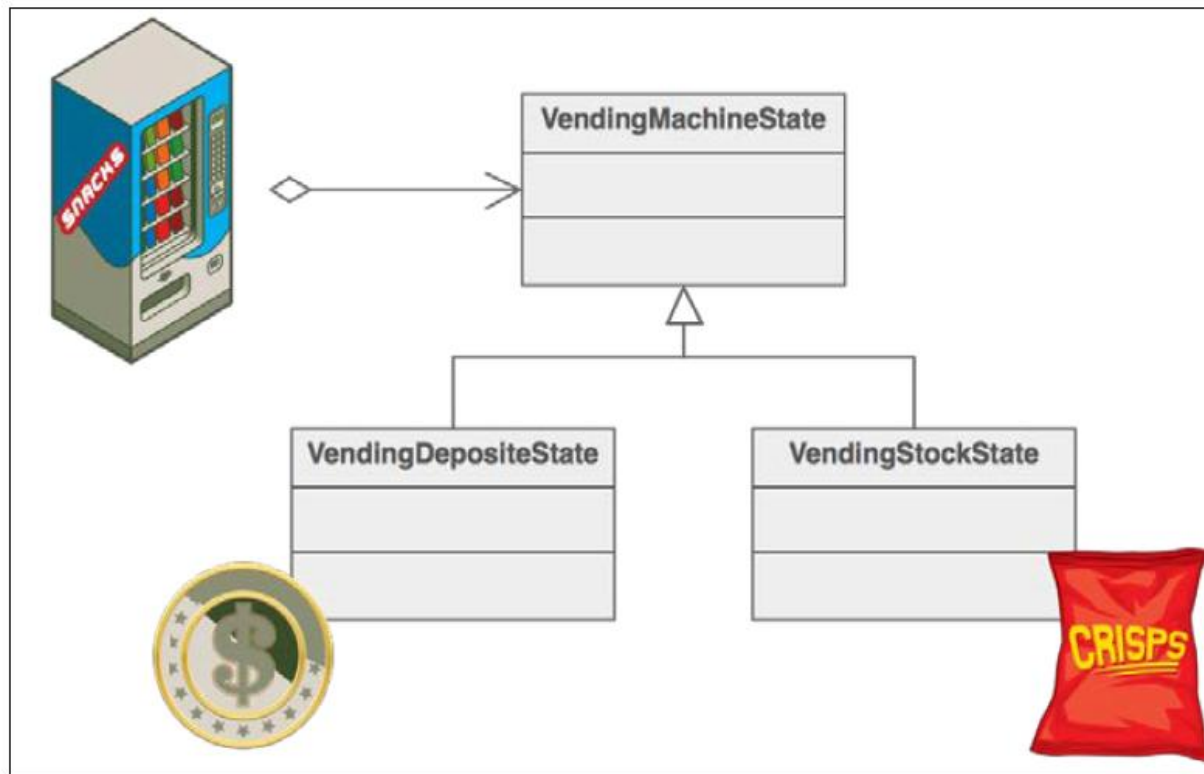
State Pattern

- Other typical states of a process are *terminated* (completed or killed), *blocked* (for example, waiting for an I/O operation to complete), and so forth. It is important to note that a state machine has only one active state at a specific point in time. For instance, a process cannot be at the same time in the state *created* and the state *running*.

State Pattern



State Pattern





State Pattern

- State machines are used to solve many computational and non-computational problems.
- Some of them are traffic lights, parking meters, hardware design, programming language parsing, and so forth. We saw how a snack vending machine relates to the way a state machine works.
- Modern software offers libraries/modules to make the implementation and usage of state machines easier.
- Django offers the third-party django-fsm package and Python also has many contributed modules.
- In fact, one of them (`state_machine`) was used in the implementation section. The **State Machine Compiler (SMC)** is yet another promising project, offering many programming language bindings (including Python).



The Strategy Pattern

- Most problems can be solved in more than one way. Take, for example, the sorting problem, which is related to putting the elements of a list in a specific order.
- There are many sorting algorithms, and, in general, none of them is considered the best for all cases
- There are different criteria that help us pick a sorting algorithm on a per-case basis.



The Strategy Pattern

- **Number of elements that need to be sorted:** This is called the input size. Almost all the sorting algorithms behave fairly well when the input size is small, but only a few of them have good performance with a large input size.
- **Best/average/worst time complexity of the algorithm:** Time complexity is (roughly) the amount of time the algorithm takes to complete, excluding coefficients and lower order terms. This is often the most usual criterion to pick an algorithm, although it is not always sufficient.

The Strategy Pattern

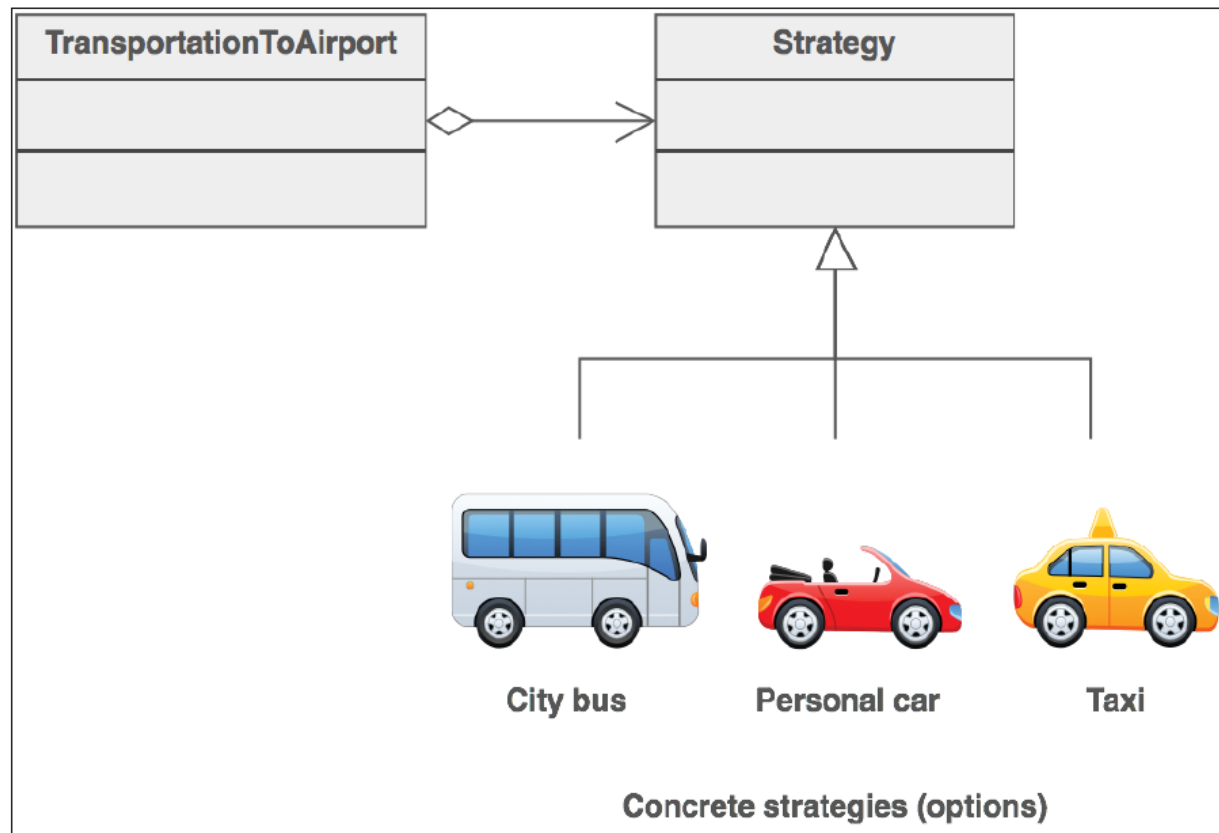
- **Space complexity of the algorithm:** Space complexity is (again roughly) the amount of physical memory needed to fully execute an algorithm. This is very important when we are working with big data or embedded systems, which usually have limited memory.
- **Stability of the algorithm:** An algorithm is considered stable when it maintains the relative order of elements with equal values after it is executed.



The Strategy Pattern

- **Code complexity of the algorithm:** If two algorithms have the same time/space complexity and are both stable, it is important to know which algorithm is easier to code and maintain.

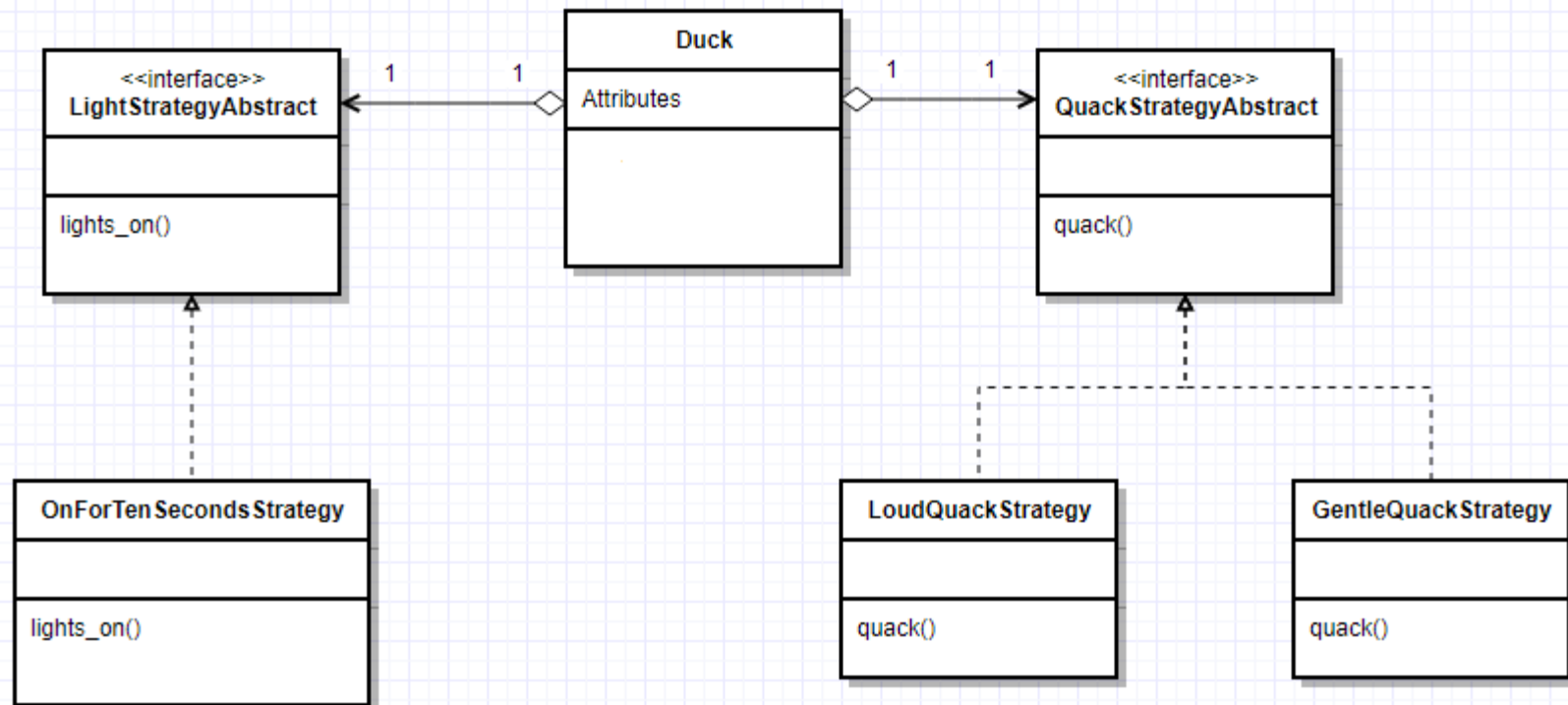
The Strategy Pattern



The Strategy Pattern



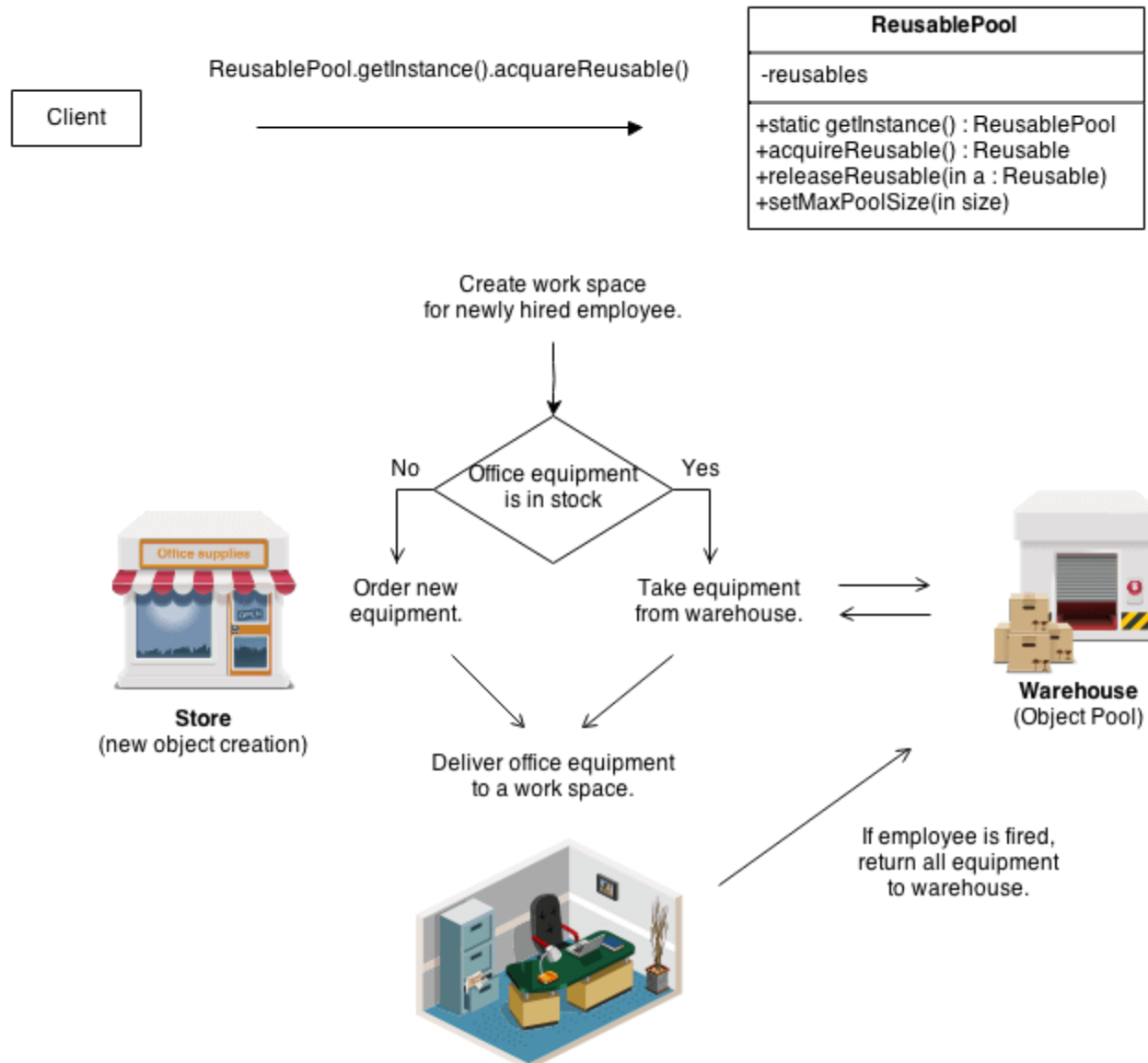
The Strategy Pattern



Object Pool Design Pattern

- Object pooling can offer a significant performance boost;
- it is most effective in situations where the cost of initializing a class instance is high, the rate of instantiation of a class is high, and the number of instantiations in use at any one time is low.
- Object pools (otherwise known as resource pools) are used to manage the object caching.
- A client with access to a Object pool can avoid creating a new Objects by simply asking the pool for one that has already been instantiated instead.
- Generally the pool will be a growing pool, i.e. the pool itself will create new objects if the pool is empty, or we can have a pool, which restricts the number of objects created.

Object Pool Design Pattern



Template Pattern

- The Template Method pattern is a behavioral design pattern that defines the program skeleton or an algorithm in a method called the Template Method.
- For example, you could define the steps to prepare a beverage as an algorithm in a Template Method.
- The Template Method pattern also helps redefine or customize certain steps of the algorithm by deferring the implementation of some of these steps to subclasses.
- This means that the subclasses can redefine their own behavior.

Template Pattern

- For example, in this case, subclasses can implement steps to prepare tea using the Template Method to prepare a beverage.
- It is important to note that the change in the steps (as done by the subclasses) don't impact the original algorithm's structure.
- Thus, the facility of overriding by subclasses in the Template Method pattern allows the creation of different behaviors or algorithms.

Template Pattern

- Abstract class is used to define the steps of the algorithm.
- These steps are also known as *primitive operations* in the context of the Template Method pattern.
- These steps are defined with abstract methods, and the Template Method defines the algorithm.
- The ConcreteClass (that subclasses the abstract class) implements subclass-specific steps of the algorithm.



Template Pattern

- The Template Method pattern is used in the following cases:
- When multiple algorithms or classes implement similar or identical logic
- The implementation of algorithms in subclasses helps reduce code duplication
- Multiple algorithms can be defined by letting the subclasses implement the behavior through overriding

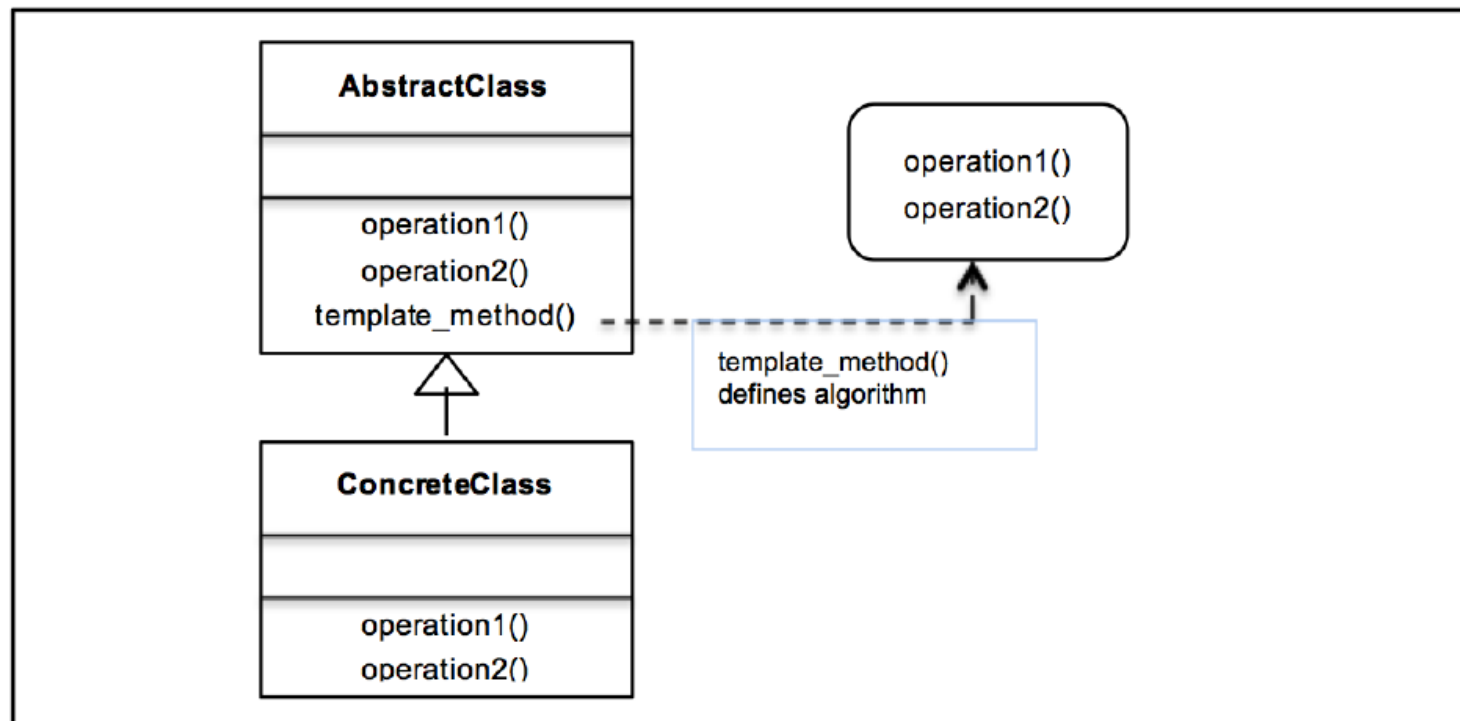
Template Pattern

- In short, the main intentions of the Template Method pattern are as follows:
- Defining a skeleton of an algorithm with primitive operations
- Redefining certain operations of the subclass without changing the algorithm's structure
- Achieving code reuse and avoiding duplicate efforts
- Leveraging common interfaces or implementations

Template Pattern

- The Template Method pattern works with the following terms—AbstractClass, ConcreteClass, Template Method, and Client:
- AbstractClass: This declares an interface to define the steps of the algorithm
- ConcreteClass: This defines subclass-specific step definitions
- `template_method()`: This defines the algorithm by calling the step methods

Template Pattern



Template Pattern

- Imagine the case of a travel agency, say, Dev Travels. Now how do they typically work?
- They define various trips to various locations and come up with a holiday package for you.
- A package is essentially a trip that you, as a customer, undertakes.
- A trip has details such as the places visited, transportation used, and other factors that define the trip itinerary.
- This same trip can be customized differently based on the needs of the customers



Template Pattern

- The Template Method pattern provides you with the following advantages:
- Code reuse happens with the Template Method pattern as it uses inheritance and not composition. Only a few methods need to be overridden.
- Flexibility lets subclasses decide how to implement steps in an algorithm.



Template Pattern

- Debugging and understanding the sequence of flow in the Template Method pattern can be confusing at times.
- You may end up implementing a method that shouldn't be implemented or not implementing an abstract method at all.
- Documentation and strict error handling has to be done by the programmer.
- Maintenance of the template framework can be a problem as changes at any level (low-level or high-level) can disturb the implementation. Hence, maintenance can be painful with the Template Method pattern.



Bridge Pattern

- The Bridge design pattern allows you to separate the abstraction from the implementation.
- It is a structural design pattern.
- **There are 2 parts in Bridge design pattern :**
 - Abstraction
 - Implementation
- **Bridge pattern is about preferring composition over inheritance. Implementation details are pushed from a hierarchy to another object with a separate hierarchy.**



Bridge Pattern

- The bridge pattern allows the Abstraction and the Implementation to be developed independently and the client code can access only the Abstraction part without being concerned about the Implementation part.
- The abstraction is an interface or abstract class and the implementor is also an interface or abstract class.
- The abstraction contains a reference to the implementor.
- Children of the abstraction are referred to as refined abstractions, and children of the implementor are concrete implementors.



Bridge Pattern

- Since we can change the reference to the implementor in the abstraction, we are able to change the abstraction's implementor at run-time.
- Changes to the implementor do not affect client code.
- It increases the loose coupling between class abstraction and its implementation.
- *Decouple an abstraction from its implementation so that the two can vary independently.*



Bridge Pattern

- Let's say that you want to build a news website and you want to show different content for different users.
- You want to give paid users full access to articles, without any ads.
- At the same time, you want to give free users some excerpts from the articles, with some ads on the page.
- Finally, you want to show a *call to action* to all free users, so you can hopefully convert them to paid users.

Composite Design Pattern

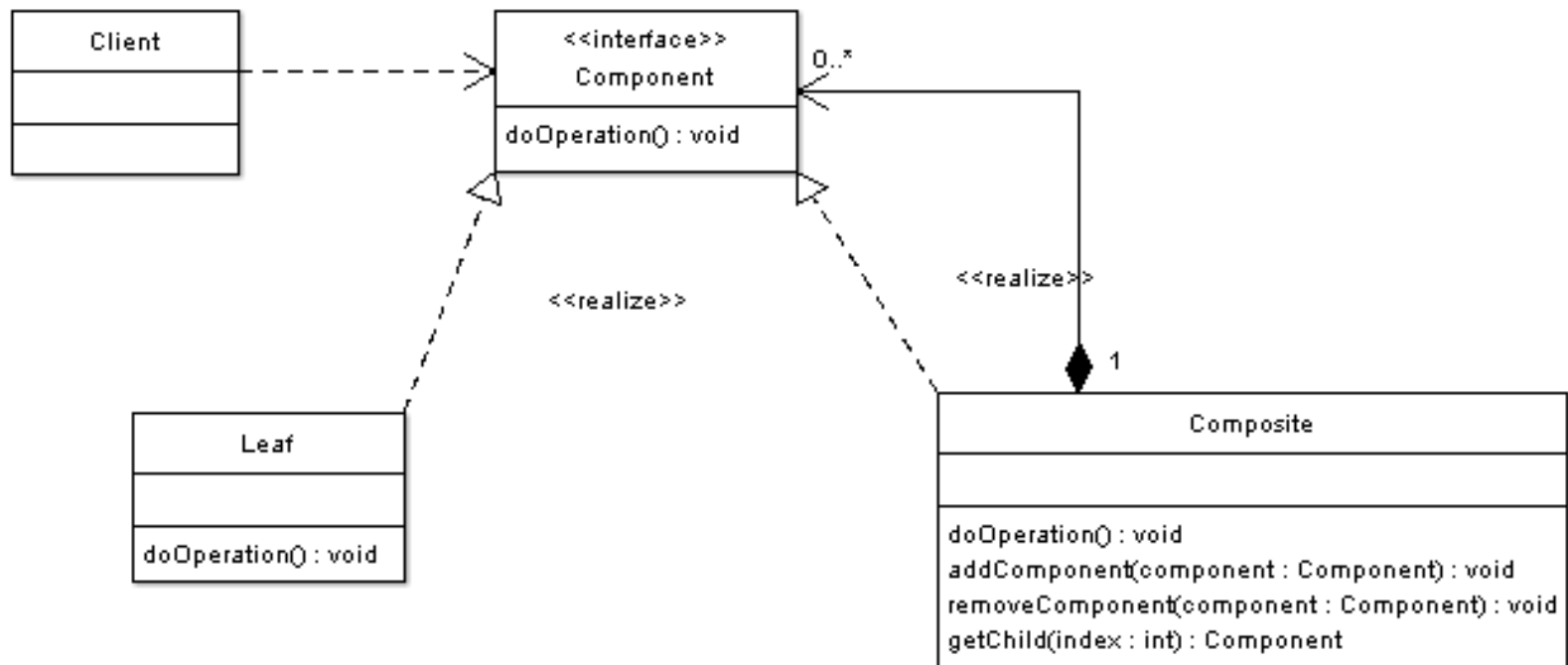
- **Composite** is a Conceptual design pattern that allows composing objects into a tree-like structure and work with the it as if it was a singular object.
- *Composite pattern lets clients to treat the individual objects in a uniform manner.*
- The Composite Pattern allows you to compose objects into a tree structure to represent the part-whole hierarchy which means you can create a tree of objects that is made of different parts, but that can be treated one big thing.
- Composite lets clients to treat individual objects and compositions of objects uniformly.



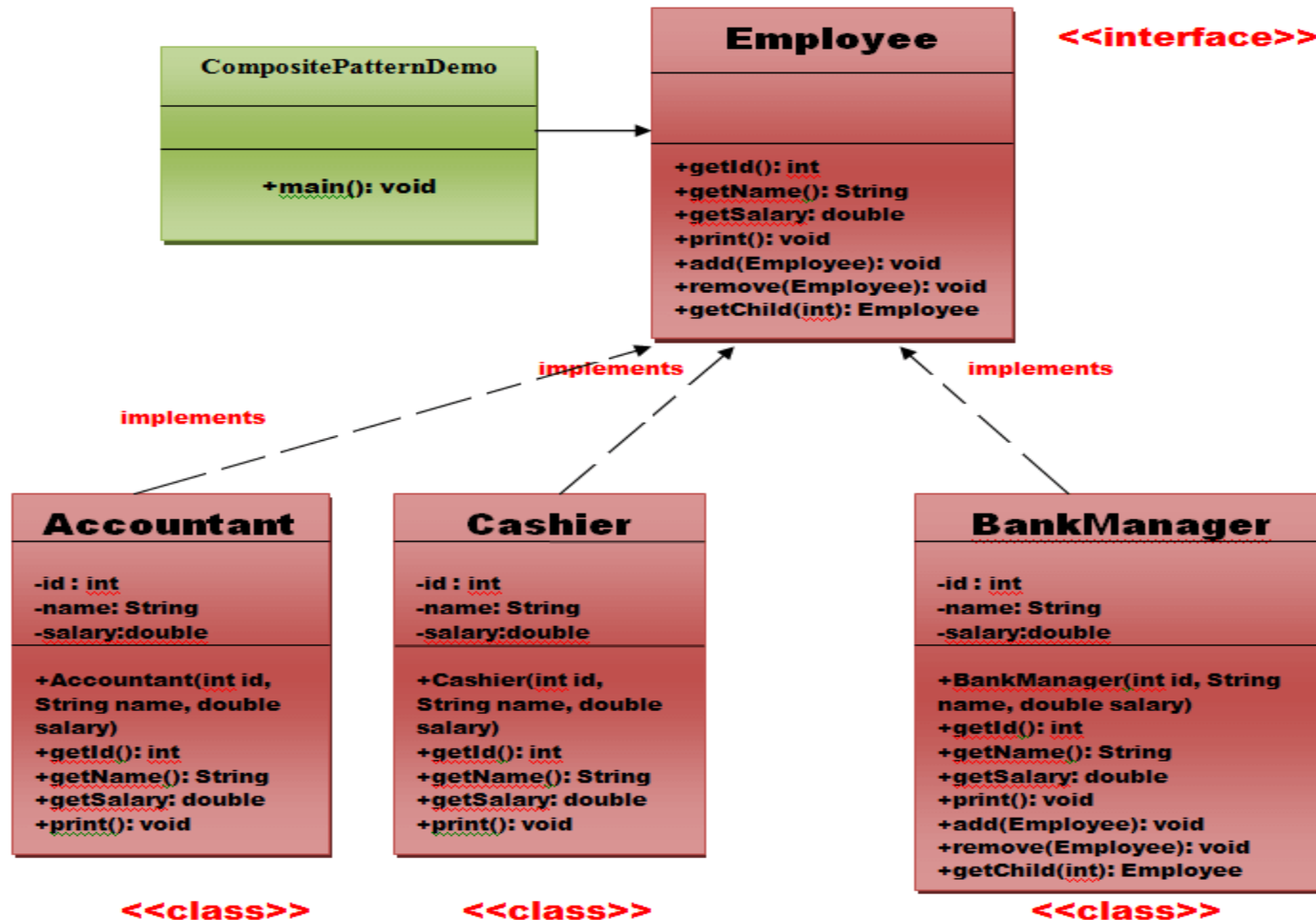
Composite Design Pattern

- For example, a program that manipulates a file system.
- A file system is a tree structure that contains Branches which are Folders as well as Leaf nodes which are Files.
- Note that a folder object usually contains one or more file or folder objects and thus is a complex object where a file is a simple object.
- Note also that since files and folders have many operations and attributes in common, such as moving and copying a file or a folder, listing file or folder attributes such as file name and size.
- It would be easier and more convenient to treat both file and folder objects uniformly by defining a File System Resource Interface

Composite Design Pattern



Composite Design Pattern



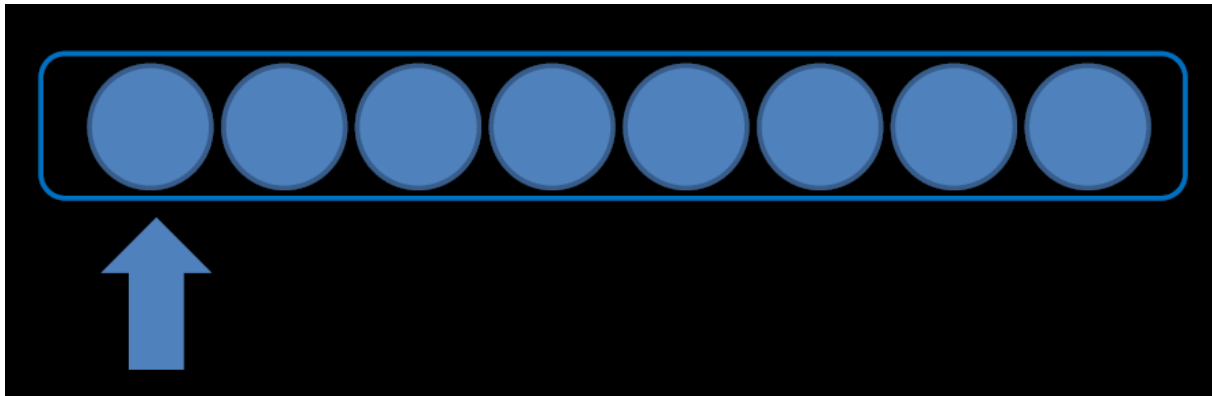


Composite Design Pattern

- Composite Pattern important points
- Composite pattern should be applied only when the group of objects should behave as the single object.
- Composite design pattern can be used to create a tree like structure.

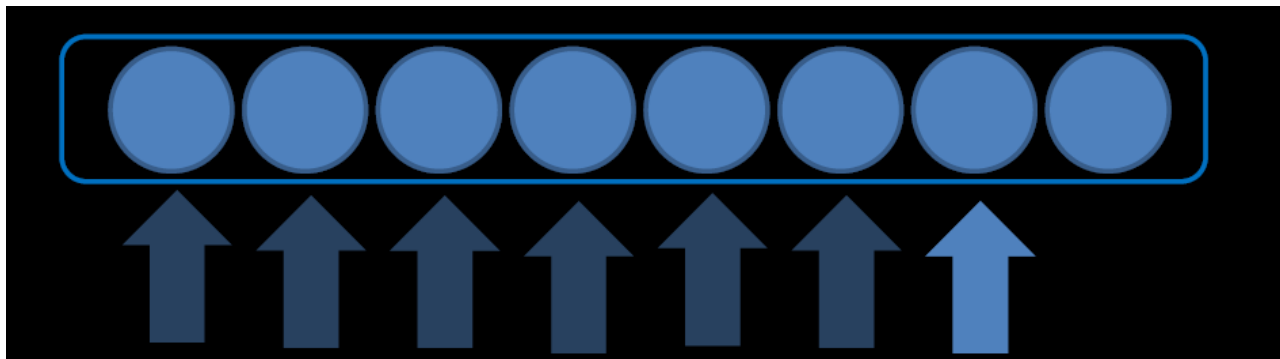
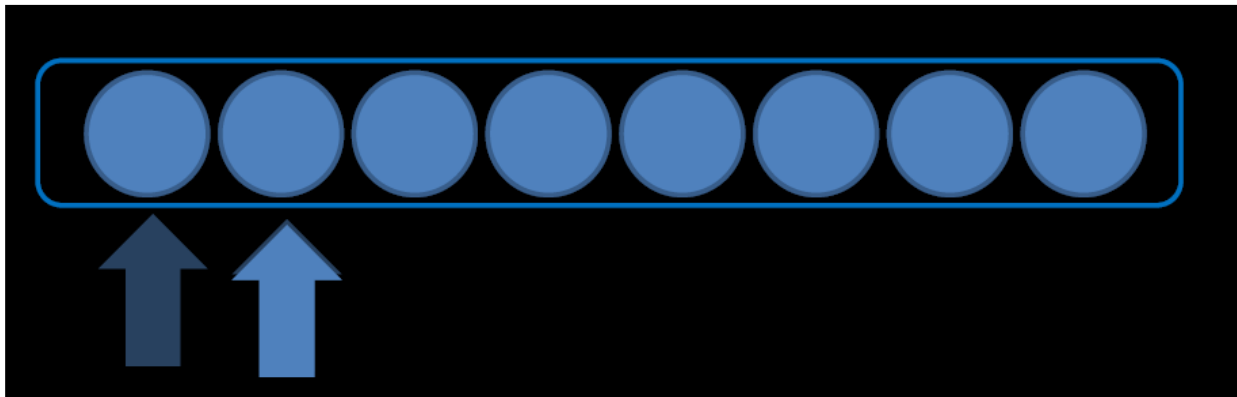
Iterator Pattern

- The iterator pattern is a design pattern in which an iterator is used to traverse a container and access the container's elements.



Iterator Pattern

The iterator pattern is a design pattern in which an iterator is used to traverse a container and access the container's elements.



Iterator Pattern

In a programming language without patterns, an iterator would have a `next()` method and a `done()` method, and the iterator loops across all the containers using these methods.

Name	Description
<code>__next__()</code>	The <code>next</code> method returns the next element from the container.
<code>StopIteration()</code>	The <code>StopIteration</code> is an exception that is raised when the last element is reached.
<code>__iter__()</code>	The <code>iter</code> method makes the object iterable, and returns an iterator.



Iterator Pattern

The iterator pattern has two parts:

- An iterable class (to create the iterator object)
- An iterator class (to traverse a container)



Iterator Pattern

Advantage of Iterator Pattern

- It supports variations in the traversal of a collection.
- It simplifies the interface to the collection.

Usage of Iterator Pattern:

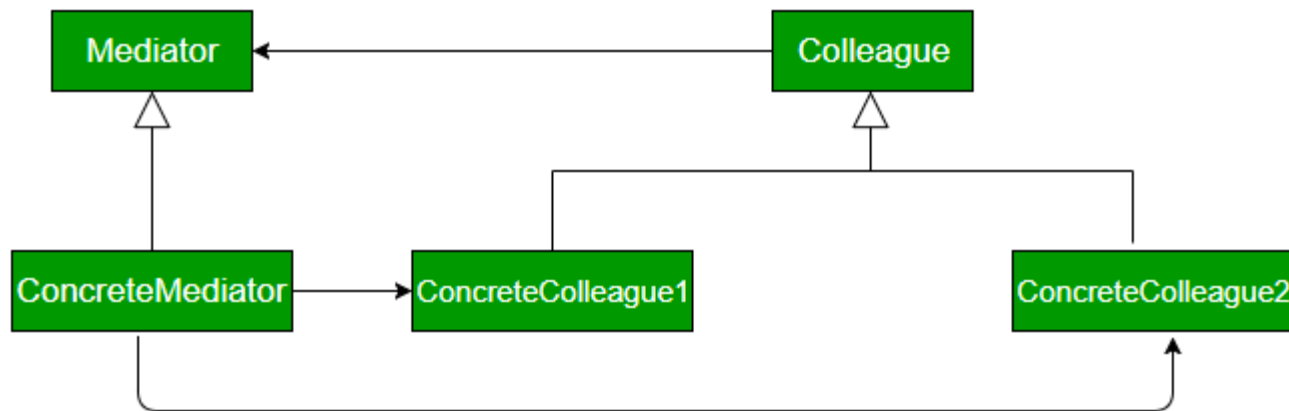
- It is used:
- When you want to access a collection of objects without exposing its internal representation.
- When there are multiple traversals of objects need to be supported in the collection.
-

Mediator Pattern

- Mediator design pattern is one of the important and widely used behavioral design pattern.
- Mediator enables decoupling of objects by introducing a layer in between so that the interaction between objects happen via the layer.
- If objects interact with each other directly, the system components are tightly-coupled with each other that makes higher maintainability cost and not hard to extend.
- Mediator pattern focuses on providing a mediator between objects for communication and help in implementing loose-coupling between objects.

Mediator Pattern

- Air traffic controller is a great example of mediator pattern where the airport control room works as a mediator for communication between different flights.
- Mediator works as a router between objects and it can have it's own logic to provide way of communication.



Mediator Pattern

Advantage

- It limits subclassing. A mediator localizes behavior that otherwise would be distributed among several objects. Changing this behaviour requires subclassing Mediator only, Colleague classes can be reused as is.

Disadvantage

- It centralizes control. The mediator pattern trades complexity of interaction for complexity in the mediator. Because a mediator encapsulates protocols, it can become more complex than any individual colleague. This can make the mediator itself a monolith that's hard to maintain

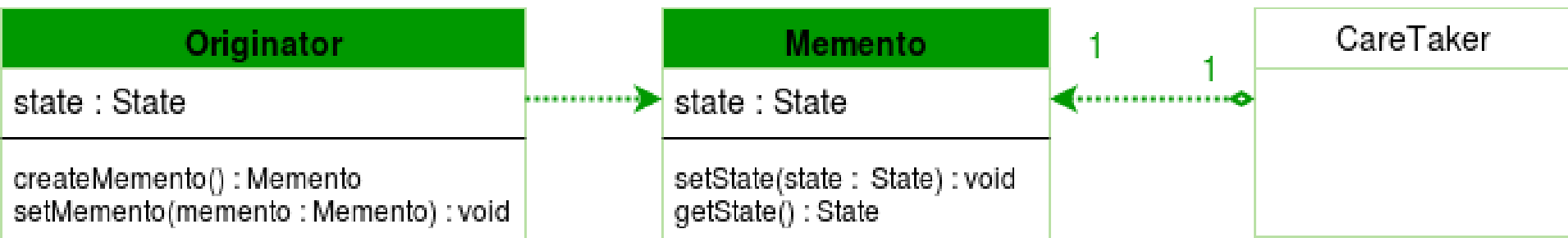


- Memento pattern is a behavioral design pattern.
- Memento pattern is used to restore state of an object to a previous state.
- As your application is progressing, you may want to save checkpoints in your application and restore back to those checkpoints later.

Memento design pattern



- **originator** : the object for which the state is to be saved. It creates the memento and uses it in future to undo.
- **memento** : the object that is going to maintain the state of originator. Its just a POJO.
- **caretaker** : the object that keeps track of multiple memento. Like maintaining savepoints.





- A **Caretaker** would like to perform an operation on the **Originator** while having the possibility to rollback.
- The caretaker calls the **createMemento()** method on the originator asking the originator to pass it a memento object.
- At this point the originator creates a memento object saving its internal state and passes the memento to the caretaker.
- The caretaker maintains the memento object and performs the operation.
- In case of the need to undo the operation, the caretaker calls the **setMemento()** method on the originator passing the maintained memento object.
- The originator would accept the memento, using it to restore its previous state.

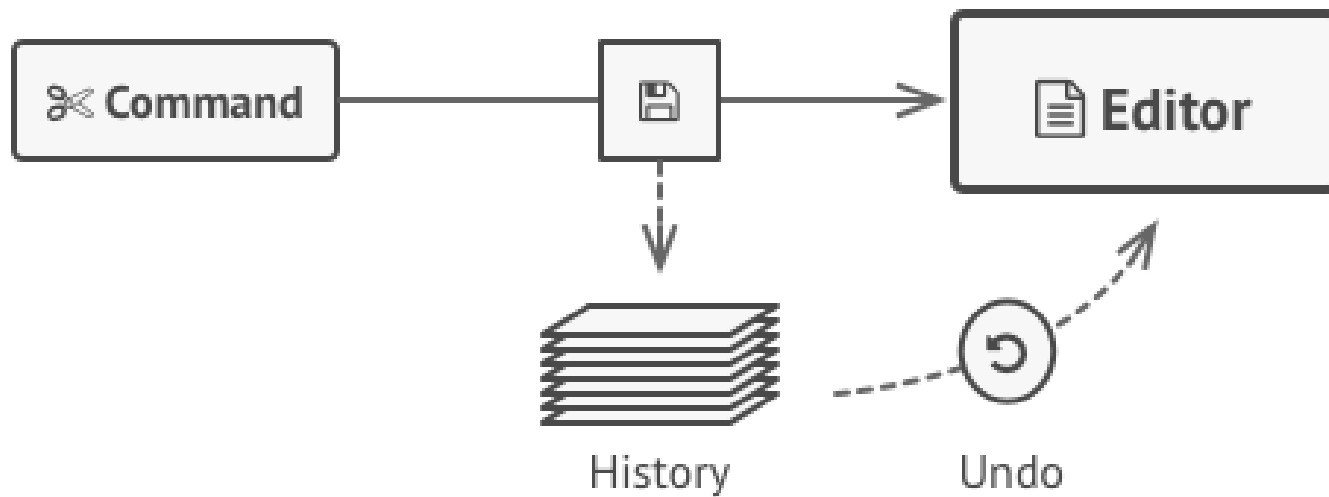


- **Advantage**
- We can use Serialization to achieve memento pattern implementation that is more generic rather than Memento pattern where every object needs to have it's own Memento class implementation.
- **Disadvantage**
- If Originator object is very huge then Memento object size will also be huge and use a lot of memory.

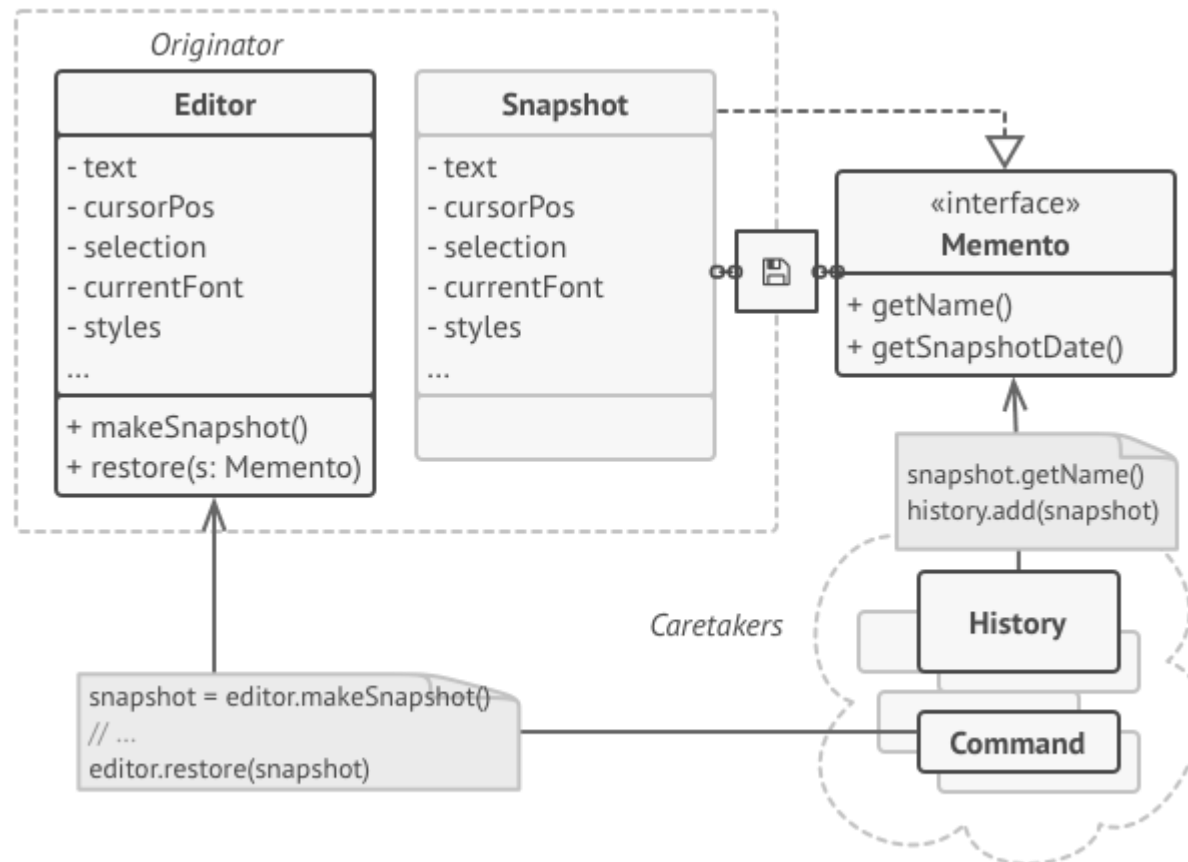
Memento design pattern



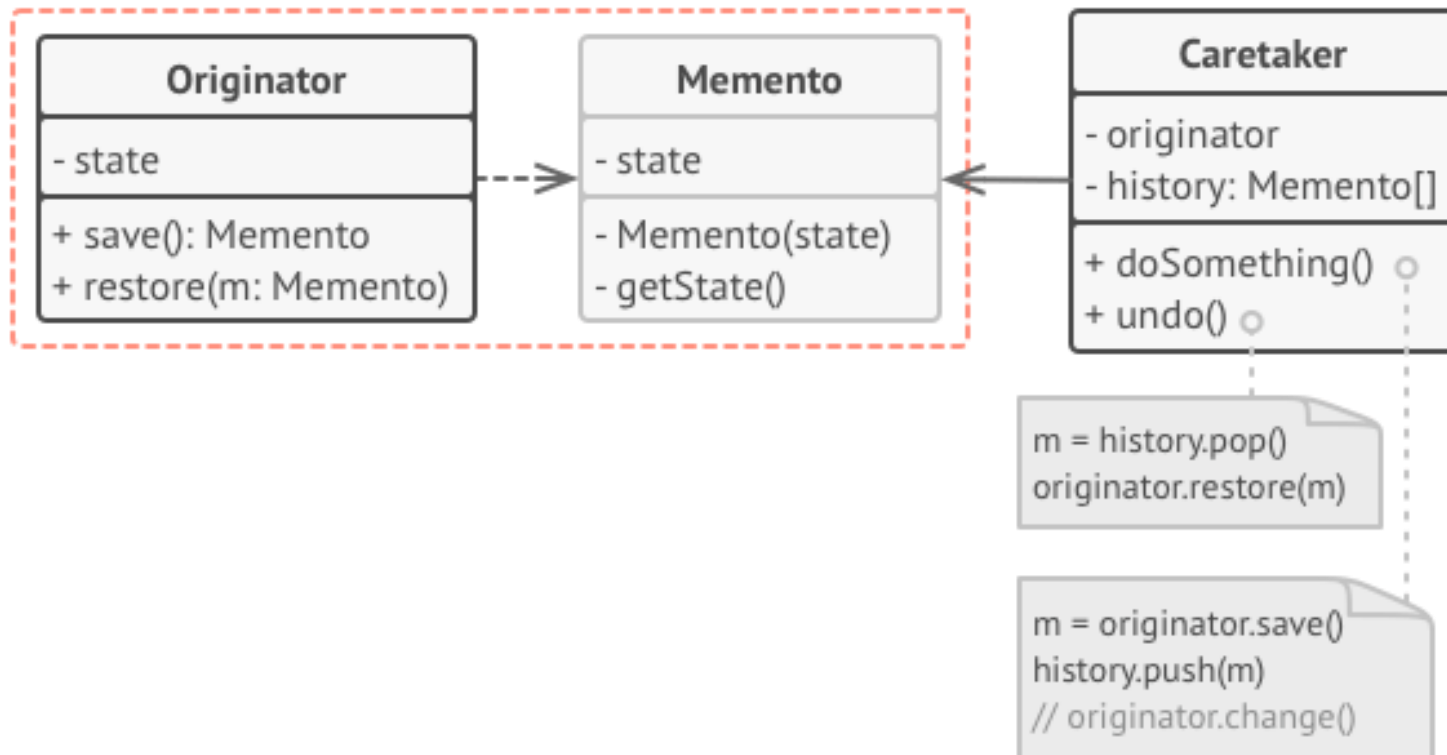
Memento design pattern



Memento design pattern



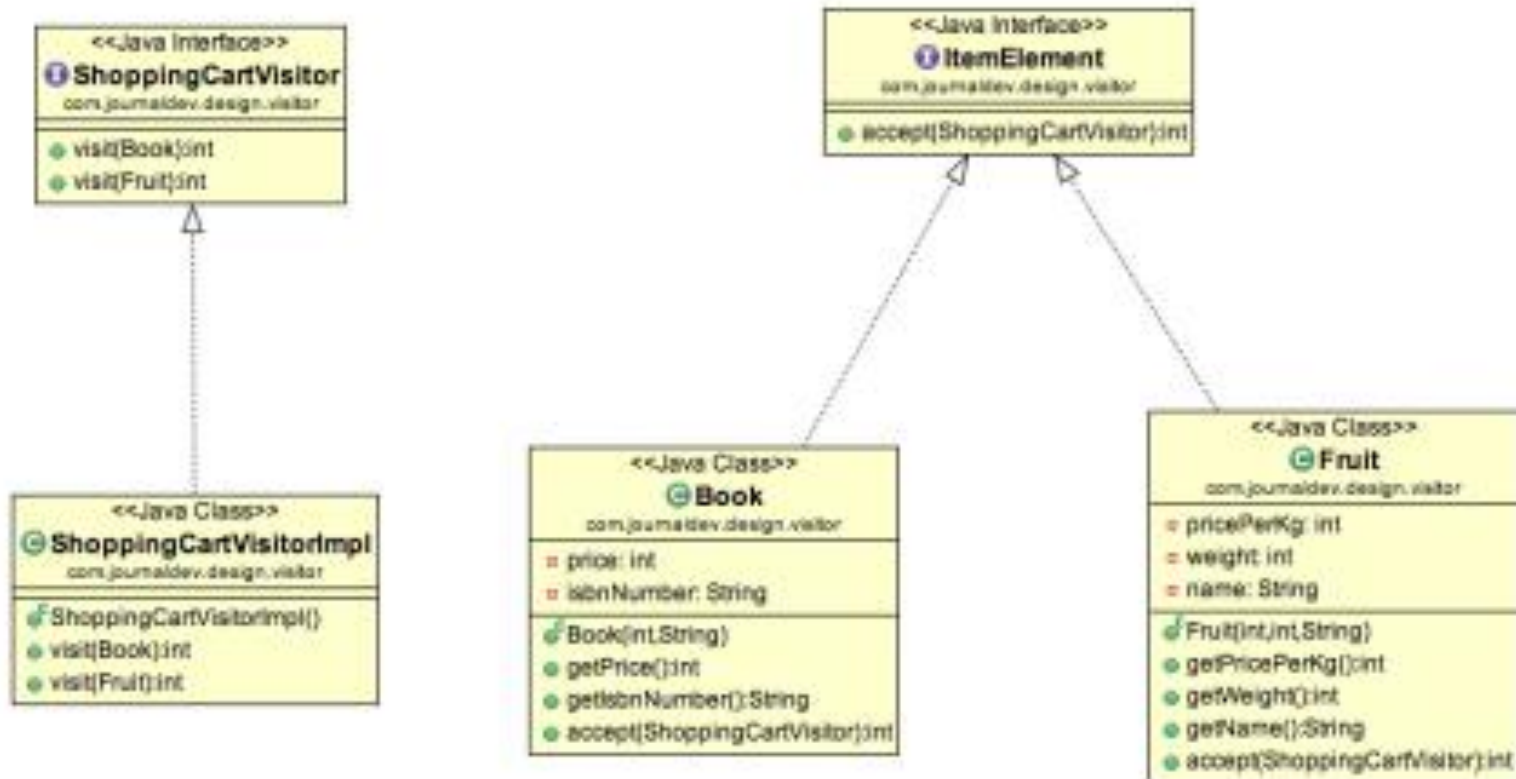
Memento design pattern



Visitor Pattern

- Visitor Design Pattern
- Visitor pattern is used when we have to perform an operation on a group of similar kind of Objects.
- With the help of visitor pattern, we can move the operational logic from the objects to another class.
- For example, think of a Shopping cart where we can add different type of items (Elements).
- When we click on checkout button, it calculates the total amount to be paid.
- Now we can have the calculation logic in item classes or we can move out this logic to another class using visitor pattern.

Visitor Pattern





Memory Management

- Memory management in Python involves a private heap containing all Python objects and data structures.
- The management of this private heap is ensured internally by the *Python memory manager*.
- The Python memory manager has different components which deal with various dynamic storage management aspects, like sharing, segmentation, preallocation or caching.



Memory Management

- At the lowest level, a raw memory allocator ensures that there is enough room in the private heap for storing all Python-related data by interacting with the memory manager of the operating system.
- On top of the raw memory allocator, several object-specific allocators operate on the same heap and implement distinct memory management policies adapted to the peculiarities of every object type.



Memory Management

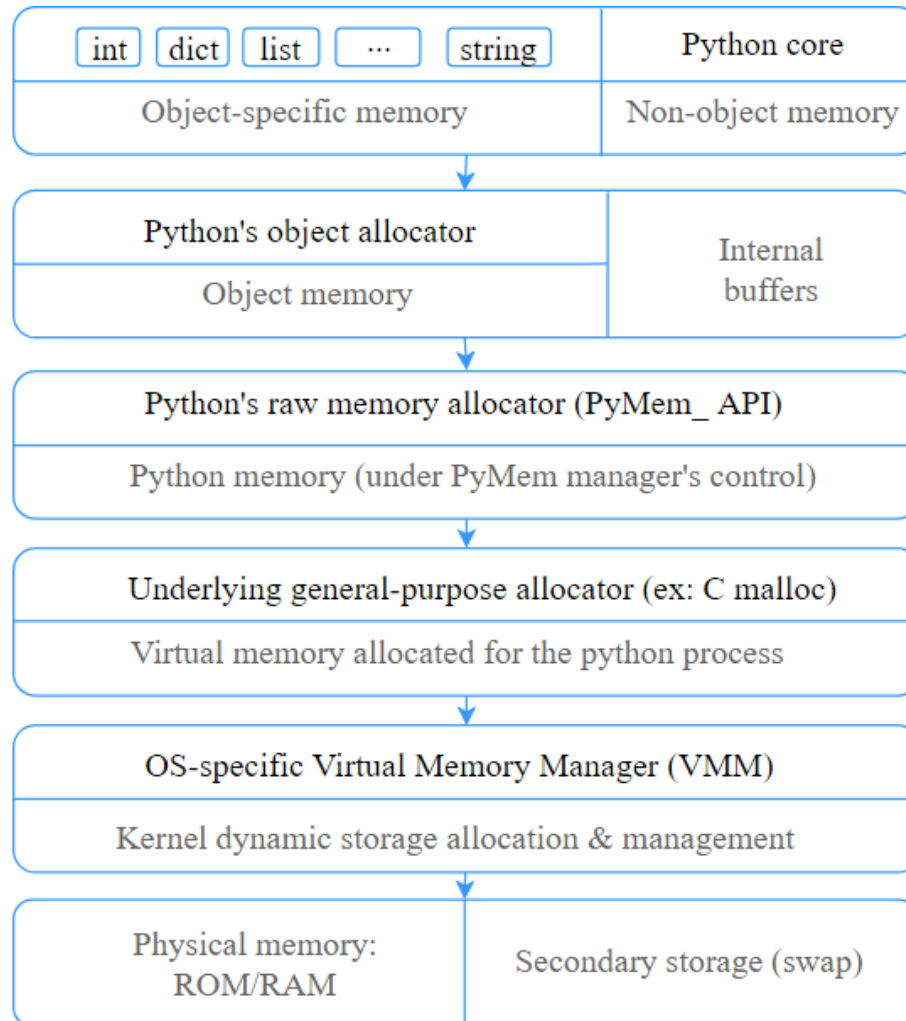
- For example, integer objects are managed differently within the heap than strings, tuples or dictionaries because integers imply different storage requirements and speed/space tradeoffs.
- The Python memory manager thus delegates some of the work to the object-specific allocators, but ensures that the latter operate within the bounds of the private heap.



Memory Management

- It is important to understand that the management of the Python heap is performed by the interpreter itself.
- User has no control over it, even if they regularly manipulate object pointers to memory blocks inside that heap.
- The allocation of heap space for Python objects and other internal buffers is performed on demand by the Python memory manager through the Python/C API functions listed in this document.

Memory Management





Memory Management

- Small object allocation
- To reduce overhead for small objects (less than 512 bytes) Python sub-allocates big blocks of memory.
- Larger objects are routed to standard C allocator. Small object allocator uses three levels of abstraction — arena, pool, and block.
- **Block**
- Block is a chunk of memory of a certain size. Each block can keep only one Python object of a fixed size. The size of the block can vary from 8 to 512 bytes and be a multiple of eight (i.e., 8-byte alignment). For convenience, such blocks are grouped in 64 size classes.

Memory Management

Request in bytes	Size of allocated block	size class idx
1-8	8	0
9-16	16	1
17-24	24	2
25-32	32	3
33-40	40	4
41-48	48	5
...	...	...
505-512	512	63

Memory Management

- **Pool**
- A collection of blocks of the same size is called a pool.
- Normally, the size of the pool is equal to the size of a memory page, i.e., 4Kb. Limiting pool to the fixed size of blocks helps with fragmentation.
- If an object gets destroyed, the memory manager can fill this space with a new object of the same size.



Memory Management

- **Each pool has three states:**
 - **used** — partially used, neither empty nor full
 - **full** — all the pool's blocks are currently allocated
 - **empty** — all the pool's blocks are currently available for allocation
- **Arena**
 - The arena is a chunk of 256kB memory allocated on the heap, which provides memory for 64 pools.



Memory Management

- **Memory deallocation**
- **Python's small object manager rarely returns memory back to the Operating System.**
- **An arena gets fully released if and only if all the pools in it are empty. For example, it can happen when you use a lot of temporary objects in a short period of time.**
- **Speaking of long-running Python processes, they may hold a lot of unused memory because of this behavior.**



Unit Testing in Python – unittest

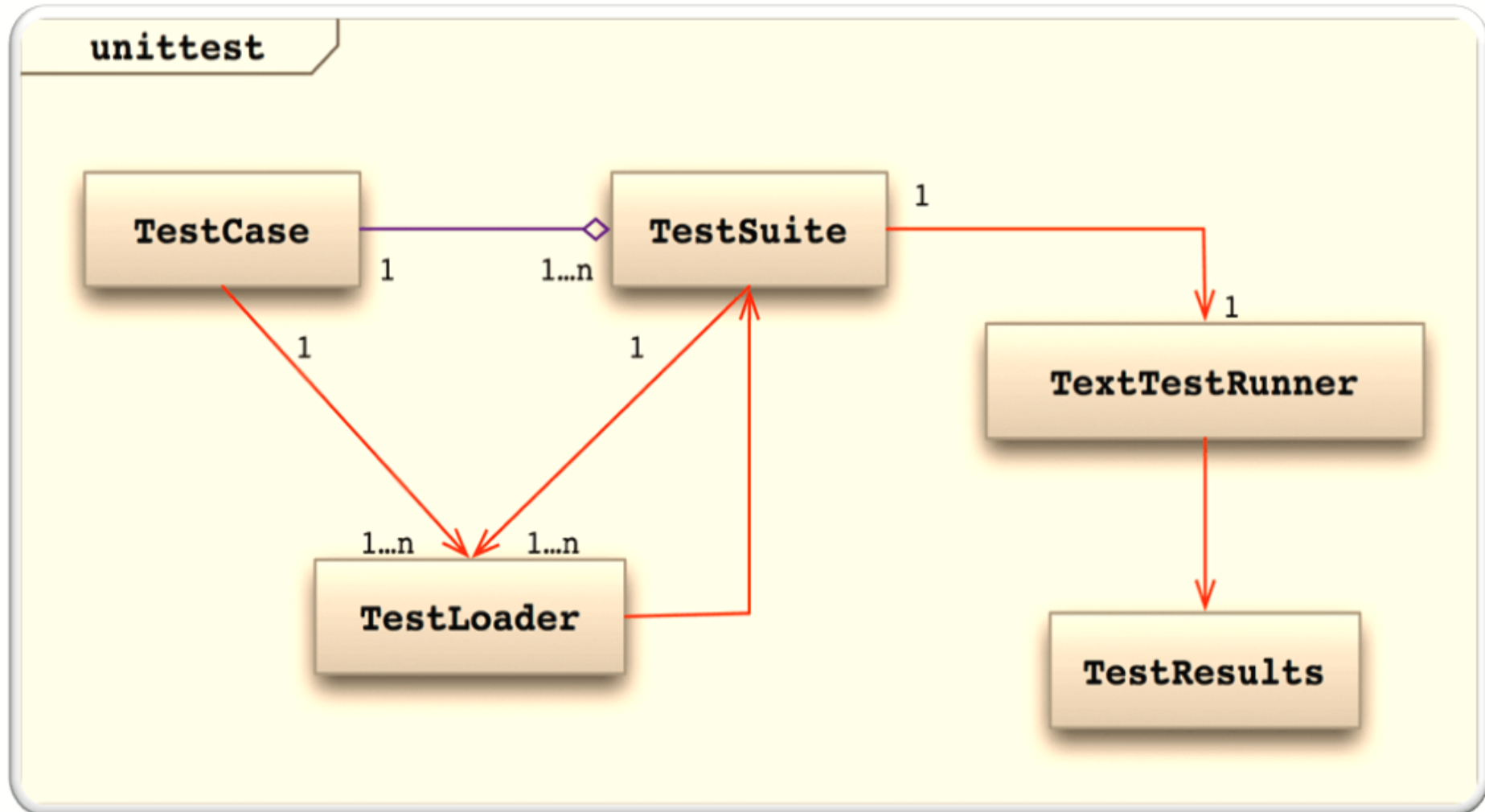
- **What is Unit Testing?**

Unit Testing is the first level of software testing where the smallest testable parts of a software are tested.

This is used to validate that each unit of the software performs as designed.

The unittest test framework is python's xUnit style framework.

Unit Testing in Python – unittest





Unit Testing in Python – unittest

- **TestCase class:** The TestCase class bears the test routines and delivers hooks for making each routine and cleaning up thereafter
- **TestSuite class:** It caters as a collection container, and it can possess multiple testcase objects and multiple testsuites objects
- **TestLoader class:** This class loads test cases and suites defined locally or from an external file. It emits a testsuite objects that possesses those suites and cases
- **TextTestRunner class:** To run the tests it caters a standard platform to execute the tests
- **The TestResults class:** It offers a standard container for the test results

Questions



Module Summary

- Regular Expressions
- Generators, Decorators and Iterators
- Socket and Network Programming
- Image Handling
- Processes and Threads
- Design Patterns

